THE NETBSD PROJECT

Руководство по pkgsrc

Система пакетов NetBSD: установка, сборка и разработка

Разработчики pkgsrc

Перевод с английского языка

Редакция от 3 августа 2026 г.

Об этом издании

Этот документ — перевод на русский язык книги *The pkgsrc guide* (https://www.netbsd.org/docs/pkgsrc/), входящей в состав документации проекта NetBSD.

Перевод выполнен автоматически с помощью больших языковых моделей и может содержать неточности. Команды, имена файлов, параметры и листинги приведены в исходном виде и не переводились. При расхождениях ориентируйтесь на оригинал.

Оригинал распространяется на условиях лицензии BSD (двухпунктовая лицензия NetBSD). Перевод распространяется на тех же условиях; к переводу применимо уведомление об авторских правах оригинального документа.

Исходный файл: pkgsrc.html · редакция 2026/08/03

Содержание

Глава 1. Что такое pkgsrc? 7
1.1. Введение 7
1.1.1. Зачем нужен pkgsrc? 7
1.1.2. Поддерживаемые платформы 8
1.2. Обзор 9
1.3. Терминология 9
1.3.1. Роли, связанные с pkgsrc 10
1.4. Типографика 11

Часть I. Руководство пользователя pkgsrc 12

Глава 2. Получение справки 13

Глава 3. Где взять pkgsrc и как поддерживать его в актуальном состоянии 14
3.1. Первое получение pkgsrc 14
3.2. Поддержание pkgsrc в актуальном состоянии 15

Глава 4. Использование pkgsrc в системах, отличных от NetBSD 17
4.1. Распространение двоичных файлов 17
4.2. Инициализация (bootstrap) pkgsrc 17

Глава 5. Использование pkgsrc 18
5.1. Использование двоичных пакетов 18
5.2. Сборка пакетов из исходных текстов 20

Глава 6. Настройка pkgsrc 24
6.1. Общая настройка 24
6.2. Переменные, влияющие на процесс сборки 24
6.3. Предпочтения для встроенного или pkgsrc-варианта программного обеспечения 25
6.4. Переменные, влияющие на процесс установки 26
6.5. Выбор и настройка компилятора 27
6.6. Настройки для разработчиков и опытных пользователей 28
6.7. Выбор параметров сборки 28

Глава 7. Создание двоичных пакетов 30
7.1. Сборка одного двоичного пакета 30
7.2. Настройки для создания двоичных пакетов 30

Глава 8. Создание двоичных пакетов для всего содержимого pkgsrc (массовые сборки) 31
8.1. Подготовка 31
8.2. Запуск сборки pbulk 31
8.3. Запуск сборки bob 33
8.4. Ускорение сканирования при массовых сборках 33
8.5. Требования к полной массовой сборке 34
8.6. Варианты массовой сборки 35
8.7. Создание коллекции пакетов на нескольких CD-ROM 39

Глава 9. Структура каталогов установленных файлов 40
9.1. Структура файловой системы в ${LOCALBASE} 40
9.2. Структура файловой системы в ${VARBASE} 41

Глава 10. Часто задаваемые вопросы 42
10.1. Существуют ли списки рассылки для обсуждения вопросов о пакетах? 42
10.2. Утилиты для управления пакетами (pkgtools) 42
10.3. Как использовать pkgsrc без прав root 43
10.4. Как возобновить передачу при загрузке дистрибутивных файлов? 44
10.5. Как установить и использовать модульный X.org из pkgsrc? 44
10.6. Как загружать файлы из-за межсетевого экрана 44
10.7. Как загружать файлы с сайтов HTTPS 44
10.8. Как указать make fetch использовать пассивный FTP? 45
10.9. Как загрузить все дистрибутивные файлы сразу 45
10.10. Что означает «Don't know how to make /usr/share/tmac/tmac.andoc»? 45
10.11. Что означает «Could not find bsd.own.mk»? 46
10.12. Использование «sudo» или «priv» с pkgsrc 46
10.13. Как изменить расположение файлов конфигурации? 46
10.14. Автоматические проверки безопасности 47
10.15. Почему некоторые пакеты игнорируют мои CFLAGS? 47
10.16. Пакет не собирается. Что делать? 48

10.17. Что означает «Makefile appears to contain unresolved cvs/rcs/??? merge conflicts»? 48

Часть II. Руководство разработчика pkgsrc 49

Глава 11. Получение помощи 51

Глава 12. Компоненты пакета — файлы, каталоги и их содержимое 52
12.1. Makefile 52
12.2. distinfo 54
12.3. patches/* 54
12.4. Другие обязательные файлы 57
12.5. Необязательные файлы 57
12.6. work* 59
12.7. files/* 59

Глава 13. Процесс сборки 60
13.1. Введение 60
13.2. Расположение программ 60
13.3. Каталоги, используемые в процессе сборки 61
13.4. Запуск фазы 61
13.5. Фаза fetch 62
13.6. Фаза checksum 64
13.7. Фаза extract 64
13.8. Фаза patch 64
13.9. Фаза tools 65
13.10. Фаза wrapper 65
13.11. Фаза configure 65
13.12. Фаза build 66
13.13. Фаза test 66
13.14. Фаза install 67
13.15. Фаза package 68
13.16. Очистка 68
13.17. Другие полезные цели 68

Глава 14. Создание нового пакета pkgsrc с нуля 74
14.1. Распространённые типы пакетов 75
14.2. Примеры 77

Глава 15. Программирование в файлах Makefile 81
15.1. Предостережения 81
15.2. Переменные Makefile 81
15.3. Фрагменты кода 82

Глава 16. Обработка опций 85
16.1. Глобальные опции по умолчанию 85
16.2. Перевод пакетов на использование bsd.options.mk 85
16.3. Имена опций 87
16.4. Определение опций зависимостей 88

Глава 17. Инструменты, необходимые для сборки и работы 89
17.1. Инструменты для сборки pkgsrc 89
17.2. Инструменты, необходимые пакетам 89
17.3. Инструменты, предоставляемые платформами 89

Глава 18. Методика buildlink 91
18.1. Перевод пакетов на buildlink3 91
18.2. Написание файлов buildlink3.mk 92
18.3. Написание файлов builtin.mk 95

Глава 19. Вопросы работы с PLIST 97
19.1. Идентификатор RCS 97
19.2. Полуавтоматическое создание PLIST 97
19.3. Настройка вывода make print-PLIST 97
19.4. Подстановка переменных в PLIST 97
19.5. Сжатие страниц руководства 99
19.6. Изменение источника PLIST с помощью PLIST_SRC 99
19.7. PLIST для разных платформ и отличающиеся PLIST 99
19.8. PLIST, зависящие от сборки 99
19.9. Совместное использование каталогов пакетами 99

Глава 20. Инфраструктура pkginstall 101
20.1. Файлы и каталоги вне префикса установки 101
20.2. Файлы конфигурации 103

20.3. Сценарии запуска системы 105
20.4. Системные пользователи и группы 106
20.5. Системные оболочки 106
20.6. Шрифты 106

Глава 21. Как заставить ваш пакет работать 108
21.1. Общие принципы работы 108
21.2. Этап fetch 119
21.3. Этап configure 121
21.4. Языки программирования 124
21.5. Этап build 126
21.6. Этап install 131
21.7. Пометка пакетов как проблемных 137

Глава 22. Упаковка и портирование GNOME 138
22.1. Метапакеты 138
22.2. Упаковка приложения GNOME 139
22.3. Обновление GNOME до более новой версии 140
22.4. Рекомендации по созданию патчей 141

Глава 23. Отправка и коммит 142
23.1. Отправка двоичных пакетов 142
23.2. Отправка пакетов с исходным кодом (для тех, кто не является разработчиком NetBSD) 142
23.3. Общие замечания при добавлении, обновлении и удалении пакетов 142
23.4. Сообщения коммитов 143
23.5. Коммит: добавление пакета в CVS 143
23.6. Обновление пакета до более новой версии 144
23.7. Переименование пакета в pkgsrc 144
23.8. Перемещение пакета в pkgsrc 145

Глава 24. Политики pkgsrc 146
24.1. Пакеты, обновление которых ограничено 146

Глава 25. Часто задаваемые вопросы 147

Часть III. Внутреннее устройство инфраструктуры pkgsrc 149

Глава 26. Устройство инфраструктуры pkgsrc 151
26.1. Смысл определений переменных 151
26.2. Как избежать проблем до их появления 151
26.3. Вычисление значений переменных 152
26.4. Как можно задавать переменные? 152
26.5. Проектирование интерфейсов для фрагментов Makefile 152
26.6. Порядок загрузки файлов 153

Глава 27. Регрессионные тесты 155
27.1. Запуск регрессионных тестов 155
27.2. Добавление нового регрессионного теста 155

Глава 28. Портирование pkgsrc 157
28.1. Портирование pkgsrc на новую операционную систему 157

Приложение A. Простой пример пакета: bison 158
A.1. files 158
A.1.1. Makefile 158
A.1.2. DESCR 158
A.1.3. PLIST 158
A.1.4. Проверка пакета с помощью pkglint 158
A.2. Этапы сборки, установки и упаковки 159

Приложение B. Усиление безопасности 162
B.1. Механизмы 162
B.1.1. Включено по умолчанию 162
B.1.2. Не включено по умолчанию 163
B.2. Предостережения 164
B.2.1. Проблемы с PKGSRC_MKPIE 164
B.2.2. Проблемы с PKGSRC_USE_FORTIFY 165
B.2.3. Проблемы с PKGSRC_USE_RELRO 165
B.2.4. Проблемы с PKGSRC_USE_SSP 166
B.3. Аудит системы 167
B.3.1. Проверка на PIE 167
B.3.2. Проверка на частичный RELRO 167
B.3.3. Проверка на полный RELRO 168

B.3.4. Проверка на SSP 168

Приложение C. Журналы сборки 169
C.1. Сборка figlet 169
C.2. Упаковка figlet 170

Приложение D. Структура каталогов FTP-сервера pkgsrc 171
D.1. distfiles: дистрибутивные файлы с исходными текстами 171
D.2. misc: разное 171
D.3. packages: двоичные пакеты 171
D.4. reports: отчёты о массовых сборках 172
D.5. current, stable, pkgsrc-20xxQy: пакеты из исходных текстов 172

Приложение E. Темы справки 173

Приложение F. Рекомендации по редактированию руководства pkgsrc 196
F.1. Цели make 196
F.2. Порядок действий 196

Глава 1. Что такое pkgsrc?

1.1. Введение

Для систем на базе Unix свободно доступно множество программ, обычно в виде исходных текстов. Прежде чем такие программы можно будет использовать, их нужно настроить под локальную систему, скомпилировать и установить — именно это и делает коллекция пакетов NetBSD (The NetBSD Packages Collection, pkgsrc). В pkgsrc есть также базовые команды для работы с двоичными пакетами, чтобы каждому пользователю не приходилось собирать пакеты самостоятельно, что отнимает много времени.

В настоящее время pkgsrc содержит несколько тысяч пакетов, в том числе:

- `www/apache24` — веб-сервер Apache
- `www/firefox` — веб-браузер Firefox
- `meta-pkgs/gnome` — рабочий стол GNOME
- `meta-pkgs/kde4` — рабочий стол KDE

… и это лишь некоторые из них.

В pkgsrc встроена поддержка работы с различными зависимостями, такими как pthreads и X11, а также расширенные возможности, например поддержка IPv6 на ряде платформ.

1.1.1. Зачем нужен pkgsrc?

pkgsrc предоставляет следующие ключевые возможности:

- Простая сборка программ из исходных текстов, а также создание и установка двоичных пакетов. Исходные тексты и последние патчи загружаются с основного сайта или зеркала, проверяется контрольная сумма, после чего выполняется сборка на вашей системе. Поддерживаются и дистрибутивы, состоящие только из двоичных файлов, — как для собственных платформ, так и для платформ, эмулируемых в NetBSD.
- Все пакеты устанавливаются в единое дерево каталогов, включая двоичные файлы, библиотеки, страницы руководства (man) и прочую документацию.
- Автоматическое отслеживание зависимостей пакетов, в том числе при выполнении обновлений, чтобы требуемые пакеты были установлены. Файлы конфигурации различных пакетов при обновлении обрабатываются автоматически, поэтому локальные изменения сохраняются.
- Как и NetBSD, pkgsrc спроектирован с расчётом на переносимость и состоит из хорошо переносимого кода. Это обеспечивает максимальную скорость разработки при портировании на новую платформу. Благодаря этой переносимости pkgsrc *одинаково ведёт себя на всех платформах*.
- Префикс установки, допустимые лицензии на программы, требования к международному шифрованию и параметры сборки для большого числа пакетов задаются в одном простом центральном файле конфигурации.
- Все исходные тексты (без дистрибутивных файлов) свободно доступны на условиях лицензии BSD, поэтому вы можете расширять и адаптировать pkgsrc под свои нужды. Поддержка

локальных пакетов и патчей доступна сразу из коробки, так что вы можете настроить её под своё окружение.

В основе pkgsrc лежат следующие принципы:

- «Работать должно только то, что сделано правильно.» — Это значит, что если пакет содержит ошибки, лучше найти их и сообщить о них, чем просто установить пакет и надеяться, что он работает. В pkgsrc есть множество проверок, которые пытаются выявить такие ошибки: средства статического анализа (`pkgtools/pkglint`), проверки во время сборки (переносимость сценариев оболочки) и проверки после установки (установленные файлы, ссылки на разделяемые библиотеки, интерпретаторы сценариев).
- «Если это работает, оно должно работать везде» — Подобно тому как NetBSD была портирована на множество аппаратных архитектур, pkgsrc портирован на множество операционных систем. Принимаются меры к тому, чтобы пакеты вели себя одинаково на всех платформах.

1.1.2. Поддерживаемые платформы

Для этих операционных систем pkgsrc поставляется как в виде дистрибутива исходных текстов, так и в виде дистрибутива двоичных файлов. Получив нужные исходные тексты или двоичные файлы, вы сможете начать работу с pkgsrc буквально за несколько минут!

pkgsrc произошёл от системы ports проекта FreeBSD и изначально разрабатывался только для NetBSD. С тех пор pkgsrc сильно вырос и теперь поддерживает следующие платформы:

Таблица 1.1. Платформы, поддерживаемые pkgsrc

Платформа	Дата добавления поддержки	Примечания
NetBSD	Авг. 1997	
Solaris	Март 1999	README.Solaris
Linux	Июнь 1999	README.Linux
Darwin / Mac OS X / OS X / macOS	Окт. 2001	README.macOS
FreeBSD	Нояб. 2002	README.FreeBSD
OpenBSD	Нояб. 2002	README.OpenBSD
IRIX	Дек. 2002	README.IRIX
BSD/OS	Дек. 2003	Удалён из pkgsrc в марте 2025
AIX	Дек. 2003	README.AIX
Interix (Microsoft Windows Services for Unix)	Март 2004	Удалён из pkgsrc в апреле 2025
DragonFlyBSD	Окт. 2004	
OSF/1	Нояб. 2004	README.OSF1
HP-UX	Апр. 2007	README.HPUX
Haiku	Сент. 2010	README.Haiku
MirBSD	Янв. 2011	Удалён из pkgsrc в марте 2025
Minix3	Нояб. 2011	README.Minix3
Cygwin	Март 2013	README.Cygwin
GNU/kFreeBSD	Июль 2013	Удалён из pkgsrc в марте 2025
Bitrig	Июнь 2014	Удалён из pkgsrc в сентябре 2022

1.2. Обзор

Этот документ разделён на три части. Первая, Руководство пользователя pkgsrc, описывает, как использовать один из пакетов коллекции: либо установив заранее собранный двоичный пакет, либо собрав собственную копию с помощью системы пакетов NetBSD. Вторая часть, Руководство разработчика pkgsrc, объясняет, как подготовить пакет, чтобы другие пользователи NetBSD могли легко его собрать, не зная подробностей устройства пакета. Третья часть, Внутреннее устройство инфраструктуры pkgsrc, предназначена для тех, кто хочет понять, как реализован pkgsrc.

Этот документ доступен в различных форматах: HTML, PDF, PS, TXT.

1.3. Терминология

До сих пор много говорилось о «портах», «пакетах» и т. п. Ниже описана вся терминология, используемая в этом документе.

Пакет

Набор файлов и инструкций по сборке, описывающих, что необходимо для сборки определённой программы с помощью pkgsrc. Пакеты традиционно хранятся в `/usr/pkgsrc`, но могут размещаться в любом месте, обозначаемом как `PKGSRCDIR`.

Система пакетов NetBSD

Это прежнее название «pkgsrc». Она является частью операционной системы NetBSD и может быть инициализирована (bootstrap) для работы в операционных системах, отличных от NetBSD. Она обеспечивает сборку (компиляцию), установку и удаление пакетов.

Дистрибутивный файл

Этим термином обозначают файл или файлы, которые автор программы предоставляет для распространения своей работы. Все изменения, необходимые для сборки в NetBSD, отражены в соответствующем пакете. Обычно дистрибутивный файл представляет собой сжатый tar-архив, но возможны и другие типы. Дистрибутивные файлы обычно хранятся в `/usr/pkgsrc/distfiles`.

Порт

Этим термином в FreeBSD и OpenBSD называют то, что мы называем пакетом. В терминологии NetBSD «порт» означает другую архитектуру.

Заранее собранный (двоичный) пакет

Набор двоичных файлов, собранных с помощью pkgsrc из дистрибутивного файла и упакованных в один файл `.tgz`, чтобы их можно было установить на машинах той же архитектуры без повторной компиляции. Пакеты обычно создаются в `/usr/pkgsrc/packages`; кроме того, архив есть на ftp.NetBSD.org.

Иногда это также называют «пакетом», особенно когда речь идёт о заранее собранных пакетах.

Программа

Программа, подлежащая установке, которая будет создана из всех файлов дистрибутивного файла действиями, определёнными в соответствующем пакете.

1.3.1. Роли, связанные с pkgsrc

пользователи pkgsrc

Пользователи pkgsrc — это люди, которые используют пакеты, предоставляемые pkgsrc. Как правило, это системные администраторы. Люди, использующие программы, находящиеся внутри пакетов (их можно назвать «конечными пользователями»), не рассматриваются в руководстве pkgsrc.

Пользователи pkgsrc бывают двух видов. Одни хотят только устанавливать заранее собранные двоичные пакеты. Другие собирают пакеты pkgsrc из исходных текстов — либо для непосредственной установки, либо для создания собственных двоичных пакетов. Пользователи pkgsrc найдут всю необходимую документацию в Часть I, «Руководство пользователя pkgsrc».

сопровождающие пакетов

Сопровождающий пакета создаёт пакеты так, как описано в Часть II, «Руководство разработчика pkgsrc».

разработчики инфраструктуры

Эти люди занимаются всеми файлами, которые находятся в каталоге `mk/` и ниже. Только им нужно читать Часть III, «Внутреннее устройство инфраструктуры pkgsrc», хотя другие тоже могут проявить любопытство.

1.4. Типографика

В примерах команд используются приглашения оболочки, показывающие, должна ли команда выполняться от имени root или достаточно прав «обычного» пользователя. Символ `#` обозначает приглашение оболочки root, `%` — приглашение оболочки пользователя (предполагается C-shell или tcsh), а `$` — приглашение Bourne shell и его производных.

Часть I. Руководство пользователя pkgsrc

Глава 2. Получение справки

Основной источник справки по pkgsrc — этот документ, руководство pkgsrc. Если здесь вы ничего не нашли, есть другие возможности:

- Встроенная справка pkgsrc, доступная после инициализации (bootstrap) pkgsrc. Выполните **bmake help topic=...**, чтобы получить справку по любой теме: имя переменной, например `BUILD_DEFS`, цель make, например **do-build**, отсутствующая функция C или C++, например **strcasecmp**, или любая другая тема.

  Доступные темы справки перечислены в Приложение E, *Темы справки*.
- Чтобы увидеть значение одной переменной, выполните **bmake show-var VARNAME=*X***.
- Чтобы увидеть значения самых распространённых переменных, выполните **bmake show-all**. Эти переменные сгруппированы по темам. Чтобы увидеть переменные одной темы, выполните **bmake show-all-*topic***, например **bmake show-all-fetch**.
- Список рассылки pkgsrc-users, на который можно подписаться, а затем задавать свои вопросы.
- Канал IRC #pkgsrc, доступный через веб-браузер или с помощью специализированной программы для общения, например XChat. Выберите любое имя пользователя и присоединитесь к каналу #pkgsrc.

Глава 3. Где взять pkgsrc и как поддерживать его в актуальном состоянии

Прежде чем загружать и распаковывать файлы, нужно решить, куда их распаковывать. При использовании pkgsrc от имени пользователя root pkgsrc обычно устанавливается в `/usr/pkgsrc`. Тем не менее вы вправе установить исходные тексты и двоичные пакеты в любое место файловой системы, при условии, что путь не содержит пробельных символов и других символов, которые особым образом интерпретируются оболочкой и некоторыми другими программами. Надёжный вариант — использовать только буквы, цифры, знаки подчёркивания и дефисы.

3.1. Первое получение pkgsrc

Прежде чем загружать какие-либо файлы pkgsrc, следует решить, нужна ли вам ветка *current* или ветка *stable*. Последняя ответвляется от текущей ветки раз в квартал и изменяется только при обновлениях безопасности. Имена стабильных веток составляются из года и квартала, например `2026Q2`.

Второй шаг — решить, *как* вы хотите загрузить pkgsrc. Это можно сделать в виде tar-файла или через CVS. Здесь описаны оба способа.

Учтите, что tar-архив содержит рабочую копию CVS. Таким образом, вы можете перейти на использование CVS в любой момент в дальнейшем.

3.1.1. В виде tar-архива

Основное место загрузки всех файлов pkgsrc — https://cdn.NetBSD.org/pub/pkgsrc/ или ftp://ftp.NetBSD.org/pub/pkgsrc/ (оба адреса указывают на одно и то же место). Там есть ряд подкаталогов для разных целей, которые подробно описаны в Приложение D, *Структура каталогов FTP-сервера pkgsrc*.

tar-архив текущей ветки находится в каталоге `current` и называется `pkgsrc.tar.gz`. Он создаётся автоматически еженедельно.

Чтобы сэкономить время загрузки, мы предоставляем архивы, сжатые bzip2 и xz, которые публикуются соответственно по адресам `pkgsrc.tar.bz2` и `pkgsrc.tar.xz`.

Те же файлы можно получить по FTP.

tar-файл стабильной ветки 2026Q2 находится в каталоге `pkgsrc-2026Q2` и также называется `pkgsrc.tar.gz`.

Чтобы загрузить последний стабильный tar-архив pkgsrc, выполните:

```
$ ftp ftp://ftp.NetBSD.org/pub/pkgsrc/pkgsrc-2026Q2/pkgsrc.tar.gz
```

При желании его можно также загрузить с помощью «wget», «curl» или веб-браузера.

Затем распакуйте его:

```
$ tar -xzf pkgsrc.tar.gz -C /usr
```

Будет создан каталог `pkgsrc/` в `/usr/`, и все исходные тексты пакетов будут размещены в `/usr/pkgsrc/`.

Чтобы загрузить pkgsrc-current, выполните:

```
$ ftp ftp://ftp.NetBSD.org/pub/pkgsrc/current/pkgsrc.tar.gz
```

3.1.2. Через анонимный CVS

Чтобы получить конкретную стабильную ветку pkgsrc, выполните:

```
$ cd /usr && cvs -q -z2 -d anoncvs@anoncvs.NetBSD.org:/cvsroot checkout -r pkgsrc-2026Q2 -P
pkgsrc
```

Будет создан каталог `pkgsrc/` в вашем каталоге `/usr/`, и все исходные тексты пакетов будут размещены в `/usr/pkgsrc/`.

Чтобы получить текущую ветку pkgsrc, выполните:

```
$ cd /usr && cvs -q -z2 -d anoncvs@anoncvs.NetBSD.org:/cvsroot checkout -P pkgsrc
```

Обратитесь к списку доступных зеркал, чтобы при необходимости выбрать более быстрое зеркало CVS.

Если вы получаете сообщения об ошибках от `rsh`, нужно задать переменную CVS_RSH. Например:

```
$ cd /usr && env CVS_RSH=ssh cvs -q -z2 -d anoncvs@anoncvs.NetBSD.org:/cvsroot checkout -P
pkgsrc
```

О том, как задать CVS_RSH=ssh постоянно, см. документацию по вашей командной оболочке. Для оболочек Bourne это можно сделать в файле `.profile` или, что лучше, глобально в `/etc/profile`:

```
# set CVS remote shell command
CVS_RSH=ssh
export CVS_RSH
```

По умолчанию CVS ведёт себя не так, как ожидает большинство людей. Но CVS можно убедить, создав в домашнем каталоге файл `.cvsrc` и сохранив в нём следующие строки. Этот файл избавит вас от множества головной боли и некоторых отчётов об ошибках, поэтому мы настоятельно его рекомендуем. Объяснение этого файла можно найти в документации по CVS.

```
# recommended CVS configuration file from the pkgsrc guide
cvs -q
checkout -P
update -dP
diff -upN
rdiff -u
release -d
```

3.2. Поддержание pkgsrc в актуальном состоянии

Предпочтительный способ поддерживать pkgsrc в актуальном состоянии — через CVS (это работает и в том случае, если вы сначала установили его из tar-файла). По сравнению с повторной загрузкой tar-файла это экономит трафик и нагрузку на жёсткий диск.

3.2.1. Через tar-файлы

> **ПРЕДУПРЕЖДЕНИЕ**
>
> При обновлении из tar-файла сначала нужно полностью удалить старый каталог pkgsrc. Иначе те файлы, которые за это время были удалены из pkgsrc, останутся на вашем локальном диске, что приведёт к несоответствиям. При удалении старых файлов все изменения, которые вы внесли в файлы pkgsrc, будут потеряны после обновления. Поэтому настоятельно рекомендуется обновляться через CVS.

Учтите, что по умолчанию дистрибутивные файлы и двоичные пакеты сохраняются в дереве pkgsrc, поэтому не забудьте спасти их перед обновлением. Можно также настроить pkgsrc так, чтобы дистрибутивные файлы и пакеты хранились в каталогах вне дерева pkgsrc, задав переменные `DISTDIR` и `PACKAGES`. Подробности см. в Глава 6, *Настройка pkgsrc*.

Чтобы обновить pkgsrc из tar-файла, загрузите tar-файл, как описано выше. Затем убедитесь, что вы не вносили изменений в файлы каталога pkgsrc. Удалите каталог pkgsrc и распакуйте новый tar-файл. Готово.

3.2.2. Через CVS

Чтобы обновить pkgsrc через CVS, перейдите в каталог `pkgsrc` и запустите cvs:

```
$ cd /usr/pkgsrc && cvs update -dP
```

Если вы получаете сообщения об ошибках от `rsh`, нужно задать переменную CVS_RSH, как описано выше. Например:

```
$ cd /usr/pkgsrc && env CVS_RSH=ssh cvs up -dP
```

3.2.2.1. Переключение между разными ветками pkgsrc

При обновлении pkgsrc программа CVS отслеживает выбранную вами ветку. Но если вы по какой-либо причине хотите переключиться со стабильной ветки на текущую, это можно сделать, добавив параметр «-A» после ключевого слова «update». Чтобы вернуться с текущей ветки на стабильную, добавьте параметр «-rpkgsrc-2026Q2».

3.2.2.2. Что происходит с моими изменениями при обновлении?

При обновлении pkgsrc программа CVS изменяет только те файлы, которые зарегистрированы в репозитории CVS. Это значит, что любые пакеты, созданные вами самостоятельно, останутся неизменными. Если вы измените файлы, управляемые CVS, при последующих обновлениях будет предпринята попытка объединить ваши изменения с изменениями, сделанными другими. Подробности см. в руководстве по CVS, глава «update».

Глава 4. Использование pkgsrc в системах, отличных от NetBSD

4.1. Распространение двоичных файлов

См. Раздел 5.1, «Использование двоичных пакетов».

4.2. Инициализация (bootstrap) pkgsrc

pkgsrc можно инициализировать (bootstrap) для работы в двух режимах: привилегированном и непривилегированном. В непривилегированном режиме, в отличие от привилегированного, все программы устанавливаются под одним определённым пользователем и не могут использовать привилегированные операции (пакеты не создают специальных пользователей, а все особые права доступа к файлам, например setuid, игнорируются).

Установка комплекта bootstrap из исходных текстов должна быть так же проста:

```
# env CVS_RSH=ssh cvs -d anoncvs@anoncvs.NetBSD.org:/cvsroot checkout -P pkgsrc
# cd pkgsrc/bootstrap
# ./bootstrap
```

Чтобы выполнить инициализацию в непривилегированном режиме, передайте флаг «--unprivileged» программе **bootstrap**

По умолчанию в привилегированном режиме pkgsrc использует `/usr/pkg` в качестве *префикса*, куда будут устанавливаться программы, и `/usr/pkg/pkgdb` в качестве каталога базы данных пакетов, где pkgsrc ведёт свой внутренний учёт; `/var` используется как *varbase*, где пакеты размещают свои постоянные данные. В непривилегированном режиме pkgsrc использует `~/pkg` в качестве *префикса*, `~/pkg/pkgdb` для базы данных пакетов и `~/pkg/var` как *varbase*.

Расположение по умолчанию можно изменить с помощью аргументов командной строки. Запустите «./bootstrap --help», чтобы узнать подробности.

> **ПРИМЕЧАНИЕ**
>
> bootstrap устанавливает инструмент **bmake**. Используйте этот **bmake** при сборке через pkgsrc. В примерах этого руководства используйте **bmake** вместо «make».

> **ПРИМЕЧАНИЕ**
>
> Можно инициализировать несколько экземпляров pkgsrc, используя непересекающиеся каталоги. Для сборки и установки пакетов используйте **bmake**, соответствующий той установке, с которой вы работаете.

Глава 5. Использование pkgsrc

В основном есть два способа использования pkgsrc. Первый — установить только инструменты для работы с пакетами и использовать двоичные пакеты, которые подготовил кто-то другой. Это «pkg» в pkgsrc. Второй способ — установить также «src» pkgsrc. Тогда вы сможете собирать собственные пакеты и по-прежнему использовать двоичные пакеты от кого-то другого.

5.1. Использование двоичных пакетов

На сайте cdn.NetBSD.org и его зеркалах есть коллекции двоичных пакетов, готовых к установке. Эти двоичные пакеты собраны с использованием значений каталогов по умолчанию, а именно:

- `/usr/pkg` для `LOCALBASE`, куда устанавливается большинство файлов,
- `/usr/pkg/etc` для файлов конфигурации,
- `/var` для `VARBASE`, куда устанавливаются файлы, которые могут изменяться после установки.

Если вы не можете использовать эти каталоги по каким-либо причинам (например, потому что вы не root), вы не можете использовать эти двоичные пакеты и должны собрать пакеты самостоятельно, как описано в Раздел 4.2, «Инициализация (bootstrap) pkgsrc».

5.1.1. Поиск двоичных пакетов

Чтобы устанавливать двоичные пакеты, сначала нужно знать, откуда их брать. В первую очередь следует искать на основном CDN pkgsrc в каталоге `/pub/pkgsrc/packages`.

Этот каталог содержит двоичные пакеты для нескольких платформ. Сначала выберите операционную систему. Затем выберите аппаратную архитектуру, а третьим шагом — версию ОС и «версию» pkgsrc.

В этом каталоге вы можете найти файл `bootstrap.tar.gz`, содержащий инструменты управления пакетами. Если файла нет, скорее всего, ваша операционная система уже предоставляет такие инструменты. Загрузите файл и распакуйте его в каталог `/`. Будут созданы каталоги `/usr/pkg` (содержащий инструменты управления двоичными пакетами и базу данных установленных пакетов).

5.1.2. Установка двоичных пакетов

В каталоге из предыдущего раздела есть подкаталог `All/`, который содержит все двоичные пакеты, доступные для платформы, за исключением тех, которые нельзя распространять по HTTP или FTP.

Чтобы установить пакеты непосредственно с сервера FTP или HTTP, выполните следующие команды в оболочке, совместимой с Bourne (предварительно обязательно перейдите в root с помощью **su**):

```
# PATH="/usr/pkg/sbin:/usr/pkg/bin:$PATH"
# PKG_PATH="https://cdn.NetBSD.org/pub/pkgsrc/packages"
# PKG_PATH="$PKG_PATH/OPSYS/ARCH/VERSIONS/All/"
# export PATH PKG_PATH
# pkg_add pkgin
```

Вместо URL можно также использовать локальные пути, например при установке с набора компакт-дисков, DVD или репозитория, смонтированного по NFS. Если вы хотите устанавливать пакеты из нескольких источников, разделите их точкой с запятой в `PKG_PATH`.

После такой подготовки установка пакета очень проста:

```
# pkgin search nginx
nginx-1.19.6         Lightweight HTTP server and mail proxy server
nginx-1.18.0nb8      Lightweight HTTP server and mail proxy server
# pkgin install zsh nginx-1.19.6 vim
```

Учтите, что **pkgin** — это удобный интерфейс к инструментам **pkg_***.

Все предварительные пакеты, необходимые для работы данного пакета, также будут установлены, если они есть в репозитории.

После установки пакетов убедитесь, что `/usr/pkg/bin` и `/usr/pkg/sbin` присутствуют в вашей переменной `PATH`, чтобы вы могли действительно запустить только что установленную программу.

5.1.3. Обновление пакетов

Для обновления двоичных пакетов рекомендуется использовать **pkgin upgrade**. Эта команда сравнит удалённый репозиторий пакетов с локально установленными пакетами и безопасно заменит любые более старые пакеты.

Учтите, что pkgsrc выпускается в виде ежеквартальных веток. Если вы обновляетесь на более новую ежеквартальную ветку pkgsrc, возможно, потребуется изменить репозиторий в `/usr/pkg/etc/pkgin/repositories.conf`.

5.1.4. Удаление пакетов

Для удаления пакета неважно, был ли он установлен из исходных текстов или из двоичного пакета. Ни команде **pkgin**, ни команде **pkg_delete** этого знать не требуется.

Чтобы удалить пакет, достаточно выполнить **pkgin remove** *`package-name`*. Имя пакета можно указать с номером версии или без него.

5.1.5. Получение сведений об установленных пакетах

pkg_info показывает сведения об установленных пакетах или файлах двоичных пакетов. Как и другие инструменты управления, эта команда работает с пакетами, установленными из исходных текстов или из двоичных файлов.

5.1.6. Проверка установленных пакетов на наличие уязвимостей

Группа безопасности pkgsrc (pkgsrc Security Team) и группы пакетов (Packages Groups) ведут список известных уязвимостей пакетов, которые входят (или входили) в pkgsrc. Список доступен на CDN NetBSD по адресу https://cdn.NetBSD.org/pub/NetBSD/packages/vulns/pkg-vulnerabilities.

Обратите внимание, что не каждая «уязвимость» с присвоенным идентификатором CVE эксплуатируема в любой конфигурации. Некоторые ошибки помечены как активные просто потому, что исправление не было помечено как таковое. Особенности усиления безопасности и защиты, специфичные для операционной системы, также могут снизить воздействие ошибок.

С помощью **pkg_admin fetch-pkg-vulnerabilities** этот список можно загрузить автоматически, после чего можно провести аудит безопасности всех пакетов, установленных в системе.

Аудит состоит из двух компонентов. Первый шаг, **pkg_admin fetch-pkg-vulnerabilities**, служит для загрузки списка уязвимостей с FTP-сайта NetBSD. Второй шаг, **pkg_admin audit**, проверяет, есть ли

среди установленных пакетов уязвимые. Если пакет уязвим, вы увидите вывод, похожий на следующий:

```
Package samba-2.0.9 has a local-root-shell vulnerability, see
    https://www.samba.org/samba/whatsnew/macroexploit.html
```

Возможно, вы захотите загружать файл vulnerabilities ежедневно, чтобы он оставался актуальным. Это можно сделать, добавив подходящую запись в crontab(5) пользователя root. Например, запись

```
# Download vulnerabilities file
0 3 * * * /usr/pkg/sbin/pkg_admin fetch-pkg-vulnerabilities >/dev/null 2>&1
# Audit the installed packages and email results to root
9 3 * * * /usr/pkg/sbin/pkg_admin audit |mail -s "Installed package audit result" \
            root >/dev/null 2>&1
```

будет обновлять список уязвимостей каждый день в 3:00, после чего в 3:09 выполняется аудит. Результаты аудита затем отправляются по электронной почте пользователю root. В NetBSD вместо этого можно добавить следующую строку в `/etc/daily.conf`:

```
fetch_pkg_vulnerabilities=YES
```

чтобы получать список уязвимостей из ежедневного сценария безопасности. По умолчанию система настроена на аудит пакетов, но при желании его можно задать явно (это не обязательно), добавив следующую строку в `/etc/security.conf`:

```
check_pkg_vulnerabilities=YES
```

дополнительные сведения см. в daily.conf(5) и security.conf(5).

5.1.7. Проверка, есть ли в pkgsrc более новые версии установленных пакетов

Установите `pkgtools/lintpkgsrc` и запустите **lintpkgsrc** с аргументом «-i», чтобы проверить, есть ли устаревшие пакеты, например

```
% lintpkgsrc -i
...
Version mismatch: 'tcsh' 6.09.00 vs 6.10.00
```

5.1.8. Другие административные функции

pkg_admin выполняет различные административные функции в системе пакетов.

5.2. Сборка пакетов из исходных текстов

После получения pkgsrc каталог `pkgsrc` содержит набор пакетов, организованных по категориям. Вы можете просмотреть индекс пакетов в интернете или выполнить **make readme** из каталога `pkgsrc`, чтобы создать локальные файлы `README.html` для всех пакетов, которые можно просматривать в любом веб-браузере, например `www/lynx` или `www/firefox`.

По умолчанию *префикс* для установленных пакетов — `/usr/pkg`. Если вы хотите это изменить, задайте `LOCALBASE` в `mk.conf`. Не следует пытаться использовать несколько разных определений `LOCALBASE` в одной системе (исключение — внутри chroot).

В остальной части этой главы предполагается, что пакет уже есть в pkgsrc. Если это не так, см. Часть II, «Руководство разработчика pkgsrc» — там описано, как создавать собственные пакеты.

5.2.1. Требования

Для сборки пакетов из исходных текстов нужен работающий компилятор C. В NetBSD необходимо установить наборы дистрибутива «comp» и «text». Если вы хотите собирать пакеты, связанные с X11, потребуются также наборы дистрибутива «xbase» и «xcomp».

5.2.2. Получение дистрибутивных файлов

Первый шаг сборки пакета — загрузка дистрибутивных файлов (то есть неизменённых исходных текстов). Если они ещё не загружены, pkgsrc загрузит их автоматически.

Если все нужные файлы уже есть в каталоге `distfiles`, подключаться не нужно. Если дистрибутивные файлы находятся на компакт-диске, можно смонтировать компакт-диск в `/cdrom` и добавить:

```
DISTDIR=/cdrom/pkgsrc/distfiles
```

в ваш `mk.conf`.

По умолчанию список сайтов распространения перемешивается случайным образом, чтобы избежать чрезмерной нагрузки на серверы, где размещены популярные пакеты (например, зеркала SourceForge.net). Поэтому каждый раз, когда нужно загрузить очередной дистрибутивный файл, все зеркала перебираются в новом (случайном) порядке. Эту возможность можно отключить, задав `MASTER_SORT_RANDOM=NO` (для `PKG_DEVELOPER` она уже отключена).

Вы можете переопределить некоторые из основных сайтов распространения, чтобы использовать сайты, близкие к вам. Задав одну или две переменные, можно изменить порядок обращения к основным сайтам. `MASTER_SORT` содержит разделённый пробельными символами список суффиксов доменов. `MASTER_SORT_REGEX` ещё более гибок: он содержит разделённый пробельными символами список регулярных выражений. Он имеет более высокий приоритет, чем `MASTER_SORT`. Загляните в `pkgsrc/mk/defaults/mk.conf`, чтобы найти примеры. Это может сэкономить вам часть трафика и времени.

Эти настройки можно изменить в окружении вашей оболочки или, если вы хотите их сохранить, отредактировав файл `mk.conf` и добавив определения в него.

Если пакет зависит от многих других пакетов (например, `meta-pkgs/kde4`), процесс сборки может чередовать периоды загрузки исходных текстов и компиляции. Чтобы заранее загрузить все исходные тексты, можно выполнить команду:

```
% make fetch-list | sh
```

которая выведет и выполнит набор команд оболочки для загрузки нужных файлов в каталог `distfiles`. Можно также загрузить файлы вручную.

5.2.3. Как собрать и установить

После загрузки программы применяются все патчи, затем она компилируется. Это может занять некоторое время в зависимости от вашего компьютера, а также от того, от скольких других пакетов зависит программа и сколько времени занимает их компиляция.

> **ПРИМЕЧАНИЕ**
>
> Если вы используете bootstrap или pkgsrc в системе, отличной от NetBSD, в примерах этого руководства используйте команду pkgsrc **bmake** вместо «make».

Например, введите

```
% cd misc/figlet
% make
```

в приглашении оболочки, чтобы собрать различные компоненты пакета.

Следующий этап — собственно установка только что скомпилированной программы в вашу систему. Для этого введите:

```
% make install
```

находясь в каталоге того пакета, который вы устанавливаете.

Для установки пакета в систему могут потребоваться права root. Однако в pkgsrc есть возможность *just-in-time-su*, которая позволяет становиться root только на время самого шага установки.

Вот и всё: программа установлена и настроена для использования. Теперь можно ввести:

```
% make clean
```

чтобы удалить скомпилированные файлы в рабочем каталоге, поскольку они вам больше не нужны. Если для компиляции вашей программы в систему были добавлены другие пакеты (зависимости), их также можно убрать с помощью команды:

```
% make clean-depends
```

Возьмём в качестве примера утилиту figlet: мы можем установить её в систему, выполнив сборку, как показано в Приложение C, *Журналы сборки*.

Программа устанавливается в корень дерева пакетов, используемый по умолчанию, — `/usr/pkg`. Если это вас не устраивает, задайте переменную `LOCALBASE` в своём окружении, и её значение будет использоваться как корень дерева пакетов. Так, чтобы использовать `/usr/local`, задайте `LOCALBASE=/usr/local` в своём окружении. Обратите внимание, что следует использовать каталог, предназначенный только для пакетов и не разделяемый с другими программами (то есть не пытайтесь использовать `LOCALBASE=/usr`). Кроме того, не следует добавлять собственные файлы или каталоги (например, `src/`, `obj/` или `pkgsrc/`) внутрь дерева `LOCALBASE`. Это делается для предотвращения возможных конфликтов между программами и другими файлами, установленными системой пакетов, и всем остальным, что могло быть установлено в этом месте.

Некоторые пакеты обращаются к `mk.conf`, чтобы изменить некоторые параметры конфигурации во время сборки. Загляните в `pkgsrc/mk/defaults/mk.conf`, чтобы получить представление о том, что там задаётся по умолчанию. Переменные окружения, такие как `LOCALBASE`, можно задать в `mk.conf`, чтобы не запоминать их и не задавать каждый раз, когда вы хотите использовать pkgsrc.

Иногда возникает желание «заглянуть внутрь», чтобы увидеть, что происходит при сборке или установке пакета. Это может быть нужно для отладки или просто из любопытства. Для этого добавлен ряд служебных значений.

1. Если вызвать команду make(1) с `PKG_DEBUG_LEVEL=2`, будет выведено огромное количество информации. Например,

   ```
   make patch PKG_DEBUG_LEVEL=2
   ```

   покажет все вызываемые команды, вплоть до этапа «patch» включительно.

2. Если вы хотите узнать значение некоторого определения make(1), следует использовать определение `VARNAME` вместе с целью show-var. Например, чтобы показать раскрытие переменной make(1) `LOCALBASE`:

   ```
   % make show-var VARNAME=LOCALBASE
   /usr/pkg
   %
   ```

Если вы хотите установить двоичный пакет, который создали сами (см. следующий раздел), который вручную поместили в pkgsrc/packages или который находится на удалённом FTP-сервере, можно использовать цель «bin-install». Эта цель установит двоичный пакет — если он доступен — с помощью pkg_add(1), иначе выполнит **make package**. Список просматриваемых удалённых FTP-сайтов хранится в переменной `BINPKG_SITES`, по умолчанию равной ftp.NetBSD.org. Любые флаги, которые следует добавить к pkg_add(1), можно указать в `BIN_INSTALL_FLAGS`. Дополнительные сведения см. в `pkgsrc/mk/defaults/mk.conf`.

И последнее предупреждение: если вы настраиваете систему с нестандартным значением `LOCALBASE`, обязательно задайте его до установки любых пакетов, поскольку нельзя использовать несколько каталогов для одной и той же цели. Иначе pkgsrc не сможет правильно определить установленные пакеты и потерпит полную неудачу. Учтите также, что заранее собранные двоичные пакеты обычно собираются с `LOCALBASE` по умолчанию, равным `/usr/pkg`, и что вам *не* следует устанавливать их, если вы используете нестандартный `LOCALBASE`.

Глава 6. Настройка pkgsrc

Вся система pkgsrc настраивается в одном файле, обычно называемом `mk.conf`. В каком каталоге pkgsrc ищет этот файл, зависит от установки. В NetBSD, если вы используете make(1) из базовой системы, он находится в каталоге `/etc/`. Во всех остальных случаях местоположение по умолчанию — `${PREFIX}/etc/`, в зависимости от того, куда вы указали программе bootstrap устанавливать двоичные пакеты.

Формат файла конфигурации — обычный для `Makefile` в стиле BSD. Вся настройка pkgsrc выполняется заданием переменных в этом файле. Учтите, что вы можете определять переменные любого рода, и никакой специальной проверки ошибок (например, орфографических) не выполняется.

6.1. Общая настройка

Следующие переменные применимы ко всем пакетам pkgsrc. Полный список переменных, которые может настроить пользователь, приведён в `mk/defaults/mk.conf` вместе с комментариями, описывающими назначение каждой переменной.

- `LOCALBASE`: куда будут устанавливаться пакеты. По умолчанию — `/usr/pkg`. Не смешивайте двоичные пакеты с разными `LOCALBASE`!
- `CROSSBASE`: куда будут устанавливаться пакеты категории «cross». По умолчанию — `${LOCALBASE}/cross`.
- `X11BASE`: куда установлен X11 в системе. По умолчанию — `/usr/X11R7`.
- `DISTDIR`: где хранить загруженные копии оригинальных дистрибутивов исходных текстов, используемых для сборки пакетов pkgsrc. По умолчанию — `${PKGSRCDIR}/distfiles`.
- `PKG_DBDIR`: где хранится база данных об установленных пакетах. По умолчанию — `/usr/pkg/pkgdb`.
- `MASTER_SITE_OVERRIDE`: если задана, переопределяет `MASTER_SITES` пакетов этим значением.
- `MASTER_SITE_BACKUP`: резервное расположение (расположения) для дистрибутивных файлов и файлов патчей, если они не найдены локально или соответственно в `${MASTER_SITES}` или `${PATCH_SITES}`. По умолчанию — `ftp://ftp.NetBSD.org/pub/pkgsrc/distfiles/${DIST_SUBDIR}/`.
- `BINPKG_SITES`: список сайтов, на которых размещены двоичные пакеты. *`rel`* и *`arch`* заменяются на выпуск ОС («2.0» и т. п.) и архитектуру («mipsel» и т. п.).
- `ACCEPTABLE_LICENSES`: список допустимых лицензий. Имена лицензий чувствительны к регистру. Всякий раз, когда вы пытаетесь собрать пакет, лицензия которого отсутствует в этом списке, вы получите сообщение об ошибке. Если условие лицензии достаточно простое, сообщение об ошибке будет содержать конкретные указания о том, как изменить эту переменную.

6.2. Переменные, влияющие на процесс сборки

- `PACKAGES`: каталог верхнего уровня для двоичных пакетов. По умолчанию — `${PKGSRCDIR}/packages`.

- `WRKOBJDIR`: каталог верхнего уровня, в котором, если он задан, будут создаваться отдельные рабочие каталоги, символически связанные с `${WRKDIR}` (см. ниже). Это полезно при сборке пакетов на нескольких архитектурах: тогда `${PKGSRCDIR}` можно смонтировать по NFS, а `${WRKOBJDIR}` будет локальным для каждой архитектуры. (Следует отметить, что `PKGSRCDIR` не должен задаваться пользователем — это внутреннее определение, указывающее на корень дерева pkgsrc. Возможно наличие множества экземпляров дерева pkgsrc.)
- `LOCALPATCHES`: каталог для локальных патчей, не входящих в pkgsrc. Дополнительные сведения см. в Раздел 12.3, «patches/*».
- `PKGMAKECONF`: расположение файла `mk.conf`, используемого Makefile пакета в стиле BSD. Если она не задана, `MAKECONF` устанавливается в `/dev/null`, чтобы не подхватывать настройки, используемые при сборках в `/usr/src`.

6.3. Предпочтения для встроенного или pkgsrc-варианта программного обеспечения

Всякий раз, когда пакет зависит от пакета, имеющего файл `builtin.mk`, зависимый пакет может использовать либо встроенную (native) версию из базовой системы, либо версию, предоставляемую pkgsrc. Это влияет только на зависимости, поэтому пакет pkgsrc `devel/pcre++` по-прежнему можно собрать, даже если другие пакеты зависят от встроенной версии pcre++.

Чтобы принудительно использовать версию, предоставляемую pkgsrc, для конкретного пакета, определите «`PREFER_PKGSRC` = *`package-ID`*» в `mk.conf`. Чтобы принудительно использовать встроенный пакет, определите «`PREFER_NATIVE` = *`package-ID`*». В обоих случаях *`package-ID`* — это идентификатор из файла `buildlink3.mk` пакета. В большинстве случаев этот идентификатор совпадает с именем каталога пакета, но, например, `devel/pcre++` имеет идентификатор пакета «pcrexx».

Для пакетов, которые не перечислены по своему идентификатору пакета, pkgsrc использует версию, предоставляемую pkgsrc, если `PREFER_PKGSRC` содержит слово «yes». В противном случае, если `PREFER_NATIVE` содержит слово «yes», pkgsrc использует встроенную версию. Например, чтобы требовать использования версий, предоставляемых pkgsrc, для всего, кроме самых базовых компонентов системы NetBSD, можно задать:

```
PREFER_PKGSRC=  yes
PREFER_NATIVE=  getopt skey tcp_wrappers
```

Пакет *обязан* иметь файл `builtin.mk`, чтобы его можно было указать в `PREFER_NATIVE`, иначе он в этом списке просто игнорируется.

`PREFER_PKGSRC` и `PREFER_NATIVE` следует задавать во время bootstrap, чтобы процесс инициализации не использовал неподходящие встроенные инструменты в качестве зависимостей для основных пакетов.

```
# ./bootstrap --prefer-pkgsrc yes --prefer-native openssl
```

Глобальное переключение между настройками в более позднее время может вызвать осложнения при разрешении зависимостей. Причина в том, что рядом оказываются установлены пакеты, собранные с противоположными предпочтениями. Поэтому при изменении любой из этих переменных после bootstrap нужно пересобрать все пакеты, зависящие от тех, чьё предпочтение было изменено. Это непросто, и этого следует избегать.

При использовании pkgsrc в системах Linux высок риск «утечки», когда установленные pkgsrc программы могут непреднамеренно использовать команду или библиотеку, установленную не pkgsrc, например установленную yum или apt. Такие чужеродные зависимости могут быть в любой момент установлены, удалены или обновлены до версии, несовместимой с пакетом pkgsrc, из-за чего пакеты pkgsrc впоследствии начнут работать неправильно. Pkgsrc не может это предотвратить, так как не управляет другими менеджерами пакетов. Ещё одна потенциальная проблема — в Red Hat Enterprise и связанных с ним системах Linux пакеты yum только патчатся и никогда не обновляются, поэтому со временем они могут оказаться слишком устаревшими для использования pkgsrc. Даже намеренное использование чужеродных зависимостей, не считающееся утечкой, может привести к таким проблемам, поэтому обычно это не рекомендуется. Чтобы свести такие проблемы к минимуму, по умолчанию в системах Linux переменная PREFER_PKGSRC равна «yes». Это гарантирует, что pkgsrc знает об изменениях зависимых пакетов и может при необходимости пересобрать или обновить всё дерево зависимостей. Это значение по умолчанию можно переопределить, задав для --prefer-pkgsrc список пакетов, а для --prefer-native — «yes».

6.4. Переменные, влияющие на процесс установки

- `PKGSRC_KEEP_BIN_PKGS`: по умолчанию двоичные пакеты собранных пакетов сохраняются в `${PACKAGES}/All`. Если задать этой переменной значение «no», сохранение выполняться не будет.

Пакеты должны поддерживать установку в подкаталог `WRKDIR`. Это позволяет собрать пакет, прежде чем будет затронута настоящая файловая система. Поддержка DESTDIR существует в двух вариантах:

- Базовая поддержка DESTDIR означает, что установка и упаковка пакета по-прежнему выполняются от имени root.
- Полная поддержка DESTDIR позволяет выполнять всю сборку, установку и упаковку как обычный пользователь. Права root нужны только для добавления пакетов.

При базовой поддержке DESTDIR команду `make clean` нужно запускать от имени root.

На примере пакета `foo/bar` полную поддержку DESTDIR можно проверить с помощью следующих команд

```
$ id
uid=1000(myusername) gid=100(users) groups=100(users),0(wheel)
$ mkdir $HOME/packages
$ cd $PKGSRCDIR/foo/bar
```

Проверка полной поддержки `DESTDIR`: права root не должны требоваться

```
$ make stage-install
```

Создание пакета без прав root

```
$ make PACKAGES=$HOME/packages package
```

Для выполнения следующей команды необходимо иметь возможность получать права root с помощью su(1)

```
$ make PACKAGES=$HOME/packages install
```

Затем, как обычный пользователь

```
$ make clean
```

6.5. Выбор и настройка компилятора

6.5.1. Выбор компилятора

По умолчанию pkgsrc использует GCC для сборки пакетов. Это можно переопределить, задав следующие переменные в /etc/mk.conf:

`PKGSRC_COMPILER`:

Здесь перечислены значения, задающие цепочку компиляторов, которые вызываются при сборке пакетов. Допустимые значения:

- `ccc`: компиляторы Compaq C (Tru64)
- `ccache`: кэш компилятора (может входить в цепочку)
- `clang`: компилятор Clang для C и Objective-C
- `distcc`: распределённая компиляция C/C++ (может входить в цепочку)
- `f2c`: компилятор Fortran 77 в C (может входить в цепочку)
- `icc`: компилятор Intel C++ (Linux)
- `ido`: cc из пакета SGI IRIS Development Option (IRIX 5)
- `gcc`: компилятор GNU C/C++
- `hp`: компиляторы C/aC++ для HP-UX
- `mipspro`: MIPSpro от Silicon Graphics, Inc. (n32/n64)
- `mipspro-ucode`: MIPSpro от Silicon Graphics, Inc. (o32)
- `sunpro`: WorkShop/Forte/Sun ONE Studio от Sun Microsystems, Inc.
- `xlc`: набор компиляторов XL C/C++ от IBM

По умолчанию используется «`gcc`». Вы можете использовать `ccache` и/или `distcc`, задав подходящее значение `PKGSRC_COMPILER`, например «`ccache gcc`». Эта переменная всегда должна заканчиваться значением настоящего компилятора. Учтите, что указывать следует только один настоящий компилятор (например, «`sunpro gcc`» недопустимо).

`GCC_REQD`:

Эта переменная задаёт минимальную версию GCC, которую нужно использовать при сборке пакетов. Если системный GCC не удовлетворяет этому требованию, pkgsrc соберёт и установит один из пакетов GCC и будет использовать его.

`PYTHON_VERSION_DEFAULT`:

Указывает, какую версию python использовать, когда доступно несколько вариантов.

`PKGSRC_FORTRAN`:

Указывает используемый компилятор Fortran. По умолчанию — `gfortran`.

`GFORTRAN_VERSION`:

Если задано `PKGSRC_FORTRAN = gfortran`, этот параметр указывает, какую версию использовать.

6.5.2. Дополнительные флаги компилятора (`CFLAGS`)

Если вы хотите задать переменную `CFLAGS`, обязательно используйте оператор `+=`, а не оператор `=`:

```
CFLAGS+=        -your -flags
```

Использование `CFLAGS=` (то есть без «+») может привести к проблемам с пакетами, которым нужно добавлять собственные флаги. Если вас интересует оптимизация именно под текущий процессор, возможно, вам стоит заглянуть в пакет `devel/cpuflags`.

6.5.3. Дополнительные флаги компоновщика (`LDFLAGS`)

Если вы хотите передать флаги компоновщику — как на этапе конфигурирования, так и на этапе сборки, — это можно сделать двумя способами: задать `LDFLAGS` или `LIBS`. Разница между ними в том, что `LIBS` добавляются в конец командной строки, а `LDFLAGS` указываются раньше. Переменная `LDFLAGS` заранее заполнена настройками rpath для машин ELF в зависимости от значения `USE_IMAKE` или от подключения `mk/x11.buildlink3.mk`. Как и в случае с `CFLAGS`, если вы не хотите переопределять эти настройки, используйте оператор `+=`:

```
LDFLAGS+=       -your -linkerflags
```

6.6. Настройки для разработчиков и опытных пользователей

- `PKG_DEVELOPER`: выполнять ряд проверок корректности, которые нужны разработчикам пакетов:
  - убедиться, что патчи применяются без размытия контекста (zero fuzz)
  - запустить check-shlibs и убедиться, что все двоичные файлы найдут свои разделяемые библиотеки.
- `CHECK_FILES_STRICT`: дополнительно проверять значения `VARBASE` и `PKG_SYSCONFDIR` в записях PLIST.
- `PKG_DEBUG_LEVEL`: уровень отладочного вывода, отображаемого при сборке и установке пакета. По умолчанию используется значение 0, при котором команды не отображаются по мере их выполнения (обычный, стандартный, тихий режим); значение 1 выводит все команды оболочки перед их вызовом, а значение 2 показывает и команды оболочки перед их вызовом, и ход их выполнения с помощью **set -x**.

6.7. Выбор параметров сборки

У некоторых пакетов есть параметры времени сборки — обычно для выбора между разными зависимостями, включения необязательной поддержки крупных зависимостей или включения экспериментальных возможностей.

Чтобы узнать, какие параметры (если они есть) поддерживает пакет и какие из них взаимоисключающие, выполните **make show-options**, например:

```
    The following options are supported by this package:
        ssl      Enable SSL support.
    Exactly one of the following gecko options is required:
```

```
        firefox  Use firefox as gecko rendering engine.
        mozilla  Use mozilla as gecko rendering engine.
    At most one of the following database options may be selected:
        mysql    Enable support for MySQL database.
        pgsql    Enable support for PostgreSQL database.

    These options are enabled by default: firefox
    These options are currently enabled: mozilla ssl
```

Следующие переменные можно определить в `mk.conf`, чтобы выбрать, какие параметры включить для пакета: `PKG_DEFAULT_OPTIONS`, с помощью которой можно включать или отключать параметры для всех пакетов, которые их поддерживают, и `PKG_OPTIONS.`*`pkgbase`*, с помощью которой можно включать или отключать параметры для конкретного пакета *`pkgbase`*. Параметры, перечисленные в этих переменных, включаются, а параметры, перед которыми стоит «-», отключаются. Несколько примеров:

```
$ grep "PKG.*OPTION" mk.conf
PKG_DEFAULT_OPTIONS=    -arts -dvdread -esound
PKG_OPTIONS.kdebase=    debug -sasl
PKG_OPTIONS.apache=     suexec
```

Важно отметить, что параметры, которые явно предложил сопровождающий пакета, нужно явно удалить, если вы не хотите включать этот параметр. Если вы не уверены, текущее состояние можно посмотреть командой **make show-options**.

Следующие настройки учитываются в указанном порядке, и используется последняя настройка, которая включает или отключает параметр:

1. параметры по умолчанию, предложенные сопровождающим пакета
2. параметры, подразумеваемые настройками устаревших переменных (см. ниже)
3. `PKG_DEFAULT_OPTIONS`
4. `PKG_OPTIONS.`*`pkgbase`*

Для групп взаимоисключающих параметров используется последний выбранный параметр, а все остальные автоматически отключаются. Если параметр группы явно отключён, используется ранее выбранный параметр, если он был. Если из обязательной группы параметров не выбран ни один, это считается ошибкой, и сборка пакета завершится неудачей.

До появления механизма параметров параметры сборки выбирались заданием переменной (часто с именем `USE_`*`FOO`*) в `mk.conf` для каждого параметра. Чтобы облегчить пользователю переход на механизм параметров, эти устаревшие переменные автоматически преобразуются в соответствующую настройку параметров (`PKG_OPTIONS.`*`pkgbase`*). Выдаётся предупреждение, чтобы пользователь обновил `mk.conf` и использовал механизм параметров напрямую. Поддержка устаревших переменных со временем будет удалена.

Глава 7. Создание двоичных пакетов

7.1. Сборка одного двоичного пакета

Собрав и установив пакет, вы можете создать *двоичный пакет*, который можно установить на другой системе с помощью pkg_add(1). Это избавляет от необходимости собирать один и тот же пакет на нескольких хостах и тратить процессорное время. Кроме того, это простой способ дать другим установить ваш пакет, если вы его распространяете.

Чтобы создать двоичный пакет, перейдите в соответствующий каталог в pkgsrc и выполните **make package**:

```
$ cd misc/figlet
$ make package
```

Эта команда соберёт ваш пакет (если это ещё не сделано) и упакует его в двоичный пакет. После этого с ним можно работать инструментами **pkg_***. Двоичные пакеты создаются в `PACKAGES`, по умолчанию это `/usr/pkgsrc/packages`, в виде сжатого tar-файла. См. Раздел C.2, «Упаковка figlet» — там продолжение приведённого выше примера с `misc/figlet`.

О том, как отправить такой двоичный пакет, см. в Глава 23, *Отправка и коммит*.

7.2. Настройки для создания двоичных пакетов

См. Раздел 13.17, «Другие полезные цели».

Глава 8. Создание двоичных пакетов для всего содержимого pkgsrc (массовые сборки)

По ряду причин вам может понадобиться собрать двоичные пакеты для большого выбранного набора пакетов pkgsrc или даже для всех пакетов pkgsrc. Например, если у вас несколько машин, на которых должно работать одно и то же программное обеспечение, они впустую потратят время, собирая свои пакеты из исходных текстов самостоятельно. Или вам может понадобиться собрать список нужных пакетов и проверить их перед развёртыванием на рабочих системах. Существует способ получить набор двоичных пакетов с помощью системы массовой сборки, например pbulk («p» означает «parallel») или bob. В этой главе описано, как их настроить.

8.1. Подготовка

Прежде всего нужно решить, собирать все пакеты или ограниченный набор. Полные массовые сборки обычно требуют гораздо больше ресурсов — и места, и времени, — чем сборки некоторого практического набора пакетов. Существует ряд особенно тяжёлых пакетов, которые на самом деле неинтересны широкой аудитории. (Приблизительный расход ресурсов для полной массовой сборки приведён в разделе 8.5, «Требования к полной массовой сборке».) Для ограниченных массовых сборок нужно составить список пакетов, которые вы хотите собрать. Учтите, что все их зависимости также будут собраны, поэтому отслеживать их вручную не нужно.

Во время сборок pbulk (но не bob) различные пакеты устанавливаются и удаляются в `/usr/pkg` (или в том каталоге, который задан в `LOCALBASE`), поэтому убедитесь, что во время сборки вам не нужен ни один пакет. По сути, следует предоставить чистую систему — окружение chroot или что-то ещё более ограниченное, в зависимости от возможностей операционной системы, — либо выделить целиком физическую машину. Как полезный побочный эффект это гарантирует, что массовые сборки не смогут ничего испортить в вашей системе. Было множество случаев, когда те или иные пакеты пытались установить файлы за пределами `LOCALBASE` или хотели изменить какие-то файлы в `/etc`.

8.2. Запуск сборки pbulk

Запуск сборки pbulk примерно выглядит так:

- Сначала соберите инфраструктуру pbulk в новом каталоге pkgsrc.
- Затем с помощью этой инфраструктуры соберите каждый из пакетов из чистого каталога установки.

8.2.1. Конфигурация

Для упрощения настройки мы предоставляем вспомогательный сценарий `mk/pbulk/pbulk.sh`.

Чтобы воспользоваться им, подготовьте чистую систему (настоящую, окружение chroot, jail, зону, виртуальную машину). Настройте сетевой доступ для загрузки дистрибутивных файлов. Создайте пользователя с именем «pbulk».

Загрузите и распакуйте pkgsrc. Используйте команду, подобную одной из этих:

```
# (cd /usr && ftp -o - https://cdn.NetBSD.org/pub/pkgsrc/current/pkgsrc.tar.gz | tar -zxf-)
# (cd /usr && fetch -o - https://cdn.NetBSD.org/pub/pkgsrc/current/pkgsrc.tar.gz | tar -zxf-)
# (cd /usr && cvs -Q -z3 -d anoncvs@anoncvs.NetBSD.org:/cvsroot get -P pkgsrc)
```

Или любой другой подходящий способ (например, curl, wget).

Разверните и настройте инструменты pbulk, например:

```
# sh pbulk.sh -n # use native make, no bootstrap kit needed (for use on NetBSD)
# sh pbulk.sh -n -c mk.conf.frag # native, apply settings from given mk.conf fragment
# sh pbulk.sh -nlc mk.conf.frag # native, apply settings, configure for limited build
```

ПРИМЕЧАНИЕ

`mk.conf.frag` — это фрагмент `mk.conf`, содержащий настройки, которые вы хотите применить к собираемым пакетам. Например,

```
PKG_DEVELOPER=          yes     # perform more checks
X11_TYPE=               modular # use pkgsrc X11
SKIP_LICENSE_CHECK=     yes     # accept all licences (useful
                                # when building all packages)
```

Если настроена работа с ограниченным списком, замените список в `/usr/pbulk/etc/pbulk.list` своим списком пакетов — по одному в строке, без пустых строк и комментариев. Например:

```
www/firefox
mail/thunderbird
misc/libreoffice4
```

На этом этапе вы также можете просмотреть конфигурацию в `/usr/pbulk/etc` и при желании внести окончательные правки.

Запустите:

```
# /usr/pbulk/bin/bulkbuild
```

После завершения `/mnt` будет заполнен дистрибутивными файлами, двоичными пакетами и отчётами, а сводка в виде обычного текста будет находиться в `/mnt/bulklog/meta/report.txt`

ПРИМЕЧАНИЕ

Сценарий `pbulk.sh` не охватывает все возможные случаи использования. Будучи готовым к запуску, он служит хорошей отправной точкой для понимания и построения более сложных конфигураций. Сценарий намеренно оставлен достаточно компактным для лучшего понимания.

ПРИМЕЧАНИЕ

Сценарий `pbulk.sh` поддерживает запуск массовой сборки без привилегий и помогает настраивать распределённые массовые сборки. Распределённые массовые сборки поддерживают либо сборку в рабочих chroot-окружениях (каждый узел — это путь к отдельному chroot), которые повторяют целевую систему, включая префикс pbulk, либо сборку на удалённых машинах (каждый узел — это IP-адрес, доступный по SSH без пароля).

8.3. Запуск сборки bob

Bob задуман как более простая и более мощная альтернатива pbulk. Для начала установите пакет `pkgtools/bob`, измените `${PKG_SYSCONFBASE}/bob/config.lua`, чтобы настроить пакеты, которые вы хотите собрать, и подогнать параметры под свою систему, затем выполните

```
# bob build
```

Эта команда соберёт все выбранные вами пакеты в чистых песочницах и в конце создаст файл `pkg_summary`, готовый к тому, чтобы `pkgtools/pkgin` обновил систему.

Каждый раз, когда вы обновляете свою рабочую копию pkgsrc, сначала выполните

```
# bob clean
```

чтобы удалить существующие метаданные сборки перед началом новой сборки.

Ещё одна полезная возможность — сначала отдельно запустить сканирование, чтобы увидеть, какие пакеты будут собраны:

```
# bob scan
# bob status
```

Вы также можете добавить произвольные пакеты в текущую сборку, указав `PKGPATH` в качестве аргумента:

```
# bob build graphics/png
```

Если пакет не удаётся собрать, вы можете просмотреть файл журнала:

```
# bob log png
```

Если вы исправили проблему и хотите пересобрать только ранее неудавшиеся пакеты, не начиная с нуля, выполните:

```
# bob rebuild -a
```

Другие полезные команды:

```
# bob dev # run an interactive sandbox for development work
# bob history # show package build history
# bob publish # generate build reports, emails, and publish packages
```

Подробнее см. https://docs.rs/bob/latest/bob/.

8.4. Ускорение сканирования при массовых сборках

В начале массовой сборки выполняется сканирование pkgsrc для получения сведений о пакетах, которые нужно собрать. Это нужно, например, для построения графа зависимостей, чтобы собирать пакеты в правильном порядке.

По сути, эта команда вызывает make(1) по одному разу для каждого пакета. Чтобы ускорить процесс, некоторые переменные можно вычислить заранее, а другие — закэшировать. Это значительно сократит время, необходимое для этапа сканирования массовой сборки.

Пакет-заглушка `pkgtools/varcache` предоставляет цель Makefile для предварительного вычисления некоторых часто используемых переменных системы сборки, которые не должны меняться в течение одной сборки. Перенаправьте его вывод в файл и подключите этот файл из `mk.conf`; например:

```
$ cd pkgtools/varcache
$ make show-varcache > /etc/varcache.mk
$ echo .include "varcache.mk" >> /etc/mk.conf
```

Некоторые сведения о системе нужны довольно рано в процессе сборки и не могут быть закэшированы таким способом. Их нужно задать в окружении для следующего ускорения:

```
$ export HOST_MACHINE_ARCH=`uname -m`
$ export NATIVE_OPSYS=`uname -s`
$ export NATIVE_OPSYS_VERSION=`uname -r | awk -F. '{major=int($1); minor=int($2); if
(minor>=100) minor=99; patch=int($3); if (patch>=100) patch=99; printf "%02d%02d%02d", major,
minor, patch}'`
$ export NATIVE_OS_VERSION=`uname -r`
```

Некоторые настройки, которые также сильно влияют на время сканирования, можно задать в `mk.conf`:

```
NATIVE_PKG_TOOLS_BIN=/usr/sbin # location on NetBSD
NO_PKGTOOLS_REQD_CHECK=yes     # assume pkg tools are up-to-date
SKIP_LICENSE_CHECK=yes         # do not check license validity
_SRC_TOP_=                     # bulk build is not running from inside /usr/src
UNPRIVILEGED_USER=pbulk        # use the appropriate user for your system
UNPRIVILEGED_GROUP=builder     # use the appropriate group for your system
UNPRIVILEGED_GROUPS=builder    # use the appropriate groups for your system
```

Наконец, некоторым пакетам нужна информация о параметрах сборки других пакетов. Эту информацию можно закэшировать, задав `PBULK_CACHE_DIRECTORY`.

```
PBULK_CACHE_DIRECTORY=/tmp/pbulk_cache
```

Для всего перечисленного следите за тем, чтобы данные оставались актуальными при изменениях в вашем окружении. Самый надёжный способ — заново создавать varcache перед каждой сборкой, проверять, что переменные окружения по-прежнему актуальны, и удалять `PBULK_CACHE_DIRECTORY` перед сборками.

8.5. Требования к полной массовой сборке

Полная массовая сборка требует много дискового пространства. Часть его может быть доступна только для чтения, а другая часть должна быть доступна для записи. Часть может находиться на удалённых файловых системах (таких как NFS), а часть должна быть локальной. Одни могут быть временными файловыми системами, другие должны пережить внезапную перезагрузку.

- 70 GB для дистрибутивных файлов (чтение-запись, удалённое, временное)
- 60 GB для двоичных пакетов (чтение-запись, удалённое, постоянное)
- 1 GB для дерева pkgsrc (только чтение, удалённое, постоянное)
- 5 GB для `LOCALBASE` (чтение-запись, локальное, временное)
- 10 GB для файлов журналов (чтение-запись, удалённое, постоянное)
- 5 GB для временных файлов (чтение-запись, локальное, временное)

8.6. Варианты массовой сборки

Чтобы убедиться, что пакеты pkgsrc работают в разных конфигурациях, имеет смысл время от времени запускать массовые сборки с нестандартными настройками. В этом разделе перечислены некоторые идеи для массовых сборок, которые намеренно допускают сбой пакетов, не следующих стилю pkgsrc.

8.6.1. Обнаружение неизвестных параметров configure

Добавьте в `mk.conf` следующую строку.

```
GNU_CONFIGURE_STRICT=   yes
```

Когда пакет не проходит эту дополнительную проверку, наиболее частая причина в том, что параметр configure был допустим для более старой версии пакета, но больше не применим. В этом случае просто удалите его.

8.6.2. Обнаружение классов ошибок принудительными предупреждениями компилятора

Задача компилятора не ограничивается созданием исполняемого кода: большинство компиляторов также обнаруживают типичные ошибки программирования. Обёртки компилятора pkgsrc позволяют легко задать параметры компилятора при сборке пакета. Это можно использовать для поиска типичных ошибок во всех пакетах, входящих в pkgsrc. Сообщая об этих ошибках авторам, вы сделаете пакеты надёжнее в следующих обновлениях.

Добавьте в `mk.conf` некоторые из следующих строк:

```
CFLAGS+=        -Werror=char-subscripts
CFLAGS+=        -Werror=implicit-function-declaration
```

Когда пакет не собирается с этими более строгими параметрами компилятора, задокументируйте обстоятельства, в которых компилятор выдал сообщение об ошибке. К ним относятся:

- Платформа (`MACHINE_PLATFORM`)
- Исходный файл
- Фрагмент кода. GCC и Clang уже делают это в составе диагностики.
- Точное сообщение об ошибке от компилятора.

Если пакет выдаёт такие сообщения об ошибках, но сам пакет в порядке, запишите это в своём локальном `mk.conf`, как показано ниже, чтобы пропустить эту проверку в следующих сборках:

```
.if ${PKGPATH} == category/package
# Version ${VERSION} failed on ${MACHINE_PLATFORM}:
```

```
# error message
# code
# Reason why the code does not need to be fixed.
BUILDLINK_TRANSFORM+=   rm:-Werror=char-subscripts
.endif
```

Если сообщения компилятора обоснованы и код нужно исправить, подготовьте локальный патч (см. `LOCALPATCHES`) и сообщите об ошибке авторам пакета, предоставив им собранные выше сведения.

Патчи, не существенные для работы пакета, следует только отправлять авторам, но не включать в pkgsrc, чтобы облегчить будущие обновления.

8.6.3. Принудительные параметры компилятора только на этапе сборки

Когда вы добавляете собственные флаги компилятора через `CFLAGS`, они применяются на всех этапах сборки пакета. Особенно на этапе configure добавление `-Werror` приводит к неверным решениям. Сценарии configure GNU передают компилятору C множество небольших тестовых программ, чтобы проверить, доступны ли определённые заголовочные файлы, определены ли функции в библиотеке и можно ли запускать программы. Во многих случаях эти программы не выдержали бы строгого прогона компилятора с `-Wall -Wextra -Werror`.

Инфраструктура pkgsrc достаточно гибкая, чтобы поддерживать добавление параметров компилятора между этапами `configure` и `build`. Это немного сложнее других примеров из этого раздела, но всё же достаточно просто.

Основная идея — использовать обёртку компилятора pkgsrc, чтобы внедрить нужные параметры компилятора. Изначальная задача обёртки компилятора — скрывать нежелательные каталоги включаемых файлов и нормализовать параметры компилятора. Она делает это, оборачивая команду компилятора и переписывая командную строку. Чтобы увидеть это в действии, выполните **bmake patch** в каталоге пакета и изучите каталог `work/.cwrappers/config`. Он содержит отдельные конфигурации для компилятора C и связанных инструментов. План состоит в том, чтобы найти точку подключения между этапами configure и build и изменить там эти файлы конфигурации.

Чтобы найти эту точку подключения, загляните в `mk/build/build.mk`, который содержит, среди прочего, `pre-build-checks-hook`. Слово `checks` подходит не совсем, но `pre-build-hook` звучит достаточно хорошо.

Код, который нужно включить в `mk.conf`:

```
# Just a few example options.
BUILD_ONLY_CFLAGS=      -Wall -Werror -O2 -DTEMPDIR='"/tmp"'

.if ${BUILD_ONLY_CFLAGS:U:M*}
pre-build-checks-hook: add-build-only-cflags

add-build-only-cflags: .PHONY
        ${RUN} cd ${CWRAPPERS_CONFIG_DIR};      \
        ${TEST} ! -f ${.TARGET} || exit 0;      \
        for flag in ${BUILD_ONLY_CFLAGS}; do    \
                ${ECHO} "append=$$flag" >> cc;  \
        done;                                   \
        > ${.TARGET}
.endif
```

(При редактировании `mk.conf` убедитесь, что команды цели `add-build-only-cflags` имеют отступ табуляцией, а не пробелами.)

Условие в операторе `.if` содержит модификатор `:U`, чтобы избежать ошибок разбора, если переменная не определена — возможно, потому что она определена только для части пакетов. Модификатор `:M*` гарантирует наличие хотя бы одного параметра компилятора, чтобы избежать синтаксической ошибки в разборщике оболочки.

Код вокруг переменной `${.TARGET}` гарантирует, что дополнительные параметры компилятора добавляются только один раз, даже если **bmake build** запускается несколько раз. Для этого он создаёт файл-маркер.

Чтобы проверить, что эта настройка работает, выполните **bmake configure** в каталоге пакета. Пока всё работает как обычно. Изучите каталог `work/.cwrappers/config` и убедитесь, что параметры компилятора из `BUILD_ONLY_CFLAGS` ещё не добавлены в файл `cc`. Посмотрите конец файла `work/.work.log` и убедитесь, что используются обычные параметры компилятора.

Теперь выполните **bmake build**. Это добавит параметры в файл `cc` и создаст файл-маркер в том же каталоге. После этого сборка начнётся как обычно, но с добавленными параметрами компилятора. Посмотрите конец файла `work/.work.log` и убедитесь, что строки, начинающиеся с `[*]`, не содержат параметров компилятора, а соответствующие строки, начинающиеся с `<.>`, заканчиваются этими параметрами.

Сборка пакетов с этим вариантом настройки и исправление возникающих ошибок выполняются так же, как в разделе 8.6.2, «Обнаружение классов ошибок принудительными предупреждениями компилятора».

8.6.4. Использование собственных каталогов

Некоторые каталоги, такие как `PREFIX`, `VARBASE`, `PKG_SYSCONFDIR`, `PKGMANDIR`, `PKG_INFODIR`, можно настроить в pkgsrc. Задайте для них произвольные пути во время начальной загрузки или позже в `mk.conf`.

```
PREFIX=         /a-random-uuid
PKG_SYSCONFDIR= /a-random-uuid
VARBASE=        /a-random-uuid
PKGMANDIR=      a-random-uuid
PKG_INFODIR=    a-random-uuid
```

8.6.5. Превращение предупреждений в ошибки

При сборке пакета предупреждения обычно игнорируются, поскольку они просто проходят мимо и не приводят к немедленному сбою сборки. Чтобы найти такие предупреждения, переопределите их как ошибки в `mk.conf`.

```
DELAYED_WARNING_MSG=    ${DELAYED_ERROR_MSG} "(was warning)"
WARNING_MSG=            ${FAIL_MSG} "(was warning)"
```

(В pkgsrc есть много других классов предупреждений, и большинство из них можно переопределить простым определением, как выше.

Если пакет предлагает добавить `USE_TOOLS+=perl` в Makefile пакета, выясните, действительно ли пакету нужен Perl. Если нужен, добавьте `USE_TOOLS+=perl` в Makefile пакета, а если нет — добавьте `TOOLS_BROKEN+=perl`.

8.6.6. Отклонение пакетов, для которых pkglint сообщает об ошибках

Использование pkglint в составе обычного процесса сборки — в основном пустая трата времени. Если вы хотите исправить некоторые предупреждения, просто запустите pkglint рекурсивно по всему дереву pkgsrc. Это займёт несколько минут (до 10), что гораздо быстрее полной массовой сборки.

8.6.7. Отклонение пакетов, содержащих запрещённые строки

Чтобы гарантировать, что двоичные пакеты не содержат ссылок на каталог сборки, уже существует `CHECK_WRKREF`. Если эта переменная включает элемент `extra`, можно определить дополнительные шаблоны, которые не должны встречаться ни в одном установленном файле. Это задаётся в `mk.conf`.

```
CHECK_WRKREF=                   extra
CHECK_WRKREF_EXTRA_DIRS+=       /usr/local
CHECK_WRKREF_EXTRA_DIRS+=       /usr/pkg
CHECK_WRKREF_EXTRA_DIRS+=       @[A-Z][A-Z]*@
```

Приведённые выше шаблоны, вероятно, дадут много ложных срабатываний, поэтому к результатам следует относиться с осторожностью.

8.6.8. Отклонение пакетов, у которых не проходят самотесты

Чтобы запускать наборы тестов, входящие в каждый пакет, добавьте в `mk.conf` эту строку.

```
PKGSRC_RUN_TEST=        yes
```

Будьте готовы к тому, что даже самые простые пакеты не проходят этот тест. При массовой сборке с этой настройкой она часто прерывается на ранней стадии, когда пакеты сканируются на предмет зависимостей, — из-за циклических зависимостей. В этой области ещё многое предстоит сделать.

8.6.9. Отклонение пакетов, использующих неопределённые переменные оболочки

Чтобы отлавливать опечатки в коротких сценариях оболочки из фрагментов Makefile, добавьте флаг `-u` к большинству команд, вставив следующую строку в `mk.conf`.

```
RUN=    @set -eu;
```

После этого убедитесь, что ни один из файлов журнала массовой сборки (даже для успешно собранных пакетов) не содержит строку `parameter not set` или иное сообщение об ошибке, которое выводит команда **sh -ceu '$undefined'**.

Существующее определение см. в `mk/misc/common.mk`.

8.6.10. Отключение подробного журналирования

Журналы сборки пакета часто бывают довольно длинными. Из-за этого сообщения об ошибках и другие интересные подробности теряются в шуме. Чтобы настоящее сообщение об ошибке было заметнее, добавьте следующие строки в `mk.conf`.

```
GNU_CONFIGURE_QUIET=    yes
MAKE_FLAGS+=            -s
```

Параметр `-s` работает как в GNU Make, так и в BSD Make. На экзотических платформах с собственной реализацией make поведение может немного отличаться.

8.7. Создание коллекции пакетов на нескольких CD-ROM

После завершения массовой сборки pkgsrc вы, возможно, захотите создать набор CD-ROM из полученных двоичных пакетов, чтобы упростить установку пакетов на других машинах. Пакет `pkgtools/cdpack` предоставляет простой инструмент для создания образов ISO 9660. **cdpack** размещает пакеты на CD-ROM так, чтобы все зависимости данного пакета находились на том же CD, что и сам пакет.

8.7.1. Пример работы с cdpack

Полная документация по cdpack приведена на странице руководства cdpack(1). В следующем коротком примере предполагается, что двоичные пакеты находятся в `/usr/pkgsrc/packages/All` и что в `/u2` достаточно места на диске для размещения образов ISO 9660.

```
# mkdir /u2/images
# pkg_add /usr/pkgsrc/packages/All/cdpack
# cdpack /usr/pkgsrc/packages/All /u2/images
```

Если вы хотите включить общий набор файлов (`COPYRIGHT`, `README` и т. д.) на каждый CD коллекции, создайте каталог, содержащий эти файлы, например:

```
# mkdir /tmp/common
# echo "This is a README" > /tmp/common/README
# echo "Another file" > /tmp/common/COPYING
# mkdir /tmp/common/bin
# echo "#!/bin/sh" > /tmp/common/bin/myscript
# echo "echo Hello world" >> /tmp/common/bin/myscript
# chmod 755 /tmp/common/bin/myscript
```

Теперь создайте образы:

```
# cdpack -x /tmp/common /usr/pkgsrc/packages/All /u2/images
```

Каждый образ будет содержать `README`, `COPYING` и `bin/myscript` в своём корневом каталоге.

Глава 9. Структура каталогов установленных файлов

Файлы, устанавливаемые pkgsrc, организованы почти так же, как в каталоге `/usr` базовой системы. Однако некоторые детали отличаются. Это связано с тем, что pkgsrc изначально пришёл из FreeBSD и унаследовал её иерархию файловой системы. Позже на него во многом повлиял NetBSD. Но независимо от того, с какой операционной системой вы используете pkgsrc, можно рассчитывать на одинаковую структуру каталогов pkgsrc.

У pkgsrc есть четыре основных корневых каталога, и все они настраиваются в сценарии `bootstrap/bootstrap`. Если pkgsrc установлен от имени root, расположения по умолчанию таковы:

```
LOCALBASE=              /usr/pkg
PKG_SYSCONFBASE=        /usr/pkg/etc
VARBASE=                /var
PKG_DBDIR=              /usr/pkg/pkgdb
```

В непривилегированном режиме (когда pkgsrc установлен от имени любого другого пользователя) расположения по умолчанию таковы:

```
LOCALBASE=              ${HOME}/pkg
PKG_SYSCONFBASE=        ${HOME}/pkg/etc
VARBASE=                ${HOME}/pkg/var
PKG_DBDIR=              ${HOME}/pkg/pkgdb
```

Назначение этих четырёх каталогов и их вид описаны ниже.

- `LOCALBASE` соответствует каталогу `/usr` в базовой системе. Это «основной» каталог, в который устанавливаются файлы; он содержит известные подкаталоги, такие как `bin`, `include`, `lib`, `share` и `sbin`.
- `VARBASE` соответствует `/var` в базовой системе. Некоторым программам (особенно играм и сетевым демонам) при обычной работе требуется доступ на запись в этот каталог.
- `PKG_SYSCONFDIR` соответствует `/etc` в базовой системе. Он содержит файлы конфигурации пакетов, а также сам файл `mk.conf` из pkgsrc.

9.1. Структура файловой системы в `${LOCALBASE}`

В типичной установке pkgsrc в `${LOCALBASE}` существуют следующие каталоги.

`bin`
: Содержит исполняемые программы, предназначенные для непосредственного использования конечным пользователем.

`emul`
: Содержит файлы для слоёв эмуляции различных других операционных систем, особенно NetBSD.

`etc` (обычное расположение `${PKG_SYSCONFDIR}`)
: Содержит файлы конфигурации.

`include`

Содержит заголовочные файлы для языков программирования C и C++.

`info`
: Содержит файлы GNU info различных пакетов.

`lib`
: Содержит разделяемые и статические библиотеки.

`libdata`
: Содержит файлы данных, которые не изменяются после установки. Остальные файлы данных должны находиться в `${VARBASE}`.

`libexec`
: Содержит программы, не предназначенные для использования конечными пользователями, например вспомогательные программы или сетевые демоны.

`libexec/cgi-bin`
: Содержит программы, предназначенные для выполнения веб-сервером в качестве CGI-сценариев.

`man` **(обычное значение `${PKGMANDIR}`)**
: Содержит краткую документацию в виде страниц руководства (man).

`sbin`
: Содержит программы, предназначенные для использования только суперпользователем.

`share`
: Содержит независимые от платформы файлы данных, которые не изменяются после установки.

`share/doc`
: Содержит файлы документации, поставляемые пакетами.

`share/examples`
: Содержит файлы примеров, поставляемые пакетами. В частности, здесь сохраняются исходные файлы конфигурации, которые при установке копируются в `${PKG_SYSCONFDIR}`.

`share/examples/rc.d`
: Содержит исходные файлы сценариев rc.d.

`var` **(обычное расположение `${VARBASE}`)**
: Содержит файлы, которые могут изменяться после установки.

9.2. Структура файловой системы в `${VARBASE}`

`db/pkg` **(обычное расположение `${PKG_DBDIR}`)**
: Содержит сведения о текущих установленных пакетах.

`games`
: Содержит файлы рекордов.

`log`
: Содержит файлы журналов.

`run`
: Содержит информационные файлы о работающих в данный момент демонах.

Глава 10. Часто задаваемые вопросы

В этом разделе собраны подсказки, советы и приёмы по особым вопросам pkgsrc, для которых в предыдущих главах не нашлось лучшего места; он адресован как пользователям, так и разработчикам pkgsrc.

10.1. Существуют ли списки рассылки для обсуждения вопросов о пакетах?

Пользователям pkgsrc могут быть интересны следующие списки рассылки:

- pkgsrc-users: список общего назначения для большинства вопросов, связанных с pkgsrc, независимо от платформы: например, просьбы о помощи в настройке pkgsrc, неожиданные сбои сборки, использование отдельных пакетов, обновление установок pkgsrc, вопросы о ветках выпуска pkgsrc и т. п. В нём также можно публиковать общие объявления и предложения об изменениях, затрагивающих сообщество пользователей pkgsrc: например, крупные изменения инфраструктуры, новые возможности, удаление пакетов и т. п.
- pkgsrc-bulk: список, в который отправляются и в котором обсуждаются результаты массовых сборок pkgsrc.
- pkgsrc-changes: этот список предназначен для тех, кто хочет получать сообщение о фиксации для каждого изменения, внесённого в pkgsrc. Он также доступен в виде дайджеста, то есть одного ежедневного сообщения, содержащего все сообщения о фиксации изменений в дереве исходных текстов пакетов за эти 24 часа.

Чтобы подписаться, выполните:

```
% echo subscribe listname | mail majordomo@NetBSD.org
```

Архивы всех этих списков рассылки доступны по адресу https://mail-index.NetBSD.org/.

10.2. Утилиты для управления пакетами (pkgtools)

Каталог `pkgsrc/pkgtools` содержит ряд полезных утилит как для пользователей, так и для разработчиков pkgsrc. Этот раздел лишь знакомит читателя с некоторыми утилитами и с тем, когда они могут пригодиться, и не повторяет документацию, поставляемую с каждым пакетом.

Утилиты, используемые pkgsrc (устанавливаются автоматически при необходимости):

- `pkgtools/x11-links`: символические ссылки для использования buildlink.

Дополнения к инструментам ОС (устанавливаются автоматически при необходимости):

- `pkgtools/digest`: вычисляет различные виды контрольных сумм (включая SHA3).
- `pkgtools/libnbcompat`: библиотека совместимости для инструментов pkgsrc.
- `pkgtools/mtree`: устанавливается в системах, отличных от BSD, из-за отсутствия собственного mtree.

- `pkgtools/pkg_install`: актуальная замена `/usr/sbin/pkg_install` либо вариант для использования в операционных системах, где pkg_install отсутствует.

Утилиты, используемые pkgsrc (не устанавливаются автоматически):

- `pkgtools/pkg_tarup`: создаёт двоичный пакет из уже установленного пакета. Используется командой **make replace** для сохранения старого пакета.
- `pkgtools/dfdisk`: добавляет в pkgsrc дополнительную функциональность, позволяя получать дистрибутивные файлы из нескольких расположений. В настоящее время поддерживаются следующие способы: несколько CD-ROM и сетевые соединения FTP/HTTP.
- `devel/cpuflags`: определяет наилучшие флаги компилятора для оптимизации кода под ваш текущий процессор и компилятор.

Утилиты для отслеживания установленных пакетов, поддержания их актуальности и т. п.:

- `pkgtools/pkgin`: средство обновления пакетов, похожее на apt(1). Позволяет легко загружать, устанавливать и обновлять двоичные пакеты.
- `pkgtools/pkg_chk`: сообщает о пакетах, установленные версии которых не соответствуют последним записям pkgsrc.
- `pkgtools/pkgdep`: строит графы зависимостей пакетов, помогая выбрать стратегию обновления.
- `pkgtools/pkgdepgraph`: строит графы по выходным данным `pkgtools/pkgdep` (использует graphviz).
- `pkgtools/pkglint`: программа pkglint(1) проверяет запись pkgsrc на наличие ошибок.
- `pkgtools/lintpkgsrc`: программа lintpkgsrc(1) выполняет различные проверки всей системы pkgsrc.
- `pkgtools/pkgsurvey`: сообщает, какие пакеты у вас установлены.

Утилиты для тех, кто сопровождает или создаёт отдельные пакеты:

- `pkgtools/pkgdiff`: автоматизирует создание и сопровождение патчей для пакета (включает pkgdiff, pkgvi, mkpatches и др.).
- `pkgtools/url2pkg`: помогает преобразовать программу в пакет pkgsrc.

Утилиты для тех, кто сопровождает pkgsrc (или: менее известные утилиты для пакетов)

- `pkgtools/pkg_comp`: собирает пакеты в области chroot.
- `pkgtools/libkver`: подменяет версию ядра для кросс-сборок в chroot.

10.3. Как использовать pkgsrc без прав root

Чтобы устанавливать пакеты из исходных текстов от имени пользователя без прав root, загрузите pkgsrc, как описано в главе 3, *Где взять pkgsrc и как поддерживать его в актуальном состоянии*, перейдите в этот каталог и выполните команду **./bootstrap/bootstrap --unprivileged**.

При этом двоичная часть pkgsrc будет установлена в `~/pkg`, а файл конфигурации pkgsrc `mk.conf` — в `~/pkg/etc`.

Подробнее см. в `mk/unprivileged.mk`.

10.4. Как возобновить передачу при загрузке дистрибутивных файлов?

По умолчанию возобновление передачи в pkgsrc отключено, но эту возможность можно включить, добавив параметр `PKG_RESUME_TRANSFERS=YES` в `mk.conf`. Если на шаге загрузки обнаруживается неполный дистрибутивный файл, pkgsrc пытается возобновить его загрузку.

Можно также использовать другую программу вместо заданной на платформе по умолчанию, изменив переменную `FETCH_USING`. Программу можно указать с помощью значений ftp, fetch, wget или curl. Кроме того, загрузку можно отключить, указав значение manual. Значение custom отключает системные значения по умолчанию и отслеживание зависимостей для программы загрузки. В этом случае необходимо задать `FETCH_CMD`, `FETCH_BEFORE_ARGS`, `FETCH_RESUME_ARGS`, `FETCH_OUTPUT_ARGS`, `FETCH_AFTER_ARGS`.

Например, если вы хотите использовать для загрузки `wget`, потребуется примерно следующее:

```
FETCH_USING=    wget
```

10.5. Как установить и использовать модульный X.org из pkgsrc?

Если вы хотите использовать модульный X.org из pkgsrc вместо собственной системы X11 (`/usr/X11R6`, `/usr/openwin`, ...), добавьте следующую строку в `mk.conf`:

```
X11_TYPE=modular
```

10.6. Как загружать файлы из-за межсетевого экрана

Если вы находитесь за межсетевым экраном, который не разрешает прямые соединения с хостами Интернета (то есть без NAT), можно указать соответствующие прокси-хосты. Это делается с помощью переменной окружения в виде URL; например, в Amdahl машина «orpheus.amdahl.com» является одним из межсетевых экранов и использует порт 80 в качестве номера порта прокси. Таким образом, переменные окружения прокси таковы:

```
ftp_proxy=ftp://orpheus.amdahl.com:80/
http_proxy=http://orpheus.amdahl.com:80/
```

10.7. Как загружать файлы с сайтов HTTPS

Некоторые инструменты загрузки не поддерживают HTTPS по умолчанию (например, инструмент в NetBSD 6.0) — как и тот, что устанавливается при начальной загрузке pkgsrc (чтобы избежать зависимости от openssl так низко в графе зависимостей).

Обычно вы этого не заметите, поскольку дистрибутивные файлы еженедельно зеркалируются на «ftp.NetBSD.org», но этого может оказаться недостаточно часто, если вы следите за pkgsrc-current. В таком случае задайте `FETCH_USING` в вашем файле `mk.conf` равной «curl» или «wget» — обе эти программы по умолчанию собраны с поддержкой HTTPS. Разумеется, чтобы использовать их таким образом, эти инструменты должны быть установлены.

10.8. Как указать make fetch использовать пассивный FTP?

Это зависит от того, какая утилита используется для получения дистрибутивных файлов. В `bsd.pkg.mk` переменной `FETCH_CMD` присваивается первая доступная команда из следующего списка:

- `${LOCALBASE}/bin/ftp`
- `/usr/bin/ftp`

При установке NetBSD по умолчанию это будет `/usr/bin/ftp`, который автоматически сначала пробует пассивные соединения и переключается на активные, если сервер отказывается от пассивных. Для остальных инструментов добавьте в свой файл `mk.conf` следующее: `PASSIVE_FETCH=1`.

Наличие этого параметра не позволит `/usr/bin/ftp` переключаться на активную передачу.

10.9. Как загрузить все дистрибутивные файлы сразу

Возможно, вы хотите загрузить все дистрибутивные файлы одним пакетом с работы или из университета, где нельзя выполнить **make fetch**. На ftp.NetBSD.org есть архив дистрибутивных файлов, но загрузка всего каталога может оказаться нецелесообразной.

Решение — выполнить **make fetch-list** в `/usr/pkgsrc` или одном из его подкаталогов, перенести полученный список на машину на работе или в школе и использовать его там. Если на работе у вас нет совместимого с NetBSD ftp(1) (например, tnftp), не забудьте задать `FETCH_CMD` как команду, загружающую URL:

Дома:

```
% cd /usr/pkgsrc
% make fetch-list FETCH_CMD=wget DISTDIR=/tmp/distfiles >/tmp/fetch.sh
% scp /tmp/fetch.sh work:/tmp
```

На работе:

```
% sh /tmp/fetch.sh
```

затем упакуйте `/tmp/distfiles` в архив tar и заберите его домой.

Если у вас есть машина под управлением NetBSD и вы хотите получить *все* дистрибутивные файлы (даже те, что предназначены не для вашей архитектуры), можно воспользоваться упомянутым выше способом с **make fetch-list** или загрузить дистрибутивные файлы напрямую, выполнив:

```
% make mirror-distfiles
```

Если вы даже решите пренебречь `NO_{SRC,BIN}_ON_{FTP,CDROM}`, то сможете получить всё, выполнив:

```
% make fetch NO_SKIP=yes
```

10.10. Что означает «Don't know how to make /usr/share/tmac/tmac.andoc»?

При сборке пакета `pkgtools/pkg_install` вы получаете от make ошибку о том, что он не знает, как создать `/usr/share/tmac/tmac.andoc`? Это означает, что на вашей машине не установлен набор

«text» (nroff, …) из базового дистрибутива NetBSD. Его рекомендуется установить, чтобы форматировать страницы руководства.

В случае пакета `pkgtools/pkg_install` можно обойтись, задав `NOMAN=YES` в окружении или в `mk.conf`.

10.11. Что означает «Could not find bsd.own.mk»?

При установке NetBSD вы не установили набор компилятора `comp.tgz`. Получите и установите его, распаковав в `/`:

```
# cd /
# tar --unlink -zxvpf .../comp.tgz
```

`comp.tgz` входит в каждый выпуск NetBSD. Возьмите тот, что соответствует вашему выпуску (определите его с помощью **uname -r**).

10.12. Использование «sudo» или «priv» с pkgsrc

При установке пакетов от имени пользователя без прав root и использовании функции su(1) с запуском по требованию (just-in-time) в pkgsrc вводить пароль root для каждого устанавливаемого пакета становится утомительно. Чтобы этого избежать, можно использовать пакет sudo, который кэширует пароль на ограниченное время. Для этого установите **sudo** или **priv** (либо как двоичный пакет, либо из `security/priv`, или `security/sudo`) и затем поместите следующее в свой `mk.conf`, где-нибудь *после* определения переменной `LOCALBASE`:

```
.if exists(${LOCALBASE}/bin/sudo)
SU_CMD=         ${LOCALBASE}/bin/sudo -E /bin/sh -c
.endif
```

Обратите внимание на флаг **-E**: он указывает **sudo** сохранять окружение. Без этого цель bin-install будет завершаться ошибкой при сборке пакета с версией python, отличной от используемой по умолчанию.

10.13. Как изменить расположение файлов конфигурации?

Как системный администратор вы можете выбрать, куда устанавливаются файлы конфигурации. По умолчанию все эти файлы попадают в `${PREFIX}/etc` или некоторые его подкаталоги; это может быть неоптимально в зависимости от ваших ожиданий (например, если `PREFIX` доступен только для чтения и экспортируется по NFS, а конфигурация установленных пакетов должна быть своей на каждой машине).

Чтобы изменить значения по умолчанию, можно изменить переменную `PKG_SYSCONFBASE` (в `mk.conf`) так, чтобы она указывала на предпочитаемый каталог конфигурации; типичные примеры — `/etc` или `/etc/pkg`.

Кроме того, это значение можно изменить для отдельного пакета, задав переменную `PKG_SYSCONFDIR.${PKG_SYSCONFVAR}`. Значение `PKG_SYSCONFVAR` обычно совпадает с именем пакета, который вы хотите изменить, то есть с содержимым `PKGBASE`.

Учтите, что после изменения этих настроек необходимо пересобрать и переустановить все затронутые пакеты.

10.14. Автоматические проверки безопасности

Учтите, что в стороннем программном обеспечении часто встречаются ошибки, и некоторые из них могут делать машину уязвимой для атак злоумышленников. Чтобы уменьшить подверженность риску, команда сопровождения пакетов NetBSD ведёт базу данных известных уязвимостей пакетов, которые когда-либо входили в pkgsrc. Базу данных можно загрузить автоматически, после чего можно провести проверку безопасности всех установленных в системе пакетов. Для этого обратитесь к следующим двум инструментам (устанавливаются вместе с пакетом `pkgtools/pkg_install`):

1. **pkg_admin fetch-pkg-vulnerabilities** — простой способ загрузить список сведений об уязвимостях безопасности. Этот список поддерживается в актуальном состоянии командой безопасности pkgsrc и распространяется с FTP-сервера NetBSD:

   https://ftp.NetBSD.org/pub/NetBSD/packages/vulns/pkg-vulnerabilities
2. **pkg_admin audit** — простой способ проверить текущую машину, проверяя каждую известную уязвимость. Если установлен уязвимый пакет, он будет показан в выводе на стандартный вывод, включая описание типа уязвимости и URL с дополнительной информацией.

Настоятельно рекомендуется использовать эти инструменты! Инструкции по автоматизации проверки и составления отчётов см. в разделе 5.1.6, «Проверка уязвимостей безопасности в установленных пакетах».

Если эта база данных установлена, сборки pkgsrc будут использовать её для проверки безопасности перед сборкой любого пакета.

10.15. Почему некоторые пакеты игнорируют мои `CFLAGS`?

Когда вы добавляете свои предпочтения в переменную `CFLAGS` в своём `mk.conf`, эти флаги передаются через переменные окружения сценариям `./configure` и программе make(1). Некоторые авторы пакетов игнорируют `CFLAGS` из переменной окружения, переопределяя их в файлах `Makefile` своего пакета.

В настоящее время эта проблема не имеет решения. Если пакету действительно нужны ваши `CFLAGS`, выполните **make patch** в каталоге пакета, а затем проверьте файлы `Makefile` и `Makefile.in` на предмет явного определения `CFLAGS`. Обычно эти строки можно удалить. Но учтите, что некоторые «умные» программисты пишут настолько плохой код, что он работает только с тем конкретным набором `CFLAGS`, который они выбрали.

Чтобы выяснить, где игнорируются CFLAGS, добавьте следующие строки в `mk.conf`:

```
CPPFLAGS+=              -Dpkgsrc___CPPFLAGS
CFLAGS+=                -Dpkgsrc___CFLAGS
CXXFLAGS+=              -Dpkgsrc___CXXFLAGS
```

Затем выполните **bmake show-all-configure show-all-build**, чтобы увидеть, передаются ли вообще указанные выше флаги в фактические команды сборки.

Чтобы выяснить, передаются ли ключи отдельным командам компилятора, загляните в файл `work/.work.log`. В большинстве случаев ключи из исходных командных строк (строк, начинающихся с `[*]`) передаются фактическому компилятору без изменений (строки, начинающиеся с `<.>`). Если ключ отсутствует в фактической команде компилятора, значит, его удалили обёртки компилятора pkgsrc.

10.16. Пакет не собирается. Что делать?

1. Убедитесь, что ваша копия pkgsrc согласована. Часто встречается ситуация, когда люди обновляют pkgsrc только частично — по соображениям производительности. Поскольку pkgsrc представляет собой одну большую систему, а не набор из множества небольших, иногда в нём появляются изменения, которые работают только при обновлении всего дерева pkgsrc.
2. Убедитесь, что у вас нет конфликтов CVS. Поищите во всех файлах pkgsrc «<<<<<<» или «>>>>>>».
3. Убедитесь, что у вас не осталось старых распакованных копий пакетов. Для проверки выполните **make clean clean-depends**.
4. Если вы разработчик пакетов и хотите приложить немного усилий, загляните в главу 21, *Как заставить пакет работать*.
5. Если проблема сохраняется, напишите письмо в список рассылки `pkgsrc-users`.

10.17. Что означает «Makefile appears to contain unresolved cvs/rcs/??? merge conflicts»?

Вы изменили файл из pkgsrc, а кто-то другой позже изменил тот же файл в репозитории CVS. Оба изменения находятся в одном и том же месте файла, поэтому при обновлении pkgsrc команда `cvs` пометила конфликтующие изменения в файле. Из-за этих пометок файл больше не является допустимым `Makefile`.

Посмотрите на этот файл: если локальные изменения вам больше не нужны, удалите его и выполните **cvs -q update -dP** в этом каталоге, чтобы загрузить текущую версию.

Часть II. Руководство разработчика pkgsrc

Эта часть книги посвящена созданию и изменению пакетов. Она начинается с руководства в стиле «HOWTO» по созданию нового пакета. Остальные главы больше похожи на справочное руководство по pkgsrc.

Глава 11. Получение помощи

Чтобы получить помощь при разработке pkgsrc, основным источником служит этот документ — руководство по pkgsrc. Если вы не нашли здесь ничего подходящего, есть другие варианты:

- Встроенная справка pkgsrc, доступная после инициализации (bootstrap) pkgsrc. Выполните **bmake help topic=...**, чтобы получить справку по любой теме: имя переменной, например `BUILD_DEFS`, цель make, например **do-build**, отсутствующая функция C или C++, например **strcasecmp**, или любая другая тема.

  Доступные темы справки перечислены в Приложение E, *Темы справки*.
- Чтобы увидеть значение одной переменной, выполните **bmake show-var VARNAME=*X***.
- Чтобы увидеть значения самых распространённых переменных, выполните **bmake show-all**. Эти переменные сгруппированы по темам. Чтобы увидеть переменные одной темы, выполните **bmake show-all-*topic***, например **bmake show-all-fetch**.
- Список рассылки tech-pkg, на который вы можете подписаться, а затем задать свои вопросы.
- Канал IRC #pkgsrc, доступный через веб-браузер или с помощью специализированной программы для общения, например XChat. Выберите любое имя пользователя и присоединитесь к каналу #pkgsrc.

Глава 12. Компоненты пакета — файлы, каталоги и их содержимое

При подготовке пакета участвует ряд файлов, которые описаны в следующих разделах.

12.1. `Makefile`

Сборка, установка и создание двоичного пакета управляются файлом `Makefile` пакета. В `Makefile` описываются различные сведения о пакете, например, откуда его взять, как настроить, собрать и установить.

Файл `Makefile` пакета содержит несколько разделов, описывающих пакет.

В первом разделе находятся следующие переменные, которые должны идти точно в указанном здесь порядке. Порядок и группировка переменных сложились в основном исторически и не несут дополнительного смысла.

- `DISTNAME` — базовое имя дистрибутивного файла, который будет загружен с веб-сайта пакета.
- `PKGNAME` — имя пакета, используемое в pkgsrc. Его нужно задать, если `DISTNAME` (который используется по умолчанию) не является подходящим именем для пакета в pkgsrc или `DISTNAME` не задан (дистрибутивный файл не требуется). Обычно это имя каталога в pkgsrc вместе с номером версии. Оно должно соответствовать регулярному выражению `^[A-Za-z0-9][A-Za-z0-9-_.+]*$`, то есть начинаться с буквы или цифры и содержать только буквы, цифры, дефисы, знаки подчёркивания, точки и знаки плюс. Новые пакеты должны иметь имена, состоящие только из строчных букв, за исключением некоторых, которые должны следовать ранее установленным соглашениям (например, пакеты R начинаются с `R-`). Чтобы привести имя пакета к нижнему регистру, можно использовать модификатор переменной `:tl`.
- `CATEGORIES` — список категорий, к которым относится пакет. В качестве категории можно выбрать любой каталог верхнего уровня pkgsrc.

  В настоящее время для `CATEGORIES` доступны следующие значения. Если используется более одного, их нужно разделять пробелами:

  ```
  archivers     cross         geography     meta-pkgs     security
  audio         databases     graphics      misc          shells
  benchmarks    devel         ham           multimedia    sysutils
  biology       editors       inputmethod   net           textproc
  cad           emulators     lang          news          time
  chat          finance       mail          parallel      wm
  comms         fonts         math          pkgtools      www
  converters    games         mbone         print         x11
  ```

- `MASTER_SITES`, `DYNAMIC_MASTER_SITES`, `DIST_SUBDIR`, `EXTRACT_SUFX` и `DISTFILES` подробно рассматриваются в разделе 13.5, «Фаза *fetch*».

Второй раздел содержит сведения об отдельно загружаемых патчах, если они есть.

- `PATCHFILES`: имена дополнительных файлов, содержащих патчи дистрибутива. Значения по умолчанию нет. pkgsrc будет искать их в `PATCH_SITES`. Перед наложением патчей они

автоматически распаковываются, если их имена заканчиваются на `.gz` или `.Z`.

- `PATCH_SITES`: основные места расположения файлов патчей дистрибутива (см. `PATCHFILES` выше), если они не найдены локально.
- `PATCH_DIST_STRIP`: аргумент для patch(1), задающий число отбрасываемых компонентов пути, что помогает найти нужные файлы для наложения патча. По умолчанию используется **-p1**.

Третий раздел содержит следующие переменные.

- `MAINTAINER` — адрес электронной почты человека, который в настоящее время отвечает за этот пакет. В теории сопровождающий знает о новых выпусках вышестоящего проекта, поддерживает пакет в актуальном состоянии и, как правило, решает проблемы. Тот, кто задумывает значительные изменения, должен обратиться к сопровождающему за советом или с просьбой просмотреть предлагаемое изменение. Это не требуется для мелких обновлений, исправления ошибок, изменений, затрагивающих многие пакеты в рамках всего pkgsrc, и прочих вещей, которые считаются бесспорными. Из этой «просьбы» следует, что сопровождающий должен отвечать на запросы достаточно оперативно (в большинстве случаев) и либо одобрять изменения, либо объяснять, почему нет. Как правило, при создании нового пакета программы в `MAINTAINER` указывают себя. Если вы не собираетесь сопровождать пакет (особенно если считаете, что у вас нет особых знаний о нём), укажите `<pkgsrc-users@NetBSD.org>`. Это особое значение — код, указывающий на отсутствие сопровождающего.
- `HOMEPAGE` — URL-адрес, по которому пользователи могут найти дополнительную информацию о пакете.
- `COMMENT` — однострочное описание пакета (не должно содержать имя пакета).
- `LICENSE` указывает лицензию (лицензии), применимые к пакету. Подробности см. в разделе 21.1.3, «Работа с лицензиями».

Другие переменные, влияющие на сборку:

- `WRKSRC`: каталог, в котором находятся нужные дистрибутивные файлы пакета. По умолчанию это `${WRKDIR}/${DISTNAME}`, что подходит для большинства пакетов.

  Если пакет не создаёт для себя подкаталог (например, так поступает большинство программ GNU), а распаковывается в текущий каталог, задайте `WRKSRC=${WRKDIR}`.

  Если пакет создаёт подкаталог не с именем `DISTNAME`, а с каким-то другим, задайте `WRKSRC` так, чтобы он указывал на нужное имя в `${WRKDIR}`, например `WRKSRC=${WRKDIR}/${DISTNAME}/unix`. Другие примеры см. в `lang/tcl` и `x11/tk`.

  Имя рабочего каталога, создаваемого pkgsrc, берётся из переменной `WRKDIR_BASENAME`. По умолчанию её значение — `work`. Если вы хотите использовать одно и то же дерево pkgsrc для сборки разных видов двоичных пакетов, переменную можно изменить по своему усмотрению. Ещё две переменные обрабатывают типичные случаи индивидуальной настройки `WRKDIR_BASENAME`. Если `OBJHOSTNAME` определена в `mk.conf`, к имени каталога добавляется первый компонент имени хоста. Если определена `OBJMACHINE`, добавляется имя платформы, и имя каталога может выглядеть как `work.i386` или `work.sparc`.

Обратите внимание на следующие подводные камни:

- Добавьте `MANCOMPRESSED`, если пакет устанавливает страницы руководства (man) в сжатом виде. Для пакетов, использующих makefile-файлы в стиле BSD, которые учитывают MANZ, есть `MANCOMPRESSED_IF_MANZ`.
- Замените `/usr/local` на «${PREFIX}» во всех файлах (см. патчи ниже).

- Если пакет устанавливает какие-либо файлы info, см. раздел 21.6.8, «Пакеты, устанавливающие файлы info».

12.2. distinfo

Файл distinfo содержит дайджест сообщения, или контрольную сумму, каждого дистрибутивного файла, необходимого пакету. Это гарантирует, что дистрибутивные файлы, полученные из Интернета, не были повреждены при передаче и не были изменены злоумышленником ради создания уязвимости. Для максимальной безопасности все дистрибутивные файлы защищаются двумя разными алгоритмами вычисления дайджеста (BLAKE2s и SHA512), а также контролем размера файла.

Файл distinfo также содержит контрольные суммы всех патчей, находящихся в каталоге patches (см. раздел 12.3, «patches/*»). Эти контрольные суммы гарантируют, что патчи применяются только намеренно и не изменяются случайно, например при объединении разных изменений. Они также следят за тем, чтобы новые патчи действительно добавлялись в CVS, а старые удалялись. Чтобы проверить, соответствуют ли патчи файлу distinfo, запустите **pkglint** после изменения патчей.

Чтобы заново сгенерировать файл distinfo, выполните команду **make distinfo**.

Некоторые пакеты имеют разные наборы дистрибутивных файлов в зависимости от платформы, например lang/openjdk8. Они хранятся в одном и том же файле distinfo, и при обновлении такого пакета следует соблюдать осторожность, чтобы не потерять сведения о дистрибутивных файлах.

12.3. patches/*

Некоторые пакеты не работают из коробки на различных платформах, поддерживаемых pkgsrc. Такие пакеты нужно пропатчить, чтобы они заработали. Файлы патчей находятся в каталоге patches/.

На фазе *patch* эти патчи применяются к файлам в каталоге WRKSRC после их распаковки, в алфавитном порядке.

12.3.1. Структура отдельного файла патча

Файлы patch-* должны быть в формате **diff -bu** и применяться без fuzz, чтобы избежать проблем. (Чтобы заставить патчи применяться с fuzz, можно задать PATCH_FUZZ_FACTOR=-F2.) Кроме того, каждый патч должен содержать изменения только для одного файла, и ни один файл не должен патчиться более чем одним файлом патча. Это помогает сохранить простоту дальнейших изменений.

Каждый файл патча устроен следующим образом: в первой строке находится RCS Id самого патча. Вторая строка из эстетических соображений должна быть пустой. После этого должен идти комментарий для каждого изменения, которое вносит патч. Есть ряд стандартных случаев:

- Патчи для широко известных уязвимостей должны упоминать идентификатор уязвимости (CAN, CVE).
- Патчи, изменяющие исходный код, должны упоминать платформу и другое окружение (например, компилятор), для которого нужен патч.

Патч следует снабдить комментариями так, чтобы любой разработчик, знающий код приложения, мог извлечь из патча пользу. Особую заботу стоит проявить о разработчиках вышестоящего проекта: мы, как правило, хотим, чтобы они приняли наши патчи, и тогда у нас будет меньше работы в будущем.

12.3.2. Создание файлов патчей

Важно отметить, что нужно следить за тем, чтобы в файлы патчей не попадали идентификаторы RCS: при последующем помещении в дерево CVS NetBSD они вызовут проблемы. Чтобы избежать этих проблем, используйте команду **pkgdiff** из пакета `pkgtools/pkgdiff`.

Для ещё большей автоматизации мы рекомендуем использовать **mkpatches** из того же пакета, чтобы создать целый набор патчей. Вам нужно лишь сохранить резервные копии файлов перед их редактированием в виде `filename.orig`, например с помощью **cp -p filename filename.orig** или, что проще, воспользовавшись **pkgvi** из того же пакета. Если вы обновляете пакет таким способом, новый набор патчей можно легко сравнить с ранее существовавшим с помощью **patchdiff**. Файлы в `patches` заменяются новыми, поэтому внимательно проверьте, хотите ли вы принять все изменения.

Закончив работу над пакетом, не забудьте сгенерировать контрольные суммы для файлов патчей с помощью команды **make makepatchsum**, см. раздел 12.2, «`distinfo`».

Добавляя патч, исправляющий проблему в дистрибутивном файле (а не, например, приводящий размещение страниц руководства к принятому в pkgsrc виду), отправьте патч как отчёт об ошибке сопровождающему. Это принесёт пользу пользователям пакета, не использующим pkgsrc, и обычно позволяет удалить патч в следующей версии.

Имена файлов патчей обычно имеют вид `patch-`*`path_to_file__with__underscores.c`*. Многие пакеты до сих пор используют прежнее соглашение `patch-`*`[a-z][a-z]`*, но новые патчи должны иметь вид, содержащий имя файла. Программа **mkpatches**, входящая в `pkgtools/pkgdiff`, задаёт имя автоматически.

При обновлении существующих файлов патчей, если файл использует старое соглашение `patch-`*`[a-z][a-z]`*, лучше не приводить его к новому виду: это вызовет лишь лишние изменения, из-за которых станет труднее отслеживать историю патчей. Аналогично, если патч теперь применяется со смещением к другим строкам, но по-прежнему применяется без изменений, обновлять его не нужно: это тоже без необходимости усложняет историю патчей.

12.3.3. Источники, из которых берутся файлы патчей

Если вы хотите совместно использовать патчи для нескольких пакетов в pkgsrc, например потому что они используют одни и те же дистрибутивные файлы, задайте `PATCHDIR` равным пути, где находятся файлы патчей, например:

```
PATCHDIR=       ../../editors/xemacs/patches
```

Файлы патчей, распространяемые автором или другими сопровождающими, можно перечислить в `PATCHFILES`.

Если нужно хранить патчи, которые не следует помещать в pkgsrc, их можно держать вне дерева pkgsrc, в каталоге `$LOCALPATCHES`. Ожидается, что дерево каталогов там имеет ту же структуру «category/package», что и pkgsrc, а патчи хранятся внутри этих каталогов (также известных как `$LOCALPATCHES/$PKGPATH`). Например, если вы хотите хранить собственный патч для `pkgsrc/graphics/png`, держите его в `$LOCALPATCHES/graphics/png/mypatch`. Все файлы в указанном каталоге должны быть файлами патчей, и *они применяются после патчей pkgsrc*.

12.3.4. Рекомендации по созданию патчей

Исправляя проблему переносимости в коде, не используйте ухищрения препроцессора для проверки текущей операционной системы или платформы. Это вредит переносимости на другие платформы, поскольку зависящие от ОС детали не абстрагируются должным образом.

Общее правило таково: вместо проверки операционной системы, под которой собирается приложение, проверяйте конкретные *возможности*, которые вам нужны. Например, вместо того чтобы считать, что kqueue доступен в NetBSD, и использовать макрос `__NetBSD__` для условного включения поддержки kqueue, добавьте проверку, которая обнаруживает сам kqueue — да, обычно это означает патч для сценария **configure**. Абсолютно ничто не мешает некоторым ОС перенимать интерфейсы других ОС (например, Linux реализует kqueue), а описанные выше проверки этого учесть не могут.

Конечно, проверка возможностей обычно требует больше работы от разработчика, но полученные изменения оказываются чище, и есть шансы, что они будут работать на многих других платформах. Не говоря уже о том, что выше вероятность их последующей интеграции в основные исходные тексты. Помните: *работает только то, что сделано правильно!*

Некоторые типичные примеры:

Таблица 12.1. Примеры патчей

Где	Неверно	Верно
сценарий configure	`case ${target_os} in netbsd*) have_kvm=yes ;; *) have_kvm=no ;; esac`	`AC_CHECK_LIB(kvm, kvm_open, have_kvm=yes, have_kvm=no)`
файл исходного кода на C	`#if defined(__NetBSD__) #  include <sys/event.h> #endif`	`#if defined(HAVE_SYS_EVENT_H) #  include <sys/event.h> #endif`
файл исходного кода на C	`int monitor_file(...) { #if defined(__NetBSD__)  int fd = kqueue();  ... #else  ... #endif }`	`int monitor_file(...) { #if defined(HAVE_KQUEUE)  int fd = kqueue();  ... #else  ... #endif }`

12.3.5. Обратная связь с автором

Всегда, всегда, **всегда** сообщайте разработчикам основного проекта обо всех *исправлениях переносимости* или улучшениях, которые вы вносите в пакет. Это единственный способ привлечь их внимание к проблемам переносимости и гарантировать, что будущие версии будут собираться из

коробки в NetBSD. Кроме того, любой пользователь, взявший более новые дистрибутивные файлы, получит исправления прямо из упакованного кода.

Обычно это подразумевает приведение патчей в порядок (потому что иногда патчи, добавляемые в pkgsrc, — это быстрые хаки), подачу отчётов об ошибках в соответствующие трекеры проектов и работу с авторами основного проекта, чтобы они приняли ваши изменения. *Крайне важно* делать это, чтобы пакеты в pkgsrc оставались простыми и дальнейшие изменения можно было вносить без лишних хлопот.

Сделав это, добавьте в комментарий к патчу URL отчёта об ошибке в вышестоящем проекте.

Поддержите идею свободного программного обеспечения!

12.4. Другие обязательные файлы

`DESCR`

Многострочное описание программы. Оно должно включать все заслуженные упоминания. Пожалуйста, помните, что другие не разделяют вашего чувства юмора (как и особенностей орфографии) и что другие прочтут всё, что вы здесь напишете.

`PLIST`

Этот файл управляет тем, какие файлы устанавливаются в вашей системе: все двоичные файлы, страницы руководства и т. д. В этот файл можно вносить и другие директивы, управляющие созданием и удалением каталогов, а также расположением устанавливаемых файлов. Дополнительные сведения см. в главе 19, *Вопросы, связанные с PLIST*.

12.5. Необязательные файлы

12.5.1. Файлы, влияющие на двоичный пакет

`INSTALL`

Этот сценарий оболочки вызывается программой pkg_add(1) дважды. В первый раз — после распаковки пакета и до того, как файлы перемещены на свои места, во второй раз — после того, как устанавливаемые файлы перемещены на свои места. Его можно использовать для любых пользовательских процедур, которые невозможно выполнить командами @exec в `PLIST`. Дополнительные сведения см. в pkg_add(1) и pkg_create(1). См. также раздел 20.1, «Файлы и каталоги вне префикса установки». Обратите внимание, что переменные в нём можно легко изменять с помощью `FILES_SUBST` в файле `Makefile` пакета:

```
FILES_SUBST+=  SOMEVAR="somevalue"
```

заменяет «@SOMEVAR@» на «somevalue» в файле `INSTALL`. По умолчанию подстановка выполняется для `PREFIX`, `LOCALBASE`, `X11BASE`, `VARBASE` и некоторых других переменных; полный список можно получить, выполнив **make help topic=FILES_SUBST**.

`DEINSTALL`

Этот сценарий выполняется до и после удаления любых файлов. Именно на этот сценарий возлагается уборка дополнительных беспорядочных деталей, связанных с установкой пакета, поскольку pkg_delete умеет только удалять файлы, созданные в исходном дистрибутиве. Дополнительные сведения см. в pkg_delete(1) и pkg_create(1). Можно использовать те же способы замены переменных, что и для файла `INSTALL`.

`MESSAGE`

Этот файл отображается после установки пакета. Хотя в прошлом его использовали часто, у него есть два недостатка: сообщение останется незамеченным, если устанавливается сразу много пакетов, а человек, устанавливающий пакет, и тот, кто использует или настраивает его, могут быть разными. Поэтому его следует применять только в исключительных случаях, когда невнимательное прочтение приведёт к долговременным негативным последствиям.

MESSAGE не следует использовать для:

- призывов прочитать документацию
- напоминаний установить файлы rc.d и задать переменные
- всего, что должно быть объяснено в документации по установке и настройке, прилагаемой к пакету

Если документацию, предоставленную вышестоящим проектом, нужно дополнить, создайте, например, файл files/README.pkgsrc и установите его в каталог документации пакета.

Учтите, что MESSAGE отображается для всех операционных систем и всех систем инициализации. Если MESSAGE необходим, его следует ограничить только теми операционными системами и системами инициализации, к которым он относится.

Учтите, что переменные в нём можно легко изменять с помощью `MESSAGE_SUBST` в файле `Makefile` пакета:

```
MESSAGE_SUBST+=  SOMEVAR="somevalue"
```

заменяет «${SOMEVAR}» на «somevalue» в файле `MESSAGE`. По умолчанию подстановка выполняется для `PKGNAME`, `PKGBASE`, `PREFIX`, `LOCALBASE`, `X11BASE`, `PKG_SYSCONFDIR`, `ROOT_GROUP` и `ROOT_USER`.

Отобразить другой или дополнительный файл можно, задав переменную `MESSAGE_SRC`. По умолчанию используется `MESSAGE`, если этот файл существует.

`ALTERNATIVES`

Этот файл используется механизмом alternatives. Он создаёт, настраивает и удаляет универсальные обёртки, служащие для запуска программ со сходными интерфейсами. Дополнительные сведения см. в pkg_alternatives(8) из pkgtools/pkg_alternatives.

Каждая строка файла содержит два имени файла: сначала обёртка, затем альтернатива, предоставляемая пакетом. Оба пути задаются относительно `PREFIX`.

12.5.2. Файлы, влияющие на процесс сборки

`Makefile.common`

Этот файл содержит произвольный материал, который мог бы находиться и в `Makefile`, но его назначение — использоваться более чем одним пакетом. Этот файл следует применять только тогда, когда пакеты, которые будут его использовать, известны заранее. В остальных случаях часто лучше написать файл `*.mk` и дать ему хорошее имя, описывающее его назначение.

`buildlink3.mk`

Этот файл содержит сведения о зависимостях для механизма buildlink3 (см. главу 18, *Методика buildlink*).

`hacks.mk`

Этот файл содержит обходные решения для ошибок компилятора и подобных вещей. Он автоматически включается инфраструктурой pkgsrc, поэтому отдельная строка `.include` для него не нужна.

`options.mk`

Этот файл содержит код для зависящих от пакета параметров (см. главу 16, *Работа с параметрами*), которые может выбирать пользователь. Если у пакета всего один-два параметра, вполне допустимо поместить код прямо в `Makefile`.

12.5.3. Файлы, ни на что не влияющие

`README*`

Эти файлы не участвуют в создании пакета и потому служат исключительно для информирования разработчика пакета.

`TODO`

Этот файл содержит то, что нужно сделать, чтобы сделать пакет ещё лучше.

12.6. `work*`

Когда вы вводите **make**, дистрибутивные файлы распаковываются в каталог, обозначаемый `WRKDIR`. Его можно удалить, выполнив **make clean**. Помимо исходных текстов, этот каталог используется и для хранения различных файлов с отметками времени. При очистке каталог *удаляется полностью*. По умолчанию это `${.CURDIR}/work` или `${.CURDIR}/work.${MACHINE_ARCH}`, если задана `OBJMACHINE`.

12.7. `files/*`

Если у вас есть файлы, которые нужно поместить в пакет до настройки или сборки, положите их сюда и используйте команду **${CP}** в цели «post-extract».

Если вы хотите таким образом совместно использовать файлы с другими пакетами, задайте переменную `FILESDIR` так, чтобы она указывала на каталог `files` другого пакета, например:

```
FILESDIR=       ../../editors/xemacs/files
```

Глава 13. Процесс сборки

13.1. Введение

В этой главе подробно описано, как собирается пакет. Сборка пакета разделена на различные *фазы* (например, `fetch`, `build`, `install`), все они описаны в следующих разделах. Каждая фаза делится на так называемые *этапы*, которые получают имя содержащей их фазы с приставкой `pre-`, `do-` или `post-`. (Примеры: `pre-configure`, `post-build`.) Большая часть фактической работы выполняется на этапах `do-*`.

Никогда не переопределяйте обычные цели (такие как `fetch`); если это необходимо, переопределяйте вместо них цели `do-*`.

Основные шаги сборки программы всегда одинаковы. Сначала исходные тексты программы (*дистрибутивный файл*) нужно доставить в локальную систему, а затем распаковать. После применения всех патчей pkgsrc, необходимых для правильной компиляции, программу можно настроить, затем собрать (обычно путём компиляции) и, наконец, поместить полученные двоичные файлы и т. п. на свои места в системе.

Чтобы получить больше подробностей о том, что происходит на каждом шаге, можно задать переменную `PKG_VERBOSE` или переменную `PATCH_DEBUG`, если вам нужны лишь подробности о шаге *patch*.

13.2. Расположение программ

Прежде чем в следующем разделе описать процесс, выполняемый системой пакетов NetBSD, кратко расскажем, куда устанавливаются программы и какие переменные на это влияют.

Автоматическая переменная `PREFIX` указывает, куда следует устанавливать все файлы итоговой программы. Обычно она задаётся равной `LOCALBASE` (`/usr/pkg`) или `CROSSBASE` для пакетов из категории `cross`. Значение `PREFIX` нужно подставить в различные места исходных текстов программы, где закодированы пути к этим файлам. Подробности см. в разделе 12.3, «`patches/*`» и разделе 21.3.1, «Разделяемые библиотеки — libtool».

Выбирая, какую из этих переменных использовать, следуйте правилам:

- `PREFIX` всегда указывает на место, куда будет установлен текущий пакет. Ссылаясь на собственный путь установки пакета, используйте «${PREFIX}».
- `LOCALBASE` — это место, куда устанавливаются все пакеты. Если нужно построить аргумент -I или -L для компилятора, чтобы найти заголовочные файлы и библиотеки, установленные другим пакетом, используйте «${LOCALBASE}». Название `LOCALBASE` происходит из FreeBSD, где все пакеты устанавливались в `/usr/local`. Поскольку pkgsrc оставляет `/usr/local` в распоряжении системного администратора, это название неточно.
- `X11BASE` — это место, куда установлен собственно дистрибутив X11 (из xsrc и т. п.). Когда нужны *стандартные* заголовочные файлы X11 (а не установленные каким-либо пакетом), используйте «${X11BASE}».
- Пакеты на основе X11, использующие imake, должны задавать `USE_IMAKE`, чтобы правильно устанавливаться в `LOCALBASE`.

- Внутри `${PREFIX}` пакеты должны устанавливать файлы в соответствии с hier(7), за исключением того, что страницы руководства помещаются в `${PREFIX}/man`, а не в `${PREFIX}/share/man`.

13.3. Каталоги, используемые в процессе сборки

При сборке пакета используются различные каталоги для хранения исходных файлов, временных файлов, внутренних файлов pkgsrc и так далее. Эти каталоги описаны здесь.

Некоторые из переменных каталогов содержат относительные пути. Для таких относительных каталогов есть два общих базовых каталога: `PKGSRCDIR/PKGPATH` используется для каталогов, относящихся к pkgsrc. `WRKSRC` используется для каталогов внутри самого пакета.

`PKGSRCDIR`
: Это абсолютный путь, указывающий на корневой каталог pkgsrc. Обычно он вам не нужен.

`PKGDIR`
: Это абсолютный путь, указывающий на текущий пакет.

`PKGPATH`
: Это путь относительно `PKGSRCDIR`, указывающий на текущий пакет. Он определяется после включения `bsd.prefs.mk` и может использоваться во фрагментах makefile, применяемых несколькими пакетами, чтобы различать эти пакеты. Другие переменные, служащие той же цели, — `PKGBASE` и `PKGNAME`, но они определяются только после включения `bsd.pkg.mk`, а это слишком поздно.

  В `mk.conf` пользователь pkgsrc может использовать `PKGPATH` для настройки таких переменных, как `MAKE_JOBS` и `CFLAGS`.

`WRKDIR`
: Это абсолютный путь, указывающий на каталог, в котором выполняется вся работа. В него распаковываются дистрибутивные файлы. В нём также находятся временные каталоги и файлы журналов, используемые различными механизмами pkgsrc, такими как *buildlink* или *wrappers*.

`WRKSRC`
: Это абсолютный путь, указывающий на каталог, в который распаковываются дистрибутивные файлы. Обычно это непосредственный подкаталог `WRKDIR`, и часто это единственная запись каталога, которая не скрыта. Эта переменная может быть изменена в `Makefile` пакета.

Определение `CREATE_WRKDIR_SYMLINK` принимает значение *yes* или *no* и по умолчанию равно *no*. Оно указывает, следует ли создавать в каталоге записи pkgsrc символическую ссылку на `WRKDIR`. Если пользователи хотят, чтобы их деревья pkgsrc вели себя так, будто они доступны только для чтения, значение `CREATE_WRKDIR_SYMLINK` следует задать равным *no*.

13.4. Запуск фазы

Вы можете запустить конкретную фазу, введя **make** *`phase`*, где *`phase`* — имя фазы. При этом автоматически выполнятся все фазы, необходимые для этой фазы. Фаза по умолчанию — `build`, то есть когда вы запускаете **make** без параметров в каталоге пакета, пакет будет собран, но не установлен.

13.5. Фаза *fetch*

Первый шаг при сборке пакета — получить дистрибутивные файлы (distfiles) с сайтов, которые их предоставляют. Это задача фазы *fetch*.

13.5.1. Что загружать и откуда

В простых случаях `MASTER_SITES` определяет все URL, откуда загружается дистрибутивный файл, имя которого выводится из переменной `DISTNAME`. Более сложные случаи описаны ниже.

Переменная `DISTFILES` задаёт список дистрибутивных файлов, которые нужно загрузить. По умолчанию её значение — `${DEFAULT_DISTFILES}`, а само оно равно `${DISTNAME}${EXTRACT_SUFX}`, так что большинству пакетов определять её вообще не нужно. `EXTRACT_SUFX` по умолчанию равен `.tar.gz`, но его можно свободно менять. Учтите: если вашему пакету нужны дополнительные дистрибутивные файлы помимо стандартного, нельзя просто дописать дополнительные имена с помощью оператора `+=`; нужно написать, например:

```
DISTFILES=      ${DEFAULT_DISTFILES} additional-files.tar.gz
```

Каждый дистрибутивный файл загружается из списка сайтов, обычно из `MASTER_SITES`. Если у пакета несколько `DISTFILES` или несколько `PATCHFILES` с разных сайтов, можно задать `SITES.`*`distfile`* равным списку URL, где можно найти файл *`distfile`* (включая суффикс).

```
DISTFILES=      ${DISTNAME}${EXTRACT_SUFX}
DISTFILES+=     foo-file.tar.gz
SITES.foo-file.tar.gz= \
https://www.somewhere.com/somehow/ \
https://www.somewhereelse.com/mirror/somehow/
```

При фактической загрузке дистрибутивных файлов к каждому элементу из `MASTER_SITES` или `SITES.*` добавляется имя каждого дистрибутивного файла, без промежуточного слэша. Поэтому все значения сайтов должны заканчиваться слэшем или другим символом-разделителем. Это позволяет, например, задать `MASTER_SITES` равным URL CGI-сценария, который получает имя дистрибутивного файла как параметр. В этом случае определение будет выглядеть так:

```
MASTER_SITES=   https://www.example.com/download.cgi?file=
```

Исключение из этого правила — URL, начинающиеся с дефиса. В этом случае URL берётся как есть, по нему выполняется загрузка, а результат сохраняется под именем дистрибутивного файла. Этот способ можно использовать, когда стиль URL загрузки не соответствует обычному случаю, описанному выше. Например, если постоянный URL загрузки представляет собой перенаправление на настоящий URL загрузки или имя загружаемого файла задаётся заголовком HTTP Content-Disposition. В следующем примере будет создан `foo-1.0.0.tar.gz` вместо файла по умолчанию `v1.0.0.tar.gz`.

```
DISTNAME=       foo-1.0.0
MASTER_SITES=   -https://www.example.com/archive/v1.0.0.tar.gz
```

Для `MASTER_SITES` есть несколько предопределённых значений, которые можно использовать в пакетах. Имена переменных говорят сами за себя.

MASTER_SITE_APACHE	MASTER_SITE_BACKUP

MASTER_SITE_CODEBERG	MASTER_SITE_CRATESIO
MASTER_SITE_CYGWIN	MASTER_SITE_DEBIAN
MASTER_SITE_FREEBSD	MASTER_SITE_FREEBSD_LOCAL
MASTER_SITE_GENTOO	MASTER_SITE_GITHUB
MASTER_SITE_GITLAB	MASTER_SITE_GNOME
MASTER_SITE_GNU	MASTER_SITE_GNUPG
MASTER_SITE_GNUSTEP	MASTER_SITE_HASKELL_HACKAGE
MASTER_SITE_IFARCHIVE	MASTER_SITE_KDE
MASTER_SITE_MOZILLA	MASTER_SITE_MOZILLA_ALL
MASTER_SITE_MYSQL	MASTER_SITE_NETLIB
MASTER_SITE_OPENBSD	MASTER_SITE_OPENOFFICE
MASTER_SITE_OSDN	MASTER_SITE_PERL_CPAN
MASTER_SITE_PGSQL	MASTER_SITE_PYPI
MASTER_SITE_RUBYGEMS	MASTER_SITE_R_CRAN
MASTER_SITE_SOURCEFORGE	MASTER_SITE_SUNSITE
MASTER_SITE_SUSE	MASTER_SITE_TEX_CTAN
MASTER_SITE_XCONTRIB	MASTER_SITE_XEMACS
MASTER_SITE_XORG	

Пояснения для некоторых из менее очевидных: `MASTER_SITE_BACKUP` содержит резервные сайты для пакетов, которые сопровождаются в ftp://ftp.NetBSD.org/pub/pkgsrc/distfiles/${DIST_SUBDIR}. `MASTER_SITE_LOCAL` содержит локальные дистрибутивы исходных текстов пакетов, которые сопровождаются в ftp://ftp.NetBSD.org/pub/pkgsrc/distfiles/LOCAL_PORTS/.

Если вы выбираете один из этих предопределённых сайтов, возможно, вам потребуется указать подкаталог этого сайта. Поскольку эти макросы могут разворачиваться более чем в один фактический сайт, для указания подкаталога *необходимо* использовать следующую конструкцию:

```
MASTER_SITES=   ${MASTER_SITE_GNU:=subdirectory/name/}
MASTER_SITES=   ${MASTER_SITE_SOURCEFORGE:=project_name/}
```

Обратите внимание на слэш в конце имени подкаталога.

13.5.2. Как загружаются файлы?

На фазе *fetch* проверяется, что все дистрибутивные файлы присутствуют в локальном каталоге (`DISTDIR`, который может задать пользователь pkgsrc). Если файлов нет, они загружаются с помощью команд вида

```
${FETCH_CMD} ${FETCH_BEFORE_ARGS} ${site}${file} ${FETCH_AFTER_ARGS}
```

где `${site}` поочерёдно принимает несколько значений: сначала пробуется `MASTER_SITE_OVERRIDE`, затем сайты, указанные в `SITES.file`, если он определён, иначе — `MASTER_SITES` или `PATCH_SITES`, в зависимости от того, что применяется, и, наконец, значение

MASTER_SITE_BACKUP. Порядок всех, кроме первого и последнего, пользователь может при желании отсортировать, задав MASTER_SORT_RANDOM и MASTER_SORT_AWK или MASTER_SORT_REGEX.

Конкретные используемые команда и аргументы зависят от параметра FETCH_USING. Пример выше приведён для FETCH_USING=custom.

Зеркало дистрибутивных файлов, поддерживаемое NetBSD Foundation, использует цель *mirror-distfiles* для зеркалирования дистрибутивных файлов, если они свободно распространяются. Пакеты, задающие NO_SRC_ON_FTP (обычно равным «${RESTRICTED}»), не будут иметь своих дистрибутивных файлов на зеркале.

13.6. Фаза *checksum*

После загрузки дистрибутивных файлов вычисляется их контрольная сумма, которая сравнивается с контрольными суммами, хранящимися в файле distinfo. Если контрольные суммы не совпадают, сборка прерывается. Это гарантирует, что для сборки используется тот же дистрибутивный файл и что он не был изменён, например, злоумышленником, намеренно изменёнными дистрибутивными файлами на основном сайте распространения или сетевыми сбоями.

13.7. Фаза *extract*

Когда дистрибутивные файлы присутствуют в локальной системе, их нужно распаковать, так как обычно они поставляются в каком-либо сжатом архивном формате.

По умолчанию распаковываются все DISTFILES. Если нужны только некоторые из них, можно задать переменную EXTRACT_ONLY равной списку этих файлов.

Распаковка файлов обычно выполняется небольшой программой mk/extract/extract, которая уже умеет распаковывать различные архивные форматы, поэтому, скорее всего, здесь ничего менять не придётся. Но если нужно, вам помогут следующие переменные:

EXTRACT_OPTS_{BIN,LHA,PAX,RAR,TAR,ZIP,ZOO}
: Используйте эти переменные, чтобы переопределить параметры команды распаковки по умолчанию, которые определены в mk/extract/extract.

EXTRACT_USING
: Эту переменную можно задать равной bsdtar, gtar, nbtar (это значение по умолчанию), pax или абсолютному пути к команде, которой следует распаковывать архивы tar. Если pax-as-tar в NetBSD недостаточно хорош, предпочтительнее выбрать bsdtar, а не gtar.

Если программа extract не отвечает вашим потребностям, можно также переопределить переменную EXTRACT_CMD, в которой хранится команда, используемая для распаковки файлов. Эта команда выполняется в каталоге ${WRKSRC}. Во время выполнения этой команды переменная оболочки extract_file содержит абсолютный путь к файлу, который будет распакован.

А если и этого недостаточно, можно переопределить цель do-extract в Makefile пакета.

13.8. Фаза *patch*

После распаковки применяются все патчи, указанные в PATCHFILES, а также те, что находятся в подкаталоге patches пакета и в $LOCALPATCHES/$PKGPATH (например, /usr/local/patches/graphics/png). Файлы патчей, оканчивающиеся на .Z или .gz, распаковываются перед применением, а файлы, оканчивающиеся на .orig или .rej, игнорируются.

Любые особые параметры для patch(1) можно передать в PATCH_DIST_ARGS. Подробности см. в разделе 12.3, «patches/*».

По умолчанию patch(1) получает особые аргументы, чтобы завершиться с ошибкой, если ожидаемый текст из контекста патча не найден в патчируемом файле. Если это произошло, исправьте файл патча, сравнив его с фактическим текстом в файле, к которому применяется патч.

13.9. Фаза *tools*

Это описано в главе 17, *Инструменты, необходимые для сборки или запуска*.

13.10. Фаза *wrapper*

На этой фазе создаются программы-обёртки для компиляторов и компоновщиков. Для настройки обёрток можно использовать следующие переменные.

ECHO_WRAPPER_MSG
: Команда, используемая для вывода сообщений о ходе работы. По умолчанию ничего не делает. Задайте ${ECHO}, чтобы видеть сообщения о ходе работы.

WRAPPER_DEBUG
: Эту переменную можно задать равной yes (по умолчанию) или no в зависимости от того, нужна ли вам дополнительная информация в файле журнала обёртки.

WRAPPER_UPDATE_CACHE
: Эту переменную можно задать равной yes или no в зависимости от того, должна ли обёртка использовать свой кэш, что повысит скорость. Значение по умолчанию — yes, но оно принудительно меняется на no, если платформа это не поддерживает.

WRAPPER_REORDER_CMDS
: Список команд изменения порядка. Команда изменения порядка имеет вид reorder:l:*lib1*:*lib2*. Она гарантирует, что -l*lib1* стоит перед -l*lib2*.

13.11. Фаза *configure*

Большинству программ нужны сведения о заголовочных файлах, системных вызовах и библиотечных функциях, доступных на платформе, где они выполняются. Процесс определения этих сведений называется настройкой и обычно автоматизирован. В большинстве случаев вместе с дистрибутивными файлами поставляется сценарий, запуск которого приводит к созданию заголовочных файлов, makefile-файлов и т. п.

Если пакет содержит сценарий configure, его можно задействовать, задав HAS_CONFIGURE равным «yes». Если сценарий configure — это сценарий GNU autoconf, вместо этого следует задать GNU_CONFIGURE равным «yes».

На этапе do-configure выполняется примерно эквивалент следующей команды. Точное определение см. в mk/configure/configure.mk, цель do-configure-script.

```
.for dir in ${CONFIGURE_DIRS}
        cd ${WRKSRC} && cd ${dir} \
        && env ${CONFIGURE_ENV} \
            ${CONFIG_SHELL} ${CONFIGURE_SCRIPT} ${CONFIGURE_ARGS}
.endfor
```

CONFIGURE_DIRS (по умолчанию: «.») — список путей относительно WRKSRC. В каждом из этих каталогов сценарий configure запускается с окружением CONFIGURE_ENV и аргументами CONFIGURE_ARGS. Переменные CONFIGURE_ENV, CONFIGURE_SCRIPT (по умолчанию: «./configure») и CONFIGURE_ARGS может изменять пакет.

Если программа использует способ настройки, принятый в Perl (главным образом модули Perl, но не только), то есть файл с именем Makefile.PL, следует включить ../../lang/perl5/module.mk. Чтобы задать параметры для Makefile.PL, используйте переменную MAKE_PARAMS (например, MAKE_PARAMS+=foo=bar

Если программа использует для настройки Imakefile, нужные шаги можно задействовать, задав USE_IMAKE равным «yes». Если вам нужен только xmkmf, добавьте его в USE_TOOLS. Переменные в окружение xmkmf можно добавить, дописав их в переменную SCRIPTS_ENV.

Если программа использует для настройки cmake, нужные шаги можно задействовать, включив ../../devel/cmake/build.mk. Переменные в окружение cmake можно добавить, дописав их в переменную CONFIGURE_ENV, а аргументы для cmake — дописав их в переменную CMAKE_CONFIGURE_ARGS. Если аргументы нужно добавить только для определённых этапов, можно использовать переменные CMAKE_CONFIGURE_ARGS, CMAKE_BUILD_ARGS и CMAKE_INSTALL_ARGS. Переменной CONFIGURE_DIRS можно задать каталоги, в которых следует запускать CMake, относительно WRKSRC. По умолчанию это «.».

Любой пакет, использующий устаревшую USE_CMAKE=yes, следует переделать. По сути, замените её включением ../../devel/cmake/build.mk, уберите создание каталога сборки и настройки для его использования, а также проверьте и исправьте настройки, которые предполагают сборку в подкаталоге.

Если шага настройки нет вообще, задайте NO_CONFIGURE равным «yes».

13.12. Фаза *build*

Для сборки пакета выполняется примерно эквивалент следующего кода; точное определение см. в mk/build/build.mk, цель do-build.

```
.for dir in ${BUILD_DIRS}
        cd ${WRKSRC} && cd ${dir} \
        && env ${MAKE_ENV} \
            ${MAKE_PROGRAM} ${MAKE_FLAGS} ${BUILD_MAKE_FLAGS} \
                -f ${MAKE_FILE} \
                ${BUILD_TARGET}
.endfor
```

BUILD_DIRS (по умолчанию: «.») — список путей относительно WRKSRC. В каждом из этих каталогов MAKE_PROGRAM запускается с окружением MAKE_ENV и аргументами BUILD_MAKE_FLAGS. Переменные MAKE_ENV, BUILD_MAKE_FLAGS, MAKE_FILE и BUILD_TARGET может изменять пакет.

Значение MAKE_PROGRAM по умолчанию — «gmake», если USE_TOOLS содержит «gmake», иначе «make». Значение MAKE_FILE по умолчанию — «Makefile», а BUILD_TARGET по умолчанию — «all».

Если шага сборки нет вообще, задайте NO_BUILD равным «yes».

13.13. Фаза *test*

Фаза test предназначена для запуска самопроверок программы. Зависимости, нужные только для тестирования, можно указать с помощью TEST_DEPENDS.

Для пакетов, использующих make(1) для сборки, можно задать TEST_TARGET. Часто подходящим значением является «check» или «test». Если нужны более подробные шаги, можно определить цель do-test.

Чтобы запустить тесты, выполните «make test».

При сборке пакета тесты не запускаются автоматически, если только PKGSRC_RUN_TEST не задана равной «yes».

13.14. Фаза *install*

Когда этап сборки завершён, последний шаг — установить программу в общедоступные каталоги, чтобы пользователи могли обращаться к программам и файлам.

На фазе *install* выполняется примерно эквивалент следующего кода; точное определение см. в mk/install/install.mk, цель do-install. Кроме того, до и после этого кода выполняются несколько проверок согласованности устанавливаемых файлов, подробности см. в mk/check/*.mk.

```
.for dir in ${INSTALL_DIRS}
        cd ${WRKSRC} && cd ${dir} \
        && env ${INSTALL_ENV} ${MAKE_ENV} \
            ${MAKE_PROGRAM} ${MAKE_FLAGS} ${INSTALL_MAKE_FLAGS} \
                -f ${MAKE_FILE} ${INSTALL_TARGET}
.endfor
```

Значения этих переменных аналогичны переменным фазы *build*. INSTALL_DIRS по умолчанию равна BUILD_DIRS. INSTALL_TARGET по умолчанию равна «install», плюс «install.man», если определена USE_IMAKE и не определена NO_INSTALL_MANPAGES.

На фазе *install* полезны следующие переменные. Все они — варианты команды install(1) с предустановленными владельцем, группой и правами доступа. INSTALL — обычная команда install. Специализированные варианты вместе с их назначением:

INSTALL_PROGRAM_DIR
: каталоги, содержащие двоичные файлы

INSTALL_SCRIPT_DIR
: каталоги, содержащие сценарии

INSTALL_LIB_DIR
: каталоги, содержащие разделяемые и статические библиотеки

INSTALL_DATA_DIR
: каталоги, содержащие файлы данных

INSTALL_MAN_DIR
: каталоги, содержащие страницы руководства

INSTALL_GAME_DIR
: каталоги, содержащие файлы данных для игр

INSTALL_PROGRAM
: двоичные файлы, из которых можно удалить отладочные символы

INSTALL_SCRIPT
: двоичные файлы, из которых нельзя удалить отладочные символы

INSTALL_GAME

двоичные файлы игр

`INSTALL_LIB`
: разделяемые и статические библиотеки

`INSTALL_DATA`
: файлы данных

`INSTALL_GAME_DATA`
: файлы данных для игр

`INSTALL_MAN`
: страницы руководства

Есть и другие переменные:

`INSTALL_UNSTRIPPED`
: Если задана равной `yes`, не запускать strip(1) при установке двоичных файлов. Все отладочные разделы и символы, присутствующие в двоичных файлах, будут сохранены.

`INSTALLATION_DIRS`
: Список каталогов относительно `PREFIX`, которые создаются pkgsrc в начале фазы *install*. Предполагается, что пакет сам создаёт все нужные каталоги перед установкой в них файлов, а все остальные каталоги перечисляет здесь.

В редких случаях, когда пакет не должен ничего устанавливать, задайте `NO_INSTALL` равным «yes». Чаще всего это относится к пакетам из категории `regress`.

13.15. Фаза *package*

Когда этап установки завершён, можно собрать двоичный пакет из установленных файлов. Такие двоичные пакеты можно использовать для быстрой установки без предварительной компиляции, например с помощью **make bin-install** или **pkg_add**.

По умолчанию двоичные пакеты создаются в `${PACKAGES}/All`, а символические ссылки — в `${PACKAGES}/`*`category`*, по одной для каждой категории из переменной `CATEGORIES`. `PACKAGES` по умолчанию равна `pkgsrc/packages`.

13.16. Очистка

Закончив работу с пакетом, вы можете очистить рабочий каталог, выполнив **make clean**. Если вы хотите очистить также рабочие каталоги всех зависимостей, используйте **make clean-depends**.

13.17. Другие полезные цели

pre/post-*
: Для любой из основных целей, описанных в предыдущем разделе (configure, build, install и т. д.), существуют две вспомогательные цели, у которых к имени основной цели добавляется префикс «„pre-“» или «„post-“». Эти цели вызываются до и после вызова основной цели, что позволяет, например, выполнить из Makefile пакета дополнительные шаги настройки или установки, которые не предусмотрены сценарием configure или целью install программы.

Около 5% пакетов pkgsrc определяют собственную цель post-extract, ещё 5% определяют pre-configure, а 10% — post-install. Остальные цели pre/post-* определяются ещё реже.

do-*

Если какая-либо из основных целей работает неправильно и нет переменной, позволяющей это исправить, её можно переопределить с помощью цели do-*. (Учтите, что переопределять саму цель вместо цели do-* — плохая идея: цели pre-* и post-* после этого перестанут вызываться и т. д.)

Около 15% пакетов pkgsrc переопределяют стандартную цель do-install, остальные цели do-* переопределяются ещё реже.

reinstall

Если вы выполнили **make install** и заметили, что какой-то файл установлен неправильно, вы можете повторить установку с помощью этой цели — она проигнорирует флаг «„already installed“».

Это значение по умолчанию для `DEPENDS_TARGET`, за исключением случаев **make update** и **make package**, где значения по умолчанию — «„package“» и «„update“» соответственно.

deinstall

Эта цель запускает pkg_delete(1) в текущем каталоге, фактически удаляя пакет. Следующие переменные позволяют настроить её поведение:

`PKG_VERBOSE`

Добавляет «-v» к команде pkg_delete(1).

`DEINSTALLDEPENDS`

Удаляет все пакеты, которые требуют (зависят от) указанный пакет. Это можно использовать для удаления пакетов, которые могли быть установлены вместе с данным пакетом: например, если выполнить **make deinstall DEINSTALLDEPENDS=1** в `pkgsrc/x11/kde`, скорее всего будет удалён весь KDE. Работает путём добавления «„-R“» в командную строку pkg_delete(1).

bin-install

Устанавливает двоичный пакет с локального диска и по FTP из списка сайтов (см. переменную `BINPKG_SITES`) и выполняет **make package**, если двоичный пакет нигде не найден. Аргументы, передаваемые **pkg_add**, можно задать с помощью `BIN_INSTALL_FLAGS`, например для подробного вывода и т. п.

install-clean

Эта цель удаляет файлы состояния для фазы «install» и последующих фаз, чтобы цель «install» можно было вызвать повторно. Это можно использовать после правки PLIST, чтобы установить пакет без его пересборки.

build-clean

Эта цель удаляет файлы состояния для фазы «build» и последующих фаз, чтобы цель «build» можно было вызвать повторно.

update

Эта цель обновляет текущий пакет до последней версии. Сначала пакет и все зависящие от него пакеты удаляются, затем текущие версии соответствующих пакетов собираются и устанавливаются. Это похоже на то, как если бы вы вручную записали, какие пакеты

установлены, а затем выполнили для них серию **make deinstall** и **make install** (или чего-то иного, в зависимости от значения `UPDATE_TARGET`).

Цель «„update“» позволяет возобновить обновление пакетов, если предыдущий **make update** был по какой-то причине прерван. Однако в этом случае убедитесь, что вы не вызываете **make clean** и не удаляете иным способом список зависимых пакетов в `WRKDIR`. В противном случае вы потеряете возможность автоматически обновить текущий пакет вместе с установленными зависимыми пакетами.

Возобновление прерванного **make update** будет работать только до тех пор, пока дерево пакетов остаётся неизменным. Если исходный код одного из обновляемых пакетов изменился, возобновление **make update** почти наверняка завершится неудачей!

Следующие переменные можно задать в командной строке или в `mk.conf`, чтобы изменить поведение **make update**:

`UPDATE_TARGET`

Цель установки, рекурсивно используемая для обновляемого пакета и зависимых пакетов. По умолчанию — `DEPENDS_TARGET`, если она задана, иначе «„install“» для **make update**. Другие подходящие цели — «„package“» или «„bin-install“». Не задавайте здесь «„update“», иначе вы застрянете в бесконечном цикле!

`NOCLEAN`

Не выполнять очистку после обновления. Полезно, если вы хотите оставить рабочие исходные тексты обновлённых пакетов для просмотра или других целей. Не забудьте в конце концов очистить дерево исходных текстов (см. цель «„clean-update“» ниже), иначе у вас могут возникнуть проблемы со старым исходным кодом, который остался лежать при следующем **make** или **make update**.

`REINSTALL`

Удалять каждый пакет перед установкой (для `DEPENDS_TARGET`). Это может понадобиться, если цель «„clean-update“» (см. ниже) была вызвана после прерывания работающего **make update**.

`DEPENDS_TARGET`

Позволяет отключить рекурсию и жёстко задать цель для пакетов. По умолчанию для цели update используется «„update“», что обеспечивает рекурсивное обновление пакетов-предварительных условий. Задавайте `DEPENDS_TARGET`, только если хотите отключить рекурсивные обновления. Вместо этого используйте `UPDATE_TARGET`, чтобы просто задать конкретную цель для каждого пакета, устанавливаемого во время **make update** (см. выше).

clean-update

Очищает дерево исходных текстов для всех пакетов, которые были бы обновлены, если бы **make update** был вызван из текущего каталога. Эту цель не следует использовать, если текущий пакет (или какие-либо из зависящих от него пакетов) уже удалены (например, после вызова **make update**), иначе вы можете потерять некоторые пакеты, которые собирались обновить. Практическое правило: используйте эту цель только *до* первого запуска **make update** и только если дерево пакетов «грязное» (например, если вы использовали `NOCLEAN`).

Если вы не уверены, что дерево чистое, выполните **make clean** в корне дерева или используйте следующую последовательность команд из каталога пакета, который хотите обновить (*до* первого запуска **make update**, иначе вы потеряете все пакеты, которые хотели обновить!):

```
# make clean-update
# make clean CLEANDEPENDS=YES
```

```
# make update
```

Следующие переменные можно задать в командной строке или в `mk.conf`, чтобы изменить поведение **make clean-update**:

`CLEAR_DIRLIST`
: После **make clean** не восстанавливать список каталогов для обновления этого пакета. Используйте это, только если **make update** успешно установил все пакеты, которые вы хотели обновить. Обычно это делается автоматически при **make update**, но могло быть отключено переменной `NOCLEAN` (см. выше).

replace
: Обновляет установку текущего пакета. Отличается от update тем, что не заменяет зависимые пакеты. Чтобы эта цель работала, вам нужно установить `pkgtools/pkg_tarup`.

Будьте осторожны при использовании этой цели! Нет никаких гарантий, что зависимые пакеты продолжат работать; в частности, они почти наверняка сломаются, если вы выполните **make replace** для библиотечного пакета, у которого основная версия разделяемой библиотеки изменилась между установленной у вас версией и новой. По этой причине данная цель официально не поддерживается и рекомендуется только опытным пользователям.

info
: Эта цель вызывает pkg_info(1) для текущего пакета. С её помощью можно узнать, какая версия пакета установлена.

index
: Это команда верхнего уровня, то есть её следует использовать в каталоге `pkgsrc`. Она создаёт базу данных всех пакетов в локальном дереве pkgsrc, включая зависимости, комментарий, сопровождающего и некоторые другие полезные сведения. Отдельные записи создаются путём выполнения **make describe** в каталогах пакетов. Этот индексный файл сохраняется как `pkgsrc/INDEX`. Его можно вывести в подробном формате, выполнив **make print-index**. Искать в нём можно с помощью **make search key=*something***. Можно извлечь список всех пакетов, зависящих от заданного, выполнив **make show-deps PKG=*somepackage***.

Выполнение этой команды занимает очень много времени — несколько часов даже на быстрых машинах!

readme
: Эта цель создаёт файл `index.html`, который можно просмотреть в браузере, например `www/firefox` или `www/links`. Созданные файлы содержат ссылки на пакеты, находящиеся в каталоге `PACKAGES` на локальном хосте. Созданные файлы можно заставить ссылаться на URL-адреса на основе `FTP_PKG_URL_HOST` и `FTP_PKG_URL_DIR`. Например, чтобы создать файлы `index.html`, указывающие на двоичные пакеты на локальной машине в каталоге `/usr/packages`, задайте `FTP_PKG_URL_HOST=file://localhost` и `FTP_PKG_URL_DIR=/usr/packages`. Каталог `${PACKAGES}` и его подкаталоги будут просканированы на предмет всех двоичных пакетов.

Цель можно запускать в корне дерева или в каталогах категорий; в последнем случае она рекурсивно обходит подкаталоги.

readme-all
: Это команда верхнего уровня, выполняйте её в `pkgsrc`. С помощью этой цели создаётся файл `README-all.html`, который содержит список всех пакетов, доступных в коллекции пакетов NetBSD, вместе с категорией, к которой они относятся, и кратким описанием. Этот файл

собирается из файлов `pkgsrc/*/index.html`, поэтому обязательно выполните это *после* **make readme**.

cdrom-readme

Это почти то же самое, что и цель «„readme“» (см. выше), но она предназначена для использования при создании дерева pkgsrc, записываемого на CD-ROM. Эта цель также создаёт файлы `index.html`, и можно сделать так, чтобы они ссылались на URL-адреса на основе `CDROM_PKG_URL_HOST` и `CDROM_PKG_URL_DIR`.

show-distfiles

Эта цель показывает, какие дистрибутивные файлы и файлы патчей нужны для сборки пакета (`ALLFILES`, которая содержит все `DISTFILES` и `PATCHFILES`, но не `patches/*`).

show-downlevel

Эта цель ничего не показывает, если пакет не установлен. Если какая-либо версия этого пакета установлена, но это не та версия, которая предоставляется в данной версии pkgsrc, выводится предупреждение. Эту цель можно использовать, чтобы узнать, какие из установленных пакетов устарели, — тогда старые версии можно удалить, а текущие добавить.

show-pkgsrc-dir

Эта цель показывает каталог в иерархии pkgsrc, из которого пакет можно собрать и установить. Он может отличаться от каталога, из которого пакет был установлен. Эта цель предназначена для тех, кто хочет обновить много пакетов на одном хосте, и её можно вызвать из Makefile верхнего уровня pkgsrc с помощью цели «„show-host-specific-pkgs“».

show-installed-depends

Эта цель показывает, какие установленные пакеты соответствуют `DEPENDS` текущего пакета. Полезно, если устаревшие зависимости вызывают проблемы при сборке.

print-build-depends-list

Эта цель показывает список пакетов, от которых зависит текущий пакет при сборке.

print-run-depends-list

Эта цель показывает список пакетов, от которых текущий пакет зависит при работе.

check-shlibs

После установки пакета проверяет все его двоичные файлы и (на платформах ELF) разделяемые библиотеки на предмет того, находят ли они нужные им разделяемые библиотеки. Выполняется по умолчанию, если `PKG_DEVELOPER` задана в `mk.conf`.

print-PLIST

После «„make install“» нового или обновлённого пакета выводит попытку сгенерировать новый `PLIST` на основе **find -newer work/.extract_done**. Делается попытка учесть разделяемые библиотеки и т. п., но результат *настоятельно* рекомендуется просмотреть, прежде чем помещать его в `PLIST`. При обновлениях полезно сравнить вывод этой команды с уже существующим файлом `PLIST`.

Если пакет устанавливает файлы через tar(1) или другими способами, не обновляющими время доступа к файлам, обязательно добавьте эти файлы в `PLIST` вручную, поскольку команда «„find -newer“», используемая этой целью, их не обнаружит!

Дополнительные сведения об этой цели см. в разделе 19.3, «Настройка вывода **make print-PLIST**».

rebuild

Если вы выполнили **make build** и заметили, что требуются дополнительные изменения исходных текстов, вы можете повторить сборку с помощью этой цели — она проигнорирует флаг «„already built“». Эта цель полезна при работе с патчами.

repackage

Если вы выполнили **make package** и заметили, что требуются дополнительные изменения исходных текстов, вы можете повторить упаковку с помощью этой цели — она проигнорирует флаг «„already packaged“». Эта цель полезна при работе с патчами, файлами PLIST* и т. п.

retest

Если вы выполнили **make test** и заметили, что требуются дополнительные изменения исходных текстов, вы можете повторить тестирование с помощью этой цели — она проигнорирует флаг «„already tested“». Эта цель полезна при работе с патчами.

Глава 14. Создание нового пакета pkgsrc с нуля

Когда вы находите пакет, которого ещё нет в pkgsrc, у вас, скорее всего, есть URL-адрес, откуда можно скачать исходный код. Начиная с этого URL-адреса, создание пакета включает всего несколько шагов.

1. В вашем `mk.conf` задайте `PKG_DEVELOPER=yes`, чтобы включить базовые проверки качества.
2. Установите пакет `meta-pkgs/pkg_developer`, который, среди прочего, установит утилиты **url2pkg**, **pkglint**, **pkgvi** и **mkpatches**:

   ```
   $ cd /usr/pkgsrc
   $ (cd meta-pkgs/pkg_developer && bmake update)
   ```

3. Выберите один из каталогов верхнего уровня как категорию, в которую вы хотите поместить свой пакет. Вы также можете создать собственный каталог (возможно, с именем `local`). Перейдите в этот каталог категории:

   ```
   $ cd category
   ```

4. Запустите программу **url2pkg**, передав ей в качестве аргумента URL дистрибутивного файла (в большинстве случаев это файл `.tar.gz`). Она скачает дистрибутивный файл и создаст необходимые файлы пакета на основе содержимого этого файла:

   ```
   $ url2pkg https://www.example.org/packages/package-1.0.tar.gz
   ```

5. Изучите извлечённые файлы, чтобы определить зависимости вашего пакета. В идеале они указаны в каком-нибудь файле `README`, но бывает по-разному. Для каждой из этих зависимостей найдите, где она находится в pkgsrc, и если в этом каталоге есть файл `buildlink3.mk`, добавьте в `Makefile` вашего пакета строку, которая подключает этот файл, непосредственно перед последней строкой. Если файла `buildlink3.mk` нет, его нужно сначала создать. Файл `buildlink3.mk` гарантирует наличие включаемых файлов и библиотек пакета.

   Если вам нужны только двоичные файлы из зависимого пакета, добавьте в Makefile строку `DEPENDS`, в которой указывается версия зависимости и место, где её можно найти в pkgsrc. Эту строку следует разместить в третьем абзаце. Если зависимость нужна только для сборки пакета, но не при его использовании, используйте `TOOL_DEPENDS` или `BUILD_DEPENDS` вместо `DEPENDS`. Разница между `TOOL_DEPENDS` и `BUILD_DEPENDS` проявляется при кросс-компиляции: `TOOL_DEPENDS` — это *нативные* пакеты, то есть пакеты для платформы, на которой выполняется сборка; `BUILD_DEPENDS` — это *целевые* пакеты, то есть пакеты для платформы, для которой выполняется сборка. Есть также `TEST_DEPENDS`, которая задаёт зависимость, используемую только для тестирования собранного пакета с помощью набора тестов, входящего в проект, на нативной платформе. Тогда ваш пакет может выглядеть так:

   ```
   [...]

   TOOL_DEPENDS+=  libxslt-[0-9]*:../../textproc/libxslt
   DEPENDS+=       screen-[0-9]*:../../misc/screen
   DEPENDS+=       screen>=4.0:../../misc/screen
   ```

```
[...]

.include "../../category/package/buildlink3.mk"
.include "../../devel/glib2/buildlink3.mk"
.include "../../mk/bsd.pkg.mk"
```

6. Запустите **pkglint**, чтобы увидеть, что ещё нужно сделать, чтобы ваш пакет стал «„хорошим“». Если вы не понимаете, о чём говорят предупреждения pkglint, попробуйте **pkglint --explain** или **pkglint -e** — они выводят дополнительные пояснения.
7. Во многих случаях пакет ещё не готов к сборке. Инструкции для наиболее распространённых случаев вы найдёте в следующем разделе — разделе 14.1, «Распространённые типы пакетов». Выполнив приведённые там инструкции, вы, будем надеяться, сможете продолжить здесь.
8. Запустите **bmake clean**, чтобы очистить рабочий каталог от извлечённых файлов. Помимо этих файлов, в рабочем каталоге сохранено множество кэшированных файлов и другой системной информации, которая могла устареть после того, как вы отредактировали `Makefile`.
9. Теперь запустите **bmake**, чтобы собрать пакет. О различных проблемах, которые могут возникнуть на этом этапе, см. главу 21, *Как заставить пакет работать*.

   Если извлечённые из пакета файлы нужно исправить, выполните несколько раз эти команды:

```
$ bmake
$ pkgvi ${WRKSRC}/some/file/that/does/not/compile
$ mkpatches
$ bmake mps
$ bmake clean
```

10. Когда пакет успешно собирается, следующий шаг — установить его. Запустите **bmake install** и надейтесь, что всё заработает.
11. До сих пор файл `PLIST`, содержащий список файлов, устанавливаемых пакетом, почти пуст. Выполните **bmake print-PLIST >PLIST**, чтобы сгенерировать предположительно правильный список. Проверьте файл в предпочитаемом вами текстовом редакторе и посмотрите, выглядит ли список файлов правдоподобно.
12. Снова запустите **pkglint**, чтобы проверить, не содержит ли сгенерированный `PLIST` мусора.
13. Когда вы выполнили **bmake install**, пакет был зарегистрирован в базе данных установленных файлов, но с пустым списком файлов. Чтобы исправить это, снова выполните **bmake deinstall** и **bmake install**. Теперь пакет зарегистрирован со списком файлов из `PLIST`.
14. Выполните **bmake clean update**, чтобы повторить всё вышеописанное за один шаг и убедиться, что PLIST правильный, а весь пакет создан как задумано.
15. Запустите **pkglint**, чтобы проверить, осталось ли что-нибудь сделать.
16. Закоммитьте пакет в pkgsrc-wip или в основной pkgsrc; см. главу 23, *Отправка и коммит*.

14.1. Распространённые типы пакетов

14.1.1. Модули и программы на Python

Пакеты с модулями и программами на Python легко создавать с помощью набора предопределённых переменных.

Если какие-то версии Python не поддерживаются программой, задайте в переменной `PYTHON_VERSIONS_INCOMPATIBLE` неподдерживаемые версии Python, например

```
PYTHON_VERSIONS_INCOMPATIBLE=       27
```

Если упаковываемое программное обеспечение является модулем Python, подключите `../../lang/python/wheel.mk`. Согласно PEP-518, минимальное и заданное по умолчанию требование для создания файлов .whl — `devel/py-setuptools`; однако проекты могут использовать и различные другие инструменты. Поэтому подключение `wheel.mk` не означает, что эти значения по умолчанию определены как `TOOL_DEPENDS`. Всё, что конкретно требуется проекту в качестве инструментов упаковки, должно быть указано в Makefile пакета.

Каталог пакета должен называться «„py-software“», а в `PKGNAME` следует задать «„${PYPKGPREFIX}-${DISTNAME}“», например

```
DISTNAME=   foopymodule-1.2.10
PKGNAME=    ${PYPKGPREFIX}-${DISTNAME}
```

Для программного обеспечения в PyPi имя должно совпадать с тем, что PyPi указывает для «pip install software».

Если это приложение, подключите «`../../lang/python/application.mk`». Чтобы правильно задать путь к интерпретатору Python, используйте переменную `REPLACE_PYTHON` и задайте в ней список файлов (пути относительно `WRKSRC`), которые нужно исправить. Например:

```
REPLACE_PYTHON=   *.py
```

14.1.2. Пакеты R

Простые пакеты R из CRAN обрабатываются автоматически программой **R2pkg**, которая доступна в `pkgtools/R2pkg`. Отдельные пакеты (и, при желании, их зависимости) можно создавать и обновлять. Пакеты R обычно имеют одинаковую структуру, и большая часть нужной информации содержится в файле `DESCRIPTION`, входящем в каждый пакет R на CRAN. Поэтому **R2pkg** скачивает эту информацию и создаёт или обновляет пакет в каноническом виде. Полученный пакет следует проверить на корректность.

14.1.3. Пакеты TeXlive

Пакеты TeXlive из CTAN обрабатываются автоматически программой **texlive2pkg**, которая доступна в `pkgtools/texlive2pkg`.

Если имя пакета TeXlive неизвестно, может помочь поиск на CTAN. Поле «„Contained in“» на странице пакета обычно указывает базовое имя файла пакета в архиве TeXlive.

Если имя пакета TeXlive известно, скачайте файлы из архива TeXlive. Для пакета `foo` вам понадобится скачать `foo.tar.xz`. У большинства пакетов TeXlive есть также связанные пакеты документации, поэтому скачайте заодно `foo.doc.tar.xz`. Эти файлы следует поместить в соответствующий каталог категории, которым часто, но не всегда, является `print`. Затем выполните следующую команду в каталоге категории.

```
texlive2pkg foo.tar.xz foo.doc.tar.xz
```

Будут созданы два пакета: `tex-foo` и `tex-foo-doc`. Обязательно проверьте, что оба пакета корректны.

Наконец, CTAN в настоящее время не включает сведения о версии в имена файлов пакетов и периодически изменяет их содержимое при обновлениях. Поэтому pkgsrc избегает скачивания дистрибутивных файлов напрямую с CTAN и вместо этого полагается на архивы pkgsrc. Для каждого нового или обновлённого пакета TeXlive, например основного и соответствующего ему пакета документации, загрузите дистрибутивные файлы следующей командой в каждом каталоге пакета.

```
make upload-distfiles
```

14.2. Примеры

14.2.1. Как пакет www/nvu попал в pkgsrc

14.2.1.1. Исходный пакет

Заглянув в файл `pkgsrc/doc/TODO`, я увидел, что пакет «„nvu“» ещё не импортирован в pkgsrc. Поскольку в описании сказано, что он связан с вебом, очевидным выбором категории было «„www“».

```
$ mkdir www/nvu
$ cd www/nvu
```

На веб-сайте сказано, что исходные тексты доступны в виде tar-файла, поэтому я передал этот URL программе **url2pkg**:

```
$ url2pkg http://cvs.nvu.com/download/nvu-1.0-sources.tar.bz2
```

Открылся мой редактор, и я добавил строку `PKGNAME` ниже строки `DISTNAME`, поскольку в имени пакета не должно быть слова «„sources“». Я также заполнил поля `MAINTAINER`, `HOMEPAGE` и `COMMENT`. После этого `Makefile` пакета выглядел так:

```
# $NetBSD $
#

DISTNAME=       nvu-1.0-sources
PKGNAME=        nvu-1.0
CATEGORIES=     www
MASTER_SITES=   http://cvs.nvu.com/download/
EXTRACT_SUFX=   .tar.bz2

MAINTAINER=     rillig@NetBSD.org
HOMEPAGE=       http://cvs.nvu.com/
COMMENT=        Web Authoring System

# url2pkg-marker (please do not remove this line.)
.include "../../mk/bsd.pkg.mk"
```

В первой строке вывода выше между NetBSD и $ добавлен искусственный пробел — это обходной путь, предотвращающий подстановку CVS в имя файла руководства.

Затем я закрыл редактор и наблюдал, как pkgsrc скачивает большой архив исходных текстов:

```
url2pkg> Running "make makesum" ...
=> Required installed package digest>=20010302: digest-20060826 found
=> Fetching nvu-1.0-sources.tar.bz2
Requesting http://cvs.nvu.com/download/nvu-1.0-sources.tar.bz2
100% |************************************| 28992 KB  150.77 KB/s00:00 ETA
29687976 bytes retrieved in 03:12 (150.77 KB/s)
url2pkg> Running "make extract" ...
=> Required installed package digest>=20010302: digest-20060826 found
=> Checksum SHA1 OK for nvu-1.0-sources.tar.bz2
=> Checksum RMD160 OK for nvu-1.0-sources.tar.bz2
work.bacc -> /tmp/roland/pkgsrc/www/nvu/work.bacc
===> Installing dependencies for nvu-1.0
===> Overriding tools for nvu-1.0
===> Extracting for nvu-1.0
url2pkg> Adjusting the Makefile.

Remember to correct CATEGORIES, HOMEPAGE, COMMENT, and DESCR when you're done!

Good luck! (See pkgsrc/doc/pkgsrc.txt for some more help :-)
```

14.2.1.2. Исправление всевозможных проблем, чтобы пакет работал

Теперь, когда пакет извлечён, посмотрим, что внутри. В пакете есть `README.txt`, но в нём говорится только что-то о mozilla, так что он, вероятно, бесполезен для выяснения зависимостей этого пакета. Но поскольку в пакете есть сценарий configure от GNU, будем надеяться, что он сообщит обо всём, что ему нужно.

```
$ bmake
=> Required installed package digest>=20010302: digest-20060826 found
=> Checksum SHA1 OK for nvu-1.0-sources.tar.bz2
=> Checksum RMD160 OK for nvu-1.0-sources.tar.bz2
===> Patching for nvu-1.0
===> Creating toolchain wrappers for nvu-1.0
===> Configuring for nvu-1.0
[...]
configure: error: Perl 5.004 or higher is required.
[...]
WARNING: Please add USE_TOOLS+=perl to the package Makefile.
[...]
```

Это сработало довольно хорошо. Поэтому я открыл Makefile пакета в своём редакторе и, поскольку в нём уже есть строка `USE_TOOLS`, просто дописал в неё «„perl“». Поскольку зависимости пакета теперь изменились, а обёртка perl устанавливается автоматически на фазе «„tools“», мне нужно собрать пакет с нуля.

```
$ bmake clean
===> Cleaning for nvu-1.0
$ bmake
[...]
*** /tmp/roland/pkgsrc/www/nvu/work.bacc/.tools/bin/make is not \
GNU Make.  You will not be able to build Mozilla without GNU Make.
[...]
```

Итак, я добавил «„gmake“» в строку `USE_TOOLS` и попробовал снова (с нуля).

```
[...]
checking for GTK - version >= 1.2.0... no
```

```
*** Could not run GTK test program, checking why...
[...]
```

Теперь о других зависимостях. Первый вопрос: где в pkgsrc спрятан пакет GTK?

```
$ echo ../../*/gtk*
[many packages ...]
$ echo ../../*/gtk
../../x11/gtk
$ echo ../../*/gtk2
../../x11/gtk2
$ echo ../../*/gtk2/bui*
../../x11/gtk2/buildlink3.mk
```

Первая попытка была явно слишком широкой. Вторая дала ровно один результат, что очень хорошо. Но с пакетами GNOME есть одна ловушка. До выхода GNOME 2 в pkgsrc уже было много пакетов GNOME 1. Чтобы можно было продолжать пользоваться этими пакетами, пакеты GNOME 2 были импортированы как отдельные пакеты, и к их именам обычно добавляется «„2“». Поэтому я проверил, так ли обстоит дело здесь, и оказалось, что так.

Поскольку у пакета GTK2 есть файл `buildlink3.mk`, добавить зависимость очень легко. Я просто вставил строку `.include` перед последней строкой `Makefile` пакета, и теперь он выглядит так:

```
[...]
.include "../../x11/gtk2/buildlink3.mk"
.include "../../mk/bsd.pkg.mk
```

После очередного **bmake clean && bmake** ответ был таким:

```
[...]
checking for gtk-config... /home/roland/pkg/bin/gtk-config
checking for GTK - version >= 1.2.0... no
*** Could not run GTK test program, checking why...
*** The test program failed to compile or link. See the file config.log for the
*** exact error that occured. This usually means GTK was incorrectly installed
*** or that you have moved GTK since it was installed. In the latter case, you
*** may want to edit the gtk-config script: /home/roland/pkg/bin/gtk-config
configure: error: Test for GTK failed.
[...]
```

В этом конкретном случае предположение, что «„каждый пакет предпочитает GNOME 2“», оказалось неверным. Первая из строк выше сообщила мне, что этот пакет действительно хотел иметь версию GTK для GNOME 1. Если бы пакет искал GTK2, он искал бы **pkg-config** вместо **gtk-config**. Поэтому я заменил `x11/gtk2` на `x11/gtk` в `Makefile` пакета и попробовал снова.

```
[...]
cc -o xpidl.o -c -DOSTYPE=\"NetBSD3\" -DOSARCH=\"NetBSD\"  [...]
In file included from xpidl.c:42:
xpidl.h:53:24: libIDL/IDL.h: No such file or directory
In file included from xpidl.c:42:
xpidl.h:132: error: parse error before "IDL_ns"
[...]
```

Пакет по-прежнему не находит все свои зависимости. Теперь вопрос: какой пакет предоставляет заголовочный файл `libIDL/IDL.h`?

```
$ echo ../../*/*idl*
../../devel/py-idle ../../wip/idled ../../x11/acidlaunch
$ echo ../../*/*IDL*
../../net/libIDL
```

Возьмём вариант из второй попытки. Итак, я подключил файл `../../net/libIDL/buildlink3.mk` и попробовал снова. Но ошибка не изменилась. Покопавшись в части кода, я пришёл к выводу, что процесс сборки пакета сломан и никогда не мог работать, но поскольку дерево исходных текстов Mozilla довольно велико, я не стал его исправлять. Поэтому я добавил в `Makefile` пакета следующее и попробовал снова:

```
CPPFLAGS+=              -I${BUILDLINK_PREFIX.libIDL}/include/libIDL-2.0
BUILDLINK_TRANSFORM+=   l:IDL:IDL-2
```

Последняя строка нужна потому, что пакет ожидает библиотеку `libIDL.so`, а доступна только `libIDL-2.so`. Поэтому я велел обёртке компилятора переписывать это на лету.

Следующая проблема была связана с недавним изменением интерфейса FreeType. Я посмотрел в `www/seamonkey`, какие файлы патчей относятся к этой проблеме, и скопировал их в каталог `patches`. Затем я повторил попытку, исправил патчи, чтобы они применялись чисто, и попробовал снова. На этот раз всё заработало.

14.2.1.3. Установка пакета

```
$ bmake CHECK_FILES=no install
[...]
$ bmake print-PLIST >PLIST
$ bmake deinstall
$ bmake install
```

Глава 15. Программирование в файлах `Makefile`

Pkgsrc состоит из множества фрагментов `Makefile`, каждый из которых образует чётко определённую часть системы pkgsrc. Использование системы make(1) в качестве языка программирования для такой большой системы, как pkgsrc, требует определённой дисциплины, чтобы код оставался правильным и понятным.

Основные составляющие программирования на `Makefile` — переменные и команды оболочки. Среди этих команд оболочки могут быть даже более сложные, например программы awk(1). Чтобы каждая команда оболочки выполнялась как задумано, необходимо правильно заключать в кавычки все переменные при их использовании.

В этой главе описаны некоторые шаблоны, которые довольно часто встречаются в `Makefile`, включая связанные с ними подводные камни.

15.1. Предостережения

- Когда вы создаёте файл как цель правила, всегда сначала записывайте данные во временный файл, а затем переименовывайте этот файл. Иначе ошибка может возникнуть в середине создания файла, и когда пользователь запустит make(1) во второй раз, файл уже существует и не будет правильно пересоздан. Пример:

```
wrong:
        @echo "line 1" > ${.TARGET}
        @echo "line 2" >> ${.TARGET}
        @false

correct:
        @echo "line 1" > ${.TARGET}.tmp
        @echo "line 2" >> ${.TARGET}.tmp
        @false
        @mv ${.TARGET}.tmp ${.TARGET}
```

  Если вы запустите **make wrong** дважды, файл `wrong` будет существовать, хотя в первом запуске было сообщение об ошибке. С другой стороны, запуск **make correct** дважды даёт сообщение об ошибке, как и ожидалось.

  Вы можете помнить, что make(1) иногда удаляет `${.TARGET}` в случае ошибки, но это происходит только тогда, когда она прерывается, например при нажатии `Ctrl+C`. Этого *не* происходит, когда одна из команд завершается неудачно (как false(1) выше).

15.2. Переменные `Makefile`

Переменные `Makefile` содержат строки, которые можно обрабатывать с помощью пяти операторов `=`, `+=`, `?=`, `:=` и `!=`, описанных на странице руководства make(1).

Когда значение переменной разбирается из `Makefile`, символ решётки `#` и символ обратной косой черты `\` обрабатываются особым образом. Если обратная косая черта является последним символом в строке, она удаляется из строки, и строка продолжается следующей строкой файла.

Символ `#` начинает комментарий, который продолжается до конца строки. Чтобы получить настоящий символ `#`, например в URL, напишите вместо него `\#`.

Вычисление переменных происходит либо сразу, либо отложенно. Оно происходит сразу, когда переменная встречается в правой части оператора `:=` или `!=`, в условии `.if` или в цикле `.for`. В остальных случаях оно вычисляется отложенно.

Некоторые модификаторы разбивают строку на слова и затем работают со словами, другие работают со строкой целиком. Когда строка разбивается на слова, двойные и одинарные кавычки интерпретируются как разделители, так же как в sh(1).

15.2.1. Соглашения об именовании

- Все имена переменных, начинающиеся с символа подчёркивания, зарезервированы для использования инфраструктурой pkgsrc. Пакеты не должны их использовать.
- В циклах **.for** для переменных итерации следует использовать имена в нижнем регистре.
- Все переменные-списки должны иметь имя во множественном числе, например `PKG_OPTIONS` или `DISTFILES`.

15.3. Фрагменты кода

15.3.1. Добавление элементов в список

При добавлении в список строки, которая может содержать пробелы или кавычки (пример 1), её необходимо заключить в кавычки с помощью модификатора `:Q`.

При добавлении в список другого списка (пример 2) его не нужно заключать в кавычки, так как его элементы уже в кавычках.

```
STRING=         foo * bar `date`
LIST=           # empty
ANOTHER_LIST=   a=b c=d

LIST+=          ${STRING:Q}       # 1
LIST+=          ${ANOTHER_LIST}   # 2
```

15.3.2. Вывод строки в точности как есть

Вывод строки, содержащей специальные символы, требует особой работы.

```
STRING=         foo bar <    > * `date` $$HOME ' "
EXAMPLE_ENV=    string=${STRING:Q} x=multiple\ quoted\ words

all:
        echo ${STRING}                  # 1
        echo ${STRING:Q}                # 2
        printf '%s\n' ${STRING:Q}''     # 3
        env ${EXAMPLE_ENV} sh -c 'echo "$$string"; echo "$$x"'   # 4
```

Пример 1 приводит к синтаксической ошибке в оболочке, так как символы просто копируются.

Пример 2 заключает строку в кавычки так, что оболочка интерпретирует её правильно. Но команда echo может дополнительно интерпретировать строки с ведущим дефисом или содержащие обратные косые черты.

Пример 3 может обрабатывать произвольные строки, поскольку printf(1) интерпретирует только строку формата, но не следующий аргумент. Завершающие одинарные кавычки обрабатывают случай, когда строка пуста. В этом случае модификатор :Q тоже дал бы пустую строку, которую затем пропустила бы оболочка. Для printf(1) это не имеет значения, но другие программы могут это учитывать.

В примере 4 переменную `EXAMPLE_ENV` не нужно заключать в кавычки, поскольку кавычки уже были расставлены при добавлении элементов в список.

15.3.3. Передача `CFLAGS` в сценарии configure от GNU

При передаче `CFLAGS` или аналогичных переменных в сценарий configure в стиле GNU (особенно в те, которые вызывают другие сценарии configure) они не должны содержать ведущих и завершающих пробелов, иначе сценарий configure запутается. Чтобы удалить ведущие и завершающие пробелы, используйте модификатор `:M`, как в следующем примере:

```
CPPFLAGS=               # empty
CPPFLAGS+=              -Wundef -DPREFIX=\"${PREFIX}\"
CPPFLAGS+=              ${MY_CPPFLAGS}

CONFIGURE_ARGS+=        CPPFLAGS=${CPPFLAGS:M*:Q}

all:
        echo x${CPPFLAGS:Q}x            # leading and trailing whitespace
        echo x${CONFIGURE_ARGS:Q}x      # properly trimmed
```

В этом примере `CPPFLAGS` содержит и ведущий, и завершающий пробел, потому что оператор `+=` всегда добавляет пробел.

15.3.4. Обработка переменных, которые могут быть пустыми

Когда переменная, которая может быть пустой, используется в программе оболочки, это может привести к синтаксической ошибке.

```
EGFILES=        # empty

install-examples:   # produces a syntax error in the shell
        for egfile in ${EGFILES}; do            \
                echo "Installing $$egfile";     \
        done
```

Оболочка видит только текст `for egfile in ; do`, поскольку `${EGFILES}` заменяется пустой строкой с помощью make(1). Чтобы исправить эту синтаксическую ошибку, используйте один из приведённых ниже фрагментов.

```
EGFILES=        # empty

install-examples:
        for egfile in ${EGFILES} ""; do         \
                [ -n "$$egfile" ] || continue;  \
                echo "Installing $$egfile";     \
        done
```

В этом случае к списку итерации добавляется пустая строка (чтобы предотвратить синтаксическую ошибку), а затем она отфильтровывается.

```
EGFILES=        # empty

install-examples:
.for egfile in ${EGFILES}
        echo "Installing ${egfile}"
.endfor
```

Если одно из имён файлов содержит специальные символы, его следует заключить в одинарные или двойные кавычки.

Чтобы команда оболочки проверила, пуста ли переменная make, используйте следующий код: `${TEST} -z ${POSSIBLY_EMPTY:Q}""`.

15.3.5. Проверка переменных yes/no в условиях

Когда переменная может принимать значения `yes` или `no`, используйте следующий шаблон для её проверки:

```
.if ${VAR:U:tl} == "yes"
# do something
.endif
```

Модификатор `:U` нужен только в том случае, если переменная может быть не определена. Если переменная гарантированно определена, `:U` можно опустить.

Модификатор `:tl` преобразует значение переменной в нижний регистр, что позволяет использовать значения `yes`, `Yes`, `YES`.

Глава 16. Обработка опций

Многие пакеты можно собирать с поддержкой различных наборов возможностей. `bsd.options.mk` — это инфраструктура в pkgsrc, обеспечивающая общую обработку опций, определяющих различные варианты сборки пакетов. Пользователь может точно указать, какие наборы опций будут включены в пакет при сборке, или разрешить применить набор глобальных опций по умолчанию.

Есть два больших класса поведения, которым можно управлять с помощью опций. Первый — включена ли некоторая конкретная возможность в программе, которая всё равно будет собрана, часто путём включения или невключения зависимости от другого пакета. Второй — будет ли в составе пакета собрана дополнительная программа. Обычно для таких дополнительных программ лучше создавать раздельный пакет, а не использовать опции, поскольку это позволяет собирать двоичные пакеты, которые затем можно устанавливать отдельно. Например, пакет foo может иметь минимальные зависимости (те пакеты, без которых foo не имеет смысла), а пакет foo-gfoo может включать программу с интерфейсом GTK — gfoo. Это лучше, чем включать в foo опцию gtk, добавляющую gfoo: если эта опция задана по умолчанию, пользователи двоичных пакетов не смогут получить foo без gfoo, а если не задана, они не смогут получить gfoo. С раздельными пакетами они могут установить foo без GTK, а позже решить установить gfoo (подтянув GTK в этот момент). Это преимущество и для пользователей, собирающих из исходных текстов, — им не нужно пересобирать пакет.

Подключаемые модули (плагины) с сильно различающимися зависимостями обычно следует выделять в отдельные пакеты, а не делать опциями.

Поддерживать раздельные пакеты часто труднее, особенно если вышестоящий пакет этого не поддерживает. Решение в пользу раздельного пакета или опции следует принимать исходя из того, насколько вероятно, что пользователи захотят получить отдельные части или будут против них, из размера включаемых зависимостей и объёма работы.

Ещё одно соображение — лицензирование. Несвободные части или части, зависящие от несвободных зависимостей (особенно плагины), почти всегда следует выделять в отдельные пакеты, если это возможно.

16.1. Глобальные опции по умолчанию

Глобальные опции по умолчанию перечислены в `PKG_DEFAULT_OPTIONS` — списке опций, которые следует включать в сборку каждого пакета, если эта опция поддерживается. Эту переменную следует задавать в `mk.conf`.

16.2. Перевод пакетов на использование `bsd.options.mk`

В следующем примере показано, как гипотетический пакет «wibble» должен использовать `bsd.options.mk` — в `Makefile` пакета или в файле, например `options.mk`, который подключается основным `Makefile` пакета.

```
PKG_OPTIONS_VAR=                PKG_OPTIONS.wibble
PKG_SUPPORTED_OPTIONS=          wibble-foo ldap
PKG_OPTIONS_OPTIONAL_GROUPS=    database
PKG_OPTIONS_GROUP.database=     mysql pgsql
```

```
PKG_SUGGESTED_OPTIONS=          wibble-foo
PKG_OPTIONS_LEGACY_VARS+=       WIBBLE_USE_OPENLDAP:ldap
PKG_OPTIONS_LEGACY_OPTS+=       foo:wibble-foo

.include "../../mk/bsd.prefs.mk"

# this package was previously named wibble2
.if defined(PKG_OPTIONS.wibble2)
PKG_LEGACY_OPTIONS+=            ${PKG_OPTIONS.wibble2}
PKG_OPTIONS_DEPRECATED_WARNINGS+= \
        "Deprecated variable PKG_OPTIONS.wibble2 used, use ${PKG_OPTIONS_VAR} instead."
.endif

.include "../../mk/bsd.options.mk"

# Package-specific option-handling

###
### FOO support
###
.if !empty(PKG_OPTIONS:Mwibble-foo)
CONFIGURE_ARGS+=    --enable-foo
.endif

###
### LDAP support
###
.if !empty(PKG_OPTIONS:Mldap)
.  include "../../databases/openldap-client/buildlink3.mk"
CONFIGURE_ARGS+=    --enable-ldap=${BUILDLINK_PREFIX.openldap-client}
.endif

###
### database support
###
.if !empty(PKG_OPTIONS:Mmysql)
.  include "../../mk/mysql.buildlink3.mk"
.endif
.if !empty(PKG_OPTIONS:Mpgsql)
.  include "../../mk/pgsql.buildlink3.mk"
.endif
```

Первый раздел содержит сведения о том, какие параметры сборки поддерживаются пакетом, и при необходимости настройки опций по умолчанию.

1. `PKG_OPTIONS_VAR` — это имя переменной make(1), которую пользователь может задать, чтобы переопределить опции по умолчанию. Ей следует присвоить PKG_OPTIONS.*`pkgbase`*. Не задавайте её как PKG_OPTIONS.${PKGBASE}, поскольку `PKGBASE` не определена в момент обработки опций.
2. `PKG_SUPPORTED_OPTIONS` — список параметров сборки, поддерживаемых пакетом.
3. `PKG_OPTIONS_OPTIONAL_GROUPS` — список имён групп взаимоисключающих опций. Опции каждой группы перечислены в `PKG_OPTIONS_GROUP.`*`groupname`*. Наиболее конкретная настройка любой опции из группы имеет приоритет над всеми остальными опциями группы. Опции из групп автоматически добавляются в `PKG_SUPPORTED_OPTIONS`.
4. `PKG_OPTIONS_REQUIRED_GROUPS` похожа на `PKG_OPTIONS_OPTIONAL_GROUPS`, но сборка пакетов завершится ошибкой, если не выбрана ни одна опция из группы.
5. `PKG_OPTIONS_NONEMPTY_SETS` — список имён наборов опций. Из каждого набора должна быть выбрана как минимум одна опция. Опции каждого набора перечислены в

PKG_OPTIONS_SET.*setname*. Опции из наборов автоматически добавляются в PKG_SUPPORTED_OPTIONS. Сборка пакета завершится ошибкой, если не выбрана ни одна опция из набора.

6. PKG_SUGGESTED_OPTIONS — список параметров сборки, включённых по умолчанию.
7. PKG_OPTIONS_LEGACY_VARS — список пар «*USE_VARIABLE*:*option*», которые сопоставляют устаревшие переменные mk.conf с соответствующими им опциями. Пары следует добавлять с помощью «+=», чтобы сохранить перечень глобальных устаревших переменных. Если пользователь использует устаревшую переменную, будет выдано предупреждение.
8. PKG_OPTIONS_LEGACY_OPTS — список пар «*old-option*:*new-option*», которые сопоставляют переименованные опции с их новыми названиями. Пары следует добавлять с помощью «+=», чтобы сохранить перечень глобальных устаревших опций. Если пользователь использует устаревшую опцию, будет выдано предупреждение.
9. PKG_LEGACY_OPTIONS — список опций, подразумеваемых используемыми устаревшими переменными. Это можно использовать в случаях, которые не под силу ни PKG_OPTIONS_LEGACY_VARS, ни PKG_OPTIONS_LEGACY_OPTS, например когда переименована PKG_OPTIONS_VAR.
10. PKG_OPTIONS_DEPRECATED_WARNINGS — список предупреждений об использовании устаревших переменных или опций и о том, что следует использовать вместо них.

Пакет никогда не должен изменять PKG_DEFAULT_OPTIONS или переменную, имя которой указано в PKG_OPTIONS_VAR. Их задаёт только пользователь. Чтобы предложить набор опций по умолчанию, используйте PKG_SUGGESTED_OPTIONS.

PKG_OPTIONS_VAR должна быть определена до подключения bsd.options.mk. Если ни одна из переменных PKG_SUPPORTED_OPTIONS, PKG_OPTIONS_OPTIONAL_GROUPS и PKG_OPTIONS_REQUIRED_GROUPS не определена (так может быть с опциями, зависящими от платформы, если ни одна из них не поддерживается на текущей платформе), PKG_OPTIONS устанавливается в пустой список, а пакет считается не использующим инфраструктуру опций.

После подключения bsd.options.mk переменная PKG_OPTIONS содержит список выбранных параметров сборки, должным образом отфильтрованный для удаления неподдерживаемых и повторяющихся опций.

Остальные разделы содержат логику, специфичную для каждой опции. Правильный способ проверить опцию — проверить, присутствует ли она в PKG_OPTIONS:

```
.if !empty(PKG_OPTIONS:M*option*)
```

16.3. Имена опций

Опции, включающие схожие возможности в разных пакетах (например, необязательная поддержка библиотеки), должны использовать общее имя во всех пакетах, которые её поддерживают (например, имя библиотеки). Если в другом пакете уже есть опция с тем же значением, используйте то же имя.

Опции, включающие возможности, специфичные для одного пакета, когда маловероятно, что другой (не связанный) пакет имеет такую же (или похожую) необязательную возможность, должны иметь имя с префиксом *pkgname-*.

Если группа связанных пакетов разделяет необязательную возможность, специфичную для этой группы, добавьте к имени префикс имени «„главного“» пакета (например, djbware-errno-hack).

Для новых опций добавьте строку в mk/defaults/options.description. Строки состоят из двух полей, разделённых табуляцией. Первое поле — имя опции, второе — её описание. Описание должно

быть целым предложением (начинаться с заглавной буквы и заканчиваться точкой), которое объясняет, что делает включение опции. Например, «„Enable ispell support.“» Файл отсортирован по именам опций.

16.4. Определение опций зависимостей

При написании файлов `buildlink3.mk` часто необходимо перечислить разные зависимости в зависимости от опций, с которыми был собран пакет. Для запроса этих опций следует использовать файл `pkgsrc/mk/pkg-build-options.mk`. Типичный пример выглядит так:

```
pkgbase := libpurple
.include "../../mk/pkg-build-options.mk"

.if !empty(PKG_BUILD_OPTIONS.libpurple:Mdbus)
...
.endif
```

Подключение `pkg-build-options.mk` здесь установит переменную `PKG_BUILD_OPTIONS.libpurple` в параметры сборки пакета libpurple, которые затем можно запрашивать так же, как `PKG_OPTIONS` в файле `options.mk`. Подробнее см. файл `pkg-build-options.mk`.

Глава 17. Инструменты, необходимые для сборки и работы

Определение `USE_TOOLS` используется как внутри pkgsrc, так и отдельными пакетами, чтобы указать, какие команды нужны для сборки пакета (как `TOOL_DEPENDS`) или для последующей работы установленного пакета (как `DEPENDS`). Если нативная система предоставляет подходящий инструмент, то во многих случаях пакет pkgsrc не используется.

При сборке пакета заменяющие инструменты становятся доступны в каталоге (в виде символических ссылок или скриптов-обёрток), который находится в начале пути поиска исполняемых файлов. Как и система buildlink, это способствует согласованной сборке.

Инструмент может понадобиться для сборки конкретного пакета. Например, могут потребоваться perl, GNU make (gmake) или yacc.

Инструмент может понадобиться и потому, что, например, поставляемый нативной системой инструмент может быть неэффективен для сборки пакета с помощью pkgsrc. Например, пакету может потребоваться GNU awk, bison (вместо yacc) или более удобный sed.

Инструменты, используемые пакетом, можно получить, выполнив **make show-tools**.

17.1. Инструменты для сборки pkgsrc

Набор инструментов, используемый pkgsrc по умолчанию, определён в `bsd.pkg.mk`. Он включает стандартные инструменты Unix, такие как **cat**, **awk**, **chmod**, **test** и т. д. Их можно увидеть, выполнив: **make show-var VARNAME=USE_TOOLS**.

Если для сборки пакету нужна конкретная программа, переменную `USE_TOOLS` можно использовать, чтобы определить необходимые инструменты.

17.2. Инструменты, необходимые пакетам

В следующих примерах :run означает, что инструмент нужен во время работы (и становится DEPENDS). По умолчанию это зависимость для сборки, которую можно задать с помощью :build. (Так что в этом примере это то же самое, что gmake:build и pkg-config:build.)

```
USE_TOOLS+=     gmake perl:run pkg-config
```

При использовании инфраструктуры инструментов определяется переменная `TOOLS_PATH.foo`, содержащая полный путь к нужному инструменту. Например, `TOOLS_PATH.bash` в системах Linux может быть «„/bin/bash“».

Если вам всегда нужна версия инструмента из pkgsrc во время работы, просто используйте вместо этого `DEPENDS`.

17.3. Инструменты, предоставляемые платформами

Улучшая или портируя pkgsrc на новую платформу, посмотрите (или создайте) соответствующий фрагмент make-файла для конкретной платформы — `pkgsrc/mk/tools/tools.${OPSYS}.mk`, в

котором определяются имена общих инструментов. Например:

```
.if exists(/usr/bin/bzcat)
TOOLS_PLATFORM.bzcat?=          /usr/bin/bzcat
.elif exists(/usr/bin/bzip2)
TOOLS_PLATFORM.bzcat?=          /usr/bin/bzip2 -cd
.endif

TOOLS_PLATFORM.true?=           true                    # shell builtin
```

Глава 18. Методика buildlink

Buildlink — это инфраструктура в pkgsrc, которая управляет тем, какие заголовочные файлы и библиотеки видят процессы configure и build пакета. Она реализована в два этапа:

1. Символические ссылки на заголовочные файлы и библиотеки зависимостей создаются в `BUILDLINK_DIR`, который по умолчанию является подкаталогом `WRKDIR`.
2. Создайте скрипты-обёртки, которые используются вместо обычных инструментов компилятора и преобразуют `-I${LOCALBASE}/include` и `-L${LOCALBASE}/lib` в ссылки на `BUILDLINK_DIR`. Скрипты-обёртки также делают нативный компилятор в некоторых операционных системах похожим на GCC, чтобы пакеты, рассчитанные на GCC, не требовали изменений для сборки этими нативными компиляторами.

Это нормализует окружение, в котором собирается пакет, чтобы его можно было собирать согласованно, независимо от того, какое ещё программное обеспечение установлено. Обратите внимание, что стандартные системные пути заголовочных файлов и библиотек, например `/usr/include`, `/usr/lib` и т. д., всегда просматриваются — buildlink3 предназначен для того, чтобы изолировать сборку пакета от программного обеспечения, не поставляемого системой.

18.1. Перевод пакетов на buildlink3

Процесс перевода пакетов на инфраструктуру buildlink3 («„bl3ifying“») довольно прост. Следует помнить о следующем:

1. Убедитесь, что при сборке всегда вызываются скрипты-обёртки, а не настоящая цепочка инструментов. Некоторые пакеты ведут себя хитро, и единственный способ точно убедиться — проверить `${WRKDIR}/.work.log` и посмотреть, вызываются ли обёртки.
2. Не переопределяйте `PREFIX` в Makefile пакета, например для виртуальных машин Java, автономных оболочек и т. п., потому что код, создающий символические ссылки на файлы в `${BUILDLINK_DIR}`, ищет файлы относительно «„pkg_info -qp *`pkgname`*“».
3. Помните, что *только* те файлы `buildlink3.mk`, которые вы перечислили в Makefile пакета, добавляются как зависимости для этого пакета.

Если зависимость от конкретного пакета требуется ради его библиотек и заголовочных файлов, то мы заменяем:

```
DEPENDS+=       foo>=1.1.0:../../category/foo
```

на

```
.include "../../category/foo/buildlink3.mk"
```

Файлы buildlink3.mk обычно определяют необходимые зависимости. Если при использовании файлов buildlink3.mk вам нужна более новая версия зависимости, её можно указать в своём Makefile; например:

```
BUILDLINK_API_DEPENDS.foo+=   foo>=1.1.0
.include "../../category/foo/buildlink3.mk"
```

В pkgsrc/mk есть несколько файлов buildlink3.mk, которые решают особые задачи пакетов:

- bdb.buildlink3.mk выбирает нативную реализацию Berkeley DB или реализацию из pkgsrc на основе значений BDB_ACCEPTED и BDB_DEFAULT.
- curses.buildlink3.mk: если в системе нет ни Curses, ни NCurses, этот файл позаботится об установке пакета devel/ncurses.
- krb5.buildlink3.mk использует значение KRB5_ACCEPTED, чтобы выбрать для пакетов, которым нужна реализация Kerberos 5, зависимость от Heimdal или от MIT-krb5.
- motif.buildlink3.mk проверяет наличие установленного в системе Motif или добавляет зависимость от x11/motif. Пользователь может задать в MOTIF_TYPE значение «„dt“» или «„motif“», чтобы выбрать используемую версию Motif.
- readline.buildlink3.mk проверяет наличие установленных в системе GNU readline или editline (libedit) либо добавляет зависимость от devel/readline, devel/editline. Пользователь может задать READLINE_DEFAULT, чтобы выбрать реализацию readline. Если вашему пакету действительно нужна библиотека GNU readline, его Makefile должен подключать devel/readline/buildlink3.mk, а не readline.buildlink3.mk.
- oss.buildlink3.mk определяет несколько переменных, которые могут использоваться пакетами, работающими с API Open Sound System (OSS).
- pgsql.buildlink3.mk принимает любую из версий Postgres, указанных в переменной PGSQL_VERSIONS_ACCEPTED, и по умолчанию использует версию PGSQL_VERSION_DEFAULT. Дополнительные сведения см. в этом файле.
- pthread.buildlink3.mk использует значение PTHREAD_OPTS и при необходимости проверяет наличие нативных pthreads или добавляет зависимость от devel/pth.
- xaw.buildlink3.mk использует значение XAW_TYPE, чтобы выбрать конкретную библиотеку виджетов Athena.

Комментарии в этих файлах buildlink3.mk дают более полное описание того, как правильно их использовать.

18.2. Написание файлов buildlink3.mk

Файл buildlink3.mk пакета подключается из Makefile, чтобы указать необходимость компиляции и компоновки с заголовочными файлами и библиотеками, предоставляемыми пакетом. Файл buildlink3.mk всегда должен содержать достаточно информации, чтобы добавить зависимость правильного типа и подключить любые другие файлы buildlink3.mk, которые нужны ему, в свою очередь, для поиска необходимых заголовочных файлов и библиотек.

Чтобы сгенерировать исходный файл buildlink3.mk для дальнейшего редактирования, настоятельно рекомендуется пакет pkgtools/createbuildlink (автор — Rene Hexel). Для большинства пакетов следующая команда создаст хорошую отправную точку для файлов buildlink3.mk:

```
% cd pkgsrc/category/pkgdir
% createbuildlink >buildlink3.mk
```

18.2.1. Анатомия файла buildlink3.mk

Следующий пример из реальной практики, `buildlink3.mk`, взят из `pkgsrc/graphics/tiff`:

```
BUILDLINK_TREE+=        tiff

.if !defined(TIFF_BUILDLINK3_MK)
TIFF_BUILDLINK3_MK:=

BUILDLINK_API_DEPENDS.tiff+=    tiff>=3.6.1
BUILDLINK_ABI_DEPENDS.tiff+=    tiff>=3.7.2nb1
BUILDLINK_PKGSRCDIR.tiff?=      ../../graphics/tiff

.include "../../devel/zlib/buildlink3.mk"
.include "../../graphics/jpeg/buildlink3.mk"
.endif # TIFF_BUILDLINK3_MK

BUILDLINK_TREE+=        -tiff
```

Заголовок и подвал работают с `BUILDLINK_TREE` — переменной, общей для всех файлов `buildlink3.mk` и служащей для отслеживания дерева зависимостей.

Основной раздел защищён от повторного включения и управляет тем, как добавляется зависимость от *pkg*. В этом разделе задаются несколько важных переменных:

- `BUILDLINK_API_DEPENDS.`*pkg* — это версия зависимости, записываемая в установленный пакет; задавать её всегда следует с помощью `+=`, чтобы гарантировать дописывание в конец уже существующего списка значений. Эту переменную нужно устанавливать в последнюю версию пакета, в которой произошло несовместимое с предыдущими версиями изменение API.
- `BUILDLINK_PKGSRCDIR.`*pkg* — это расположение каталога *pkg* в pkgsrc.
- `BUILDLINK_DEPMETHOD.`*pkg* (не показана выше) определяет, используется ли `BUILD_DEPENDS` или `DEPENDS` для добавления зависимости от *pkg*. Зависимость времени сборки выбирается установкой `BUILDLINK_DEPMETHOD.`*pkg* в «„build“». По умолчанию используется полная зависимость.
- `BUILDLINK_INCDIRS.`*pkg* и `BUILDLINK_LIBDIRS.`*pkg* (не показаны выше) — это списки подкаталогов `${BUILDLINK_PREFIX.`*pkg*`}`, добавляемых к путям поиска заголовочных файлов и библиотек. По умолчанию они равны «„include“» и «„lib“» соответственно.
- `BUILDLINK_CPPFLAGS.`*pkg* (не показана выше) — это список флагов препроцессора, добавляемых к `CPPFLAGS`, которые передаются на этапы конфигурации и сборки. Опции «„-I“» следует избегать и вместо этого обрабатывать её с помощью `BUILDLINK_INCDIRS.`*pkg*, как описано выше.

Все перечисленные ниже переменные необязательно определяются во втором разделе (защищённом от повторного включения) и управляют тем, какие файлы пакета создаются как символические ссылки в `${BUILDLINK_DIR}` и как их имена преобразуются при этом:

- `BUILDLINK_FILES.`*pkg* (не показана выше) — это шаблон подстановки оболочки, заданный относительно `${BUILDLINK_PREFIX.`*pkg*`}`, для файлов, которые должны быть слинкованы в `${BUILDLINK_DIR}`, например `include/*.h`.
- `BUILDLINK_FILES_CMD.`*pkg* (не показана выше) — это конвейер оболочки, выводящий в stdout список файлов относительно `${BUILDLINK_PREFIX.`*pkg*`}`. Полученные файлы должны быть слинкованы в `${BUILDLINK_DIR}`. По умолчанию берётся `+CONTENTS` пакета *pkg* и фильтруется через `${BUILDLINK_CONTENTS_FILTER.`*pkg*`}`.

- BUILDLINK_CONTENTS_FILTER.*pkg* (не показана выше) — это команда фильтрации, преобразующая входные данные +CONTENTS в список файлов относительно ${BUILDLINK_PREFIX.*pkg*} в stdout. По умолчанию BUILDLINK_CONTENTS_FILTER.*pkg* выводит содержимое каталогов include и lib в файле +CONTENTS пакета.
- BUILDLINK_FNAME_TRANSFORM.*pkg* (не показана выше) — это список аргументов sed, используемых для преобразования имени исходного файла в имя целевого файла, например **-e "s|/curses.h|/ncurses.h|g"**.

Этот раздел может дополнительно включать любые файлы buildlink3.mk, необходимые для библиотечных зависимостей *pkg*. Включение этих файлов buildlink3.mk означает, что заголовочные файлы и библиотеки этих зависимостей также создаются как символические ссылки в ${BUILDLINK_DIR} всякий раз, когда включается относящийся к *pkg* файл buildlink3.mk. Зависимости добавляются только для непосредственно включённых файлов buildlink3.mk.

Создавая buildlink3.mk и включая в него другие файлы buildlink3.mk, добавляйте только необходимые, то есть те, чьи библиотеки или заголовочные файлы автоматически становятся доступны при использовании пакета.

В частности, если с библиотекой компонуется только исполняемый файл (bin/foo), эту библиотеку не нужно распространять через файл buildlink3.mk.

Следующие шаги помогут вам решить, нужно ли включать файл buildlink3.mk:

- Посмотрите на установленные заголовочные файлы: какие заголовки они включают? Пакеты, предоставляющие эти файлы, должны быть подключены через buildlink.
- Запустите ldd для всех установленных библиотек и посмотрите, с какими другими библиотеками они слинкованы. Некоторые из пакетов, предоставляющих их, вероятно, нужно подключить через buildlink; однако это происходит не автоматически, поскольку, например, GTK в одних системах подтягивает библиотеки X, и они появляются в выводе ldd, а в других (например, OS X) — нет. Поэтому вывод ldd можно использовать лишь как подсказку.

18.2.2. Обновление BUILDLINK_API_DEPENDS.*pkg* и BUILDLINK_ABI_DEPENDS.*pkg* в файлах buildlink3.mk

Обе переменные задают нижнюю границу версии этого пакета. Различаются они тем, что одна описывает совместимость на уровне исходных текстов (API), а другая — двоичную совместимость (ABI). Разница в том, что изменение API нарушает компиляцию программ, а из-за изменения ABI скомпилированные программы перестают запускаться.

Переменную BUILDLINK_API_DEPENDS.*pkg* в файле buildlink3.mk следует менять очень редко. (Один из возможных сценариев: если всем пакетам, использующим этот пакет, нужна версия выше указанной в buildlink3.mk, то BUILDLINK_API_DEPENDS.*pkg* можно обновить до этой более высокой версии.)

С другой стороны, изменения BUILDLINK_ABI_DEPENDS.*pkg* происходят чаще. Эту переменную придётся обновлять каждый раз, когда меняется основная версия одной из разделяемых библиотек пакета или происходит любое другое изменение, после которого двоичный файл, собранный с предыдущей версией пакета, больше не будет работать с новой версией.

В таком случае необходимо увеличить BUILDLINK_ABI_DEPENDS.*pkg* пакета, чтобы потребовать новую версию пакета. Затем нужно увеличить PKGREVISION всех пакетов *foo*, зависящих от этого пакета, а если у них есть файлы buildlink3.mk, то и BUILDLINK_ABI_DEPENDS.*foo* в их файлах buildlink3.mk тоже нужно увеличить до новой версии. Это требуется для того, чтобы пакет подтягивал версии пакетов, использующих новый ABI, и чтобы значения PKGREVISION пакетов

однозначно определяли пакеты, собранные с новым ABI. С этими обновлениями может помочь пакет `pkgtools/revbump`.

См. раздел 21.1.5, «Обработка зависимостей» — там приведены дополнительные сведения о зависимостях от других пакетов, включая определения `BUILDLINK_API_DEPENDS`.

Прежде чем корректировать `BUILDLINK_API_DEPENDS.pkg` или `BUILDLINK_ABI_DEPENDS.pkg` в файле `buildlink3.mk`, тщательно всё взвесьте: нам не нужны лишние удаления и пересборки пакетов. Во многих случаях новые версии пакетов прекрасно работают со старыми зависимостями.

Кроме того, не нужно задавать `BUILDLINK_ABI_DEPENDS.pkg`, если она совпадает с `BUILDLINK_API_DEPENDS.pkg`.

Отметим также ещё одно различие: пользователи могут отключить принудительное соблюдение зависимостей ABI с помощью переменной `USE_ABI_DEPENDS`, но для зависимостей API аналогичной возможности нет.

18.3. Написание файлов `builtin.mk`

Некоторые пакеты в pkgsrc устанавливают заголовочные файлы и библиотеки, совпадающие с теми, что уже есть в базовой системе. Помимо файла `buildlink3.mk`, такие пакеты должны включать файл `builtin.mk` с необходимыми проверками, позволяющими решить, что уместнее использовать — встроенное программное обеспечение или программное обеспечение из pkgsrc.

К файлу builtin.mk для *pkg* предъявляются только следующие требования:

1. После включения он должен устанавливать `USE_BUILTIN.pkg` в «„yes“» или «„no“».
2. Он *не должен* переопределять `USE_BUILTIN.pkg`, если она уже задана до включения файла `builtin.mk`.
3. Он должен быть написан так, чтобы допускать многократное включение. Это *очень* важно и требует внимательного отношения к написанию кода в `Makefile`.

18.3.1. Структура файла `builtin.mk`

Ниже приведён рекомендуемый шаблон для файлов builtin.mk:

```
.if !defined(IS_BUILTIN.foo)
#
# IS_BUILTIN.foo is set to "yes" or "no" depending on whether "foo"
# genuinely exists in the system or not.
#
IS_BUILTIN.foo?=        no

# BUILTIN_PKG.foo should be set here if "foo" is built-in and its package
# version can be determined.
#
.  if !empty(IS_BUILTIN.foo:M[yY][eE][sS])
BUILTIN_PKG.foo?=       foo-1.0
.  endif
.endif  # IS_BUILTIN.foo

.if !defined(USE_BUILTIN.foo)
USE_BUILTIN.foo?=       ${IS_BUILTIN.foo}
.  if defined(BUILTIN_PKG.foo)
.    for _depend_ in ${BUILDLINK_API_DEPENDS.foo}
.      if !empty(USE_BUILTIN.foo:M[yY][eE][sS])
USE_BUILTIN.foo!=                                                       \
        ${PKG_ADMIN} pmatch '${_depend_}' ${BUILTIN_PKG.foo}            \
        && ${ECHO} "yes" || ${ECHO} "no"
```

```
.      endif
.    endfor
.  endif
.endif  # USE_BUILTIN.foo

CHECK_BUILTIN.foo?=     no
.if !empty(CHECK_BUILTIN.foo:M[nN][oO])
#
# Here we place code that depends on whether USE_BUILTIN.foo is set to
# "yes" or "no".
#
.endif  # CHECK_BUILTIN.foo
```

Первый раздел устанавливает `IS_BUILTIN.pkg` в зависимости от того, действительно ли *`pkg`* присутствует в базовой системе. Здесь не имеется в виду программное обеспечение базовой системы со сходными с *`pkg`* функциями; значение «„yes“» должно выставляться, только если сам пакет входит в состав базовой системы. Эта переменная используется только внутри файла `builtin.mk`.

Второй раздел устанавливает `BUILTIN_PKG.pkg` в версию *`pkg`*, присутствующую в базовой системе, если она есть (то есть если `IS_BUILTIN.pkg` равна «„yes“»). Эта переменная используется только внутри файла `builtin.mk`.

Третий раздел устанавливает `USE_BUILTIN.pkg` и *обязателен* во всех файлах `builtin.mk`. Код в этом разделе должен определить, достаточно ли встроенного программного обеспечения, чтобы удовлетворить зависимости, перечисленные в `BUILDLINK_API_DEPENDS.pkg`. Обычно это делается путём сравнения `BUILTIN_PKG.pkg` с каждой из зависимостей в `BUILDLINK_API_DEPENDS.pkg`. К концу файла `builtin.mk` переменной `USE_BUILTIN.pkg` *должно* быть присвоено правильное значение. Учтите, что `USE_BUILTIN.pkg` может иметь значение «yes», даже если `IS_BUILTIN.pkg` равно «no», поскольку можно прийти к выводу, что встроенная версия программы достаточно близка к требуемой и может использоваться в качестве замены.

Последний раздел защищён условием `CHECK_BUILTIN.pkg` и содержит код, использующий значение `USE_BUILTIN.pkg`, заданное в предыдущем разделе. Обычно он включает, например, добавление дополнительных ограничений на зависимости и перечисление дополнительных файлов, которые нужно слинковать в `${BUILDLINK_DIR}` (через `BUILDLINK_FILES.pkg`).

Глава 19. Вопросы работы с PLIST

Файл `PLIST` содержит «„список упаковки“» пакета, то есть список файлов, принадлежащих пакету (относительно каталога `${PREFIX}`, в который он установлен), а также некоторые дополнительные инструкции — полный список см. на странице руководства pkg_create(1). В этой главе рассматриваются некоторые вопросы, на которые нужно обратить внимание при работе с файлом `PLIST` (или файлами, см. ниже!).

19.1. Идентификатор RCS

Обязательно добавляйте строку с идентификатором RCS первой в любой создаваемый вами файл `PLIST`:

```
@comment $NetBSD $
```

Искусственный пробел между NetBSD и $ добавлен здесь как обходной приём, чтобы CVS не подставил имя файла руководства. При добавлении идентификатора RCS этот пробел нужно опустить.

19.2. Полуавтоматическое создание `PLIST`

С помощью команды **make print-PLIST** можно вывести PLIST, соответствующий всем новым файлам, появившимся после распаковки пакета. Дополнительные сведения об этой цели см. в разделе 13.17, «Другие полезные цели».

19.3. Настройка вывода make print-PLIST

Переменная `PRINT_PLIST_AWK` принимает набор шаблонов и действий AWK, используемых для фильтрации вывода print-PLIST. К ней можно *дописывать* любые фрагменты сценариев AWK, но будьте осторожны с кавычками.

Например, чтобы удалить из итогового PLIST все файлы из каталога `libdata/foo`:

```
PRINT_PLIST_AWK+=       /^libdata\/foo/ { next; }
```

Преобразования `PRINT_PLIST_AWK` выполняются после сортировки списка файлов и списка каталогов. `EARLY_PRINT_PLIST_AWK` действует так же, как `PRINT_PLIST_AWK`, но до сортировки списка файлов и списка каталогов.

19.4. Подстановка переменных в PLIST

При установке пакета в системе в PLIST автоматически подставляется ряд переменных. К ним относятся следующие переменные:

`${MACHINE_ARCH}`, `${MACHINE_GNU_ARCH}`

Некоторые пакеты, например emacs и perl, встраивают сведения о том, для какой архитектуры они собраны, в имена путей, по которым устанавливаются их файлы. Чтобы учесть этот случай, PLIST предварительно обрабатывается, и символ «`${MACHINE_ARCH}`» заменяется на то, что выдаёт **uname -p**. То же делается, если в PLIST где-либо встречается строка `${MACHINE_GNU_ARCH}`, — используйте её в пакетах со сценариями configure, созданными GNU autoconf.

> **ЗАМЕЧАНИЕ ОБ УСТАРЕВШИХ ВЕРСИЯХ**
>
> Раньше существовал символ «`$ARCH`», который заменялся на вывод **uname -m**, но это больше не поддерживается и было удалено.

`${OPSYS}`, `${LOWER_OPSYS}`, `${OS_VERSION}`

Некоторые пакеты хотят встраивать имя и версию операционной системы в некоторые пути. Для этого используйте в `PLIST` следующие переменные:

- `${OPSYS}` — вывод «**uname -s**»
- `${LOWER_OPSYS}` — общепринятое название в нижнем регистре (например, «„solaris“»)
- `${OS_VERSION}` — «**uname -r**»

Список значений, подставляемых по умолчанию, можно получить из вывода **make help topic=PLIST_SUBST**, а также поиском в каталоге `pkgsrc/mk` с помощью **grep** строки `PLIST_SUBST`.

Если вы хотите изменить другие переменные, не перечисленные выше, можно добавить переменные и их раскрытия в эту переменную следующим образом, аналогично `MESSAGE_SUBST` (см. раздел 12.5, «Необязательные файлы»):

```
PLIST_SUBST+=   SOMEVAR="somevalue"
```

Это заменяет все вхождения «„${SOMEVAR}“» в файле `PLIST` на «„somevalue“».

Переменную `PLIST_VARS` можно использовать, чтобы упростить распространённый случай условного включения некоторых записей `PLIST`. Для этого нужно добавить `PLIST_VARS+=foo` и установить соответствующую переменную `PLIST.foo` в `yes`, если запись должна быть включена. При этом «`${PLIST.foo}`» в файле `PLIST` заменяется либо на «`""`», либо на «`"@comment "`». Например, в `Makefile`:

```
PLIST_VARS+=    foo
.if condition
PLIST.foo=      yes
.else
```

А затем в `PLIST`:

```
@comment $NetBSD $
bin/bar
man/man1/bar.1
${PLIST.foo}bin/foo
${PLIST.foo}man/man1/foo.1
${PLIST.foo}share/bar/foo.data
```

Искусственный пробел между NetBSD и $ добавлен здесь как обходной приём, чтобы CVS не подставил имя файла руководства. При добавлении идентификатора RCS этот пробел нужно

опустить.

19.5. Сжатие страниц руководства

Страницы руководства должны устанавливаться в сжатом виде, если задана `MANZ` (в `bsd.own.mk`), и в несжатом — в противном случае. Чтобы учесть это в файле `PLIST`, суффикс «„.gz“» автоматически добавляется к страницам руководства или удаляется у них в зависимости от того, заданы ли `MANZ` и `MANCOMPRESSED`; подробности см. выше. Это изменение файла `PLIST` выполняется на его копии, а не на самом `PLIST`.

19.6. Изменение источника PLIST с помощью `PLIST_SRC`

Чтобы использовать один или несколько файлов в качестве источника `PLIST` при создании двоичного пакета, задайте переменной `PLIST_SRC` имена этих файлов. Позже файлы объединяются с помощью cat(1), и порядок следования имеет значение. Значение по умолчанию для `PLIST_SRC` — `${PKGDIR}/PLIST`.

19.7. PLIST для разных платформ и отличающиеся PLIST

Некоторые пакеты устанавливают разный набор файлов в зависимости от используемой операционной системы. Такие различия можно обрабатывать автоматически с помощью следующих файлов:

- `PLIST.common`
- `PLIST.${OPSYS}`
- `PLIST.${MACHINE_ARCH}`
- `PLIST.${OPSYS}-${MACHINE_ARCH}`
- `PLIST.common_end`

19.8. PLIST, зависящие от сборки

Некоторые пакеты при установке создают имена файлов, которые трудно предугадать и трудно точно зафиксировать.

В таких случаях можно задать переменную `GENERATE_PLIST` как код оболочки, завершаемый точкой с запятой, который выведет записи PLIST, добавляемые в конец PLIST

Один пример можно найти в editors/xemacs:

```
GENERATE_PLIST+=        ${ECHO} bin/${DISTNAME}-`${WRKSRC}/src/xemacs -sd`.dmp ;
```

который добавит что-то вроде `bin/xemacs-21.4.23-54e8ea71.dmp` к файлу `PLIST`.

19.9. Совместное использование каталогов пакетами

«„Общий каталог“» — это каталог, в который несколько (и не связанных между собой) пакетов устанавливают файлы. Такие каталоги создавали проблемы, поскольку приходилось добавлять в PLIST особые ухищрения для их условного удаления или поручать их обработку какому-нибудь централизованному пакету.

В pkgsrc теперь это просто: каждый пакет должен создавать каталоги и устанавливать файлы по мере необходимости; **pkg_delete** удалит все каталоги, оставшиеся пустыми после удаления пакета.

Если пакету для работы нужен пустой каталог, создайте его при установке как обычно и добавьте запись в PLIST:

```
@pkgdir path/to/empty/directory
```

или обратите внимание на `MAKE_DIRS` и `OWN_DIRS`.

Глава 20. Инфраструктура pkginstall

В этой главе описывается инфраструктура под названием `pkginstall`, ключевые возможности которой перечислены ниже:

- Универсальная установка каталогов и файлов вне дерева, управляемого pkgsrc, — в `LOCALBASE` — и работа с ними.
- Автоматическая обработка файлов конфигурации при установке при условии, что пакеты спроектированы правильно.
- Создание и установка сценариев запуска системы.
- Регистрация системных пользователей и групп.
- Регистрация системных оболочек.
- Автоматическое обновление баз данных шрифтов.

В следующих разделах каждый из перечисленных пунктов рассматривается подробно.

Возможно, вы думаете, что многое из описанного здесь легко сделать простым кодом в цели пакета, выполняемой после установки (`post-install`). *Это неверно*: такой код выполняется только при сборке из исходных текстов. Машины, использующие двоичные пакеты, вообще не получат от него пользы (поскольку сам код может быть недоступен). Поэтому единственный способ реализовать любой из перечисленных выше пунктов — сценарии установки, которые автоматически создаются pkginstall.

20.1. Файлы и каталоги вне префикса установки

Как вы уже знаете, файл `PLIST` содержит список файлов и каталогов, принадлежащих пакету. Имена в нём задаются относительно префикса установки (`${PREFIX}`), а значит, зарегистрировать файлы вне этого каталога он не может (абсолютные имена путей не допускаются). Несмотря на это ограничение, некоторым пакетам нужно устанавливать файлы вне данного места, например в `${VARBASE}` или `${PKG_SYSCONFDIR}`. Единственный способ добиться этого — создавать такие файлы во время установки с помощью сценариев установки.

Универсальные сценарии установки — это сценарии оболочки, которые могут содержать произвольный код. Список сценариев для выполнения берётся из переменной `INSTALL_FILE`, по умолчанию равной `INSTALL`. Аналогичная переменная есть для удаления пакета (`DEINSTALL_FILE`, по умолчанию — `DEINSTALL`). Эти сценарии могут выполнять произвольные команды, поэтому они способны создавать файлы в любом месте файловой системы и управлять ими.

Использовать такие общие файлы установки не рекомендуется, хотя в некоторых особых случаях они могут понадобиться. Одна из причин избегать их — пользователю приходится доверять упаковщику в том, что в сценарий установки не включён нежелательный или просто ошибочный код. Кроме того, раньше существовало много похожих сценариев с одинаковой функциональностью, и исправление общей ошибки требовало найти и изменить их все.

Инфраструктура pkginstall предлагает другой, стандартизованный путь. Она предоставляет универсальные сценарии, абстрагирующие работу с такими файлами и каталогами на основе переменных, заданных в `Makefile` пакета. Остальная часть этого раздела описывает эти переменные.

20.1.1. Работа с каталогами

Следующие переменные можно задать, чтобы запросить создание каталогов в любом месте файловой системы:

- `MAKE_DIRS` и `OWN_DIRS` содержат список каталогов, которые должны создаваться и которые сценарии установки должны пытаться удалить. Разница между ними в том, что последняя предлагает администратору удалить каталоги, которые могли остаться после удаления пакета (поскольку были непустыми), а первая этого не делает. Пример:

```
MAKE_DIRS+=             ${VARBASE}/foo/private
```

- `MAKE_DIRS_PERMS` и `OWN_DIRS_PERMS` содержат список кортежей, описывающих, какие каталоги должны создаваться и которые сценарии установки должны пытаться удалить. Каждый кортеж содержит следующие значения, разделённые пробелами: имя каталога, его владелец, его группа и числовой режим доступа. Например:

```
MAKE_DIRS_PERMS+=       ${VARBASE}/foo/private \
                        ${REAL_ROOT_USER} ${REAL_ROOT_GROUP} 0700
```

  Разница между ними точно такая же, как у их аналогов без `PERMS`.

20.1.2. Работа с файлами

Создание непустых файлов вне префикса установки — непростая задача, поскольку `PLIST` требует, чтобы все файлы находились внутри него. Единственное решение этой проблемы — извлечь файл в известное место (то есть внутрь префикса установки) и скопировать его в нужное место при установке (это делают сценарии установки, создаваемые pkginstall). Файл, извлекаемый в известное место, в дальнейшем мы будем называть *эталонным файлом*; в следующих абзацах описаны переменные, позволяющие автоматически и единообразно обрабатывать файлы вне префикса установки:

- `REQD_FILES` — это список пар из эталонного и целевого файлов. При установке эталонный файл копируется в целевой тогда и только тогда, когда последний не существует. При удалении пакета эталонный файл удаляется при условии, что он не был изменён при установке.
- `REQD_FILES_PERMS` содержит кортежи, описывающие эталонные и целевые файлы, включая владельца, группу и числовые права доступа, которые нужно установить. Например:

```
REQD_FILES_PERMS+=      ${PREFIX}/share/somefile ${VARBASE}/somefile \
                        ${REAL_ROOT_USER} ${REAL_ROOT_GROUP} 0700
```

- `CONF_FILES` и `CONF_FILES_PERMS` имеют тот же синтаксис, что `REQD_FILES` и `REQD_FILES_PERMS` соответственно. Разница в том, что эти переменные предназначены специально для работы с файлами конфигурации, к которым применяются дополнительные соглашения и ограничения. Подробнее см. в разделе 20.2, «Файлы конфигурации». Обратите особое внимание: хотя пользователь может отключить обработку файлов конфигурации (см. раздел 20.2.5, «Отключение обработки файлов конфигурации»), эта настройка не влияет на `REQD_FILES` и `REQD_FILES_PERMS`.

Чтобы установить пустой файл, можно использовать эти макросы, указав `/dev/null` в качестве эталонного файла.

20.2. Файлы конфигурации

Обработку файлов конфигурации в pkgsrc определяют два принципа: во-первых, конфигурация пользователя не должна теряться или перезаписываться при обновлениях и переустановках; во-вторых, конфигурация по умолчанию всегда должна быть доступна для справки. Для этого в pkgsrc есть специальная инфраструктура для работы с файлами конфигурации. Как правило, файлы конфигурации устанавливаются в `${PREFIX}/share/examples` и оттуда копируются в соответствующий каталог `etc` отдельным шагом и только при необходимости.

Чтобы всё заработало, нужно выполнить три вещи, и любая из них может потребовать внесения патчей. Во-первых, пакет нужно научить читать свою конфигурацию из правильного места во время работы; это место — `${PKG_SYSCONFDIR}` (или, возможно, его подкаталог), который часто, но не всегда совпадает с `${PREFIX}/etc`. Во-вторых, пакет нужно научить устанавливать свои файлы конфигурации по умолчанию в `${PREFIX}/share/examples/${PKGBASE}`; именно на этом этапе часто требуется патчинг, поскольку это место не совпадает с местом, используемым во время работы. В-третьих, файлы конфигурации нужно объявить в makefile пакета; это самая простая часть.

Когда всё это на месте, всё пойдёт как надо: этап установки пакета и полученный двоичный пакет не будут трогать `${PKG_SYSCONFDIR}`, а конфигурация по умолчанию будет скопирована на место инфраструктурой pkgsrc, только если её там ещё нет. Точно так же при удалении пакета конфигурация не будет удалена, если она была изменена. (Это означает, что при переустановке и обновлении неизменённые файлы конфигурации обновляются, а изменения никогда не отбрасываются.)

В следующих разделах описано, как этого добиться, а также другие полезные настройки, доступные в инфраструктуре pkgsrc.

20.2.1. Как задаётся `PKG_SYSCONFDIR`

Как уже говорилось, переменная `PKG_SYSCONFDIR` задаёт, куда должны устанавливаться файлы конфигурации. Её значение задаётся на основе следующих переменных:

- `PKG_SYSCONFBASE`: корневой каталог конфигурации. По умолчанию — `${PREFIX}/etc`, хотя пользователь может переопределить его, указав предпочтительное место (например, `/etc`, `/etc/pkg` и т. п.). Пакеты не должны использовать её напрямую.
- `PKG_SYSCONFSUBDIR`: подкаталог `PKG_SYSCONFBASE`, в который должны устанавливаться файлы конфигурации собираемого пакета. Определение этой переменной имеет смысл только в `Makefile` пакета (то есть пользователь не может её настраивать).

  В качестве примера рассмотрим пакет Apache, `www/apache24`, который размещает свои файлы конфигурации в подкаталоге `httpd/` каталога `PKG_SYSCONFBASE`. Это следует задавать в Makefile пакета.
- `PKG_SYSCONFVAR`: задаёт имя переменной, содержащей каталог конфигурации этого пакета (если он отличается от `PKG_SYSCONFBASE`). По умолчанию равно значению `PKGBASE` и всегда используется с префиксом `PKG_SYSCONFDIR`.
- `PKG_SYSCONFDIR.${PKG_SYSCONFVAR}`: содержит каталог, в который должны помещаться файлы конфигурации пакета, определяемого значением `PKG_SYSCONFVAR`.

На основе перечисленных выше переменных pkginstall определяет значение `PKG_SYSCONFDIR` — *единственной* переменной, которую можно использовать внутри пакета для ссылки на его каталог конфигурации. Алгоритм установки её значения в основном таков:

1. Если `PKG_SYSCONFDIR.${PKG_SYSCONFVAR}` задана, используется её значение.

2. Если предыдущая переменная не определена, но `PKG_SYSCONFSUBDIR` задана в `Makefile` пакета, результирующее значение — `${PKG_SYSCONFBASE}/${PKG_SYSCONFSUBDIR}`.
3. В противном случае она равна `${PKG_SYSCONFBASE}`.

Стоит отметить, что `${PKG_SYSCONFDIR}` автоматически добавляется в `OWN_DIRS`. Благодаря этому каталог при необходимости создаётся автоматически. Подробности см. в разделе 20.1.1, «Работа с каталогами». Это не распространяется на подкаталоги `${PKG_SYSCONFDIR}`: их нужно создавать вручную с помощью `OWN_DIRS` или `MAKE_DIRS`.

20.2.2. Как сообщить программе, где находятся файлы конфигурации

Поскольку pkgsrc (да и пользователи!) ожидает, что файлы конфигурации находятся в известном месте, нужно научить каждый пакет, куда устанавливать свои файлы. В некоторых случаях для этого придётся патчить Makefile пакета. Если повезёт, всё может свестись к передаче дополнительного флага сценарию конфигурации. Так обстоит дело с пакетами, использующими GNU autoconf:

```
CONFIGURE_ARGS+= --sysconfdir=${PKG_SYSCONFDIR}
```

Учтите, что так задаётся место, где пакет должен *искать* свои файлы конфигурации, а не место, куда они будут установлены. К счастью, для второго есть другой способ, описанный в следующем разделе, хотя на первый взгляд это сочетание довольно запутанно.

20.2.3. Патчинг установки

Как обсуждалось выше, **сами пакеты не должны напрямую изменять содержимое** `${PKG_SYSCONFDIR}`. Плохая новость: многие сценарии установки программного обеспечения из коробки вмешиваются в содержимое этого каталога. Так как же правильно исправить эту проблему?

Нужно научить пакет (обычно путём ручного патчинга) устанавливать все файлы конфигурации в иерархию examples — `share/examples/${PKGBASE}/`. Тогда `PLIST` зарегистрирует их, а у администратора всегда будут под рукой исходные копии.

Оказывается, для пакетов, использующих GNU autoconf, можно получить нужный эффект, задав `sysconfdir` в командной строке make во время установки. Рассмотрим пример из `mail/mutt`:

```
EGDIR=                  ${PREFIX}/share/examples/mutt
INSTALL_MAKE_FLAGS=     ${MAKE_FLAGS} sysconfdir=${EGDIR}
```

Учтите, что переменная `EGDIR`, хотя обычно и используется для этой цели, является локальной для этого пакета и вне его смысла не имеет.

20.2.4. Объявление файлов конфигурации

Когда нужные файлы конфигурации находятся на месте (то есть в иерархии examples), инфраструктура pkginstall может использовать их как эталонные копии при установке пакета, чтобы обновить содержимое `${PKG_SYSCONFDIR}`. Для этого используются переменные `CONF_FILES` и `CONF_FILES_PERMS`. Дополнительные сведения об их синтаксисе и назначении см. в разделе 20.1.2, «Работа с файлами». Вот пример из пакета `mail/mutt`:

```
EGDIR=                  ${PREFIX}/share/examples/mutt
CONF_FILES=             ${EGDIR}/Muttrc ${PKG_SYSCONFDIR}/Muttrc
```

Учтите, что (как и в примере из предыдущего раздела) переменная EGDIR специфична для пакета и вне его смысла не имеет.

Для справки, полный пример из Mutt выглядит так:

```
CONFIGURE_ARGS+=        --sysconfdir=${PKG_SYSCONFDIR}
EGDIR=                  ${PREFIX}/share/examples/mutt
CONF_FILES=             ${EGDIR}/Muttrc ${PKG_SYSCONFDIR}/Muttrc
INSTALLATION_DIRS+=     ${EGDIR}
INSTALL_MAKE_FLAGS=     ${MAKE_FLAGS} sysconfdir=${EGDIR}
```

20.2.5. Отключение обработки файлов конфигурации

Автоматическое копирование файлов конфигурации можно включить или отключить, задав переменную среды PKG_CONFIG перед установкой пакета.

20.3. Сценарии запуска системы

Сценарии запуска системы — особые файлы, поскольку они должны устанавливаться в место, известное базовой операционной системе, обычно вне префикса установки. Поэтому действуют те же правила, что описаны в разделе 20.1, «Файлы и каталоги вне префикса установки», и можно применять те же решения. Однако pkginstall предоставляет специальный механизм для работы с такими файлами.

Чтобы предоставить сценарии запуска системы, пакет должен:

1. Хранить сценарий в ${FILESDIR} с добавленным суффиксом .sh. Например, пакет print/cups-base содержит в своём каталоге files файл cupsd.sh.
2. Сообщить pkginstall, что его нужно обрабатывать, добавив имя сценария без расширения в переменную RCD_SCRIPTS. Продолжая предыдущий пример:

   ```
   RCD_SCRIPTS+=   cupsd
   ```

После этого pkginstall автоматически выполнит для каждого сценария следующие шаги:

1. Обработает файл, найденный в каталоге files, применив все подстановки, описанные в переменной FILES_SUBST.
2. Скопирует сценарий из каталога files в иерархию examples — ${PREFIX}/share/examples/rc.d/. Учтите, что этот эталонный файл должен быть явно зарегистрирован в PLIST.
3. Добавит в сценарии установки код, копирующий сценарий запуска из иерархии examples в общесистемный каталог сценариев запуска.

20.3.1. Отключение обработки сценариев запуска системы

Автоматическое копирование файлов конфигурации можно включить или отключить, задав переменную среды PKG_RCD_SCRIPTS перед установкой пакета. Учтите, что сценарии всегда будут копироваться в иерархию examples — ${PREFIX}/share/examples/rc.d/, — каким бы ни было значение этой переменной.

20.4. Системные пользователи и группы

Если пакету нужно создать специальных пользователей и (или) группы при установке, он может сделать это с помощью инфраструктуры pkginstall.

Пользователей можно создавать, добавляя записи в переменную `PKG_USERS`. Каждая запись имеет следующий синтаксис:

```
user:group
```

Дополнительные сведения о пользователе можно задать с помощью переменных, специфичных для каждого пользователя. `PKG_UID.user` — числовой UID пользователя. `PKG_GECOS.user` — описание или комментарий пользователя. `PKG_HOME.user` — домашний каталог пользователя; если он не задан, по умолчанию используется `/nonexistent`. `PKG_SHELL.user` — оболочка пользователя; если она не задана, по умолчанию используется `/sbin/nologin`.

Аналогично группы можно создавать, добавляя записи в переменную `PKG_GROUPS`, синтаксис которой:

```
group
```

Числовой GID группы можно задать, определив `PKG_GID.group`.

Если пакету нужно создать пользователей и группы на более раннем этапе, он может задать `USERGROUP_PHASE` равной `configure`, `build` или `pre-install`, указав этап, до которого создаются пользователи и группы. В этом случае числовые UID и GID созданных пользователей и групп автоматически жёстко прописываются в итоговых сценариях установки.

20.5. Системные оболочки

Пакеты, устанавливающие системные оболочки, должны регистрировать их в базе данных оболочек `/etc/shells`, чтобы облегчить жизнь администратору. Это нужно делать из сценариев установки, чтобы двоичные пакеты работали в любой системе. pkginstall предоставляет простой способ выполнить эту задачу.

Когда пакет предоставляет интерпретатор оболочки, он должен задать переменную `PKG_SHELL` равной абсолютному имени файла интерпретатора. Это добавит в сценарии установки некоторые перехватчики для его обработки. Рассмотрим следующий пример из `shells/zsh`:

```
PKG_SHELL=      ${PREFIX}/bin/zsh
```

20.5.1. Отключение регистрации оболочек

Администратор может отключить автоматическую регистрацию интерпретаторов оболочек, установив переменную среды `PKG_REGISTER_SHELLS` в `NO`.

20.6. Шрифты

Пакеты, устанавливающие шрифты X11, должны обновлять файлы баз данных, индексирующие шрифты в каждом каталоге шрифтов. Это легко сделать в рамках инфраструктуры pkginstall.

Когда пакет устанавливает шрифты X11, он должен перечислить каталоги, в которые устанавливаются шрифты, в переменных `FONTS_DIRS.type`, где *`type`* может быть одним из значений

«„ttf“», «„type1“» или «„x11“». Это добавит в сценарии установки перехватчики, запускающие нужные команды для обновления файлов баз данных шрифтов в каждом из этих каталогов. Для удобства, если путь к каталогу относительный, он считается относительным префикса установки пакета. Рассмотрим следующий пример из `fonts/dbz-ttf`:

```
FONTS_DIRS.ttf= ${PREFIX}/share/fonts/X11/TTF
```

20.6.1. Отключение автоматического обновления баз данных шрифтов

Администратор может отключить автоматическое обновление баз данных шрифтов, установив переменную среды `PKG_UPDATE_FONTS_DB` в `NO`.

Глава 21. Как заставить ваш пакет работать

21.1. Общие принципы работы

Одна из привлекательных черт pkgsrc — работа на множестве различных платформ. Поэтому важно, насколько это возможно, обеспечивать переносимость пакетов в pkgsrc. В этой главе упоминаются некоторые особенности, на которые следует обращать внимание при работе с pkgsrc.

21.1.1. Как подхватить переменные, задаваемые пользователем, из `mk.conf`

Пользователь pkgsrc может настраивать pkgsrc, переопределяя ряд переменных в файле, на который указывает `MAKECONF`, — по умолчанию это `mk.conf`. Если вы хотите использовать эти переменные в директивах препроцессора make(1) (например, `.if` или `.for`), перед этим нужно включить файл `../../mk/bsd.prefs.mk`, который, в свою очередь, загружает пользовательские настройки.

Но учтите, что некоторые переменные могут оказаться определёнными не полностью после включения `../../mk/bsd.prefs.mk`, поскольку они могут содержать ссылки на ещё не определённые переменные. В командах оболочки (строках `Makefile`, начинающихся с табуляции) это не проблема, так как переменные раскрываются только при использовании. Но в упомянутых выше директивах препроцессора и в строках зависимостей (вида `target: dependencies`) переменные раскрываются во время загрузки.

> **ПРИМЕЧАНИЕ**
>
> Чтобы проверить, можно ли использовать переменную во время загрузки, запустите **pkglint -Wall** для своего пакета.

21.1.2. Взаимодействие с пользователем

Иногда пакетам требуется взаимодействие с пользователем, и это может происходить по-разному:

- При получении дистрибутивных файлов некоторым пакетам требуется участие пользователя: ввод имени и пароля или принятие лицензии на веб-странице.
- При распаковке дистрибутивных файлов некоторые пакеты могут запрашивать пароли.
- помощь в настройке пакета перед сборкой
- помощь в процессе сборки
- помощь при установке пакета

Пакет может задать переменную `INTERACTIVE_STAGE`, чтобы определить, на каких этапах требуется взаимодействие. Это следует делать в `Makefile` пакета, например:

```
INTERACTIVE_STAGE=      configure install
```

После этого пользователь может решить пропустить такой пакет, задав переменную `BATCH`. Пакеты, требующие взаимодействия, также исключаются из массовых сборок.

21.1.3. Работа с лицензиями

Авторы программного обеспечения могут выбирать лицензию, по которой его можно копировать. Фонд свободного программного обеспечения (Free Software Foundation) объявил некоторые лицензии «свободными», а инициатива Open Source Initiative даёт определение «открытого исходного кода» (Open Source).

По умолчанию pkgsrc разрешает сборку пакетов со свободными лицензиями или лицензиями открытого исходного кода. Чтобы разрешить сборку пакетов и с другими лицензиями, пользователь pkgsrc должен добавить эти лицензии в переменную `ACCEPTABLE_LICENSES` в `mk.conf`. Учтите, что эта переменная влияет только на то, какие пакеты можно *собирать*, тогда как условия лицензии часто также ограничивают фактическое использование пакета и его распространение.

Может возникнуть желание устанавливать только пакеты с лицензией BSD или GPL, но не с другой. Свободные лицензии добавляются в переменную `ACCEPTABLE_LICENSES` по умолчанию. Пользователь pkgsrc может переопределить значения по умолчанию, задав переменную `ACCEPTABLE_LICENSES` с помощью «=» вместо «+=». Лицензии, принимаемые по умолчанию, определены в переменной `DEFAULT_ACCEPTABLE_LICENSES` в файле `pkgsrc/mk/license.mk`.

Механизм меток лицензий предназначен для решения вопросов авторского права, связанных со сборкой, установкой и использованием пакета, а не для вопросов распространения (см. `RESTRICTED` и `NO_SRC_ON_FTP` и т. п.). Пакеты с ограничениями на распространение должны устанавливать эти метки.

Указание на то, что пакет можно копировать по определённой лицензии, делается так: лицензия помещается в `pkgsrc/licenses`, а переменной `LICENSE` присваивается строка, идентифицирующая лицензию, например в `graphics/xv`:

```
LICENSE=        xv-license
```

При попытке сборки пользователь получит уведомление о том, что пакет распространяется по лицензии, которая не была внесена в переменную `ACCEPTABLE_LICENSES`:

```
% make
===> xv-3.10anb9 has an unacceptable license: xv-license.
===>     To view the license, enter "/usr/bin/make show-license".
===>     To indicate acceptance, add this line to your /etc/mk.conf:
===>     ACCEPTABLE_LICENSES+=xv-license
*** Error code 1
```

Если пакету требуется несколько лицензий, можно добавить их все в переменную LICENSE, соединив ключевым словом в верхнем регистре `AND`. Если пользователь может выбрать одну из нескольких лицензий, можно соединить их ключевым словом в верхнем регистре `OR`, например:

```
LICENSE=        isc AND apache-2.0
LICENSE=        2-clause-bsd OR ruby-license
```

Лицензию можно просмотреть с помощью **make show-license**, и, если пользователь того пожелает, напечатанную выше строку можно добавить в `mk.conf`, чтобы сообщить pkgsrc, что в дальнейшем он не должен завершаться ошибкой из-за этой лицензии:

```
ACCEPTABLE_LICENSES+=xv-license
```

Использование `LICENSE=shareware`, `LICENSE=no-commercial-use` и подобных формулировок устарело, поскольку они не указывают чётко на конкретный текст лицензии. Ещё одна проблема такого использования в том, что оно не позволяет пользователю сказать pkgsrc, чтобы тот продолжил работу для одного пакета, не сообщая при этом pkgsrc продолжать для всех пакетов с этой меткой.

21.1.3.1. Добавление пакета с новой лицензией

При добавлении пакета с новой лицензией необходимо выполнить следующие шаги:

1. Проверьте, является ли лицензия свободной или относится к открытому исходному коду, по страницам Различные лицензии и комментарии к ним и Лицензии по названиям | Open Source Initiative. Если это так, имя файла в `pkgsrc/licenses/` не нуждается в суффиксе `-license`, а имя лицензии следует добавить в:
   - DEFAULT_ACCEPTABLE_LICENSES в `pkgsrc/mk/license.mk`
   - default_acceptable_licenses в `pkgsrc/pkgtools/pkg_install/files/lib/license.c`
2. Текст лицензии следует добавить в `pkgsrc/licenses` для отображения. Список известных лицензий можно увидеть в этом каталоге.

21.1.3.2. Изменение лицензии

Когда лицензия меняется (не только по оформлению), убедитесь, что у новой лицензии другое имя (например, добавьте номер версии, если он есть, или дату). То, что пользователь разрешил pkgsrc собирать программы по предыдущей версии лицензии, вовсе не означает, что pkgsrc должен собирать программы по новой лицензии. Более общий момент: pkgsrc не оценивает лицензии на разумность; единственная проверка — механическая: одобрен ли конкретный текст одной из двух организаций (FSF или OSI).

21.1.4. Пакеты с ограничениями

Некоторые лицензии ограничивают повторное распространение программного обеспечения. Объявив эти ограничения, инструменты работы с пакетами могут автоматически воздерживаться, например, от размещения двоичных пакетов на FTP-сайтах.

Возможны четыре ограничения — все сочетания того, что исходные тексты (дистрибутивные файлы) и двоичные файлы не размещаются на FTP-сайтах и компакт-дисках. Поскольку в лицензиях редко встречаются именно такие формулировки, а несвободные лицензии обычно различаются, pkgsrc принимает собственное определение FTP и CD-ROM. «FTP» означает предоставление исходного или двоичного файла через Интернет бесплатно. «CD-ROM» означает предоставление исходных текстов или двоичных файлов на каком-либо носителе вместе с другими исходными и двоичными пакетами, который продаётся за плату за распространение.

Чтобы закодировать эти ограничения, система пакетов определяет пять переменных make, которые можно задать для отметки этих ограничений:

- `RESTRICTED`

  Эту переменную следует задавать всякий раз, когда есть ограничение (независимо от его вида). Присвойте этой переменной строку с причиной ограничения. Следует понимать, что желающим разобраться в ограничении придётся прочитать лицензию и, возможно, обратиться за консультацией к юристу.

- `NO_BIN_ON_CDROM`

  Двоичные файлы нельзя размещать на компакт-диске, содержащем другие двоичные пакеты, за который может взиматься плата за распространение. В этом случае задайте этой переменной

значение ${RESTRICTED}.

- NO_BIN_ON_FTP

  Двоичные файлы нельзя предоставлять в Интернете бесплатно. В этом случае задайте этой переменной значение ${RESTRICTED}. Если эта переменная задана, двоичные пакеты не будут включены на ftp.NetBSD.org.

- NO_SRC_ON_CDROM

  Дистрибутивные файлы нельзя размещать на компакт-диске вместе с другими дистрибутивными файлами, за который может взиматься плата. В этом случае задайте этой переменной значение ${RESTRICTED}.

- NO_SRC_ON_FTP

  Дистрибутивные файлы нельзя предоставлять по FTP бесплатно. В этом случае задайте этой переменной значение ${RESTRICTED}. Если эта переменная задана, дистрибутивные файлы не будут зеркалироваться на ftp.NetBSD.org.

Обратите внимание: пакеты удаляются из pkgsrc, если дистрибутивные файлы не подлежат распространению и их нельзя получить в течение одного полного квартального цикла. Пакеты с ручным (интерактивным) получением файлов должны иметь сопровождающего, и именно на нём лежит ответственность за это.

21.1.5. Работа с зависимостями

Ваш пакет может зависеть от присутствия другого пакета, и выразить эту зависимость можно разными способами. pkgsrc поддерживает определения DEPENDS, BUILD_DEPENDS, TOOL_DEPENDS и TEST_DEPENDS, определение USE_TOOLS, а также зависимости через buildlink3.mk — предпочтительный способ работы с зависимостями, использующий перечисленные выше переменные. Дополнительные сведения см. в главе 18, *Методика buildlink*.

Основное различие в том, что определение DEPENDS регистрирует эту предпосылку в двоичном пакете, поэтому она будет подтянута при последующей установке двоичного пакета, а определения BUILD_DEPENDS, TOOL_DEPENDS и TEST_DEPENDS — нет, помечая зависимость, нужную только для сборки или тестирования итогового пакета. Дополнительные сведения см. также в главе 14, *Создание нового пакета pkgsrc с нуля*.

Это значит, что если пакет нужен только во время сборки или тестирования, его следует указать как TOOL_DEPENDS, BUILD_DEPENDS или TEST_DEPENDS. При кросс-компиляции TOOL_DEPENDS — это *собственные* пакеты, то есть пакеты для архитектуры, на которой выполняется сборка; BUILD_DEPENDS — это *целевые* пакеты, то есть пакеты для архитектуры, для которой выполняется сборка.

Формат определения DEPENDS, BUILD_DEPENDS, TOOL_DEPENDS и TEST_DEPENDS:

```
<pre-req-package-name>:../../<category>/<pre-req-package>
```

Обратите внимание, что «„pre-req-package-name“» может включать любые шаблоны номеров версий, распознаваемые pkg_info(1).

1. Если вашему пакету для сборки и работы нужны двоичные файлы или библиотеки другого пакета и у этого пакета есть файл buildlink3.mk, используйте его:

   ```
   .include "../../graphics/jpeg/buildlink3.mk"
   ```

2. Если вашему пакету двоичные файлы или библиотеки другого пакета нужны только для сборки и у этого пакета есть файл `buildlink3.mk`, используйте его:

```
.include "../../graphics/jpeg/buildlink3.mk"
```

   но задайте `BUILDLINK_DEPMETHOD.`*`jpeg`*`?=build`, чтобы сделать её только зависимостью времени сборки. Такой случай довольно редок.

3. Если вашему пакету для сборки нужны двоичные файлы из другого пакета, используйте определение `TOOL_DEPENDS`:

```
TOOL_DEPENDS+= itstool-[0-9]*:../../textproc/itstool
```

4. Если вашему пакету для сборки нужны статические библиотеки для компоновки, заголовочные файлы для включения и т. п. из другого пакета, используйте определение `BUILD_DEPENDS`.
5. Если вашему пакету нужна библиотека для компоновки, а файла `buildlink3.mk` нет, создайте его. Использования `DEPENDS` будет недостаточно, поскольку включаемые файлы и библиотеки окажутся скрыты от компилятора.
6. Если вашему пакету для правильной работы нужен некий исполняемый файл, а файла `buildlink3.mk` нет, это указывается с помощью переменной `DEPENDS`. Пакету `print/lyx` при работе нужно уметь запускать двоичный файл latex из пакета tex-latex-bin, и это задаётся так:

```
DEPENDS+=        tex-latex-bin-[0-9]*:../../print/tex-latex-bin
```

7. Если ваш пакет включает набор тестов, у которого есть дополнительные зависимости, нужные только для этой цели (часто его можно запустить как цель «„make test“»), используйте переменную `TEST_DEPENDS`.
8. В зависимостях пакетов можно использовать шаблоны. Учтите, что такие зависимости с шаблонами сохраняются при создании двоичных пакетов. Зависимость проверяется при установке двоичного пакета, и будет использован любой подходящий под шаблон пакет. К использованию зависимостей с шаблонами следует относиться осторожно.

   Следует использовать «„-[0-9]*“» вместо «„-*“», чтобы избежать потенциально неоднозначных совпадений, например когда «„tk-postgresql“» соответствует «„tk-*“» в `DEPENDS`.

   Шаблоны можно также использовать, чтобы указать, что пакет будет собираться только с определённой минимальной версией предварительно необходимого пакета:

```
DEPENDS+=       ImageMagick>=6.0:../../graphics/ImageMagick
```

   Это означает, что пакет будет собран с ImageMagick версии 6.0 или новее. Такая зависимость может быть оправданна, если, например, изменились параметры командной строки исполняемого файла.

   Если нужно зависеть от минимальных версий библиотек, задайте `BUILDLINK_API_DEPENDS.`*`pkg`* равной соответствующему шаблону до включения файла `buildlink3.mk`, например

```
BUILDLINK_API_DEPENDS.jpeg+=    jpeg>=9.0
.include "../../graphics/jpeg/buildlink3.mk"
```

При исправлениях безопасности, пожалуйста, обновляйте файл уязвимостей пакета. Дополнительные сведения см. в разделе 21.1.9, «Работа с пакетами, имеющими проблемы безопасности».

Если вашему пакету для сборки нужны файлы из другого пакета, добавьте соответствующие дистрибутивные файлы в `DISTFILES`, чтобы они распаковывались автоматически. Пример см. в пакете `print/ghostscript`. (Он рассчитывает на то, что исходные тексты jpeg присутствуют в виде исходников во время сборки.)

21.1.6. Работа с конфликтами с другими пакетами

Ваш пакет может конфликтовать с другими пакетами, которые пользователи, возможно, уже установили в своей системе, например если ваш пакет устанавливает тот же набор файлов, что и другой пакет в дереве pkgsrc.

Например, `x11/libXaw3d` и `x11/Xaw-Xpm` устанавливают одну и ту же разделяемую библиотеку, поэтому в `pkgsrc/x11/libXaw3d/Makefile` задаётся:

```
CONFLICTS=      Xaw-Xpm-[0-9]*
```

а в `pkgsrc/x11/Xaw-Xpm/Makefile`:

```
CONFLICTS=      libXaw3d-[0-9]*
```

pkg_add(1) умеет обнаруживать попытки установить пакеты, конфликтующие с уже имеющимися, и прерывать их. Однако во многих ситуациях это происходит слишком поздно. Менеджеры двоичных пакетов узнают о конфликте только при попытке установить пакет — уже скачав его и все его зависимости. Пользователи также могут потратить время на сборку пакета и его зависимостей, чтобы в конце обнаружить, что он конфликтует с другим установленным пакетом.

Чтобы избежать этих проблем, записи `CONFLICTS` следует добавлять во всех случаях, когда известно, что пакеты конфликтуют друг с другом. Эти записи `CONFLICTS` экспортируются в файлы pkg_summary(5) и используются менеджерами двоичных пакетов, чтобы сообщать пользователям, что пакеты нельзя установить в целевую систему.

21.1.7. Пакеты, которые нельзя или не следует собирать

Есть несколько причин, по которым пакету может быть предписано не собираться при определённых обстоятельствах. Если пакет собирается и работает на большинстве платформ, исключения следует отмечать с помощью `BROKEN_ON_PLATFORM`. Если пакет собирается и работает лишь на немногих платформах, вместо этого задайте `BROKEN_EXCEPT_ON_PLATFORM`. И `BROKEN_ON_PLATFORM`, и `BROKEN_EXCEPT_ON_PLATFORM` — это тройки ОС (ОС-версия-платформа), в которых можно использовать шаблоны в стиле glob.

Если пакет не подходит для некоторых платформ (в отличие от случая, когда он просто сломан), следует использовать другой набор переменных, поскольку это влияет на отчёты о сбоях и статистику. Если пакет подходит для большинства платформ, исключения следует отмечать с помощью `NOT_FOR_PLATFORM`. Если пакет подходит лишь для нескольких платформ (часто ровно для одной), вместо этого задайте `ONLY_FOR_PLATFORM`. И `ONLY_FOR_PLATFORM`, и `NOT_FOR_PLATFORM` представляют собой тройки ОС (OS-version-platform), в которых можно использовать подстановочные знаки в стиле glob.

Некоторые пакеты жёстко привязаны к конкретной версии операционной системы, например LKM или `sysutils/lsof`. Такие двоичные пакеты несовместимы с другими версиями ОС, и их следует загружать в отдельный каталог для каждой версии на FTP-сервере. Отмечайте такие пакеты, задавая `OSVERSION_SPECIFIC` в значение «yes». В настоящее время эта переменная не используется внутри системы пакетов, но может использоваться в будущем.

Если пакет нужно пропустить (например, потому что он предоставляет функциональность, уже имеющуюся в системе), задайте `PKG_SKIP_REASON` в виде описательного сообщения. Если пакет должен завершиться с ошибкой из-за невыполненных предварительных условий, задайте `PKG_FAIL_REASON` в виде описательного сообщения.

21.1.8. Пакеты, которые нельзя удалять после установки

Чтобы пакет нельзя было удалить после установки, в Makefile пакета следует задать определение `PKG_PRESERVE`. Оно будет перенесено в любой двоичный пакет, созданный из этой записи pkgsrc. «Сохранённый» пакет не будет удалён с помощью pkg_delete(1), если не использовать параметр «-f».

21.1.9. Работа с пакетами, имеющими проблемы безопасности

При обнаружении уязвимости это следует отметить в `pkgsrc/doc/pkg-vulnerabilities`. Записи в этом файле состоят из трёх частей:

- шаблон версии пакета
- тип уязвимости (по возможности скопируйте существующую запись)
- URL с дополнительной информацией о проблеме

Для шаблона версии пакета всегда используйте `<' для обозначения верхней границы (не `<='!). Это позволит избежать возможных проблем из-за несвязанных с исправлениями безопасности обновлений `PKGREVISION`. Нижние границы тоже можно добавлять, используя '>' или '>='. Например, «`foo>=1<1.2`» обозначит версии 1.0 (включительно) — 1.2 (исключительно) пакета «`foo`» как подверженные проблеме безопасности.

Записи всегда следует добавлять в конец файла.

Исправляя пакеты, изменяйте верхнюю границу соответствующей записи. Продолжая предыдущий пример: если исправление было перенесено в версию 1.1nb2, измените предыдущий шаблон на «`foo>=1<1.1nb2`».

Чтобы локально проверить шаблон версии пакета на соответствие `PKGNAME`, можно использовать команду **pkg_admin pmatch**.

URL должен быть как можно более постоянным и содержать как можно больше информации о проблеме. Предпочтительны записи CVE.

После фиксации этого файла попросите pkgsrc-security@NetBSD.org обновить файл на ftp.NetBSD.org.

После исправления уязвимости патчем следует увеличить её `PKGREVISION` (разумеется, это не требуется, если проблема исправлена использованием более нового выпуска ПО), а шаблон в файле pkg-vulnerabilities необходимо обновить.

Кроме того, если исправление должно быть применено к стабильной ветке pkgsrc, обязательно отправьте запрос на перенос исправления (pullup)!

Двоичные пакеты, уже находящиеся на ftp.NetBSD.org, будут обрабатываться полуавтоматически еженедельным заданием cron.

Если проблема безопасности оспаривается, обратитесь по адресу pkgsrc-security@NetBSD.org.

21.1.10. Как обрабатывать увеличение версий при исправлении существующего пакета

При внесении исправлений в существующий пакет может быть полезно изменить номер версии в `PKGNAME`. Чтобы избежать конфликта с будущими версиями от исходного автора, можно использовать суффиксы «nb1», «nb2», … для версий пакета, задавая `PKGREVISION=1` (2, …). Инструменты пакетов обрабатывают «nb» так же, как «.». Например,

```
DISTNAME=       foo-17.42
PKGREVISION=    9
```

даст в результате `PKGNAME` со значением «foo-17.42nb9». Если вы хотите использовать исходное значение `PKGNAME` без суффикса «nbX», например для задания `DIST_SUBDIR`, используйте `PKGNAME_NOREV`.

Когда выходит новый выпуск пакета, `PKGREVISION` следует удалить, например при новом минорном выпуске вышеуказанного пакета должно быть так:

```
DISTNAME=       foo-17.43
```

`PKGREVISION` следует увеличивать при любом нетривиальном изменении в результирующем двоичном пакете. Без увеличения `PKGREVISION` тот, у кого установлена предыдущая версия, никак не узнает, что его пакет устарел. Поэтому изменения без увеличения `PKGREVISION` по сути помечаются как «это настолько тривиально, что ни один разумный человек не захочет обновляться», и это примерный критерий того, когда увеличивать `PKGREVISION` уместно. Примеры изменений, не заслуживающих увеличения `PKGREVISION`:

- Изменение `HOMEPAGE`, `MAINTAINER` или комментариев в Makefile.
- Изменение переменных сборки, если результирующий двоичный пакет остаётся тем же.
- Изменение `DESCR`.
- Добавление `PKG_OPTIONS`, если параметры по умолчанию не меняются.

Примеры изменений, которые заслуживают увеличения `PKGREVISION`:

- Исправления безопасности
- Изменения или дополнения в файле патча
- Изменения в `PLIST`
- Зависимость изменена или переименована.

PKGREVISION также необходимо увеличивать, когда у зависимостей меняется ABI.

21.1.11. Подстановка текста переменных в файлах пакета (фреймворк SUBST)

Когда нужно заменить один и тот же текст в нескольких файлах или несколько раз в одном файле, поддерживать для этого файл патча неудобно. Здесь на помощь приходит фреймворк SUBST. Он предоставляет простой в использовании интерфейс для замены текста в файлах. Ему нужна лишь следующая информация:

- На каком этапе цикла сборки пакета должна происходить замена?
- В каких файлах должна происходить замена?
- Какой текст на что следует заменить?

Эта информация задаётся блоком переменных `SUBST`. Минимальный пример:

```
SUBST_CLASSES+=         paths
SUBST_STAGE.paths=      pre-configure
SUBST_FILES.paths=      src/*.c
SUBST_SED.paths=        -e 's,/usr/local,${PREFIX},g'
```

Если перевести это на обычный язык: на этапе pre-configure (то есть после применения патчей из каталога patches/ и до запуска сценария configure и проверки переносимости) заменить текст `/usr/local` содержимым переменной `PREFIX`.

Каждый блок SUBST начинается с добавления идентификатора в `SUBST_CLASSES` (обратите внимание на `+=`). Этот идентификатор пакет может выбирать свободно. Если когда-нибудь появятся совпадающие идентификаторы, инфраструктура pkgsrc это обнаружит и сразу завершит работу с ошибкой, так что о совпадении имён не беспокойтесь.

Кроме `SUBST_CLASSES`, все переменные в блоке SUBST параметризованы этим идентификатором. В оставшейся части раздела эти параметризованные переменные записываются как `SUBST_STAGE.*`.

```
SUBST_CLASSES+=         paths
SUBST_STAGE.paths=      pre-configure
SUBST_MESSAGE.paths=    Fixing absolute paths.
SUBST_FILES.paths=      src/*.c
SUBST_FILES.paths+=     scripts/*.sh
SUBST_SED.paths=        -e 's,"/usr/local,"${PREFIX},g'
SUBST_SED.paths+=       -e 's,"/var/log,"${VARBASE}/log,g'
SUBST_VARS.paths=       LOCALBASE PREFIX PKGVERSION
```

Чтобы получить полную картину подстановок SUBST, выполните **bmake show-all-subst**. Если что-то работает не так, как ожидалось, запустите pkglint для пакета — он обнаруживает несколько типичных ошибок, связанных с блоками SUBST. Если после этого останутся вопросы, загляните в `mk/subst.mk`.

21.1.11.1. Выбор момента выполнения подстановок

Переменная `SUBST_STAGE.*` принимает одно из значений {pre,do,post}-{extract,patch,configure,build,test,install}. Из них `pre-configure` используется чаще всего, и с большим отрывом. Наиболее популярные этапы в хронологическом порядке:

`post-extract`

Подстановки применяются сразу после извлечения дистрибутивных файлов. Выполнение **bmake extract** для пакета не оставит следов исходных файлов.

Если подстановка применяется к файлам, для которых в каталоге `patches/` также есть патч, это означает, что патчи будут вычислены на основе результата подстановки. Когда позже эти патчи отправят сопровождающему upstream, чтобы исправить их в исходном пакете, они могут уже не соответствовать тому, к чему привык автор upstream. Поэтому `pre-configure` часто является лучшим выбором.

`pre-configure`

Подстановки применяются после патчей из каталога `patches/`. Это позволяет выполнить для пакета **bmake patch**, после чего патчи можно отредактировать с помощью инструментов pkgvi и mkpatches из пакета `pkgtools/pkgdiff`.

При обновлении патчей полезно явно разделить **bmake patch** и **bmake configure** и редактировать патчи только между этими командами. Иначе подстановки из блока SUBST попадут в файл патча. Когда это происходит совсем очевидным образом, pkglint будет ругаться на

патчи, содержащие жёстко заданный `/usr/pkg` вместо правильного и предполагаемого `@PREFIX@`, но обнаружить он может только эти действительно очевидные случаи.

`do-configure`
: Этот этап следует использовать, только если пакет сам определяет действие `pre-configure` и подстановка должна выполняться после него. Типичные примеры — пакеты, которые используют этап `pre-configure`, чтобы заново сгенерировать сценарий configure GNU из `configure.ac`.

`post-configure`
: Этот этап используется для исправления ошибок, допущенных на этапе configure.

`pre-build`
: Этот этап следует использовать только для подстановок, явно связанных со сборкой пакета, а не для исправления конфигурации. Подстановки для имён путей (например, замена `/usr/local` на `${PREFIX}`) или имён пользователей (например, замена `@MY_USER@` на фактическое имя пользователя) должны выполняться на этапах pre-configure или post-configure.

`post-build`
: Так же как и в случае pre-build, этот этап следует использовать только для подстановок, явно связанных со сборкой пакета, а не для исправления конфигурации. Подстановки для имён путей (например, замена `/usr/local` на `${PREFIX}`) или имён пользователей (например, замена `@MY_USER@` на фактическое имя пользователя) должны выполняться на этапах pre-configure или post-configure.

  Типичное применение — обновление файлов pkg-config, чтобы они включали параметры компилятора rpath.

`pre-install`
: В целом этап install должен быть как можно проще. Как и на этапах pre-build и post-build, его не следует использовать для исправления имён путей или имён пользователей — это относится к pre-configure. Для применения подстановок на этом этапе есть лишь несколько законных случаев.

21.1.11.2. Выбор файлов, в которых выполняются подстановки

Переменная `SUBST_FILES.*` содержит список шаблонов имён файлов. Эти шаблоны задаются относительно `WRKSRC`, поскольку именно там происходит большинство подстановок. Типичный пример:

```
SUBST_FILES.path=       Makefile */Makefile */*/Makefile *.[ch]
```

Приведённые выше шаблоны, особенно последний, довольно широки. Реализация SUBST проверяет, что каждый упомянутый здесь шаблон имени файла даёт эффект. Например, если ни один из файлов `*/*/Makefile` не содержит искомого и заменяемого текста, этот шаблон избыточен и его следует убрать. По умолчанию фреймворк SUBST сообщит об этом ошибкой. Если заменяемый текст встречается лишь в части файлов, подходящих под один шаблон, а не во всех, это совершенно нормально, и фреймворк SUBST выведет лишь сообщение уровня INFO для этих файлов.

Если есть веская причина для избыточных шаблонов имён файлов, задайте `SUBST_NOOP_OK.*` в значение `yes`.

Ещё один популярный способ выбора файлов для подстановок — с помощью команды оболочки, например так:

```
C_FILES_CMD=            cd ${WRKSRC} && ${FIND} . -name '*.c'
```

```
SUBST_FILES.path=       ${C_FILES_CMD:sh}
```

Имя переменной `C_FILES_CMD` в этом примере выбирается свободно и не зависит от фреймворка SUBST.

В этом варианте переменная `SUBST_FILES.*` перечисляет каждый файл по отдельности. Из-за этого выше шанс, что среди шаблонов имён файлов найдутся такие, в которых подстановка не происходит. Поскольку фреймворк SUBST не может знать, были ли шаблоны имён файлов в `SUBST_FILES.*` явно перечислены в Makefile (где любой избыточный шаблон был бы подозрительным) или сгенерированы командой оболочки (где избыточные шаблоны более вероятны и ожидаемы), он будет сообщать о таких избыточных шаблонах. Поэтому блокам SUBST, которые используют команду оболочки для создания списка шаблонов имён файлов, часто требуется задавать `SUBST_NOOP_OK.*` в значение `yes`.

21.1.11.3. Выбор того, что подставлять

В большинстве случаев подстановки задаются с помощью одной или нескольких команд sed(1), например так:

```
SUBST_SED.path=         -e 's|/usr/local|${PREFIX}|g'
```

Перед каждой командой sed нужно указывать параметр `-e`, и каждую следует располагать на отдельной строке, чтобы короткие команды sed не скрывались в конце строки.

Поскольку команды sed часто содержат в качестве разделителя метасимволы оболочки (`|` в приведённом выше примере), их принято заключать в одинарные кавычки.

Обычная подстановка — замена заполнителей вида `@VARNAME@` на соответствующую переменную pkgsrc `${VARNAME}`. Типичный пример:

```
SUBST_VARS.path=        PREFIX
```

Такую подстановку обычно выполняют сценарии configure GNU на этапе do-configure, но в некоторых случаях её нужно переопределить. Тот же шаблон используется, когда пакет определяет патчи, которые сначала заменяют жёстко заданные пути вроде `/usr/local` заполнителем `@PREFIX@`, а затем на этапе pre-configure он заменяется фактическим `${PREFIX}`. Во многих из этих случаев столь же хорошо просто использовать фреймворк SUBST, чтобы напрямую заменить `/usr/local` на `${PREFIX}`, вообще отказавшись от промежуточного файла патча.

Если описанного выше недостаточно для гибкости, можно вообще не использовать sed для подстановки, а задать совершенно другую команду, например так:

```
SUBST_FILTER_CMD.path=  LC_ALL=C ${TR} -d '\r'
```

Это применяется для нескольких оставшихся пакетов, в которых распространяемые файлы используют окончания строк в стиле Windows, и их нужно преобразовать в окончания строк в стиле UNIX.

21.1.11.4. Другие переменные SUBST

Когда блок SUBST применяется во время сборки пакета, в журнал записывается сообщение. Сообщение по умолчанию подходит для большинства случаев, но его можно переопределить, задав `SUBST_MESSAGE.*` в виде отдельного сообщения.

21.2. Этап *fetch*

21.2.1. Пакеты, чьи дистрибутивные файлы недоступны для обычного скачивания

Если нужно скачивать с динамического URL, можно задать `DYNAMIC_MASTER_SITES`, и **make fetch** вызовет `files/getsite.sh`, передав в качестве аргумента имя каждого файла для скачивания, и ожидает, что он выведет URL каталога, из которого этот файл нужно скачать. Пример такого использования — `graphics/ns-cult3d`.

Если скачивание нельзя автоматизировать — потому что пользователь должен отправить личные данные, чтобы получить пароль, или оплатить исходный код, или что-то ещё, — можно задать `FETCH_MESSAGE` в виде списка строк, которые показываются пользователю перед прерыванием сборки. Пример:

```
FETCH_MESSAGE=  "Please download the files"
FETCH_MESSAGE+= "    "${DISTFILES:Q}
FETCH_MESSAGE+= "manually from "${MASTER_SITES:Q}"."
```

21.2.2. Как обрабатывать изменённые дистрибутивные файлы со «старым» именем

Иногда авторы программного пакета вносят изменения после выпуска программы и выкладывают новый дистрибутивный файл, не меняя номер версии пакета. Если пакет к тому времени уже есть в pkgsrc, контрольная сумма перестанет совпадать. Прежде чем что-либо менять, сравните содержимое нового дистрибутивного файла со старым, чтобы убедиться, что файл действительно обновлён намеренно и что в него не проник троянский конь или что-то подобное. Пожалуйста, укажите в сообщении коммита, что дистрибутивные файлы сравнивались и что при этом обнаружилось.

Тогда правильный способ обойти это — задать `DIST_SUBDIR` в виде уникального имени каталога, обычно основанного на `PKGNAME_NOREV` (но будьте осторожны с пакетами Python или Ruby, где `PKGNAME` содержит префикс-переменную). Все `DISTFILES` и `PATCHFILES` для этого пакета будут помещены в этот подкаталог локального каталога дистрибутивных файлов. (Подробности см. в разделе 21.1.10, «Как обрабатывать увеличение версий при исправлении существующего пакета».) Если это случается чаще, можно использовать `PKGNAME` (тем самым включая суффикс `nbX`) или добавить отметку даты, например `${PKGNAME_NOREV}-YYYYMMDD`.

`DIST_SUBDIR` также используется, когда имя дистрибутивного файла не содержит версии и сам файл может измениться. В случаях, когда вероятность этого очень мала, `DIST_SUBDIR` может не понадобиться. Кроме того, `DIST_SUBDIR` нельзя удалять, если не изменилось имя дистрибутивного файла, даже если пакет перемещается или переименовывается.

Не забудьте после этого заново сгенерировать файл `distinfo`, поскольку он содержит путь `DIST_SUBDIR` в именах файлов. Также увеличьте PKGREVISION, если установленный пакет отличается. Кроме того, уместно написать авторам пакета письмо о том, что менять дистрибутивные файлы после выпуска, не меняя имён файлов, — плохая практика.

21.2.3. Пакеты, размещённые на github.com

Для пакетов, размещённых на github.com, есть вспомогательные методы: их имена дистрибутивных файлов часто совпадают с именами других пакетов, например `1.0.tar.gz`. Используйте один из трёх рецептов ниже:

21.2.3.1. Скачивание на основе помеченного выпуска

Если URL вашего дистрибутивного файла похож на `https://github.com/username/example/archive/v1.0.zip`, значит вы оформляете помеченный выпуск.

```
DISTNAME=       example-1.0
MASTER_SITES=   ${MASTER_SITE_GITHUB:=username/}
#GITHUB_PROJECT= example    # can be omitted if same as DISTNAME
GITHUB_TAG=     v${PKGVERSION_NOREV}
EXTRACT_SUFX=   .zip
```

Здесь DISTNAME вместе с использованием GITHUB_TAG приводит к тому, что инфраструктура скачивания файлов сохраняет полученный файл локально как `example-1.0.zip`.

21.2.3.2. Скачивание на основе конкретного коммита до первого выпуска

Если ваш дистрибутивный файл похож на `https://github.com/username/example/archive/988881adc9fc3655077dc2d4d757d480b5ea0e11` и взят из коммита до первого выпуска, задайте версию пакета 0.0.0.N, где N — число коммитов в репозитории, а GITHUB_TAG — хеш коммита. Это (почти) гарантирует, что у первого помеченного выпуска версия будет больше этой, так что обновление пакетов будет работать правильно.

```
DISTNAME=       example-0.0.0.347
MASTER_SITES=   ${MASTER_SITE_GITHUB:=username/}
#GITHUB_PROJECT= example    # can be omitted if same as DISTNAME
GITHUB_TAG=     988881adc9fc3655077dc2d4d757d480b5ea0e11
```

21.2.3.3. Скачивание на основе конкретного коммита после выпуска

Если ваш дистрибутивный файл похож на `https://github.com/username/example/archive/988881adc9fc3655077dc2d4d757d480b5ea0e11` и взят из коммита после выпуска, включите в версию пакета версию последнего выпуска и число коммитов после этого выпуска, а GITHUB_TAG задайте равным хешу коммита. Последний выпуск и число коммитов показаны в выводе «git describe --tags»:

```
# git clone https://github.com/username/example
# cd example
# git describe --tags
1.2.3-5-g988881a
```

```
DISTNAME=       example-1.2.3.5
MASTER_SITES=   ${MASTER_SITE_GITHUB:=username/}
#GITHUB_PROJECT= example    # can be omitted if same as DISTNAME
GITHUB_TAG=     988881adc9fc3655077dc2d4d757d480b5ea0e11
```

21.2.3.4. Скачивание на основе выпуска

Если URL вашего дистрибутивного файла похож на `https://github.com/username/example/releases/download/rel-1.6/offensive-1.6.zip`, значит вы оформляете выпуск.

```
DISTNAME=       offensive-1.6
PKGNAME=        ${DISTNAME:S/offensive/proper/}
MASTER_SITES=   ${MASTER_SITE_GITHUB:=username/}
```

```
GITHUB_PROJECT= example
GITHUB_RELEASE= rel-${PKGVERSION_NOREV} # usually just set this to ${DISTNAME}
EXTRACT_SUFX=   .zip
```

21.3. Этап *configure*

21.3.1. Разделяемые библиотеки — libtool

pkgsrc поддерживает множество разных машин с разными форматами объектных файлов, такими как a.out и ELF, и с разными возможностями работы с разделяемыми библиотеками и динамической загрузкой. В связи с этим компилятору, компоновщику и т. д. приходится передавать разные команды и параметры, чтобы получить нужный результат, что может быть довольно утомительно, особенно если под рукой нет всех машин для проверки. Здесь может помочь пакет `devel/libtool`: он просто «знает», как собирать и статические, и динамические библиотеки из набора исходных файлов, оставаясь независимым от платформы.

Вот как использовать libtool в пакете за семь простых шагов:

1. Добавьте в Makefile пакета `USE_LIBTOOL=yes`.
2. Для объектов библиотеки используйте «${LIBTOOL} --mode=compile ${CC}» вместо «${CC}». Можно даже добавить это в определение `CC`, если в данном Makefile собираются только библиотеки. Эта одна команда соберёт объекты библиотеки как PIC, так и не-PIC, поэтому отдельные правила для разделяемой и неразделяемой библиотек вам не нужны.
3. Для компоновки библиотеки удалите все команды «ar», «ranlib» и «ld -Bshareable», а вместо них используйте:

```
${LIBTOOL} --mode=link \
    ${CC} -o ${.TARGET:.a=.la} \
        ${OBJS:.o=.lo} \
        -rpath ${PREFIX}/lib \
        -version-info major:minor
```

Учтите, что библиотека получает расширение `.la`, а объекты — расширение `.lo`. При необходимости измените `OBJS`. Это автоматически создаёт в каталоге сборки все символьные ссылки `.a`, `.so.major.minor` и ELF (если нужно). Обязательно указывайте «-version-info», особенно когда major и minor равны нулю, иначе libtool отбросит версию разделяемой библиотеки.

Из руководства libtool:

```
So, libtool library versions are described by three integers:

CURRENT
The most recent interface number that this library implements.

REVISION
The implementation number of the CURRENT interface.

AGE
The difference between the newest and oldest interfaces that
this library implements.  In other words, the library implements
all the interface numbers in the range from number `CURRENT -
AGE' to `CURRENT'.
```

```
If two libraries have identical CURRENT and AGE numbers, then the
dynamic linker chooses the library with the greater REVISION number.
```

Параметр «-release» даст разные результаты для a.out и ELF (не считая символьных ссылок) лишь в одном случае. Библиотека ELF вида «libfoo-release.so.*x*.*y*» получит на платформе a.out символьную ссылку «libfoo.so.*x*.*y*». Это обрабатывается автоматически.

Аргумент «-rpath» — это каталог установки собираемой библиотеки.

В `PLIST` включайте только файл `.la`, остальные файлы будут добавлены автоматически.

4. При компоновке общих объектных файлов (`.so`), то есть файлов, которые загружаются через dlopen(3), а НЕ разделяемых библиотек, используйте «-module -avoid-version», чтобы к ним не приписывалась версия.

   В файл `PLIST` попадает запись `foo.so`.

5. При компоновке программ, которые зависят от этих библиотек *до* их установки, поставьте перед строкой cc(1) или ld(1) «${LIBTOOL} --mode=link», и она найдёт нужные библиотеки (статические или разделяемые), но учтите, что libtool не позволит указать относительный путь в -L (например, «-L../somelib»), поскольку ожидает, что вы замените этот аргумент на файл `.la`. Например,

   ```
   ${LIBTOOL} --mode=link ${CC} -o someprog -L../somelib -lsomelib
   ```

   следует заменить на:

   ```
   ${LIBTOOL} --mode=link ${CC} -o someprog ../somelib/somelib.la
   ```

   и он правильно поступит с библиотеками.

6. При установке библиотек поставьте перед командой install(1) или cp(1) «${LIBTOOL} --mode=install» и измените имя библиотеки на `.la`. Например,

   ```
   ${LIBTOOL} --mode=install ${BSD_INSTALL_LIB} ${SOMELIB:.a=.la} ${PREFIX}/lib
   ```

   Это установит статическую `.a`, разделяемую библиотеку, необходимые символьные ссылки и запустит ldconfig(8).

7. В вашем `PLIST` включайте только файл `.la` (это изменение по сравнению с предыдущим поведением).

21.3.2. Использование libtool в пакетах GNU, которые уже поддерживают libtool

Добавьте в Makefile пакета `USE_LIBTOOL=yes`. В большинстве случаев это переопределит собственный libtool пакета. В старых пакетах, использующих libtool, libtool создаётся сценарием ltconfig на шаге do-configure; проверить расположение сценария libtool можно так: **make configure; find work*/ -name libtool**.

`LIBTOOL_OVERRIDE` задаёт, какие сценарии libtool (относительно `WRKSRC`) переопределять. По умолчанию она равна «libtool */libtool */*/libtool». Если это не соответствует расположению сценария (сценариев) libtool пакета, задайте её нужным образом.

Если вам не нужно, чтобы статические библиотеки `*.a` собирались и устанавливались, используйте вместо неё `SHLIBTOOL_OVERRIDE`.

Если ваш пакет использует независимую от платформы библиотеку для загрузки динамических общих объектов, поставляемую вместе с libtool (libltdl), включите devel/libltdl/buildlink3.mk.

Некоторые пакеты используют libtool неправильно, из-за чего пакет может не работать или не собираться в некоторых условиях. Вот некоторые из наиболее распространённых ошибок:

- Включение общего объекта (-module) в качестве зависимой библиотеки в исполняемый файл или библиотеку. Само по себе это не проблема, если выполнено одно из двух условий:
  1. Общий объект назван правильно, то есть `libfoo.la`, а не `foo.la`
  2. При компоновке исполняемого файла используется параметр -dlopen.
- Использование libltdl без правильных вызовов процедур инициализации. Следует вызывать функцию lt_dlinit() и включать в исполняемые файлы макрос `LTDL_SET_PRELOADED_SYMBOLS`.

21.3.3. GNU Autoconf/Automake

Если пакету нужно, чтобы для регенерации сценария `configure` и шаблонов makefile `Makefile.in` из `configure.ac` и `Makefile.am` выполнялись GNU autoconf или automake, их следует запускать в цели pre-configure:

```
USE_TOOLS+=     autoconf automake autoreconf
GNU_CONFIGURE=  yes
...

pre-configure:
        set -e; cd ${WRKSRC} && autoreconf -fi
...
```

Пакетам, использующим GNU Automake, иногда требуется GNU Make (`gmake` в `USE_TOOLS`), но не всегда. Учтите, что autoreconf нужно выполнять, только если изменены `configure.ac` или makefile-файлы, либо `configure` отсутствует.

Иногда процесс configure вносит дополнительные изменения в сгенерированные файлы, из-за чего процесс сборки пытается повторно выполнить последовательность automake. Это предотвращается обновлением времени изменения различных файлов на этапе configure. Если это создаёт проблемы с вашим пакетом, можно задать в Makefile пакета `AUTOMAKE_OVERRIDE=NO`.

21.3.4. Meson / ninja

Пакеты, использующие Meson для конфигурации, должны включать:

```
.include "../../devel/meson/build.mk"
```

Почти во всех случаях (любая программа с зависимостями) в `USE_TOOLS` нужно добавить pkg-config. Если пакет устанавливает файлы переводов для языков, отличных от английского, добавьте также msgfmt и xgettext:

```
USE_TOOLS+=     pkg-config msgfmt xgettext
```

Если в Meson нужно передать какие-либо параметры, используйте `MESON_ARGS` вместо `CONFIGURE_ARGS`:

```
MESON_ARGS+=    -Dx11=false
```

21.3.5. Определение порядка байтов

Чтобы определить порядок байтов целевой платформы, загляните в `mk/endian.mk`.

21.4. Языки программирования

21.4.1. C, C++ и Fortran

Компиляторы языков C и C++ входят в базовую систему NetBSD. По умолчанию pkgsrc считает, что пакет написан на C, и скрывает все остальные компиляторы (через фреймворк обёрток, см. главу 18, *Методика buildlink*).

Чтобы объявить, какие языки должны быть доступны через обёртки компилятора pkgsrc, используйте переменную `USE_LANGUAGES`. Допустимые значения на данный момент:

```
c, c++, fortran, fortran77, java, objc, obj-c++, and ada.
```

(и любые их комбинации). По умолчанию — «c». Пакетам, использующим сценарии configure GNU, даже если они написаны на C++, обычно нужен компилятор C на этапе configure.

Чтобы объявить, какие возможности требуются пакету от компилятора, задайте `USE_CC_FEATURES` или `USE_CXX_FEATURES`. Допустимые значения `USE_CC_FEATURES` на данный момент:

```
c11, c99, has_include
```

Допустимые значения `USE_CXX_FEATURES` на данный момент:

```
c++11, c++14, c++17, c++20, has_include, regex, filesystem,
charconv, parallelism_ts, unique_ptr, put_time,
is_trivially_copy_constructible
```

Учтите, что сейчас эти переменные влияют только на использование GCC, но не других компиляторов.

Варианты языка, такие как `gnu99` и `c++11`, можно указать в `FORCE_C_STD` и `FORCE_CXX_STD`, если пакет явно не задаёт `-std=...` при компиляции (то есть пакет предполагает, что компилятор по умолчанию использует C++11 или другой стандарт). Это распространённая ошибка в системах сборки upstream.

Допустимые значения `FORCE_C_STD` на данный момент:

```
c90, c99, c11, gnu90, gnu99, gnu11
```

Допустимые значения `FORCE_CXX_STD` на данный момент:

```
c++03, c++11, c++14, c++17, c++20,
gnu++03, gnu++11, gnu++14, gnu++17, gnu++20
```

Учтите, что сейчас эти переменные влияют только на использование GCC и Clang.

21.4.2. Java

Если программа написана на Java, используйте фреймворк Java в pkgsrc. Пакет должен включать `../../mk/java-vm.mk`. Этот фрагмент Makefile предоставляет следующие переменные:

- USE_JAVA определяет, добавляется ли зависимость сборки от JDK. Если USE_JAVA задана как «run», то от JDK будет только зависимость времени выполнения. По умолчанию — «yes», что также добавляет зависимость сборки от JDK.
- Задайте USE_JAVA2, чтобы объявить, что пакету нужна реализация Java2. Поддерживаемые значения — «yes», «1.4» и «1.5». «yes» принимает любую реализацию Java2, «1.4» требует версии 1.4 или выше, а «1.5» принимает только версии 1.5 или выше. По умолчанию эта переменная не задана.
- PKG_JAVA_HOME автоматически задаётся равной расположению используемой зависимости от реализации Java во время выполнения. Её можно использовать, чтобы задать JAVA_HOME правильным значением, если программе нужно, чтобы эта переменная была определена.

21.4.3. Go

Если программа написана на Go и имеет зависимости от других модулей Go, пакет должен включать ../../lang/go/go-module.mk.

1. Создайте список этих зависимостей командой **make clean && make patch && make show-go-modules > go-modules.mk**.
2. Поставьте .include "go-modules.mk" перед любыми другими .include.
3. Включите эти модули в distinfo с помощью **make makesum**.

21.4.4. Rust

Если программа написана на Rust и для сборки используется Cargo, пакет должен включать ../../lang/rust/cargo.mk.

1. Обычно она зависит от множества другого кода (распространяемого в так называемых crate). Создайте список этих зависимостей командой **make extract && make print-cargo-depends > cargo-depends.mk**.
2. Поставьте .include "cargo-depends.mk" перед любыми другими .include.
3. Включите эти модули в distinfo с помощью **make makesum**.

21.4.5. Пакеты, содержащие сценарии Perl

Если ваш пакет содержит интерпретируемые сценарии Perl, добавьте «perl» в переменную USE_TOOLS и задайте REPLACE_PERL, чтобы был установлен правильный путь к интерпретатору. REPLACE_PERL должен содержать список сценариев относительно WRKSRC, которые нужно скорректировать. Каждое вхождение */bin/perl в строке she-bang будет заменено полным путём к исполняемому файлу Perl.

Если нужна конкретная версия Perl, задайте переменной PERL5_REQD номер версии. По умолчанию — «5.0».

Информацию о работе с модулями Perl см. в разделе 21.6.6, «Пакеты, устанавливающие модули Perl».

Существует также переменная REPLACE_PERL6 для языка, который теперь называется Raku.

21.4.6. Пакеты, содержащие сценарии оболочки

REPLACE_SH, REPLACE_BASH, REPLACE_CSH и REPLACE_KSH можно использовать для замены she-bang оболочки в файлах. Пожалуйста, используйте подходящую из них, отдавая предпочтение REPLACE_SH, когда этой оболочки достаточно. Каждая из них должна содержать список сценариев относительно WRKSRC, которые нужно скорректировать. Каждое вхождение соответствующей оболочки в строке

she-bang будет заменено полным путём к исполняемому файлу оболочки. При использовании REPLACE_BASH не забудьте добавить bash в USE_TOOLS.

21.4.7. Другие языки программирования

Есть и другие подобные переменные REPLACE, например REPLACE_AWK для пакетов, содержащих сценарии awk, и REPLACE_R для R. У этих двух, как и у остальных упомянутых выше, действия определены централизованно в mk/configure/replace-interpreter.mk. Другие языки определяют действия этих переменных в своей собственной части дерева: например, REPLACE_PHP обрабатывается в lang/php/replace.mk, а REPLACE_PYTHON — в lang/python/application.mk. Для остальных языков обратитесь к mk-файлам в их каталогах (соглашение об именовании бывает разным) или проверьте список в приложении E, *Темы справки*.

В настоящее время особая обработка других языков в pkgsrc различается. Если пакет компилятора предоставляет файл buildlink3.mk, включите его; в противном случае просто добавьте (сборочную) зависимость от соответствующего пакета компилятора.

21.5. Этап *build*

Самые частые сбои при сборке пакета — когда некоторые платформы не предоставляют определённые заголовочные файлы, функции или библиотеки либо предоставляют функции в библиотеке, о которой автор исходного пакета не знал. Чтобы обойти это, в большинстве случаев можно переписать исходный код так, чтобы он не использовал отсутствующие функции или предоставлял функцию-замену.

21.5.1. Условная компиляция кода C и C++

Если пакет уже поставляется со сценарием configure GNU, предпочтительный способ исправить сбой сборки — изменить сценарий configure, а не код. В остальных случаях можно воспользоваться препроцессором C, который определяет некоторые макросы в зависимости от операционной системы и архитектуры оборудования, для которых идёт компиляция. Эти макросы можно проверять, например, так: #if defined(__i386). Почти у каждой операционной системы, архитектуры оборудования и компилятора есть собственный макрос. Например, если определены макросы __GNUC__, __i386__ и __NetBSD__, вы знаете, что используете NetBSD на процессоре, совместимом с i386, а ваш компилятор — GCC.

Список приведённых ниже макросов для оборудования и операционной системы зависит от используемого компилятора. Например, если вы хотите условно компилировать код на Solaris, не используйте __sun__, так как компилятор SunPro его не определяет. Используйте вместо него __sun.

21.5.1.1. Макросы препроцессора C для определения операционной системы

Чтобы различать конкретные версии NetBSD, следует использовать следующий код.

```
#ifdef __NetBSD__
#include <sys/param.h>
#if __NetBSD_Prereq__(9,99,17)
/* use a newer feature */
#else
/* older code */
#endif
#endif

#ifndef _WIN32
```

```
/* Unix-like specific code */
#endif
```

Чтобы отличить системы, производные от 4.4 BSD, от всех остальных, следует использовать следующий код.

```
#include <sys/param.h>
#if (defined(BSD) && BSD >= 199306)
/* BSD-specific code goes here */
#else
/* non-BSD-specific code goes here */
#endif
```

Можно также проверять следующие макросы:

```
Cygwin      __CYGWIN__
DragonFly   __DragonFly__
FreeBSD     __FreeBSD__
Haiku       __HAIKU__
IRIX        __sgi (TODO: get a definite source for this)
Linux       __linux
Mac OS X    __APPLE__
Minix3      __minix
NetBSD      __NetBSD__
OpenBSD     __OpenBSD__
Solaris     sun, __sun
```

21.5.1.2. Макросы препроцессора C для определения архитектуры оборудования

```
i386        i386, __i386, __i386__
x86-64      __amd64__, __x86_64__
ARM         __arm__
MIPS        __mips
SPARC       sparc, __sparc
PowerPC     __powerpc
```

21.5.1.3. Макросы препроцессора C для определения компилятора

```
GCC         __GNUC__ (major version), __GNUC_MINOR__
MIPSpro     _COMPILER_VERSION (0x741 for MIPSpro 7.41)
SunPro      __SUNPRO_C (0x570 for Sun C 5.7)
SunPro C++  __SUNPRO_CC (0x580 for Sun C++ 5.8)
```

21.5.2. Как обходить ошибки компилятора

Некоторые исходные файлы вызывают ошибки в компиляторе — в зависимости от сочетания версии компилятора и архитектуры и почти всегда в связи с включённой оптимизацией. Типичные симптомы — внутренние ошибки gcc или бесконечная компиляция файла.

Как правило, обходной путь включает проверку `MACHINE_ARCH` и версии компилятора с отключением оптимизации для такого сочетания файла, `MACHINE_ARCH` и компилятора.

В прошлом это было большой проблемой, но сейчас требуется редко, поскольку технологии компиляторов созрели. Если вам всё же нужно добавить обходной путь для конкретного компилятора, сделайте это в файле `hacks.mk` и опишите симптом и версию компилятора как можно подробнее.

21.5.3. No such file or directory

Иногда компиляция завершается с сообщением об ошибке вроде этого:

```
.../x11/gtk3/work/gtk+-3.24.12/gdk/gdktypes.h:35:10:
    fatal error: pango/pango.h: No such file or directory
```

Правильный способ исправить эту проблему зависит от типа заголовочного файла, что описано в следующих разделах.

21.5.3.1. Заголовочные файлы из других пакетов

Если по имени заголовочного файла видно, что он из другого пакета, этот другой пакет следует включить через фреймворк buildlink3.

Сначала посмотрите, есть ли этот заголовочный файл где-нибудь в каталоге buildlink3 внутри `WRKDIR`. В приведённом выше случае с отсутствующим заголовочным файлом Pango:

```
$ find work/.buildlink/ -print | grep -F pango/pango.h
```

В случае Pango вывод будет таким:

```
work/.buildlink/include/pango-1.0/pango/pango.h
```

Если бы файл `pango/pango.h` был помещён непосредственно в каталог `.buildlink`, он был бы найден автоматически. Однако там есть дополнительный компонент пути `pango-1.0`, а значит, командная строка компилятора должна содержать параметр вида `-I${BUILDLINK3_PREFIX.pango}/include/pango-1.0`. В большинстве случаев этот параметр генерируется сценарием configure, который можно изучить с помощью:

```
$ $ grep -o '[-]I[^[:space:]]*/pango[^[:space:]]*' work/*/Makefile
-I/usr/pkg/include/pango-1.0
-I/usr/pkg/include/pango-1.0
-I/usr/pkg/include/pango-1.0
-I/usr/pkg/include/pango-1.0
-I/usr/pkg/include/pango-1.0
```

Это выглядит хорошо. Эти параметры преобразуются обёрткой buildlink так, чтобы ссылаться на правильный путь внутри `work/.buildlink`.

Однако поскольку компиляция всё же не удаётся, изучите командные строки компилятора в `work/.work.log`, чтобы увидеть, включён ли параметр `-I` в конкретную командную строку.

Чтобы глубже проанализировать ситуацию, выполните **bmake build-env** — эта команда создаёт интерактивное окружение, близкое к реальному, со всеми командами-обёртками pkgsrc и переменными окружения. В нём попробуйте скомпилировать несколько простых примеров программ, использующих этот заголовочный файл.

21.5.3.2. Заголовочные файлы, генерируемые во время сборки

Если имя заголовочного файла, похоже, относится к самому пакету, а сборка выполняется с параллельными заданиями, в пакете могут быть необъявленные зависимости между файлами `.c` и `.h`, и файл C компилируется до того, как сгенерирован нужный ему заголовочный файл.

Чтобы узнать, выполняется ли сборка с параллельными заданиями, запустите **bmake show-all-build | grep JOBS**. Вывод будет выглядеть так:

```
usr   MAKE_JOBS=              7
pkg   MAKE_JOBS_SAFE          # undefined
def   _MAKE_JOBS_N=           7
```

В этом случае пользователь pkgsrc попросил pkgsrc собирать пакеты с 7 параллельными заданиями (`MAKE_JOBS`). Пакет мог бы отключить параллельную сборку, задав `MAKE_JOBS_SAFE` в значение `no`, но в данном случае он этого не сделал.

Чтобы выяснить, вызван ли сбой сборки параллельными сборками, сначала сохраните точное сообщение об ошибке и немного контекста — возможно, они понадобятся вам позже для сообщения об ошибке. Затем выполните:

```
MAKE_JOBS_SAFE=no bmake clean build
```

Если это удастся, сообщите об ошибке в пакете pkgsrc, указав точное сообщение об ошибке и содержимое вашего файла `mk.conf`.

21.5.3.3. Символьные ссылки

Pkgsrc не работает надёжно, если любой из каталогов `LOCALBASE`, `VARBASE` или `WRKDIR` содержит символьную ссылку. Начиная с 2019Q2 программа начальной загрузки pkgsrc не позволяет устанавливать pkgsrc в каталоги, основанные на символьных ссылках. Существующие установки pkgsrc при этом на символьные ссылки не проверяются.

Сообщения об ошибке «No such file or directory» — типичный симптом символьных ссылок, и выяснить, что настоящая причина именно в этом, довольно трудно.

21.5.3.4. Устаревшие рабочие каталоги

При сборке иерархии пакетов может случиться, что один пакет собран, а затем pkgsrc обновлён. Такая ситуация может вызвать разнообразные трудно диагностируемые ошибки сборки. Чтобы исправить положение:

```
$ (cd ../../ && cat mk/bsd.pkg.mk >/dev/null && rm -rf */*/work)
```

(Единственное назначение `bsd.pkg.mk` — не дать выполнить эту команду в неправильном каталоге.)

Если вы задали `WRKOBJDIR` в `mk.conf`, удалите и этот каталог.

21.5.3.5. Другие возможные причины

На платформах, отличных от BSD, сторонние пакеты устанавливаются в `/usr/include` вместе с базовой системой. Это означает, что pkgsrc не может отличить заголовочные файлы, предоставляемые базовой системой (которые ему нужны), от заголовочных файлов сторонних пакетов (которые тоже часто включены в pkgsrc). Это может приводить к трудноуловимым несоответствиям версий.

В установках pkgsrc, работающих уже несколько лет, может случиться, что некоторые файлы были удалены вручную. Чтобы исключить эту маловероятную причину, выполните **pkg_admin check**.

Может помочь выполнение **pkg_admin rebuild-tree** для проверки и исправления зависимостей.

Если ничего из перечисленного не помогает, контактную информацию см. в главе 2, *Получение помощи*. Будьте готовы описать, что вы уже пробовали и какими были сообщения об ошибках.

21.5.4. Неопределённая ссылка на «...»

Это сообщение об ошибке часто означает, что пакет не скомпоновался с нужной ему разделяемой библиотекой. Известно, что следующие функции раз за разом вызывают это сообщение об ошибке.

Функция	Библиотека	Затронутые платформы
accept, bind, connect	-lsocket	Solaris
crypt	-lcrypt	DragonFly, NetBSD
dlopen, dlsym	-ldl	Linux
gethost*	-lnsl	Solaris
inet_aton	-lresolv	Solaris
nanosleep, sem_*, timer_*	-lrt	Solaris
openpty	-lutil	Linux

Чтобы исправить эти ошибки компоновщика, часто достаточно добавить `LIBS.`*`OperatingSystem`*`+= -l`*`foo`* в `Makefile` пакета, а затем выполнить **bmake clean; bmake**.

21.5.4.1. Компилятор SunPro и встроенные функции

Если вы используете компилятор SunPro, возможна ещё одна причина. Этот компилятор не справляется со следующим кодом:

```
extern int extern_func(int);

static inline int
inline_func(int x)
{
        return extern_func(x);
}

int main(void)
{
        return 0;
}
```

Он генерирует код для `inline_func`, даже если эта функция никогда не используется. Затем этот код ссылается на `extern_func`, которую обычно не удаётся разрешить. Чтобы решить эту проблему, можно попробовать указать пакету отключить встраивание функций.

21.5.4.2. Отсутствующие атомарные функции

При сборке для старых архитектур (например, i386, PowerPC) сборка может сбоить, потому что пакет ожидает современные 64-битные атомарные функции, которые нижележащее оборудование либо не поддерживает, либо поддерживает только с определёнными параметрами компилятора. Обычно это решается включением mk/atomic64.mk.

21.5.5. Нехватка памяти

Иногда пакеты не собираются, потому что компилятор упирается в определённый операционной системой программный предел. С помощью переменной `UNLIMIT_RESOURCES` можно указать pkgsrc, чтобы он снял ограничения на ресурсы. Допустимые значения — любая комбинация «cputime», «datasize», «memorysize», «stacksize» и «virtualsize». Задать эту переменную — то же самое, что

выполнить встроенную в оболочку команду **ulimit**, подняв соответственно максимальный размер сегмента данных или максимальный размер стека процесса до их жёстких пределов.

21.6. Этап *install*

21.6.1. Создание нужных каталогов

Совместимая с BSD команда **install**, поставляемая с некоторыми операционными системами, не может создать более одного каталога за раз. Поэтому вызывать `${INSTALL_*_DIR}` следует так:

```
${INSTALL_DATA_DIR} ${PREFIX}/dir1
${INSTALL_DATA_DIR} ${PREFIX}/dir2
```

Вместо прямого выполнения команд **install** можно также дописать «`dir1 dir2`» к переменной `INSTALLATION_DIRS` — тогда всё нужное произойдёт автоматически.

21.6.2. Куда устанавливать документацию

Как правило, документацию следует устанавливать в `${PREFIX}/share/doc/${PKGBASE}` или `${PREFIX}/share/doc/${PKGNAME_NOREV}` (последний вариант включает номер версии пакета).

Многие современные пакеты, использующие GNU autoconf, позволяют задать каталог установки документации HTML с помощью параметра «--with-html-dir». Иногда этот параметр необходим, иначе документация оказывается в `${PREFIX}/share/doc/html` или в других местах. В pkgsrc документация HTML должна попадать в каталог конкретного пакета, как и любая другая документация.

Исключение из сказанного выше — документация API библиотек, созданная инструментами `textproc/gtk-doc` для использования специальными браузерами (devhelp): её следует оставлять в расположении по умолчанию, то есть в `${PREFIX}/share/gtk-doc`. Такую документацию можно распознать по файлам с расширением `.devhelp` или `.devhelp2`. (Допустимо также устанавливать такие файлы в `${PREFIX}/share/doc/${PKGBASE}` или `${PREFIX}/share/doc/${PKGNAME}`; тогда файл `.devhelp*` должен находиться непосредственно в этом каталоге, дополнительный уровень подкаталогов в этом случае не допускается. Обычно этого добиваются с помощью «--with-html-dir=${PREFIX}/share/doc». Тем не менее предпочтителен `${PREFIX}/share/gtk-doc`.)

21.6.3. Установка файлов рекордов

Некоторые пакеты, большинство из них в категории games, устанавливают файл рекордов, позволяющий всем пользователям системы записывать свои лучшие результаты. Чтобы это работало, двоичные файлы нужно устанавливать с битом setgid, а файлы рекордов должны принадлежать соответствующей группе и/или владельцу (традиционно пользователю и группе «games»). Задайте `USE_GAMESGROUP` в значение yes, чтобы включить эту поддержку. Этим поведением управляют следующие переменные, более подробно описанные в `mk/defaults/mk.conf`: `GAMEDATAMODE`, `GAMEDIRMODE`, `GAMES_GROUP`, `GAMEMODE`, `GAME_USER`. Другие полезные переменные: `GAMEDIR_PERMS`, `GAMEDATA_PERMS` и `SETGID_GAMES_PERMS`.

Пример, иллюстрирующий некоторые из описанных выше переменных, — `games/moon-buggy`. Переменная `OWN_DIRS_PERMS` используется, чтобы правильно задать права доступа к каталогу, в котором сохраняется файл рекордов, `REQD_FILES_PERMS` — чтобы создать фиктивный файл рекордов (`mbscore`) с нужными правами, а `SPECIAL_PERMS` — чтобы установить игровой двоичный файл с битом setgid:

```
USE_GAMESGROUP=         yes

BUILD_DEFS+=            VARBASE

OWN_DIRS_PERMS+=        ${VARBASE}/games/moon-buggy ${GAMEDIR_PERMS}
REQD_FILES_PERMS+=      /dev/null ${VARBASE}/games/moon-buggy/mbscore ${GAMEDATA_PERMS}
SPECIAL_PERMS+=         ${PREFIX}/bin/moon-buggy ${SETGID_GAMES_PERMS}
```

Также доступны различные переменные `INSTALL_*`: `INSTALL_GAME` для установки игровых двоичных файлов с битом setgid, `INSTALL_GAME_DIR` для установки игровых каталогов, к которым должен иметь доступ процесс с битом setgid, и `INSTALL_GAME_DATA` для установки файлов рекордов.

Поэтому пакет никогда не должен жёстко задавать владельца файлов или права доступа, а должен полагаться на `*_PERMS`, как описано выше, либо на `INSTALL_GAME`, `INSTALL_GAME_DATA` и `INSTALL_GAME_DIR`, чтобы задать их правильно.

21.6.4. Добавление поддержки DESTDIR в пакеты

Поддержка `DESTDIR` означает, что пакет устанавливается в промежуточный каталог, а не в окончательное расположение файлов. Затем создаётся двоичный пакет, который можно использовать для установки как обычно. Есть два способа: либо пакет должен устанавливаться от имени root («destdir»), либо пакет может устанавливаться от имени обычного пользователя («user-destdir»).

- `PKG_DESTDIR_SUPPORT` нужно задать как «destdir» или «user-destdir». По умолчанию `PKG_DESTDIR_SUPPORT` задана как «user-destdir», чтобы помочь выявить больше потенциальных проблем упаковки. Если в Makefile включается bsd.prefs.mk, `PKG_DESTDIR_SUPPORT` нужно задать до этого включения.
- Все операции установки должны иметь префикс `${DESTDIR}`.
- automake в основном правильно обрабатывает DESTDIR автоматически. Многие правила, написанные вручную, а также шаги pre/post-install часто ошибочны; исправляйте их.
- Если файлы устанавливаются с особым владельцем или группой, используйте `SPECIAL_PERMS`.
- В целом пакеты должны поддерживать `UNPRIVILEGED`, чтобы иметь возможность использовать DESTDIR.

21.6.5. Пакеты с жёстко заданными путями к другим интерпретаторам

Ваш пакет может также содержать сценарии с жёстко заданными путями к другим интерпретаторам помимо Perl (или наряду с ним). Чтобы исправить полный путь к интерпретатору сценариев, нужно задать в вашем `Makefile` следующие определения (в этом примере мы будем использовать **tclsh**):

```
REPLACE_INTERPRETER+=   tcl
REPLACE.tcl.old=        .*/bin/tclsh
REPLACE.tcl.new=        ${PREFIX}/bin/tclsh
REPLACE_FILES.tcl=      # list of tcl scripts which need to be fixed,
# relative to ${WRKSRC}, just as in REPLACE_PERL
```

21.6.6. Пакеты, устанавливающие модули Perl

Makefile пакетов, предоставляющих модули perl5, должны включать фрагмент Makefile `../../lang/perl5/module.mk`. Он предоставляет цель **do-configure** для стандартной конфигурации

perl для таких модулей, а также различные хуки для настройки этой конфигурации. Подробности см. в комментариях в этом файле.

Модули perl5 будут устанавливаться в разные места в зависимости от версии perl, использованной при сборке. Чтобы учесть это, pkgsrc добавит в `PLIST` строки, соответствующие файлам, перечисленным в установленном файле `.packlist`, который создаёт большинство модулей perl5. Для этого нужно задать `PERL5_PACKLIST` в виде разделённого пробелами списка файлов packlist относительно `PERL5_PACKLIST_DIR` (по умолчанию `PERL5_INSTALLVENDORARCH`), например:

```
PERL5_PACKLIST= auto/Pg/.packlist
```

Переменные конфигурации perl5 `installarchlib`, `installscript`, `installvendorbin`, `installvendorscript`, `installvendorarch`, `installvendorlib`, `installvendorman1dir` и `installvendorman3dir` обозначают те места, в которые могут устанавливаться компоненты модулей perl5; они предоставляются как переменные в верхнем регистре с префиксом `PERL5_`, например `PERL5_INSTALLARCHLIB`, и могут использоваться пакетами perl5, у которых нет packlist. В `PLIST` эти переменные также подставляются в верхнем регистре с префиксом `PERL5_SUB_`.

21.6.7. Пакеты, устанавливающие файлы pkg-config

Некоторые пакеты, обычно те, что предоставляют библиотеки, устанавливают файлы pkg-config, чтобы их заголовочные файлы и библиотеки было легко найти. Имена таких файлов заканчиваются на `.pc`.

Чаще всего эти файлы предоставляют только параметры компоновки для библиотеки, но не включают параметры задания rpath, чтобы библиотеки можно было найти и во время выполнения. Поскольку это очень распространено, pkgsrc предоставляет для этого `PKGCONFIG_OVERRIDE`. Многие пакеты генерируют `.pc` из `.pc.in`; в этом случае добавьте такие файлы в переменную `PKGCONFIG_OVERRIDE`:

```
PKGCONFIG_OVERRIDE+=   foo.pc.in
```

Для пакетов, использующих meson, файлы генерируются во время сборки, и вам также нужно изменить этап, на котором выполняется замена. Например:

```
PKGCONFIG_OVERRIDE+=            output/meson-private/foo.pc
PKGCONFIG_OVERRIDE_STAGE=       post-build
```

21.6.8. Пакеты, устанавливающие файлы info

Некоторые пакеты устанавливают файлы info или используют команды «makeinfo» или «install-info». В Makefile пакета следует определить `INFO_FILES`, чтобы были созданы сценарии `INSTALL` и `DEINSTALL`, регистрирующие файлы info в файле каталога Info. Команда «install-info», используемая для регистрации файлов info, предоставляется либо системой, либо специальным пакетом, автоматически добавляемым как зависимость при необходимости.

`PKGINFODIR` — это каталог внутри `${PREFIX}`, где в первую очередь находятся файлы info. По умолчанию `PKGINFODIR` равна «info» и может быть переопределена пользователем.

Файлы info пакета должны быть перечислены в `PLIST` пакета; однако разбитые файлы info перечислять не нужно.

Пакет, которому при сборке нужна команда «makeinfo», должен добавить «makeinfo» в `USE_TOOLS` в своём Makefile. Если требуется минимальная версия команды «makeinfo», её следует указать с помощью переменной `TEXINFO_REQD` в файле `Makefile` пакета. По умолчанию требуется версия не ниже 3.12. Если система не предоставляет команду **makeinfo** или она не соответствует требуемому минимуму, автоматически будет добавлена зависимость сборки от пакета `devel/gtexinfo`.

Процесс сборки и установки программного обеспечения, предоставляемого пакетом, не должен использовать команду **install-info**, так как регистрация info-файлов — задача сценария `INSTALL` пакета, а он должен использовать соответствующую команду **makeinfo**.

Для достижения этой цели инфраструктура pkgsrc создаёт переопределяющие сценарии для команд **install-info** и **makeinfo** в каталоге, указанном в начале `PATH`.

Сценарий, переопределяющий **install-info**, не делает ничего, кроме записи сообщения в журнал. Сценарий, переопределяющий **makeinfo**, записывает сообщение и в зависимости от значения `TEXINFO_REQD` либо запускает соответствующую команду **makeinfo**, либо завершается с ошибкой.

21.6.9. Пакеты, устанавливающие страницы руководства (man)

Все пакеты, устанавливающие страницы руководства, должны устанавливать их в один и тот же каталог, чтобы было одно общее место для их поиска. В pkgsrc это `${PREFIX}/${PKGMANDIR}`, и именно это выражение следует использовать в пакетах. По умолчанию `PKGMANDIR` имеет значение «`man`». Другое часто используемое значение — «`share/man`».

> **ПРИМЕЧАНИЕ**
>
> Поддержка нестандартного `PKGMANDIR` далека от завершения.

В файлах `PLIST` можно просто использовать `man/` как каталог верхнего уровня для записей файлов страниц руководства, и система pkgsrc выполнит нужное преобразование. Во всех остальных местах необходимо использовать правильный `PKGMANDIR`.

Пакеты, в которых `GNU_CONFIGURE` имеет значение «yes», по умолчанию используют ключ --mandir команды `./configure`, чтобы задать, куда устанавливать страницы руководства. Путь задаётся переменной `GNU_CONFIGURE_MANDIR`, которая по умолчанию равна `${PREFIX}/${PKGMANDIR}`.

Пакеты, которые используют `GNU_CONFIGURE`, но не используют --mandir, могут установить `CONFIGURE_HAS_MANDIR` в значение «no». Если же сценарий `./configure` использует --mandir нестандартным образом, можно задать `GNU_CONFIGURE_MANDIR` по необходимости.

См. Раздел 19.5, «Сжатие страниц руководства», где описана установка сжатых страниц руководства.

21.6.10. Пакеты, устанавливающие шрифты X11

Если пакет устанавливает файлы шрифтов, вам нужно будет пересобирать базу данных шрифтов в каталоге, куда они устанавливаются, при установке и удалении. Это можно делать автоматически с помощью фреймворка pkginstall.

Вы можете перечислить каталоги, в которые устанавливаются шрифты, в переменных `FONTS_DIRS.`*`type`*, где *`type`* может быть одним из значений «ttf», «type1» или «x11». Также убедитесь, что файл базы данных `fonts.dir` не указан в PLIST.

Учтите, что не следует создавать новые каталоги для шрифтов; используйте стандартные, чтобы пользователю не приходилось вручную настраивать свой X-сервер для их поиска.

21.6.11. Пакеты, устанавливающие данные SGML или XML

Если пакет устанавливает файлы данных SGML или XML, которые нужно регистрировать в системных каталогах (таких как DTD, подкаталоги и т. п.), необходимо выполнить несколько дополнительных действий:

1. Включите `../../textproc/xmlcatmgr/catalogs.mk` в свой `Makefile` — он обеспечит регистрацию этих файлов в системных каталогах при установке и удалении.
2. Задайте в `SGML_CATALOGS` полный путь ко всем каталогам SGML, устанавливаемым пакетом.
3. Задайте в `XML_CATALOGS` полный путь ко всем каталогам XML, устанавливаемым пакетом.
4. Задайте в `SGML_ENTRIES` отдельные записи, которые нужно добавить в каталог SGML. Они задаются группами из трёх строк; подробности см. в xmlcatmgr(1) (а именно аргументы, распознаваемые действием «add»). Учтите, что обычно эта переменная не используется.
5. Задайте в `XML_ENTRIES` отдельные записи, которые нужно добавить в каталог XML. Они задаются группами из трёх строк; подробности см. в xmlcatmgr(1) (а именно аргументы, распознаваемые действием «add»). Учтите, что обычно эта переменная не используется.

21.6.12. Пакеты, устанавливающие расширения базы данных MIME

Если пакет предоставляет расширения базы данных MIME, устанавливая файлы `.xml` в `${PREFIX}/share/mime/packages`, необходимо выполнить несколько дополнительных действий, чтобы база данных оставалась согласованной с учётом этих новых файлов:

1. Включите `../../databases/shared-mime-info/mimedb.mk` (не используйте файл `buildlink3.mk` из этого же каталога — он предназначен для включения из других файлов `buildlink3.mk`). Он обеспечивает пересборку базы данных MIME при установке и удалении, а также запрещает прямой доступ к ней из пакета.
2. Проверьте PLIST и удалите все записи в каталоге `share/mime`, *кроме* файлов, сохранённых в `share/mime/packages`. Первые обрабатываются автоматически программой update-mime-database, а вторые зависят от пакета и должны удаляться тем пакетом, который их установил.
3. Удалите из PLIST все каталоги `share/mime/*`. Их обработкой займётся пакет shared-mime-info.

21.6.13. Пакеты, использующие intltool

Если пакет использует intltool при сборке, добавьте `intltool` в `USE_TOOLS` — это заставит его использовать пакет intltool, предоставляемый pkgsrc, вместо того, который входит в дистрибутивный файл.

Так отслеживаются зависимости intltool времени сборки и используется последняя доступная версия; благодаря этому пакет получает все исправления ошибок, появившиеся после его выпуска.

21.6.14. Пакеты, устанавливающие сценарии запуска

Если пакет содержит сценарий rc.d, по умолчанию он не будет скопирован в каталог запуска (`/etc/rc.d`), но копирование можно включить, задав параметр `PKG_RCD_SCRIPTS=YES` в `mk.conf`. При `PKG_RCD_SCRIPTS=YES` сценарии rc.d копируются в `/etc/rc.d` при установке пакета, но только если целевой файл ещё не существует. Копии в `/etc/rc.d` будут автоматически удалены только в том случае, если они не изменялись.

Учтите, что одного этого недостаточно для включения службы: её всё равно нужно добавить в `/etc/rc.conf`.

21.6.15. Пакеты, устанавливающие модули TeX

Если пакет устанавливает пакеты TeX в дерево texmf, базу данных `ls-R` этого дерева нужно обновить.

> **ПРИМЕЧАНИЕ**
>
> За исключением основных пакетов TeX, таких как kpathsea, пакеты должны устанавливать файлы в `${PREFIX}/share/texmf-dist`, а не в `${PREFIX}/share/texmf`.

1. Включите `../../print/kpathsea/texmf.mk`. Он обеспечивает пересборку базы данных `ls-R` при установке и удалении.
2. Если ваш пакет устанавливает файлы в дерево texmf, отличное от `${PREFIX}/share/texmf-dist`, задайте в `TEX_TEXMF_DIRS` список всех деревьев texmf, требующих обновления базы данных.

   Если ваш пакет также устанавливает файлы карт шрифтов, которые нужно зарегистрировать с помощью **updmap**, включите `../../print/tex-tetex/map.mk` и задайте в `TEX_MAP_FILES` и/или `TEX_MIXEDMAP_FILES` список всех таких файлов карт шрифтов. Тогда **updmap** будет запускаться автоматически при установке/удалении, чтобы включать или отключать файлы карт шрифтов для драйверов вывода TeX.
3. Убедитесь, что ни одна из баз данных `ls-R` не включена в `PLIST`, так как они удаляются только пакетом kpathsea.

21.6.16. Пакеты, обеспечивающие запуск двоичных файлов в режиме эмуляции

Есть пакеты, предоставляющие библиотеки и исполняемые файлы для запуска двоичных файлов одной операционной системы в другой (если последняя это поддерживает). Один из примеров — запуск двоичных файлов Linux в NetBSD.

Пакет `pkgtools/rpm2pkg` помогает распаковывать и упаковывать rpm-пакеты Linux.

`CHECK_SHLIBS` можно установить в no, чтобы избежать выполнения цели **check-shlibs**, проверяющей, находит ли динамический компоновщик все библиотеки для каждого установленного исполняемого файла. Поскольку при этом запускается стандартный динамический компоновщик, для пакетов эмуляции проверка не проходит, так как библиотеки, используемые эмуляцией, находятся не в стандартных каталогах.

21.6.17. Пакеты, устанавливающие значки hicolor

Если пакет устанавливает изображения в `share/icons/hicolor` и/или обновляет базу данных `share/icons/hicolor/icon-theme.cache`, необходимо выполнить несколько дополнительных действий, чтобы общий каталог темы обрабатывался правильно и база данных кэша пересобиралась:

1. Включите `../../graphics/hicolor-icon-theme/buildlink3.mk`.
2. Проверьте `PLIST` и удалите запись, относящуюся к кэшу темы.
3. Убедитесь, что PLIST не удаляет общие каталоги значков из иерархии `share/icons/hicolor`, поскольку они обрабатываются автоматически.

Лучший способ проверить, что PLIST верен с учётом последних двух пунктов, — пересоздать его с помощью **make print-PLIST**.

21.6.18. Пакеты, устанавливающие файлы desktop

Если пакет устанавливает файлы `.desktop` в `share/applications` и они содержат информацию MIME (ключ MimeType), необходимо выполнить дополнительные действия, чтобы они были

зарегистрированы в базе данных MIME:

1. Включите `../../sysutils/desktop-file-utils/desktopdb.mk`.
2. Проверьте PLIST и удалите запись, относящуюся к файлу `share/applications/mimeinfo.cache`. Он обрабатывается автоматически.

Лучший способ проверить, что PLIST верен с учётом последнего пункта, — пересоздать его с помощью **make print-PLIST**.

21.7. Пометка пакетов как проблемных

В некоторых случаях нет времени сразу решить проблему. Тогда пакет можно просто пометить как сломанный. Для этого достаточно задать переменной `BROKEN` значение, описывающее причину поломки пакета (аналогично переменной `PKG_FAIL_REASON`). Пользователь, пытающийся собрать пакет, сразу увидит это сообщение, и сборка даже не будет предпринята.

Пакеты со статусом `BROKEN` время от времени удаляются из pkgsrc.

Глава 22. Упаковка и портирование GNOME

Цитата с веб-сайта GNOME:

> *Проект GNOME предоставляет две вещи: среду рабочего стола GNOME — интуитивно понятный и привлекательный рабочий стол для пользователей, и платформу разработки GNOME — обширный фреймворк для создания приложений, интегрирующихся с остальным рабочим столом.*

pkgsrc обеспечивает простой способ автоматической сборки и установки полноценного окружения GNOME *на многих различных платформах*. Можно с уверенностью сказать, что pkgsrc — одна из самых продвинутых систем сборки и упаковки для GNOME благодаря включённым в неё технологиям buildlink3, фреймворку обёрток и инструментов и автоматическому управлению файлами конфигурации. Много усилий вкладывается в то, чтобы удаление установленных компонентов программного обеспечения было полностью чистым.

Поскольку pkgsrc — официальная система упаковки NetBSD, сказанное выше также означает, что прилагаются большие усилия, чтобы GNOME работал в этой операционной системе.

Эта глава адресована разработчикам pkgsrc и другим людям, желающим помочь в портировании и упаковке GNOME. В ней приведены инструкции по управлению существующими пакетами и некоторые важные сведения об их внутреннем устройстве.

22.1. Метапакеты

pkgsrc включает три метапакета, связанных с GNOME:

- `meta-pkgs/gnome-base`: предоставляет базовую среду рабочего стола GNOME. Он включает только необходимые компоненты для корректной загрузки, хотя в нём может не хватать важной функциональности для повседневной работы. Идея этого пакета в том, чтобы конечные пользователи строили свои конфигурации поверх него: сначала устанавливали этот метапакет для получения работоспособной системы, а затем добавляли отдельные приложения.
- `meta-pkgs/gnome`: обеспечивает полную установку платформы и рабочего стола GNOME в том виде, как их определяет проект GNOME; он основан на компонентах, распространяемых в каталогах `platform/x.y/x.y.z/sources` и `desktop/x.y/x.y.z/sources` официального FTP-сервера. Инструменты, предназначенные только для разработчиков и находящиеся в этих каталогах, не устанавливаются, если они не требуются для правильной работы какого-либо другого компонента. Аналогично пакеты из набора bindings (`bindings/x.y/x.y.z/sources`) не подключаются, если они не нужны как зависимость для компонента конечного пользователя. Этот пакет «расширяет» `meta-pkgs/gnome-base`.
- `meta-pkgs/gnome-devel`: устанавливает все инструменты, необходимые для сборки компонента GNOME, полученного из репозитория CVS. Они нужны, чтобы сценарии **autogen.sh** работали должным образом.

Во всех этих пакетах строки `DEPENDS` упорядочены так, чтобы облегчить обновления: пакет может зависеть от других пакетов, перечисленных перед ним, но не от перечисленных после него. Очень

важно сохранять этот порядок для удобства обновлений, поэтому… *не приводите его к алфавитному порядку!*

22.2. Упаковка приложения GNOME

Почти все приложения GNOME написаны на языке C и используют в качестве системы сборки общий набор инструментов. Иначе обстоит дело с новыми привязками к другим языкам (например, Python), но ниже даётся общее представление о минимально необходимых инструментах:

- Почти все приложения GNOME используют в качестве системы сборки GNU Autotools. Как правило, нужно сообщить об этом своему пакету:

```
GNU_CONFIGURE=  yes
USE_LIBTOOL=    yes
USE_TOOLS+=     gmake
```

- Если пакет использует pkg-config для определения зависимостей, добавьте этот инструмент в список требуемых утилит:

```
USE_TOOLS+=     pkg-config
```

  Также используйте `pkgtools/verifypc` в конце процесса сборки, чтобы убедиться, что вы не пропустили ни одной зависимости в своём пакете и что все требования к версиям указаны правильно.
- Если пакет использует intltool, обязательно добавьте `intltool` в `USE_TOOLS`, чтобы обрабатывать зависимости и заставить пакет использовать последнюю доступную версию.
- Если пакет использует gtk-doc (утилиту генерации документации), *не* добавляйте зависимость от неё. Этот инструмент довольно большой, а дистрибутивный файл в любом случае должен содержать заранее сгенерированную документацию; если это не так, это ошибка, о которой следует сообщить. Для таких пакетов нужно отключить gtk-doc (если это не значение по умолчанию):

```
CONFIGURE_ARGS+=        --disable-gtk-doc
```

  Расположение устанавливаемых HTML-файлов по умолчанию (`share/gtk-doc/<package-name>`) верно, и его не следует менять, если только пакет не настаивает на установке в другое место. Иначе такие программы, как **devhelp**, не смогут их открыть. Сделать это можно с помощью записи, подобной следующей:

```
CONFIGURE_ARGS+=        --with-html-dir=${PREFIX}/share/gtk-doc/...
```

GNOME использует несколько *общих* каталогов и файлов в префиксе установки для ведения баз данных. Здесь «общие» означает, что одни и те же каталоги и файлы используются несколькими разными пакетами, что приводит к конфликтам в `PLIST`. Сейчас pkgsrc включает функциональность для обработки наиболее частых случаев, поэтому вам придётся забыть об использовании строк `@unexec ${RMDIR}` в списках файлов и об исключении из них общих файлов. Если вы всё-таки это делаете, *скорее всего, ваш пакет неправильный*.

В следующей таблице перечислены типичные ситуации, приводящие к использованию общих каталогов или файлов. Для каждой из них приводится подходящее решение. Применив решение,

обязательно *пересоздайте список файлов пакета* с помощью **make print-PLIST** и убедитесь, что он верен.

Таблица 22.1. Обработка PLIST для пакетов GNOME

Если пакет…	То…
Устанавливает значки в иерархию `share/icons/hicolor` или обновляет `share/icons/hicolor/icon-theme.cache`.	См. Раздел 21.6.17, «Пакеты, устанавливающие значки hicolor».
Устанавливает файлы в `share/mime/packages`.	См. Раздел 21.6.12, «Пакеты, устанавливающие расширения базы данных MIME».
Устанавливает файлы `.desktop` в `share/applications`, и они содержат информацию MIME.	См. Раздел 21.6.18, «Пакеты, устанавливающие файлы desktop».

22.3. Обновление GNOME до более новой версии

Если рассматривать GNOME в целом, существует два вида обновлений:

Мажорное обновление

Учитывая, что до GNOME 3 ещё очень далеко (если он вообще появится), мажорным мы считаем обновление с версии `2.X` до версии `2.Y`, где `Y` чётное и больше `X`. Такие обновления выполнять сложно: они вносят множество изменений в код компонентов, и почти все дистрибутивные файлы GNOME обновляются до более новых версий. Некоторые из них могут даже нарушить совместимость API и ABI с предыдущей серией мажорных версий. В результате обновление приходится выполнять сразу целиком, чтобы свести поломки к минимуму.

Мажорное обновление обычно состоит примерно из 80 обновлений пакетов и добавления нескольких новых.

Минорное обновление

Минорным мы считаем обновление с версии `2.A.X` до версии `2.A.Y`, где `Y` больше `X`. Такие обновления выполнять легко: они не обновляют все компоненты GNOME, могут выполняться постепенно и не нарушают совместимость API или ABI.

Минорное обновление обычно состоит примерно из 50 обновлений пакетов, хотя эти числа могут сильно различаться.

Чтобы обновить компоненты GNOME в pkgsrc до нового стабильного выпуска (мажорного или минорного), следует выполнить следующие шаги:

1. Получите список всех архивов, составляющих новый выпуск, с помощью следующих команд. Они сохранят полный список дистрибутивных файлов компонентов в файле `list.txt`:

```
% echo ls "*.tar.bz2" | \
    ftp -V ftp://ftp.gnome.org/pub/gnome/platform/x.y/x.y.z/sources/ | \
    awk '{ print $9 }' >list.txt
% echo ls "*.tar.bz2" | \
    ftp -V ftp://ftp.gnome.org/pub/gnome/desktop/x.y/x.y.z/sources/ | \
    awk '{ print $9 }' >>list.txt
```

2. Откройте `Makefile` каждого метапакета и поднимите их версию до выпуска, до которого вы их обновляете. Все три метапакета должны всегда иметь согласованные версии. Разумеется, удалите

любые PKGREVISION, которые в них могут быть.

3. Для каждого метапакета обновите все его строки DEPENDS, чтобы они соответствовали последним версиям, указанным предыдущими командами. *Не* указывайте более новые версии (даже если они есть на FTP), потому что метапакеты должны перечислять точные версии, образующие конкретный выпуск GNOME. Исключения здесь допустимы, если более новая версия решает серьёзную проблему в работе рабочего стола в целом; обычно это выражается в поднятии ревизии в pkgsrc, а не в более новых версиях от разработчиков.

   Пакеты, не перечисленные в файле list.txt, следует обновить до последней доступной версии (если она есть в pkgsrc). Так обстоит дело, например, с зависимостями от GNU Autotools в метапакете meta-pkgs/gnome-devel.

4. Сгенерируйте патч из изменённых метапакетов и извлеките список «новых» строк. Это даст вам план того, какие пакеты нужно обновить в pkgsrc и в каком порядке:

```
% cvs diff -u gnome-devel gnome-base gnome | grep '^+D' >todo.txt
```

5. При мажорных обновлениях рабочего стола на этом этапе рекомендуется удалить все установленные пакеты и начать с нуля.
6. Теперь наступает самый длинный этап: пройдите по содержимому todo.txt и по порядку обновите перечисленные в нём пакеты. При мажорных обновлениях рабочего стола ничего из этого не следует коммитить, пока не завершён весь набор, так как есть вероятность сломать ещё не обновлённые пакеты.
7. Когда пакеты обновлены и работают, закоммитьте их в дерево по одному с подходящими сообщениями журнала. В конце закоммитьте обновления трёх метапакетов и все соответствующие изменения в файлах doc/CHANGES-<YEAR> и pkgsrc/doc/TODO.

22.4. Рекомендации по созданию патчей

GNOME — очень крупный компонент pkgsrc, насчитывающий около 100 пакетов. Очень важно, чтобы вы всегда, всегда, **всегда** передавали любые исправления переносимости, которые вы вносите в пакет GNOME, разработчикам основной ветки (см. Раздел 12.3.5, «Обратная связь с автором»). Это единственный способ привлечь их внимание к вопросам переносимости и гарантировать, что будущие версии можно будет собрать сразу после установки в NetBSD. Чем меньше собственных патчей в pkgsrc, тем проще дальнейшие обновления. Разработчики, отвечающие за выпуск мажорных обновлений GNOME, будут вам благодарны.

Чаще всего сообщения об ошибках направляют в GitLab GNOME и GitLab freedesktop.org. Не все компоненты используют их для отслеживания ошибок, но большинство — да. Не будьте кратки в своих отчётах: всегда подробно объясняйте текущий сбой, как его можно улучшить для максимальной переносимости и, если это вообще возможно, приложите патч относительно основной ветки Git. Чем подробнее вы опишете, тем выше шансы, что ваш патч примут.

Кроме того, пожалуйста, избегайте использования препроцессорной магии для исправления проблем переносимости. Хотя участники проекта GNOME во FreeBSD отлично справляются с портированием GNOME на свою операционную систему, официальные исходные тексты GNOME теперь наводнены условными конструкциями, проверяющими __FreeBSD__ и подобные макросы. Это вредит переносимости. Подробности см. в наших рекомендациях по созданию патчей (Раздел 12.3.4, «Рекомендации по созданию патчей»).

Глава 23. Отправка и коммит

23.1. Отправка двоичных пакетов

Наша политика состоит в том, что мы принимаем двоичные файлы только от разработчиков pkgsrc, чтобы гарантировать, что пакеты не содержат троянских программ и т. п. Это делается не для того, чтобы кого-то раздражать, а для защиты наших пользователей! Вы по-прежнему вольны выкладывать свои самодельные двоичные пакеты и рассказывать всем, откуда их взять. Разработчикам NetBSD, выполняющим массовые сборки и желающим их выложить, следует обратиться к Главе 8, *Создание двоичных пакетов для всего в pkgsrc (массовые сборки)*.

23.2. Отправка пакетов с исходным кодом (для тех, кто не является разработчиком NetBSD)

Во-первых, вы можете импортировать новые пакеты в pkgsrc-wip («pkgsrc work-in-progress»); подробности см. на домашней странице https://pkgsrc.org/wip/.

Затем проверьте, что ваш пакет полон, компилируется и хорошо работает; см. Главу 14, *Создание нового пакета pkgsrc с нуля* и остальную часть этого документа. Запустите инструмент `pkgtools/pkglint` и исправьте все появляющиеся ошибки.

Наконец, отправьте отчёт в систему отслеживания ошибок pkgsrc: либо командой send-pr(1), либо, если её нет, на веб-страницу https://www.NetBSD.org/support/send-pr.html, где есть некоторые инструкции и ссылка на форму, через которую можно отправить пакеты. В качестве замены любого из двух указанных выше инструментов также доступен пакет `sysutils/gtk-send-pr`.

В форме отчёта о проблеме категория должна быть «pkg», в кратком описании следует указать имя пакета и номер версии, а поле описания должно содержать короткое описание вашего пакета (подойдёт содержимое переменной COMMENT или файла DESCR).

23.3. Общие замечания при добавлении, обновлении и удалении пакетов

Пожалуйста, отмечайте все добавления, обновления, перемещения и удаления пакетов в `pkgsrc/doc/CHANGES-`*`YYYY`*. Очень важно поддерживать этот файл в актуальном состоянии и в существующем формате, потому что он используется сценариями для автоматического обновления страниц на www.NetBSD.org и других сайтах. Кроме того, проверьте файл `pkgsrc/doc/TODO` и удалите запись о пакете, который вы обновили или удалили, если она там упоминалась.

Когда повышается `PKGREVISION` пакета, изменение следует отразить в `pkgsrc/doc/CHANGES-`*`YYYY`*, если оно связано с безопасностью или иным образом существенно. Массовые повышения, вызванные обновлением зависимости, упоминать не нужно. Во всех остальных случаях решение принимает разработчик.

Есть цель make, помогающая создавать правильные записи в `CHANGES-`*`YYYY`*: **make changes-entry**. Она использует необязательные переменные `CTYPE` и `NETBSD_LOGIN_NAME`. Обычно нужно сначала убедиться, что ваш файл `CHANGES-`*`YYYY`* актуален (чтобы позже не пришлось разрешать конфликты), а затем перейти с помощью **cd** в каталог пакета. При обновлении пакета достаточно **make changes-entry**. Для новых пакетов, а также при перемещении или удалении пакетов задайте переменной `CTYPE` в командной строке значение «Added», «Moved» или «Removed». Если ваше локальное имя для

входа не совпадает с именем для входа в NetBSD, можно задать `NETBSD_LOGIN_NAME` в `mk.conf`. Кроме того, эта цель автоматически удаляет записи о пакете в файле `TODO`, если они там есть. Не забудьте зафиксировать изменения, например с помощью **make commit-changes-entry**! Если вы используете не рабочую копию непосредственно с cvs.NetBSD.org, а, скажем, локальную копию репозитория, можно задать USE_NETBSD_REPO=yes. Тогда команды cvs будут обращаться к основному репозиторию.

23.4. Сообщения коммитов

В течение нескольких лет существуют зеркала pkgsrc в fossil, git и hg. Стандартная практика при использовании этих инструментов — делать первую строку сообщения коммита кратким резюме, которое можно прочитать отдельно от остального, как это обычно делается с «git log --oneline». По этой причине у нас есть следующие рекомендации для сообщений коммитов pkgsrc:

- Начните сообщение коммита строкой, которая объясняет общую картину, не длиннее 65 символов. Если коммит относится к одному пакету, укажите имя пакета. Для обновлений укажите версию, до которой выполняется обновление.
- Оставьте следующую строку пустой.
- Затем идут подробности коммита (изменения в этом пакете, причина изменения) и любые относящиеся к делу PR. Разбивайте эту часть на строки.

Вот пример:

```
libxslt: update to 1.0.30

Changes since 1.0.29:
...
```

Вот другой пример:

```
mk/bsd.pkg.mk: enable SSP by default on NetBSD

(rationale)
```

Сообщения коммитов окончательны: в репозитории pkgsrc не допускается «cvs admin» для изменения сообщений коммитов.

23.5. Коммит: добавление пакета в CVS

Этот раздел представляет интерес только для разработчиков pkgsrc, имеющих право записи в репозиторий pkgsrc.

Когда пакет готов, выполните «cvs add» для файлов. Сначала добавьте каталог, а затем файлы в нём. Не забудьте добавить новый пакет в `Makefile` категории. Убедитесь, что не пропустили ни одного файла; проверить это можно с помощью «cvs status». Пример:

```
$ cd .../pkgsrc/category
$ cvs add pkgname
$ cd pkgname
$ cvs add DESCR Makefile PLIST distinfo buildlink3.mk patches
$ cvs add patches/p*
$ cvs status | less
```

```
$ cvs commit
$ cd ..
$ vi Makefile # add SUBDIR+=pkgname line
$ cvs commit Makefile
$ cd pkgname
$ make CTYPE=Added commit-changes-entry
```

Сообщение коммита первоначального импорта должно включать часть файла `DESCR`, чтобы читающие списки рассылки понимали, что это за пакет и что он делает.

Также упомяните новый пакет в `pkgsrc/doc/CHANGES-20xx`.

Раньше предлагалось использовать «cvs import», но в нём гораздо легче ошибиться, чем в «cvs add».

23.6. Обновление пакета до более новой версии

Пожалуйста, всегда помещайте в журнал коммита краткое, уместное и содержательное описание изменений между старой и новой версиями при обновлении пакета. Для этого есть несколько причин:

- URL непостоянен и может меняться со временем. Он может вовсе исчезнуть, а его информация — быть перезаписана более новой.
- Наличие информации об изменениях между старой и новой версиями в нашем репозитории CVS очень полезно для тех, кто использует cvs или anoncvs.
- Наличие информации об изменениях между старой и новой версиями в нашем репозитории CVS очень полезно для тех, кто читает список рассылки pkgsrc-changes: так они могут принимать тактические решения о том, когда обновлять пакет.

Пожалуйста, также учтите: то, что вышла новая версия пакета, ещё не означает, что её нужно автоматически обновлять в репозитории CVS. Мы предпочитаем консервативный подход к пакетам, включаемым в pkgsrc, — разрабатываемые или бета-пакеты не самое лучшее решение для большинства мест, где используется pkgsrc. Пожалуйста, руководствуйтесь собственным суждением о том, что должно попасть в pkgsrc, и помните, что стабильность предпочтительнее новых и, возможно, непроверенных возможностей.

23.7. Переименование пакета в pkgsrc

Переименовывать пакеты не рекомендуется.

При переименовании пакетов обязательно исправьте все ссылки на старое имя в других Makefile, параметрах, файлах buildlink и т. д.

Кроме того, при переименовании пакета добавьте имя пакета и шаблон(ы) версии предыдущего пакета в `SUPERSEDES`. Это можно повторять при нескольких переименованиях. Новый пакет станет точной заменой. Пример:

```
SUPERSEDES+=    p5-IO-Compress-Zlib<2.017
SUPERSEDES+=    optcomp-[0-9]*
```

Учтите, что «преемник» в файле CHANGES-*YYYY* не обязательно означает, что он *вытесняет* прежний, так как этот преемник может и не быть точной заменой, а лишь предложением для заменяемой функциональности.

23.8. Перемещение пакета в pkgsrc

Предпочтительно, чтобы пакеты не переименовывались и не перемещались, но если это необходимо, выполните следующие шаги.

1. Сделайте копию каталога в другом месте.
2. Удалите все каталоги CVS.

   Вместо первых двух шагов можно также выполнить:

   ```
   % cvs -d user@cvs.NetBSD.org:/cvsroot export -D today pkgsrc/category/package
   ```

   и использовать это для дальнейшей работы.
3. Исправьте `CATEGORIES` и все пути `DEPENDS`, в которых указано просто «../package» вместо «../../category/package».
4. В Makefile изменённого пакета рассмотрите возможность задать `PREV_PKGPATH` равным прежнему пути category/package. `PREV_PKGPATH` может использоваться инструментами для выполнения обновления с помощью сборки pkgsrc; например, он может искать в базе данных pkg_summary(5) значение `PREV_PKGPATH` (если нет `SUPERSEDES`), а затем использовать соответствующий новый `PKGPATH` для этого перемещённого пакета. Учтите, что совпадений может быть несколько, поэтому инструмент должен проверять также `PKGBASE`. Вероятно, `PREV_PKGPATH` не имеет значения, если только не задан `SUPERSEDES`, то есть `PKGBASE` остаётся прежним.
5. Выполните **cvs add** для изменённого пакета на новом месте.
6. Проверьте, зависит ли от него какой-либо пакет:

   ```
   % cd /usr/pkgsrc
   % grep /package */*/Makefile* */*/buildlink*
   ```

7. Исправьте пути в пакетах из шага 5, чтобы они указывали на новое место.
8. Выполните **cvs rm (-f)** для пакета на старом месте.
9. Удалите из `oldcategory/Makefile`.
10. Добавьте в `newcategory/Makefile`.
11. Закоммитьте изменённые и удалённые файлы:

    ```
    % cvs commit oldcategory/package oldcategory/Makefile newcategory/Makefile
    ```

    (и, разумеется, любые пакеты из шага 5).

Глава 24. Политики pkgsrc

24.1. Пакеты, обновление которых ограничено

В прошлом некоторые пакеты вызывали больше отказов сборки, чем другие, и в будущем мы хотим это сократить.

По этой причине pkgsrc-pmc помечает некоторые пакеты `POLICY_UPDATE_LIMITED`. Возможные значения сейчас таковы:

- `abi` — для пакетов, в которых изменения ABI/API часто ломали зависящие от них пакеты
- `bootstrap` — для пакетов, используемых при начальной загрузке pkgsrc

`pkglint` будет выдавать предупреждение при коммите обновлений этих пакетов.

24.1.1. Ограниченные обновления — ABI

Прежде чем коммитить обновления не микро-версий пакетов, помеченных `POLICY_UPDATE_LIMITED=abi`, нужно выполнить ограниченную массовую сборку `meta-pkgs/bulk-test-${PACKAGE}` и опубликовать результат в списке рассылки tech-pkg, отметив, какие пакеты перестанут собираться (если такие есть).

В зависимости от результата pkgsrc-pmc решает:

- продолжить обновление
- дождаться, пока пакеты X, Y, Z будут исправлены (в вышестоящем проекте или локально) с обновлённой версией, которая тем временем помещается в wip
- Во втором случае всем разработчикам pkgsrc предлагается заняться исправлением — это задача не только того, кто выполняет обновление.

Решение о ожидании пакетов может быть пересмотрено.

24.1.2. Ограниченные обновления — начальная загрузка

При обновлении пакетов, используемых при начальной загрузке, то есть помеченных `POLICY_UPDATE_LIMITED=bootstrap`, проверьте процесс начальной загрузки и, желательно, некоторые базовые пакеты, а затем отправьте патч в список рассылки tech-pkg, чтобы его могли протестировать и на других платформах. Дайте не менее двух недель на отзывы и тестирование другими.

Глава 25. Часто задаваемые вопросы

В этом разделе содержатся ответы на вопросы, которые могут возникнуть при написании пакета. Если вы не нашли здесь ответа на свой вопрос, сначала загляните в другие главы, а если ответа так и нет, задайте его в списке рассылки `pkgsrc-users`.

25.1. **В чём разница между MAKEFLAGS, .MAKEFLAGS и MAKE_FLAGS?**

25.2. **В чём разница между MAKE, GMAKE и MAKE_PROGRAM?**

25.3. **В чём разница между CC, PKG_CC и PKGSRC_COMPILER?**

25.4. **Почему make show-var VARNAME=BUILDLINK_PREFIX.foo сообщает, что значение пусто?**

25.5. **Что означает ${MASTER_SITE_SOURCEFORGE:=package/}? Я не понимаю, что делают символы := внутри.**

25.6. **Какие списки рассылки существуют для разработчиков пакетов?**

25.7. **Где находится документация по pkgsrc?**

25.8. **У меня есть немного свободного времени. Чем заняться?**

25.1.	В чём разница между `MAKEFLAGS`, `.MAKEFLAGS` и `MAKE_FLAGS`?
	`MAKEFLAGS` — это флаги, передаваемые внутренним вызовам make(1) в pkgsrc. `MAKE_FLAGS` — это флаги, передаваемые `MAKE_PROGRAM` при сборке пакета. `.MAKEFLAGS` — внутренняя переменная **bmake**, и пакетам не следует её использовать.
25.2.	В чём разница между `MAKE`, `GMAKE` и `MAKE_PROGRAM`?
	`MAKE` — путь к программе make(1), которая используется в инфраструктуре pkgsrc. `GMAKE` — путь к GNU Make, но чтобы его использовать, нужно указать `USE_TOOLS+=gmake`. `MAKE_PROGRAM` — путь к программе Make, используемой для сборки пакета.
25.3.	В чём разница между `CC`, `PKG_CC` и `PKGSRC_COMPILER`?
	`CC` — путь к настоящему компилятору C, который может настроить пользователь pkgsrc. `PKG_CC` — путь к обёртке компилятора. `PKGSRC_COMPILER` — *не* путь к компилятору, а тип компилятора, который следует использовать. Дополнительные сведения об этой последней переменной см. в `mk/compiler.mk`.
25.4.	Почему **make show-var VARNAME=BUILDLINK_PREFIX.*foo*** сообщает, что значение пусто?
	По причинам оптимизации некоторые переменные доступны только на фазе «wrapper» и позже. Чтобы «смоделировать» фазу wrapper, добавьте **PKG_PHASE=wrapper** к указанной выше команде.
25.5.	Что означает `${MASTER_SITE_SOURCEFORGE:=package/}`? Я не понимаю, что делают символы `:=` внутри.
	`:=` — не оператор присваивания, хотя может показаться таковым. На самом деле это вырожденная форма `${LIST:`*`old_string`*`=`*`new_string`*`}`, которая описана на странице руководства make(1) и обычно используется в виде `${SRCS:.c=.o}`. В случае `MASTER_SITE_*` *`old_string`* — пустая строка, а *`new_string`* — `package/`. Именно здесь `:` и `=` сливаются воедино.
25.6.	Какие списки рассылки существуют для разработчиков пакетов?

	tech-pkg Это список для технических обсуждений, связанных с разработкой pkgsrc: например, запрос отзывов об изменениях в инфраструктуре pkgsrc, предложения новых возможностей, вопросы о портировании pkgsrc на новую платформу, советы по сопровождению пакета, патчи, затрагивающие множество пакетов, просьбы о помощи, перенесённые из pkgsrc-users при обнаружении ошибки в инфраструктуре, и т. п. **pkgsrc-bugs** Сюда попадают все отчёты об ошибках категории «pkg», отправленные с помощью send-pr(1). Пожалуйста, не сообщайте о своих ошибках здесь напрямую; используйте один из других списков рассылки.
25.7.	Где находится документация по pkgsrc?
	Есть много мест, где можно найти документацию по pkgsrc: • Руководство pkgsrc (этот документ) — собрание глав, объясняющих большую часть pkgsrc, но некоторые главы имеют свойство устаревать. Какие именно — сказать трудно. • В архивах списков рассылки (см. https://mail-index.NetBSD.org/) можно найти обсуждения определённых возможностей, объявления о новых частях инфраструктуры pkgsrc и иногда даже объявления о том, что какая-то возможность помечена как устаревшая. Преимущество в том, что каждое сообщение имеет дату. • Многие файлы в каталоге `mk/` начинаются с комментария, описывающего назначение файла и то, как его могут использовать пользователь pkgsrc и авторы пакетов. Простой способ найти эту документацию — выполнить **bmake help**. • Сообщения журнала CVS — богатый источник информации, но они, как правило, сильно сокращены, особенно для часто повторяющихся действий. Некоторые содержат подробное описание того, что изменилось, но они рассчитаны на других разработчиков pkgsrc, а не на рядового пользователя pkgsrc. К тому же они документируют только *изменения*, поэтому если вы не знаете, что было раньше, эти сообщения могут оказаться для вас не слишком полезными. • Некоторые части pkgsrc лишь «неявно документированы», то есть документация существует только в голове разработчика, написавшего код. Чтобы получить эту информацию, воспользуйтесь командой **cvs annotate**, чтобы узнать, кто это написал, и задайте вопрос в списке рассылки `tech-pkg`, чтобы другие могли позже найти ваши вопросы (см. выше). Чтобы быть уверенным, что ответственный разработчик прочитает письмо, можете поставить его или её в копию.
25.8.	У меня есть немного свободного времени. Чем заняться?
	Это ещё не совсем часто задаваемый вопрос, но ответ всё же есть. • Запустите **pkg_chk -N** (из пакета `pkgtools/pkg_chk`). Он сообщит о доступных более новых версиях установленных пакетов, которые ещё не обновлены в pkgsrc. • Просмотрите `pkgsrc/doc/TODO` — там есть список предлагаемых новых пакетов, а также список желательных улучшений и наведений порядка в pkgsrc. • Проверьте пакеты, для которых запросили проверку в списке рассылки tech-pkg.

Часть III. Внутреннее устройство инфраструктуры pkgsrc

Эта часть руководства посвящена всему в инфраструктуре, что стоит за интерфейсами, описанными в руководстве разработчика. Обычному сопровождающему пакетов вряд ли понадобится что-либо из этой части.

Глава 26. Устройство инфраструктуры pkgsrc

Инфраструктура pkgsrc состоит из множества небольших фрагментов Makefile. Каждый такой фрагмент должен иметь правильно заданный интерфейс. В этой главе объясняется, как выглядит такой интерфейс.

26.1. Смысл определений переменных

Когда в инфраструктуре pkgsrc определяется переменная, место и способ определения многое говорят о предполагаемом использовании этой переменной. Кроме того, дополнительную документацию можно найти в комментарии в начале файла или в этом руководстве pkgsrc.

Особый файл — `mk/defaults/mk.conf`, в котором перечислены все переменные, предназначенные для определения пользователем. Они либо определяются с помощью оператора `?=`, либо остаются неопределёнными, поскольку определение их в любое значение фактически означало бы «yes». Все эти переменные могут быть переопределены пользователем pkgsrc в файле `MAKECONF`.

Вне этого файла действуют следующие соглашения: переменные, определённые с помощью оператора `?=`, могут быть переопределены пакетом.

Переменные, определённые с помощью оператора `=`, могут использоваться только для чтения во время выполнения.

Переменные, имя которых начинается с символа подчёркивания, вообще не должны использоваться вне инфраструктуры pkgsrc. Они могут изменяться без дополнительного уведомления.

> **ПРИМЕЧАНИЕ**
>
> В настоящее время эти соглашения применяются ко всей инфраструктуре pkgsrc непоследовательно.

26.2. Как избежать проблем до их появления

Все переменные, содержащие списки чего-либо, по умолчанию должны быть пустыми. Два примера, не следующих этому правилу, — `USE_LANGUAGES` и `DISTFILES`. Эти переменные нельзя просто изменять с помощью оператора `+=` в `Makefile` пакетов (или других включаемых ими файлах), так как нет гарантии, определена ли переменная и каково её значение. В случае `DISTFILES` пакеты «знают» значение по умолчанию и просто определяют его, как в следующем примере.

```
DISTFILES=      ${DISTNAME}${EXTRACT_SUFX} additional-files.tar.gz
```

Из-за выбора этого значения по умолчанию одно и то же значение встречается во многих Makefile пакетов. То же самое с `USE_LANGUAGES`, но в этом случае значение по умолчанию («`c`») настолько короткое, что не бросается в глаза. Тем не менее оно упоминается во многих файлах.

26.3. Вычисление значений переменных

26.3.1. Во время загрузки

Вычисление значений переменных происходит либо во время загрузки, либо во время выполнения, в зависимости от контекста. К контекстам, в которых переменные вычисляются во время загрузки, относятся:

- Правая часть операторов `:=` и `!=`,
- Директивы Make, такие как `.if` или `.for`,
- Строки зависимостей.

Особое исключение — ссылки на переменные итерации циклов `.for`, которые раскрываются на месте, в каком бы контексте они ни встречались.

Поскольку значения переменных могут меняться во время загрузки, нужно соблюдать осторожность, чтобы не вычислить их случайно. Типичные примеры переменных, которые не следует вычислять во время загрузки, — `DEPENDS` и `CONFIGURE_ARGS`. Чтобы эффект был понятнее, вот пример:

```
CONFIGURE_ARGS=         # none
CFLAGS=                 -O
CONFIGURE_ARGS+=        CFLAGS=${CFLAGS:Q}

CONFIGURE_ARGS:=        ${CONFIGURE_ARGS}

CFLAGS+=                -Wall
```

Этот код показывает, как использование оператора `:=` может быстро привести к неожиданным результатам. Первый фрагмент — довольно обычный код. Во втором фрагменте вычисляется переменная `CONFIGURE_ARGS`, что даёт `CFLAGS=-O`. В третьем фрагменте `-Wall` добавляется к `CFLAGS`, но это добавление не появится в `CONFIGURE_ARGS`. В реальном коде три приведённых выше фрагмента обычно находятся в совершенно разных файлах.

26.3.2. Во время выполнения

После загрузки всех файлов значения переменных больше нельзя изменить. Переменные, используемые в командах оболочки, раскрываются на этом этапе.

26.4. Как можно задавать переменные?

Есть много способов ограничить определение и использование переменной, чтобы выявлять ошибки и нарушения (в основном неписаных) правил. Пакет можно проверить с помощью `pkglint -Wall`, чтобы узнать, соблюдает ли он эти правила.

26.5. Проектирование интерфейсов для фрагментов Makefile

Большинство файлов `.mk` относятся к одному из следующих классов. Случаев, когда файл относится более чем к одному классу, следует избегать, так как это часто приводит к трудноуловимым ошибкам.

26.5.1. Процедуры с параметрами

В традиционном императивном языке программирования некоторые файлы `.mk` можно было бы описать как процедуры. Они принимают некоторые входные параметры и — после включения — выдают результат в выходных параметрах. Поскольку все переменные в `Makefile` имеют глобальную область видимости, нужно следить за тем, чтобы не использовать имена параметров, уже имеющие другое значение. Например, `PKGNAME` — плохой выбор для имени параметра.

Процедуры полностью вычисляются на этапе предварительной обработки. То есть при вызове процедуры все входные параметры должны быть полностью определены. Например, `CONFIGURE_ARGS` никогда не должен быть входным параметром, так как весьма вероятно, что после вызова процедуры будет добавлен дальнейший текст, и тогда процедура фактически применится только к части переменной. Кроме того, ссылки на другие переменные будут изменены после вызова процедуры.

Процедура может объявить свои выходные параметры либо как пригодные для использования в директивах предварительной обработки, либо как доступные только во время выполнения. Последний вариант предназначен для переменных, содержащих ссылки на другие переменные времени выполнения.

Процедуры должны быть написаны так, чтобы их можно было вызывать более одного раза. То есть файл не должен содержать защиты от многократного включения.

Примеры процедур — `mk/bsd.options.mk` и `mk/buildlink3/bsd.builtin.mk`. Чтобы показать, что параметры вычисляются во время загрузки, их следует назначать с помощью оператора `:=`, который должен использоваться только для этой цели.

26.5.2. Действия, выполняемые от имени параметров

Файлы действий принимают некоторые входные параметры и могут определять переменные времени выполнения. Они не должны определять переменные времени загрузки. Некоторые файлы действий включаются инфраструктурой pkgsrc неявно, а другие должны включаться явно.

Пример файла действий — `mk/subst.mk`.

26.6. Порядок загрузки файлов

`Makefile` пакетов обычно состоят из набора определений переменных и включают файл `../../mk/bsd.pkg.mk` в самой последней строке. До этого они могут также включать различные другие файлы `*.mk`, если нужно узнать о наличии определённых возможностей, например типа компилятора или реализации X11. Из-за активного использования директив предварительной обработки, таких как `.if` и `.for`, порядок загрузки файлов имеет значение.

В этом разделе описывается, на каком этапе загружаются различные файлы, и объясняются причины такого порядка.

26.6.1. Порядок в `bsd.prefs.mk`

Самое первое действие в `bsd.prefs.mk` — определение некоторых основных переменных, таких как `OPSYS`, `OS_VERSION` и `MACHINE_ARCH`.

Затем из файла, указанного в `MAKECONF` (обычно это `mk.conf`), загружаются пользовательские настройки. После этого из `mk/defaults/mk.conf` загружаются те переменные, которые не были переопределены пользователем.

После пользовательских настроек загружаются системные настройки и настройки платформы, которые могут переопределить пользовательские.

Затем загружаются определения инструментов. Обёртки инструментов пока не действуют. Это происходит только при сборке пакета, поэтому вместо прямых имён инструментов нужно использовать соответствующие переменные.

На последних этапах загружаются некоторые основные переменные из обёртки и варианта пакетной системы, а также переменные, закэшированные на более ранних этапах сборки пакета.

26.6.2. Порядок в `bsd.pkg.mk`

Сначала загружается `bsd.prefs.mk`.

Затем загружаются различные файлы `*-vars.mk`, которые заполняют значения по умолчанию для переменных, не определённых пакетом. Эти переменные позже могут использоваться даже в несвязанных файлах.

Затем файл `bsd.pkg.error.mk` предоставляет цель `error-check`, которая добавляется как особая зависимость ко всем другим целям, использующим `DELAYED_ERROR_MSG` или `DELAYED_WARNING_MSG`.

Затем включаются специфичные для пакета доработки из `hacks.mk`.

Затем следуют различные другие файлы. Большинство из них не зависит от того, что должно быть включено до или после них, хотя некоторые зависят.

Затем выполняется код проверки `PKG_FAIL_REASON` и `PKG_SKIP_REASON`, который ограничивает использование этих переменных всеми файлами, включёнными ранее. Появления в более поздних файлах будут молча игнорироваться.

Затем включаются файлы для основных целей в порядке их последующего выполнения, хотя фактический порядок не должен иметь значения.

Наконец, включается ещё несколько файлов, которые не задают никаких интересных переменных, а лишь определяют цели make для выполнения.

Глава 27. Регрессионные тесты

Инфраструктура pkgsrc — это обширная кодовая база, и в ней немало мест, где каждая мелочь в файле продумана до тонкостей, поэтому pkgsrc, скорее всего, сломается, стоит что-то изменить рядом с такими местами. Чтобы большинство изменений ничего не ломало, каждую важную часть инфраструктуры pkgsrc должен сопровождать набор регрессионных тестов. В этой главе рассказано, как устроены регрессионные тесты в pkgsrc и как добавить новые тесты.

27.1. Запуск регрессионных тестов

Сначала нужно установить пакет `pkgtools/pkg_regress`, предоставляющий команду **pkg_regress**. После этого достаточно просто запустить эту команду — она выполнит все тесты в каталоге `regress/`.

27.2. Добавление нового регрессионного теста

Каждый каталог внутри `regress/`, содержащий файл с именем `spec`, считается регрессионным тестом. Этот файл — программа для оболочки, которую подключает команда **pkg_regress**. Приведённые ниже функции можно переопределить по своему усмотрению.

27.2.1. Переопределяемые функции

Эти функции не принимают параметров. Хотя они вызываются в режиме «set -e», они не останавливаются на первой неудачной команде. Подробности см. в этом вопросе на Stack Overflow.

`do_setup`
: Эта функция подготавливает окружение для теста. По умолчанию она ничего не делает.

`do_test`
: Эта функция выполняет собственно тест. По умолчанию она вызывает `TEST_MAKE` с аргументами `MAKEARGS_TEST` и записывает свой вывод, включая сообщения об ошибках, в файл `TEST_OUTFILE`.

  Определяя эту функцию, убедитесь, что весь вывод, который нужно проверить, записывается в правильный выходной файл. Пример:

```
do_test() {
        echo "Example output"
} 1>$TEST_OUTFILE 2>&1
```

`check_result`
: Эта функция выполняется после теста и обычно служит для сравнения фактического вывода с ожидаемым. В ней можно использовать различные вспомогательные функции из следующего раздела. Пример:

```
check_result() {
        exit_status 0
        output_require "Example"
        output_require "^[[:alpha:]+[[:space:]][[:alpha:]]{6}$"
        output_prohibit "no such file or directory"
```

```
        regress_fail "expected $expected but got $actual for input $input"
}
```

`do_cleanup`

Эта функция выполняет очистку после завершения теста. По умолчанию она ничего не делает.

27.2.2. Вспомогательные функции

`regress_fail` *`message...`*

Эта функция приводит к ошибке теста с заданным сообщением об ошибке.

`exit_status expected`

Эта функция сравнивает код возврата функции **do_test** с её первым параметром. Если они различаются, тест завершается с ошибкой.

`output_require regex...`

Эта функция проверяет для каждого из своих параметров, соответствует ли вывод функции **do_test** расширенному регулярному выражению. Если не соответствует, тест завершается с ошибкой. Пример:

```
output_require "looks fine"
output_require "^[[:alpha:]+[[:space:]][[:alpha:]]{6}$"
```

`output_prohibit regex...`

Эта функция проверяет для каждого из своих параметров, что вывод функции **do_test()** *не* соответствует расширенному регулярному выражению. Если совпадает хотя бы одно из регулярных выражений, тест завершается с ошибкой.

Глава 28. Портирование pkgsrc

Система pkgsrc уже портирована на многие операционные системы, аппаратные архитектуры и компиляторы. В этой главе объясняются необходимые шаги, чтобы сделать pkgsrc ещё более переносимой.

28.1. Портирование pkgsrc на новую операционную систему

Чтобы портировать pkgsrc на новую операционную систему (в этом примере она называется `MyOS`), нужно затронуть следующие файлы:

`pkgtools/bootstrap-mk-files/files/mods/MyOS.sys.mk`
: Этот файл содержит некоторые базовые определения, например имя компилятора C.

`mk/bsd.prefs.mk`
: Вставьте код, определяющий переменные `OPSYS`, `OS_VERSION`, `OPSYS_VERSION`, `LOWER_VENDOR`, `MACHINE_ARCH`, `OBJECT_FMT`, `APPEND_ELF` и остальные переменные, встречающиеся в этом файле.

`mk/platform/MyOS.mk`
: Этот файл содержит определения, специфичные для платформы, которые используются pkgsrc. Начните с копирования одного из других файлов и отредактируйте его под свои нужды.

`mk/tools/tools.MyOS.mk`
: Этот файл определяет пути ко всем инструментам, которые нужны тем или иным пакетам в pkgsrc, а также самому pkgsrc. Выясните, где эти инструменты находятся на вашей платформе, и добавьте их.

Теперь вы должны суметь собрать некоторые базовые пакеты, например `lang/perl5`, `shells/bash`.

Приложение A. Простой пример пакета: bison

Мы поискали программу, которой ещё не было в коллекции пакетов, и выбрали GNU bison. Зачем кому-то может понадобиться **bison**, когда в дереве уже есть **yacc** от Berkeley, нам не понять, но для целей этого упражнения он полезен.

A.1. files

A.1.1. Makefile

```
# $NetBSD: pkgsrc.html,v 1.278 2026/08/03 13:14:54 wiz Exp $
#

DISTNAME=       bison-1.25
CATEGORIES=     devel
MASTER_SITES=   ${MASTER_SITE_GNU:=bison/}

MAINTAINER=     pkgsrc-users@NetBSD.org
HOMEPAGE=       https://www.gnu.org/software/bison/bison.html
COMMENT=        GNU yacc clone

GNU_CONFIGURE=  yes
INFO_FILES=     yes

.include "../../mk/bsd.pkg.mk"
```

A.1.2. DESCR

```
GNU version of yacc.  Can make re-entrant parsers, and numerous other
improvements.  Why you would want this when Berkeley yacc(1) is part
of the NetBSD source tree is beyond me.
```

A.1.3. PLIST

```
@comment $NetBSD: pkgsrc.html,v 1.278 2026/08/03 13:14:54 wiz Exp $
bin/bison
man/man1/bison.1.gz
share/bison.simple
share/bison.hairy
```

A.1.4. Проверка пакета с помощью pkglint

В состав системы пакетов NetBSD входит `pkgtools/pkglint`, который помогает проверять содержимое этих файлов. После установки им очень легко пользоваться: достаточно перейти в каталог пакета, который вы хотите проверить, и запустить **pkglint**:

```
$ pkglint
ERROR: Makefile: Each package must define its LICENSE.
WARN: Makefile:9: HOMEPAGE should migrate from http to https.
NOTE: PLIST:3: The .gz extension is unnecessary for manual pages.
WARN: PLIST:5: "share/bison.hairy" should be sorted before "share/bison.simple".
1 error, 2 warnings and 1 note found.
(Run "pkglint -e" to show explanations.)
(Run "pkglint -fs" to show what can be fixed automatically.)
(Run "pkglint -F" to automatically fix some issues.)
```

В зависимости от переданных аргументов командной строки (см. pkglint(1)) выполняется больше проверок. Например, для очень тщательной проверки используйте **pkglint -Wall**.

A.2. Этапы сборки, установки и упаковки

Создайте каталог, в котором будет находиться пакет, а также все вспомогательные каталоги:

```
# cd /usr/pkgsrc/lang
# mkdir bison
# cd bison
# mkdir patches
```

Создайте `Makefile`, `DESCR` и `PLIST` (см. Глава 12, *Компоненты пакета — файлы, каталоги и их содержимое*), после чего переходите к получению дистрибутивного файла:

```
# make fetch
>> bison-1.25.tar.gz doesn't seem to exist on this system.
>> Attempting to fetch from ftp://prep.ai.mit.edu/pub/gnu//.
Requesting ftp://prep.ai.mit.edu/pub/gnu//bison-1.25.tar.gz (via ftp://orpheus.amdahl.com:80/)
ftp: Error retrieving file: 500 Internal error

>> Attempting to fetch from ftp://wuarchive.wustl.edu/systems/gnu//.
Requesting ftp://wuarchive.wustl.edu/systems/gnu//bison-1.25.tar.gz (via
ftp://orpheus.amdahl.com:80/)
ftp: Error retrieving file: 500 Internal error

>> Attempting to fetch from ftp://ftp.freebsd.org/pub/FreeBSD/distfiles//.
Requesting ftp://ftp.freebsd.org/pub/FreeBSD/distfiles//bison-1.25.tar.gz (via
ftp://orpheus.amdahl.com:80/)
Successfully retrieved file.
```

Сгенерируйте контрольную сумму дистрибутивного файла в `distinfo`:

```
# make makedistinfo
```

Теперь скомпилируйте:

```
# make
>> Checksum OK for bison-1.25.tar.gz.
===>  Extracting for bison-1.25
===>  Patching for bison-1.25
===>   Ignoring empty patch directory
===>  Configuring for bison-1.25
creating cache ./config.cache
checking for gcc... cc
checking whether we are using GNU C... yes
```

```
checking for a BSD compatible install... /usr/bin/install -c -o bin -g bin
checking how to run the C preprocessor... cc -E
checking for minix/config.h... no
checking for POSIXized ISC... no
checking whether cross-compiling... no
checking for ANSI C header files... yes
checking for string.h... yes
checking for stdlib.h... yes
checking for memory.h... yes
checking for working const... yes
checking for working alloca.h... no
checking for alloca... yes
checking for strerror... yes
updating cache ./config.cache
creating ./config.status
creating Makefile
===>  Building for bison-1.25
cc -c -DSTDC_HEADERS=1 -DHAVE_STRING_H=1 -DHAVE_STDLIB_H=1 -DHAVE_MEMORY_H=1 -DHAVE_ALLOCA=1 -
DHAVE_STRERROR=1 -I./../include  -g LR0.c
cc -c -DSTDC_HEADERS=1 -DHAVE_STRING_H=1 -DHAVE_STDLIB_H=1 -DHAVE_MEMORY_H=1 -DHAVE_ALLOCA=1 -
DHAVE_STRERROR=1 -I./../include  -g allocate.c
cc -c -DSTDC_HEADERS=1 -DHAVE_STRING_H=1 -DHAVE_STDLIB_H=1 -DHAVE_MEMORY_H=1 -DHAVE_ALLOCA=1 -
DHAVE_STRERROR=1 -I./../include  -g closure.c
cc -c -DSTDC_HEADERS=1 -DHAVE_STRING_H=1 -DHAVE_STDLIB_H=1 -DHAVE_MEMORY_H=1 -DHAVE_ALLOCA=1 -
DHAVE_STRERROR=1 -I./../include  -g conflicts.c
cc -c -DSTDC_HEADERS=1 -DHAVE_STRING_H=1 -DHAVE_STDLIB_H=1 -DHAVE_MEMORY_H=1 -DHAVE_ALLOCA=1 -
DHAVE_STRERROR=1 -I./../include  -g derives.c
cc -c -DXPFILE=\"/usr/pkg/share/bison.simple\"  -DXPFILE1=\"/usr/pkg/share/bison.hairy\" -
DSTDC_HEADERS=1 -DHAVE_STRING_H=1 -DHAVE_STDLIB_H=1 -DHAVE_MEMORY_H=1 -DHAVE_ALLOCA=1 -
DHAVE_STRERROR=1  -g  ./files.c
cc -c -DSTDC_HEADERS=1 -DHAVE_STRING_H=1 -DHAVE_STDLIB_H=1 -DHAVE_MEMORY_H=1 -DHAVE_ALLOCA=1 -
DHAVE_STRERROR=1 -I./../include  -g getargs.c
cc -c -DSTDC_HEADERS=1 -DHAVE_STRING_H=1 -DHAVE_STDLIB_H=1 -DHAVE_MEMORY_H=1 -DHAVE_ALLOCA=1 -
DHAVE_STRERROR=1 -I./../include  -g gram.c
cc -c -DSTDC_HEADERS=1 -DHAVE_STRING_H=1 -DHAVE_STDLIB_H=1 -DHAVE_MEMORY_H=1 -DHAVE_ALLOCA=1 -
DHAVE_STRERROR=1 -I./../include  -g lalr.c
cc -c -DSTDC_HEADERS=1 -DHAVE_STRING_H=1 -DHAVE_STDLIB_H=1 -DHAVE_MEMORY_H=1 -DHAVE_ALLOCA=1 -
DHAVE_STRERROR=1 -I./../include  -g lex.c
cc -c -DSTDC_HEADERS=1 -DHAVE_STRING_H=1 -DHAVE_STDLIB_H=1 -DHAVE_MEMORY_H=1 -DHAVE_ALLOCA=1 -
DHAVE_STRERROR=1 -I./../include  -g main.c
cc -c -DSTDC_HEADERS=1 -DHAVE_STRING_H=1 -DHAVE_STDLIB_H=1 -DHAVE_MEMORY_H=1 -DHAVE_ALLOCA=1 -
DHAVE_STRERROR=1 -I./../include  -g nullable.c
cc -c -DSTDC_HEADERS=1 -DHAVE_STRING_H=1 -DHAVE_STDLIB_H=1 -DHAVE_MEMORY_H=1 -DHAVE_ALLOCA=1 -
DHAVE_STRERROR=1 -I./../include  -g output.c
cc -c -DSTDC_HEADERS=1 -DHAVE_STRING_H=1 -DHAVE_STDLIB_H=1 -DHAVE_MEMORY_H=1 -DHAVE_ALLOCA=1 -
DHAVE_STRERROR=1 -I./../include  -g print.c
cc -c -DSTDC_HEADERS=1 -DHAVE_STRING_H=1 -DHAVE_STDLIB_H=1 -DHAVE_MEMORY_H=1 -DHAVE_ALLOCA=1 -
DHAVE_STRERROR=1 -I./../include  -g reader.c
cc -c -DSTDC_HEADERS=1 -DHAVE_STRING_H=1 -DHAVE_STDLIB_H=1 -DHAVE_MEMORY_H=1 -DHAVE_ALLOCA=1 -
DHAVE_STRERROR=1 -I./../include  -g reduce.c
cc -c -DSTDC_HEADERS=1 -DHAVE_STRING_H=1 -DHAVE_STDLIB_H=1 -DHAVE_MEMORY_H=1 -DHAVE_ALLOCA=1 -
DHAVE_STRERROR=1 -I./../include  -g symtab.c
cc -c -DSTDC_HEADERS=1 -DHAVE_STRING_H=1 -DHAVE_STDLIB_H=1 -DHAVE_MEMORY_H=1 -DHAVE_ALLOCA=1 -
DHAVE_STRERROR=1 -I./../include  -g warshall.c
cc -c -DSTDC_HEADERS=1 -DHAVE_STRING_H=1 -DHAVE_STDLIB_H=1 -DHAVE_MEMORY_H=1 -DHAVE_ALLOCA=1 -
DHAVE_STRERROR=1 -I./../include  -g version.c
cc -c -DSTDC_HEADERS=1 -DHAVE_STRING_H=1 -DHAVE_STDLIB_H=1 -DHAVE_MEMORY_H=1 -DHAVE_ALLOCA=1 -
DHAVE_STRERROR=1 -I./../include  -g getopt.c
cc -c -DSTDC_HEADERS=1 -DHAVE_STRING_H=1 -DHAVE_STDLIB_H=1 -DHAVE_MEMORY_H=1 -DHAVE_ALLOCA=1 -
DHAVE_STRERROR=1 -I./../include  -g getopt1.c
cc  -g -o bison LR0.o allocate.o closure.o conflicts.o derives.o files.o         getargs.o
gram.o lalr.o lex.o                                 main.o nullable.o output.o print.o
reader.o reduce.o symtab.o   warshall.o version.o getopt.o getopt1.o
./files.c:240: warning: mktemp() possibly used unsafely, consider using mkstemp()
```

```
rm -f bison.s1
sed -e "/^#line/ s|bison|/usr/pkg/share/bison|" < ./bison.simple > bison.s1
```

Похоже, всё в порядке, поэтому установите файлы:

```
# make install
>> Checksum OK for bison-1.25.tar.gz.
===>  Installing for bison-1.25
sh ./mkinstalldirs /usr/pkg/bin /usr/pkg/share  /usr/pkg/info /usr/pkg/man/man1
rm -f /usr/pkg/bin/bison
cd /usr/pkg/share; rm -f bison.simple bison.hairy
rm -f /usr/pkg/man/man1/bison.1 /usr/pkg/info/bison.info*
install -c  -o bin -g bin -m 555 bison /usr/pkg/bin/bison
/usr/bin/install -c -o bin -g bin -m 644 bison.s1 /usr/pkg/share/bison.simple
/usr/bin/install -c -o bin -g bin -m 644 ./bison.hairy /usr/pkg/share/bison.hairy
cd .; for f in bison.info*;  do /usr/bin/install -c -o bin -g bin -m 644 $f /usr/pkg/info/$f;
done
/usr/bin/install -c -o bin -g bin -m 644 ./bison.1 /usr/pkg/man/man1/bison.1
===>  Registering installation for bison-1.25
```

Теперь можно пользоваться bison, а также — если вы так решите — удалить его командой **pkg_delete bison**. Если вы решите, что вам нужен двоичный пакет, сделайте это сейчас:

```
# make package
>> Checksum OK for bison-1.25.tar.gz.
===>  Building package for bison-1.25
Creating package bison-1.25.tgz
Registering depends:.
Creating gzip'd tar ball in '/u/pkgsrc/lang/bison/bison-1.25.tgz'
```

Теперь, когда исходные и объектные файлы больше не нужны, выполните очистку:

```
# make clean
===>  Cleaning for bison-1.25
```

Приложение B. Усиление безопасности

В pkgsrc есть ряд механизмов для повышения безопасности итоговой системы. На этой странице описаны эти механизмы и даны подсказки о том, как обнаруживать и устранять проблемы.

Механизмы можно включать по отдельности в `mk.conf`; каждый из них описан ниже.

Как правило, при первом включении какая-либо возможность заставляет некоторые программы переставать собираться или работать. Причиной могут быть скрытые проблемы в самой программе, а также иные причины. После достаточного тестирования, дающего уверенность, что проблемы у пользователей будут довольно редкими, отдельные механизмы включаются по умолчанию.

Для каждого механизма см. раздел «Предостережения» ниже: там объясняется, что может пойти не так на этапе компиляции и во время выполнения, а также как заметить и устранить эти проблемы.

B.1. Механизмы

B.1.1. Включено по умолчанию

B.1.1.1. PKGSRC_USE_FORTIFY

Это позволяет подставлять обёртки вместо некоторых часто используемых библиотечных функций, в которых нет встроенной проверки границ — хотя в ряде случаев она могла бы быть.

Доступны два уровня защиты:

- «weak» включает проверки только на этапе компиляции.
- «strong» включает проверки на этапе компиляции и во время выполнения.

«strong» включён по умолчанию начиная с pkgsrc-2017Q3.

B.1.1.2. PKGSRC_USE_SSP

Это включает защиту от переполнения стека. Она реализуется добавлением защитной переменной к функциям, содержащим уязвимые объекты. Защитные переменные инициализируются при входе в функцию и проверяются при выходе из неё. Если переменная за это время была изменена, проверка не проходит, и программа принудительно завершается. Это может случиться при переполнении буфера или повреждении памяти — и тогда такие ошибки становятся заметны.

Доступны различные уровни защиты:

- «yes» — защищаются только функции, которые компилятор считает уязвимыми;
- «all» — защищаются все функции;
- «strong» (по умолчанию) — обеспечивает лучший баланс между двумя предыдущими вариантами.

Эта защита поддерживается и GCC, и clang. Возможно, её поддерживают и другие компиляторы, возможно, под другим названием. Она особенно полезна для небезопасных языков программирования, таких как C/C++.

- «yes» включён по умолчанию там, где такая поддержка известна, начиная с pkgsrc-2017Q3.
- «strong» включён по умолчанию там, где такая поддержка известна, начиная с pkgsrc-2021Q4.

Более подробные сведения можно найти здесь:

- Защита от переполнения буфера в Википедии

B.1.1.3. PKGSRC_MKPIE

Это предписывает создавать для всех исполняемых файлов PIE (Position Independent Executables) — исполняемые файлы, независимые от позиции. Механизм PIE обычно применяется к разделяемым библиотекам, чтобы их можно было загружать во время выполнения по разным адресам. Сам по себе PIE не даёт полезных свойств безопасности, однако он необходим для полноценного использования некоторых из них, например ASLR. Некоторые операционные системы поддерживают рандомизацию размещения адресного пространства (ASLR), из-за которой при каждом запуске программы используются разные адреса. Это затрудняет для атакующего угадывание адресов и тем самым усложняет построение эксплойтов. С PIE ASLR действительно применяется ко всей программе, а не только к стеку и куче.

Исполняемые файлы PIE будут создаваться только для тех инструментальных сред, которые, как известно, поддерживают PIE. Сейчас это означает NetBSD на x86, ARM, SPARC64, m68k и MIPS.

`PKGSRC_MKPIE` был включён по умолчанию после появления ветви pkgsrc-2021Q3.

B.1.1.4. PKGSRC_USE_RELRO

Это также в ряде случаев затрудняет эксплуатацию некоторых уязвимостей безопасности.

Доступны два разных уровня защиты:

- partial (по умолчанию): разделы ELF переупорядочиваются так, чтобы внутренние разделы данных шли перед собственными разделами данных программы, а GOT, не относящийся к PLT, доступен только для чтения;
- full: в дополнение к partial RELRO каждая релокация выполняется сразу при запуске программы, что позволяет сделать весь GOT доступным только для чтения. Это может сильно замедлить запуск больших программ.

В настоящее время это поддерживается GCC. Многие дистрибутивы программного обеспечения теперь включают эту возможность по умолчанию на уровне «partial».

Более подробные сведения можно найти здесь:

- Усиление защиты двоичных файлов ELF с помощью Relocation Read-Only (RELRO)

B.1.2. Не включено по умолчанию

B.1.2.1. PKGSRC_MKREPRO

С этим параметром pkgsrc постарается собирать пакеты воспроизводимо. Это позволяет пакетам, собранным из одного и того же дерева и с одними и теми же параметрами, давать побитово идентичные результаты. Этот параметр следует сочетать с ASLR и `PKGSRC_MKPIE`, чтобы избежать предсказуемых смещений адресов для атакующих, пытающихся использовать уязвимости безопасности.

Более подробные сведения можно найти здесь:

- Reproducible Builds — набор практик разработки программного обеспечения, обеспечивающих независимо проверяемый путь от исходного кода к двоичному

Вероятно, прежде чем pkgsrc станет полностью воспроизводимым, предстоит выполнить ещё много работы.

B.1.2.2. PKGSRC_USE_STACK_CHECK

Здесь используется `-fstack-check` в GCC как ещё одна мера защиты стека.

Компилятору поручается генерировать код, проверяющий, что он не повреждает стек. Согласно странице руководства GCC, это действительно полезно только для многопоточных программ.

B.2. Предостережения

B.2.1. Проблемы с PKGSRC_MKPIE

B.2.1.1. Пакеты, которые не собираются

Ряд пакетов может не собираться при включённом параметре. Ошибки часто связаны с отсутствием флага компиляции `-fPIC` при сборке библиотек или исполняемых файлов (а в последнем случае в идеале — `-fPIE`). Этот флаг уже добавляется в `CFLAGS`, но пакет должен действительно его поддерживать.

B.2.1.1.1. Как это исправить

Эти инструкции приведены только для справки; скорее всего, для многих пакетов их придётся адаптировать по отдельности.

Для пакетов, использующих `Makefiles`:

```
MAKE_FLAGS+=    CFLAGS=${CFLAGS:Q}
MAKE_FLAGS+=    LDFLAGS=${LDFLAGS:Q}
```

Для пакетов, использующих `Imakefiles`:

```
MAKE_FLAGS+=    CCOPTIONS=${CFLAGS:Q}
MAKE_FLAGS+=    LOCAL_LDFLAGS=${LDFLAGS:Q}
```

B.2.1.2. Сбои при выполнении

Некоторые программы могут не запускаться или случайным образом аварийно завершаться, будучи собраны как PIE. Возможны, по сути, два сценария. Почти всегда это вызвано ошибкой в программе, которая проявляется из-за ASLR.

B.2.1.3. Отключение PKGSRC_MKPIE для отдельного пакета

В идеале пакеты следует исправлять для совместимости с MKPIE. Однако в некоторых случаях это очень трудно из-за сложных систем сборки, пакетов, использующих нестандартные инструментальные среды, или языков программирования со странными механизмами начальной загрузки.

Чтобы отключить `PKGSRC_MKPIE` для отдельного пакета, задайте `MKPIE_SUPPORTED= no` в Makefile пакета до включения `bsd.prefs.mk`.

B.2.2. Проблемы с PKGSRC_USE_FORTIFY

B.2.2.1. Пакеты, которые не собираются

Эта возможность использует директивы препроцессора для поиска усиленных альтернативных реализаций важных библиотечных вызовов. В результате некоторые программы могут не собраться; обычно это происходит с теми, что слишком стараются быть переносимыми или иным образом злоупотребляют определениями из стандартной библиотеки.

B.2.2.2. Сбои при выполнении

Эта возможность может приводить к аварийному завершению некоторых программ, что обычно указывает на реальную ошибку в программе. Исправление, как правило, требует патча исходного кода программы.

B.2.2.3. Требуется оптимизация

По крайней мере в случае GCC FORTIFY будет применяться только в том случае, если при компиляции включена оптимизация. Это означает, что `CFLAGS` должен также содержать `-O`, `-O2` или другой уровень оптимизации. Это непросто применить ко всем пакетам сразу, поскольку некоторым из них могут требоваться особые уровни оптимизации.

B.2.2.4. Отключение FORTIFY для отдельного пакета

> **ПРИМЕЧАНИЕ**
>
> Не следует отключать FORTIFY, чтобы обойти аварийные завершения программы! Это очень плохая идея, которая подвергнет вас уязвимостям безопасности.

Чтобы отключить FORTIFY для отдельного пакета, задайте следующее в `Makefile` пакета до включения `bsd.prefs.mk`:

```
FORTIFY_SUPPORTED=      no
```

B.2.3. Проблемы с PKGSRC_USE_RELRO

B.2.3.1. Влияние на производительность

Для лучшей защиты полный RELRO требует, чтобы все символы разрешались при запуске программы, а не просто по мере необходимости во время выполнения. Это сильнее сказывается на программах, использующих много символов или связанных с библиотеками, которые экспортируют много символов. Поэтому демоны и другие программы, работающие в фоне, затрагиваются только при запуске. Программы, загружающие подключаемые модули во время выполнения, затрагиваются при загрузке этих модулей.

Ожидается, что на современном оборудовании это влияние будет незаметным, кроме некоторых случаев с большими программами.

B.2.3.2. Сбои при выполнении

Некоторые программы работают с подключаемыми модулями и зависимостями так, что это конфликтует с RELRO: например, когда процедура инициализации перечисляет все необходимые ей подключаемые модули. При полном RELRO отсутствующие символы разрешаются до того, как процедура инициализации сможет выполниться, и динамический загрузчик не сможет найти их напрямую, а потому аварийно завершится. К сожалению, именно так Xorg загружает свои драйверы. В этом случае вместо него можно применять частичный RELRO.

B.2.3.3. Отключение RELRO для отдельного пакета

Чтобы отключить RELRO для отдельного пакета, задайте следующее в `Makefile` пакета до включения `bsd.prefs.mk`:

```
RELRO_SUPPORTED=        no
```

Можно также разрешить не более чем частичный RELRO, задав для `RELRO_SUPPORTED` значение `partial`.

B.2.4. Проблемы с PKGSRC_USE_SSP

B.2.4.1. Пакеты, которые не собираются

Защита от переполнения стека, обеспечиваемая этим параметром, не работает для некоторых программ. Чаще всего это происходит, когда программа размещает переменные в стеке, причём размер определяется во время выполнения.

B.2.4.2. Сбои при выполнении

Опять же, эта возможность может приводить к аварийному завершению некоторых программ по сигналу `SIGABRT`, что обычно указывает на реальную ошибку в программе.

В NetBSD отправляются сообщения syslog уровня `LOG_CRIT`, которые по умолчанию дописываются в `/var/log/messages`, например:

```
Jan  6 15:42:51 hostname -: hostname program - - - buffer overflow detected; terminated
```

(где `hostname` — это hostname(1), а `program` — basename(1) программы, завершившейся аварийно).

После этого требуется пропатчить исходную программу.

Пересоберите пакет с помощью:

```
% env CFLAGS=-g INSTALL_UNSTRIPPED=yes make replace
```

а анализ обратной трассировки дампа памяти в отладчике должен указать проблемный вызов — достаточно посмотреть на кадр, из которого вызывается функция _chk() (SSP).

B.2.4.3. Влияние на производительность

При использовании этой возможности компилятор генерирует дополнительный код: при входе в функции и выходе из них выполняется проверка переполнения буфера, для чего требуется дополнительная переменная в стеке. Впрочем, уровень защиты можно настроить так, чтобы он

затрагивал только те функции, которые компилятор считает более чувствительными (с помощью -fstack-protector вместо -fstack-protector-all).

Ожидается, что на современном оборудовании влияние будет незаметным. Однако программы, для которых критично работать с максимально возможной скоростью, должны избегать использования этой возможности или библиотек, собранных с ней.

B.2.4.4. Отключение SSP для отдельного пакета

> **ПРИМЕЧАНИЕ**
>
> Не следует отключать SSP, чтобы обойти аварийные завершения программы! Это очень плохая идея, которая подвергнет вас уязвимостям безопасности.

Чтобы отключить SSP для отдельного пакета, задайте следующее в `Makefile` пакета до включения `bsd.prefs.mk`:

```
SSP_SUPPORTED=  no
```

B.3. Аудит системы

Иллюзия безопасности хуже, чем полное её отсутствие. В этом разделе перечислены некоторые способы убедиться, что запрошенные функции безопасности действительно работают.

Эти инструкции были получены и проверены в системе на основе NetBSD 7 (amd64). Результаты могут отличаться.

B.3.1. Проверка на PIE

У двоичных файлов, собранных как PIE, меняется используемый тип исполняемого файла ELF; без PIE:

```
$ file /path/to/bin/ary
/path/to/bin/ary: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked
(uses shared libs), for NetBSD 7.0, not stripped
```

в отличие от следующего двоичного файла, собранного как PIE:

```
$ file /path/to/pie/bin/ary
/path/to/pie/bin/ary: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically
linked (uses shared libs), for NetBSD 7.0, not stripped
```

Именно последний результат и ожидается.

B.3.2. Проверка на частичный RELRO

Следующая команда должна вывести раздел с именем RELRO:

```
$ objdump -p /path/to/bin/ary

/path/to/bin/ary:     file format elf64-x86-64
```

```
Program Header:
[...]
   RELRO off    0x0000000000000d78 vaddr 0x0000000000600d78 paddr 0x0000000000600d78 align
2**0
```

Теперь эта проверка выполняется автоматически, если задана `PKG_DEVELOPER` и включён RELRO.

B.3.3. Проверка на полный RELRO

Динамический загрузчик применит RELRO сразу же, обнаружив флаг `BIND_NOW`:

```
$ objdump -x /path/to/bin/ary

/path/to/bin/ary:     file format elf64-x86-64

Dynamic Section:
[...]
  BIND_NOW             0x0000000000000000
```

Чтобы это было полностью эффективно, это нужно сочетать с частичным RELRO (см. выше).

Теперь эта проверка выполняется автоматически (там, где это поддерживается), если задана `PKG_DEVELOPER`.

B.3.4. Проверка на SSP

> **ПРИМЕЧАНИЕ**
>
> Проверка SSP этим методом работает только там, где операционная система использует `libssp`. В последних версиях NetBSD/FreeBSD/Linux `libssp` не используется.

Сборка объектных файлов, двоичных файлов и библиотек с SSP влияет на наличие в итоговом файле дополнительных символов:

```
$ nm /path/to/bin/ary
[...]
                 U __stack_chk_fail
0000000000600ea0 B __stack_chk_guard
```

Это признак того, что программа действительно была собрана с поддержкой SSP.

Теперь эта проверка выполняется автоматически (там, где это поддерживается), если задана `PKG_DEVELOPER` и включён SSP.

Приложение C. Журналы сборки

C.1. Сборка figlet

```
# make
===> Checking for vulnerabilities in figlet-2.2.1nb2
=> figlet221.tar.gz doesn't seem to exist on this system.
=> Attempting to fetch figlet221.tar.gz from ftp://ftp.figlet.org/pub/figlet/program/unix/.
=> [172219 bytes]
Connected to ftp.plig.net.
220 ftp.plig.org NcFTPd Server (licensed copy) ready.
331 Guest login ok, send your complete e-mail address as password.
230-You are user #5 of 500 simultaneous users allowed.
230-
230-  ___ _              _ _
230- |  _| |_ ___   ___| |_|___   ___ ___ ___
230- |  _|  _| . |_| . | | | . |_| . |  _| . |
230- |_| |_| |  _|_|  _|_|_|_  |_|___|_| |_  |
230-         |_|   |_|     |___|         |___|
230-
230-** Welcome to ftp.plig.org **
230-
230-Please note that all transfers from this FTP site are logged. If you
230-do not like this, please disconnect now.
230-
230-This archive is available via
230-
230-HTTP:  http://ftp.plig.org/
230-FTP:   ftp://ftp.plig.org/     (max 500 connections)
230-RSYNC: rsync://ftp.plig.org/   (max  30 connections)
230-
230-Please email comments, bug reports and requests for packages to be
230-mirrored to ftp-admin@plig.org.
230-
230-
230 Logged in anonymously.
Remote system type is UNIX.
Using binary mode to transfer files.
200 Type okay.
250 "/pub" is new cwd.
250-"/pub/figlet" is new cwd.
250-
250-Welcome to the figlet archive at ftp.figlet.org
250-
250-    ftp://ftp.figlet.org/pub/figlet/
250-
250-The official FIGlet web page is:
250-    http://www.figlet.org/
250-
250-If you have questions, please mailto:info@figlet.org. If you want to
250-contribute a font or something else, you can email us.
250
250 "/pub/figlet/program" is new cwd.
250 "/pub/figlet/program/unix" is new cwd.
local: figlet221.tar.gz remote: figlet221.tar.gz
502 Unimplemented command.
227 Entering Passive Mode (195,40,6,41,246,104)
150 Data connection accepted from 84.128.86.72:65131; transfer starting for figlet221.tar.gz
```

```
(172219 bytes).
38% |**************                          | 65800      64.16 KB/s    00:01 ETA
226 Transfer completed.
172219 bytes received in 00:02 (75.99 KB/s)
221 Goodbye.
=> Checksum OK for figlet221.tar.gz.
===> Extracting for figlet-2.2.1nb2
===> Required installed package ccache-[0-9]*: ccache-2.3nb1 found
===> Patching for figlet-2.2.1nb2
===> Applying pkgsrc patches for figlet-2.2.1nb2
===> Overriding tools for figlet-2.2.1nb2
===> Creating toolchain wrappers for figlet-2.2.1nb2
===> Configuring for figlet-2.2.1nb2
===> Building for figlet-2.2.1nb2
gcc -O2 -DDEFAULTFONTDIR=\"/usr/pkg/share/figlet\"  -DDEFAULTFONTFILE=\"standard.flf\"
figlet.c zipio.c crc.c inflate.c -o figlet
chmod a+x figlet
gcc -O2 -o chkfont chkfont.c
=> Unwrapping files-to-be-installed.
#
# make install
===> Checking for vulnerabilities in figlet-2.2.1nb2
===> Installing for figlet-2.2.1nb2
install -d -o root -g wheel -m 755 /usr/pkg/bin
install -d -o root -g wheel -m 755 /usr/pkg/man/man6
mkdir -p /usr/pkg/share/figlet
cp figlet /usr/pkg/bin
cp chkfont /usr/pkg/bin
chmod 555 figlist showfigfonts
cp figlist /usr/pkg/bin
cp showfigfonts /usr/pkg/bin
cp fonts/*.flf /usr/pkg/share/figlet
cp fonts/*.flc /usr/pkg/share/figlet
cp figlet.6 /usr/pkg/man/man6
===> Registering installation for figlet-2.2.1nb2
#
```

C.2. Упаковка figlet

```
# make package
===> Checking for vulnerabilities in figlet-2.2.1nb2
===> Packaging figlet-2.2.1nb2
===> Building binary package for figlet-2.2.1nb2
Creating package /home/cvs/pkgsrc/packages/i386/All/figlet-2.2.1nb2.tgz
Using SrcDir value of /usr/pkg
Registering depends:.
#
```

Приложение D. Структура каталогов FTP-сервера pkgsrc

Как и в других больших проектах, структура каталогов pkgsrc довольно сложна для новичков. В этой главе объясняется, где на FTP-сервере что находится. Базовый каталог на `ftp.NetBSD.org` — `/pub/pkgsrc/`. На других серверах он может быть иным, но внутри этого каталога всё должно выглядеть одинаково, независимо от того, на каком сервере вы находитесь. Этот каталог содержит несколько подкаталогов, которые описаны ниже.

D.1. `distfiles`: дистрибутивные файлы с исходными текстами

Каталог `distfiles` содержит множество архивных файлов из всех пакетов pkgsrc, которые здесь зеркалируются. Подкаталоги называются по именам пакетов и используются, когда дистрибутивные файлы имеют имена, явно не содержащие номера версии или иным образом слишком общие (например, `release.tar.gz`).

D.2. `misc`: разное

Этот каталог содержит вещи, которые отдельные разработчики pkgsrc считают достойными публикации.

D.3. `packages`: двоичные пакеты

Этот каталог содержит двоичные пакеты для различных платформ, поддерживаемых pkgsrc. Каждый подкаталог имеет вид *OPSYS*/*ARCH*/*OSVERSION_TAG*. Значения этих переменных таковы:

- `OPSYS` — имя операционной системы, для которой собраны пакеты. Имя берётся из вывода команды **uname**, поэтому оно может отличаться от того, к которому вы привыкли.
- `ARCH` — аппаратная архитектура платформы, для которой собраны пакеты. Она также включает `ABI` (двоичный интерфейс приложения) для платформ, у которых их несколько.
- `OSVERSION` — версия операционной системы. Обычно это версия, определяющая ABI, например 9.0.
- `TAG` — это либо `20`*YY*`Q`*N* для стабильной ветви, либо `current` для пакетов, собранных из ветви HEAD. Последнее следует использовать только тогда, когда пакеты регулярно обновляются.

Смысл этой схемы в том, что пользователи могут сначала искать по OPSYS и ARCH, поскольку они более или менее неизменны. После этого остаётся набор каталогов управления с OSVERSION и TAG.

Дополнительные сведения, в том числе о символических ссылках, см. также в `README.md` в `packages`.

В каждом из этих каталогов находится целая коллекция двоичных пакетов для конкретной платформы. В нём есть каталог `All`, содержащий все двоичные пакеты.

D.4. `reports`: отчёты о массовых сборках

Здесь находятся отчёты о массовых сборках — для тех, кто хочет исправить пакеты, которые не собрались на некоторых платформах. Структура подкаталогов должна быть такой же, как в Раздел D.3, «`packages`: двоичные пакеты».

D.5. `current`, `stable`, `pkgsrc-20`*`xx`*`Q`*`y`*: пакеты из исходных текстов

Эти каталоги содержат «настоящий» pkgsrc, то есть файлы, которые определяют, как создавать двоичные пакеты из архивов исходных текстов.

Каждый из каталогов `current`, `stable` и `pkgsrc-20`*`xx`*`Q`*`y`* имеет одинаковую структуру. В каждом из них есть каталог `pkgsrc` и файл `pkgsrc.tar.{bz,gz,xz}`.

Каталог `pkgsrc` содержит снимок репозитория CVS, который регулярно обновляется. Файл `pkgsrc.tar.{bz,gz,xz}` содержит то же самое, что и каталог, и готов к загрузке целиком.

Каталог `current` содержит файлы, относящиеся к ветви HEAD репозитория CVS. В этом каталоге есть дополнительный файл `pkgsrc-readmes.tar.{bz,gz,xz}`, содержащий все файлы `README` pkgsrc со сведениями о категориях и пакетах.

Каталог `stable` — символическая ссылка на последний `pkgsrc-20`*`xx`*`Q`*`y`*.

Каталоги `pkgsrc-20`*`xx`*`Q`*`y`*. содержат файлы, относящиеся к стабильной ветви `-20`*`xx`*`Q`*`y`* репозитория CVS. В этих каталогах есть дополнительный файл с именем `pkgsrc-20`*`xx`*`Q`*`y`*`.tar.{bz,gz,xz}`, который содержит состояние pkgsrc на момент создания ветви.

Приложение E. Темы справки

Следующий список содержит все темы справки, доступные при запуске **bmake help topic=:index**.

#!	-lintl
32bit	64bit
ABI	ACCEPTABLE_LICENSES
ACROREAD_FONTPATH	AFAIK
AIX	ALLOW_NETWORK_ACCESS
ALLOW_NEWER_COMPILER	ALLOW_VULNERABLE_PACKAGES
ALL_ENV	ALSA_PC
ALTERNATIVES_SRC	AMANDA_TMP
AMANDA_USER	AMANDA_VAR
APACHE_GROUP	APACHE_MODULE_NAME
APACHE_MODULE_SRC	APACHE_MODULE_SRCDIR
APACHE_PKG_PREFIX	APACHE_SUEXEC_CONFIGURE_ARGS
APACHE_SUEXEC_DOCROOT	APACHE_USER
ARGMAX_REQ	ARG_MAX
ARLA_CACHE	AUDIT_PACKAGES_FLAGS
AUTOCONF_REQD	AUTOMAKE_OVERRIDE
AUTOMAKE_REQD	AUTO_MKDIRS
AUX	BDB185_DEFAULT
BDBBASE	BDB_ACCEPTED
BDB_DEFAULT	BDB_LIBS
BDB_TYPE	BINCIMAP_GROUP
BINCIMAP_LOG_USER	BINCIMAP_USER
BIND_DIR	BIND_GROUP
BIND_USER	BINOWN
BINPKG_SITES	BIN_INSTALL_FLAGS
BISON_PKGDATADIR	BLAS
BLAS_ACCEPTED	BLAS_C_INTERFACE
BLAS_INCLUDES	BLAS_INDEX64
BLAS_LIBS	BLAS_PC
BLAS_TYPE	BOOTSTRAP_DEPENDS

BROKEN	BROKEN_EXCEPT_ON_PLATFORM
BROKEN_ON_PLATFORM	BSDSRCDIR
BSD_MAKE_ENV	BUILDLINK_AUTO_DIRS
BUILDLINK_AUTO_VARS	BUILDLINK_CFLAGS
BUILDLINK_CONTENTS_FILTER	BUILDLINK_CPPFLAGS
BUILDLINK_DEPMETHOD	BUILDLINK_FILES
BUILDLINK_FILES_CMD	BUILDLINK_FNAME_TRANSFORM
BUILDLINK_LDFLAGS	BUILDLINK_LIBS
BUILDLINK_OPSYS_SUPPORT_PTHREAD	BUILDLINK_PKGNAME
BUILDLINK_PREFIX	BUILDLINK_RPATHDIRS
BUILDLINK_TREE	BUILD_DEFS
BUILD_DEFS_EFFECTS	BUILD_DEPENDS
BUILD_DIRS	BUILD_ENV_SHELL
BUILD_MAKE_CMD	BUILD_MAKE_FLAGS
BUILD_TARGET	BUILTIN_FIND_FILES
BUILTIN_FIND_FILES_VAR	BUILTIN_FIND_GREP
BUILTIN_FIND_HEADERS	BUILTIN_FIND_HEADERS_VAR
BUILTIN_FIND_PKGCONFIG	BUILTIN_FIND_PKGCONFIG_VAR
BUILTIN_PKG	BUILTIN_TEST_CURSES_DEFINES
BUILTIN_TEST_CURSES_FUNCS	BUILTIN_VERSION_SCRIPT
BUILTIN_X11_TYPE	BUILTIN_X11_VERSION
CACTI_GROUP	CACTI_USER
CANNA_GROUP	CANNA_USER
CAT	CBLAS_LIBS
CBLAS_PC	CCACHE_BASE
CCACHE_DIR	CCACHE_LOGFILE
CCACHE_RECACHE	CC_VERSION
CDRECORD_CONF	CDROM_PKG_URL_DIR
CDROM_PKG_URL_HOST	CHANGES
CHECKOUT_DATE	CHECK_COMPILER
CHECK_FAKEHOME	CHECK_FILES
CHECK_FILES_ONLY_PREFIX	CHECK_FILES_SKIP
CHECK_FILES_STRICT	CHECK_HEADERS
CHECK_HEADERS_SKIP	CHECK_INTERPRETER
CHECK_INTERPRETER_SKIP	CHECK_PERMS
CHECK_PERMS_AUTOFIX	CHECK_PERMS_SKIP

CHECK_PIE	CHECK_PIE_SKIP
CHECK_PIE_SUPPORTED	CHECK_PORTABILITY
CHECK_PORTABILITY_EXPERIMENTAL	CHECK_PORTABILITY_SKIP
CHECK_RELRO	CHECK_RELRO_SKIP
CHECK_RELRO_SUPPORTED	CHECK_SHLIBS
CHECK_SHLIBS_SKIP	CHECK_SHLIBS_SUPPORTED
CHECK_SHLIBS_TOXIC	CHECK_SSP
CHECK_SSP_SKIP	CHECK_SSP_SUPPORTED
CHECK_STRIPPED	CHECK_STRIPPED_SKIP
CHECK_WRKREF	CHECK_WRKREF_EXTRA_DIRS
CHECK_WRKREF_SKIP	CLAMAV_DBDIR
CLAMAV_GROUP	CLAMAV_USER
CLANGBASE	CLEANDEPENDS
CLT	CMAKE_DEPENDENCIES_REWRITE
CMAKE_INSTALL_NAME_DIR	CMAKE_INSTALL_PREFIX
CMAKE_MODULE_PATH_OVERRIDE	CMAKE_PKGSRC_BUILD_FLAGS
CMAKE_PREFIX_PATH	CMAKE_USE_GNU_INSTALL_DIRS
CODEBERG_PROJECT	CODEBERG_RELEASE
CODEBERG_SUBMODULES	CODEBERG_TAG
CODEBERG_TYPE	COMMON_LISP_DOCFILES
COMMON_LISP_EXAMPLES	COMMON_LISP_EXTRAFILES
COMMON_LISP_PACKAGES	COMMON_LISP_SYSTEM
COMPILER_RPATH_FLAG	COMPILER_USE_SYMLINKS
CONFIGURE_ARGS	CONFIGURE_DIRS
CONFIGURE_ENV	CONFIGURE_ENV_SHELL
CONFIGURE_HAS_INFODIR	CONFIGURE_HAS_MANDIR
CONFIGURE_SCRIPT	CONFIG_SHELL
CONFIG_SHELL_FLAGS	CONF_FILES
CONF_FILES_MODE	CONF_FILES_PERMS
CONSERVER_DEFAULTHOST	CONSERVER_DEFAULTPORT
CP	CPE
CPE_EDITION	CPE_LANG
CPE_OTHER	CPE_PART
CPE_PRODUCT	CPE_SW_EDITION
CPE_TARGET_HW	CPE_TARGET_SW
CPE_UPDATE	CPE_URI

CPE_VENDOR	CPE_VERSION
CPP	CPP_PRECOMP_FLAGS
CREATE_WRKDIR_SYMLINK	CROSSBASE
CTFCONVERT	CTF_FILES_SKIP
CTF_SUPPORTED	CTYPE
CUPS_GROUP	CUPS_SYSTEM_GROUPS
CUPS_USER	CURSES_DEFAULT
CURSES_TYPE	CVS_EXTRACTDIR
CVS_MODULE	CVS_PROJECT
CVS_REPOSITORIES	CVS_ROOT
CVS_ROOT_GNU	CVS_ROOT_NONGNU
CVS_ROOT_SOURCEFORGE	CVS_TAG
CYRUS_GROUP	CYRUS_IDLE
CYRUS_USER	DAEMONTOOLS_GROUP
DAEMONTOOLS_LOG_USER	DARWIN_NO_SYSTEM_LIBS
DARWIN_REQUIRES_FILTER	DBUS_GROUP
DBUS_USER	DEFANG_GROUP
DEFANG_USER	DEFAULT_ACCEPTABLE_LICENSES
DEFAULT_DISTFILES	DEFAULT_IRC_SERVER
DEFAULT_SERIAL_DEVICE	DEF_UMASK
DEINSTALLDEPENDS	DEINSTALL_SRC
DEINSTALL_TEMPLATES	DELAYED_ERROR_MSG
DELAYED_WARNING_MSG	DEPENDS
DEPENDS_TARGET	DEPENDS_TYPE
DESTDIR	DESTDIR_VARNAME
DIALER_GROUP	DIGEST_REQD
DISTDIR	DISTFILES
DISTINFO_FILE	DISTNAME
DIST_PATH	DIST_SUBDIR
DJBDNS_AXFR_USER	DJBDNS_CACHE_USER
DJBDNS_DJBDNS_GROUP	DJBDNS_LOG_USER
DJBDNS_RBL_USER	DJBDNS_TINY_USER
DLOPEN_REQUIRE_PTHREADS	DL_AUTO_VARS
DL_CFLAGS	DL_LDFLAGS
DL_LIBS	DNS
DOCOWN	DOWNLOADED_DISTFILE

DQCACHE_GROUP	DQCACHE_USER
DT_LAYOUT	DYNAMIC_SITES_CMD
DYNAMIC_SITES_SCRIPT	ECHO
ECHO_N	ELK_GUI
EMACS_TYPE	EMULDIR
EMULSUBDIR	EMULSUBDIRSLASH
EMUL_ARCH	EMUL_DISTRO
EMUL_EXEC_FMT	EMUL_IS_NATIVE
EMUL_MODULES	EMUL_OPSYS
EMUL_PKG_FMT	EMUL_PLATFORM
EMUL_PLATFORMS	EMUL_PREFER
EMUL_REQD	EMUL_TYPE
ERROR_MSG	EXIM_GROUP
EXIM_USER	EXPORT_SYMBOLS_LDFLAGS
EXTRACTOR	EXTRACT_CMD
EXTRACT_CMD_DEFAULT	EXTRACT_DIR
EXTRACT_ELEMENTS	EXTRACT_ENV
EXTRACT_ONLY	EXTRACT_OPTS
EXTRACT_SUFX	EXTRACT_USING
FAIL	FAILOVER_FETCH
FAIL_MSG	FAKE_NCURSES
FAM	FAM_ACCEPTED
FAM_DEFAULT	FAM_SERVER
FCPATH	FETCH_AFTER_ARGS
FETCH_BEFORE_ARGS	FETCH_CMD
FETCH_INSECURE	FETCH_OUTPUT_ARGS
FETCH_PROXY	FETCH_RESUME_ARGS
FETCH_TIMEOUT	FETCH_USE_IPV4_ONLY
FETCH_USING	FILES_SUBST
FILES_SUBST_SED	FIX_SYSTEM_HEADERS
FONTDIR	FONTS_DIRS
FONTS_VERBOSE	FOO_HACKS_MK
FORCE_CXX_STD	FORCE_C_STD
FOSSIL_EXTRACTDIR	FOSSIL_REPO
FOSSIL_REPOSITORIES	FOSSIL_VERSION
FOX_USE_XUNICODE	FREEWNN_GROUP

FREEWNN_USER	FTP_PKG_URL_DIR
FTP_PKG_URL_HOST	GAMEDATAMODE
GAMEDATA_PERMS	GAMEDIRMODE
GAMEDIR_PERMS	GAMEGRP
GAMEMODE	GAMEOWN
GAMES_GROUP	GAMES_USER
GCC	GCCBASE
GCC_REQD	GCC_VERSION_SUFFIX
GEM_BUILD	GEM_CLEANBUILD
GEM_CLEANBUILD_EXTENSIONS	GEM_DOCDIR
GEM_EXTSDIR	GEM_HOME
GEM_KEEPBUILD	GEM_LIBDIR
GEM_NAME	GEM_PATH
GEM_PLUGINSDIR	GEM_SPECFILE
GENERATE_PLIST	GHC
GHOSTSCRIPT_REQD	GITHUB_PROJECT
GITHUB_RELEASE	GITHUB_SUBMODULES
GITHUB_TAG	GITHUB_TYPE
GITLAB_PROJECT	GITLAB_RELEASE
GITLAB_SUBMODULES	GITLAB_TAG
GITLAB_TYPE	GIT_BRANCH
GIT_ENV	GIT_EXTRACTDIR
GIT_REPO	GIT_REPOSITORIES
GIT_REVISION	GIT_TAG
GNAT	GNU
GNU_CONFIGURE	GNU_CONFIGURE_ICONV
GNU_CONFIGURE_INFODIR	GNU_CONFIGURE_MANDIR
GNU_CONFIGURE_QUIET	GNU_CONFIGURE_STRICT
GOFLAGS	GO_BUILD_PATTERN
GO_DIST_BASE	GO_EXTRA_MOD_DIRS
GO_MODULE_EXTRACT	GO_MODULE_FILES
GO_SRCPATH	GROUP_SPECIFIC_PKGS
GRUB_NETWORK_CARDS	GRUB_PRESET_COMMAND
GRUB_SCAN_ARGS	GZIP
HASKELL_COMPILER	HASKELL_DISABLE_EXECUTABLES
HASKELL_DISABLE_LIBRARIES	HASKELL_ENABLE_DYNAMIC_EXECUTABLE

HASKELL_ENABLE_HADDOCK_DOCUMENTATION	HASKELL_ENABLE_LIBRARY_PROFILING
HASKELL_ENABLE_SHARED_LIBRARY	HASKELL_ENABLE_TESTS
HASKELL_OPTIMIZATION_LEVEL	HASKELL_PKG_NAME
HASKELL_UNRESTRICT_DEPENDENCIES	HEADER_TEMPLATES
HG_REPO	HG_REPOSITORIES
HG_TAG	HOMEPAGE
HOST_PKGTOOLS_ARGS	HOST_SPECIFIC_PKGS
HOWL_GROUP	HOWL_USER
ICCBASE	ICECAST_CHROOTDIR
ICON_THEMES	IDOBASE
IGNORE_CCACHE	IGNORE_INFO_DIRS
IGNORE_INTERACTIVE_FETCH	IMAKE
IMAKEOPTS	IMAKE_MAKE
IMAKE_MANINSTALL	IMAP_UW_CCLIENT_MBOX_FMT
IMDICTDIR	INCOMPAT_CURSES
INFO_DIR	INFO_FILES
INFO_MSG	INIT_SYSTEM
INN_DATA_DIR	INN_GROUP
INN_USER	INSTALLATION_DIRS
INSTALLATION_DIRS_FROM_PLIST	INSTALL_ENV
INSTALL_ENV_SHELL	INSTALL_SH_OVERRIDE
INSTALL_SRC	INSTALL_TEMPLATES
INSTALL_UNSTRIPPED	IRCD_HYBRID_IRC_GROUP
IRCD_HYBRID_IRC_USER	IRCD_HYBRID_MAXCONN
IRCD_HYBRID_NICLEN	IRCD_HYBRID_SYSLOG_EVENTS
IRCD_HYBRID_SYSLOG_FACILITY	IRCD_HYBRID_TOPICLEN
IRIX	IRRD_USE_PGP
JABBERD_GROUP	JABBERD_LOGDIR
JABBERD_PIDDIR	JABBERD_SPOOLDIR
JABBERD_USER	JAVA_APP_PATH
JAVA_APP_TARGETS	JAVA_BINPREFIX
JAVA_CLASSPATH	JAVA_HOME
JAVA_LD_LIBRARY_PATH	JAVA_NAME
JAVA_UNLIMIT	JAVA_WRAPPERS
JPEG_ACCEPTED	JPEG_DEFAULT
KERBEROS	KERMIT_SUID_UUCP

KJS_USE_PCRE	KNEWS_DOMAIN_FILE
KNEWS_DOMAIN_NAME	KRB5_ACCEPTED
KRB5_DEFAULT	LANGUAGES
LAPACKE_LIBS	LAPACKE_PC
LAPACK_LIBS	LAPACK_PC
LATEX2HTML_ICONPATH	LDCONFIG_ADD_CMD
LDCONFIG_REMOVE_CMD	LDD
LEAFNODE_DATA_DIR	LEAFNODE_GROUP
LEAFNODE_USER	LIBDVDCSS_HOMEPAGE
LIBDVDCSS_MASTER_SITES	LIBIMAGEQUANT_TYPE
LIBRSVG_TYPE	LIBTOOL
LIBTOOLIZE_PLIST	LIBTOOL_M4_OVERRIDE
LIBUSB_TYPE	LICENSE
LINKER_RPATH_FLAG	LINK_RPATH_FLAG
LINUX_BASE_NODEPS	LINUX_BASE_PREFERRED
LINUX_BASE_REQUIRED	LINUX_LOCALES
LOCALBASE	LOCALBASE_LOCKTYPE
LOCALPATCHES	LOVE_DATA
LOVE_GAME	LOVE_VERSION
LOWER_OPSYS	LOWER_OPSYS_VERSUFFIX
LOWER_VARIANT_VERSION	LOWER_VENDOR
LP32PLATFORMS	LP64PLATFORMS
LUA_BUSTED_ARGS	LUA_CDIR
LUA_COMPILER	LUA_DOCDIR
LUA_EXAMPLESDIR	LUA_INCDIR
LUA_INTERPRETER	LUA_LDIR
LUA_LINKER_MAGIC	LUA_PKGPREFIX
LUA_SELF_CONFLICT	LUA_USE_BUSTED
LUA_VERSIONS_ACCEPTED	LUA_VERSIONS_INCOMPATIBLE
LUA_VERSION_DEFAULT	LUA_VERSION_REQD
Lua	MACHINE_PLATFORM
MAILAGENT_DOMAIN	MAILAGENT_EMAIL
MAILAGENT_FQDN	MAILAGENT_ORGANIZATION
MAJORDOMO_HOMEDIR	MAJOR_OS_VERSION
MAKEINFO_ARGS	MAKE_DIRS
MAKE_DIRS_PERMS	MAKE_ENV

MAKE_FILE	MAKE_FLAGS
MAKE_JOBS	MAKE_JOBS_METHOD
MAKE_JOBS_SAFE	MAKE_PROGRAM
MANCOMPRESSED_IF_MANZ	MANINSTALL
MANZ	MARIADB_VERSIONS_ALL
MASTER_SITES	MASTER_SITE_BACKUP
MASTER_SITE_LOCAL	MASTER_SITE_MOZILLA
MASTER_SITE_OVERRIDE	MASTER_SITE_PERL_CPAN
MASTER_SORT	MASTER_SORT_RANDOM
MASTER_SORT_REGEX	MECAB_CHARSET
MEDIATOMB_GROUP	MEDIATOMB_USER
MIPSPROBASE	MIREDO_GROUP
MIREDO_USER	MISSING_FEATURES
MKDIR	MKPIE_SUPPORTED
MLDONKEY_GROUP	MLDONKEY_HOME
MLDONKEY_USER	MONOTONE_GROUP
MONOTONE_USER	MOTIFBASE
MOTIF_TYPE	MOTIF_TYPE_DEFAULT
MPI_TYPE	MSGFMT_STRIP_MSGCTXT
MSGFMT_STRIP_MSGID_PLURAL	MTOOLS_ENABLE_FLOPPYD
MUST	MV
MYSQL_CHARSET	MYSQL_DATADIR
MYSQL_EXTRA_CHARSET	MYSQL_GROUP
MYSQL_USER	MYSQL_VERSION
MYSQL_VERSIONS_ACCEPTED	MYSQL_VERSION_DEFAULT
NAGIOSCMD_GROUP	NAGIOSDIR
NAGIOS_GROUP	NAGIOS_USER
NATIVE_EXEC_FMT	NATIVE_LOWER_OPSYS
NATIVE_LOWER_VENDOR	NATIVE_MACHINE_PLATFORM
NATIVE_OBJECT_FMT	NATIVE_OPSYS_VERSION
NATIVE_OS_VARIANT	NATIVE_OS_VERSION
NBPAX_PROGRAM_PREFIX	NETBSD_LOGIN_NAME
NMH_EDITOR	NMH_MTA
NMH_PAGER	NODE_VERSIONS_ACCEPTED
NODE_VERSIONS_INCOMPATIBLE	NODE_VERSION_DEFAULT
NODE_VERSION_REQD	NOLOGIN

NOTE	NOT_FOR_PLATFORM
NOT_PAX_ASLR_SAFE	NOT_PAX_MPROTECT_SAFE
NO_BUILD	NO_CHECKSUM
NO_CONFIGURE	NO_SKIP
NSD_GROUP	NSD_LOG_FACILITY
NSD_USER	NS_PREFERRED
NULLMAILER_GROUP	NULLMAILER_USER
OBJECT_FMT	OBJHOSTNAME
OBJMACHINE	OCAML_FINDLIB_DIRS
OCAML_FINDLIB_REGISTER	OCAML_FINDLIB_REGISTER_VERBOSE
OCAML_SITELIBDIR	OMF
ONLY_FOR_COMPILER	ONLY_FOR_PLATFORM
OPENSSH_CHROOT	OPENSSH_GROUP
OPENSSH_USER	OPSYS
OPSYS_EMULDIR	OPSYS_VERSION
OS	OSS_TYPE
OSX	OSX_PATH_TO_M4
OSX_PATH_TO_YACC	OS_HAVE_ALSA
OS_HAVE_RCD	OS_VARIANT
OS_VERSION	OTF_FONTS_DIR
OVERRIDE_DIRDEPTH	OVERRIDE_GEMSPEC
OVERRIDE_GNU_CONFIG_SCRIPTS	OVERRIDE_ROCKSPEC
OWN_DIRS	OWN_DIRS_PERMS
P4GROUP	P4PORT
P4ROOT	P4USER
PACKAGES	PALMOS_DEFAULT_SDK
PAMBASE	PAM_DEFAULT
PAPERSIZE	PASSIVE_FETCH
PATCHDIR	PATCHFILES
PATCH_ARGS	PATCH_DEBUG
PATCH_DIST_ARGS	PATCH_DIST_CAT
PATCH_DIST_STRIP	PATCH_FUZZ_FACTOR
PATCH_STRIP	PATH
PCCBASE	PEAR
PEAR_CHANNEL	PEAR_CHANNEL_ALIAS
PEAR_CHANNEL_VERSION	PEAR_CMD

PEAR_LIB	PEAR_PACKAGE_XML
PEP-518	PERL5
PERL5_CONFIGURE	PERL5_CONFIGURE_DIRS
PERL5_LDFLAGS	PERL5_LICENSE
PERL5_MODULE_TYPE	PERL5_PACKLIST
PERL5_PACKLIST_DIR	PERL5_PERLBASE
PERL5_SITEBASE	PERL5_USE_PACKLIST
PERL5_VENDORBASE	PFCTL
PFVAR_H	PF_VERSION
PGGROUP	PGHOME
PGPKGSRCDIR	PGSQL_TYPE
PGSQL_VERSION	PGSQL_VERSIONS_ACCEPTED
PGSQL_VERSION_DEFAULT	PGUSER
PG_LIB_EXT	PHP
PHPCOMMON_MK	PHPPKGSRCDIR
PHP_BASE_VERS	PHP_EGDIR
PHP_EXTENSION_DIR	PHP_INCDIR
PHP_LIBDIR	PHP_MAJOR_VERS
PHP_MINIMUM_TEENY	PHP_MINOR_VERS
PHP_PKG_PREFIX	PHP_REQUIRE_MODULE
PHP_SHAREDIR	PHP_VER
PHP_VERSION	PHP_VERSIONS_ACCEPTED
PHP_VERSIONS_INCOMPATIBLE	PHP_VERSION_DEFAULT
PHP_VERSION_REQD	PILRC_USE_GTK
PKGCONFIG_BASE	PKGCONFIG_FILE
PKGCONFIG_OVERRIDE	PKGCONFIG_OVERRIDE_STAGE
PKGDIR	PKGGNUDIR
PKGINFODIR	PKGLOCALEDIR
PKGMANDIR	PKGNAME
PKGNAME_REQD	PKGPATH
PKGREVISION	PKGSRC_BLAS_TYPES
PKGSRC_CHANGES	PKGSRC_COMPILER
PKGSRC_KEEP_BIN_PKGS	PKGSRC_LOCKTYPE
PKGSRC_MAKE_ENV	PKGSRC_MESSAGE_RECIPIENTS
PKGSRC_MKPIE	PKGSRC_MKREPRO
PKGSRC_OVERRIDE_MKPIE	PKGSRC_RUN_TEST

PKGSRC_SETENV	PKGSRC_SLEEPSECS
PKGSRC_TODO	PKGSRC_USE_CTF
PKGSRC_USE_FORTIFY	PKGSRC_USE_MKTOOLS
PKGSRC_USE_RELRO	PKGSRC_USE_SSP
PKGSRC_USE_STACK_CHECK	PKGTASKS_DATAFILE
PKGTOOLS_ARGS	PKGTOOLS_ENV
PKG_ALTERNATIVES	PKG_APACHE
PKG_APACHE_ACCEPTED	PKG_APACHE_DEFAULT
PKG_BEST_EXISTS	PKG_BUILD_OPTIONS
PKG_COMPRESSION	PKG_CONFIG
PKG_CONFIG_PERMS	PKG_CREATE_USERGROUP
PKG_DBDIR	PKG_DB_TMPDIR
PKG_DEBUG_LEVEL	PKG_DEFAULT_OPTIONS
PKG_DESTDIR_SUPPORT	PKG_DEVELOPER
PKG_DISABLED_OPTIONS	PKG_FATAL_ERRORS
PKG_FC	PKG_FILELIST_CMD
PKG_GECOS	PKG_GID
PKG_GROUPS	PKG_GROUPS_VARS
PKG_HOME	PKG_INIT_SCRIPTS
PKG_JAVA_HOME	PKG_JVM
PKG_JVMS_ACCEPTED	PKG_JVM_DEFAULT
PKG_LEGACY_OPTIONS	PKG_LIBTOOL
PKG_MKSYMLINKS	PKG_OPTIONS
PKG_OPTIONS_DEPRECATED_WARNINGS	PKG_OPTIONS_LEGACY_OPTS
PKG_OPTIONS_LEGACY_VARS	PKG_OPTIONS_NONEMPTY_SETS
PKG_OPTIONS_OPTIONAL_GROUPS	PKG_OPTIONS_REQUIRED_GROUPS
PKG_OPTIONS_VAR	PKG_RCD_SCRIPTS
PKG_REFCOUNT_DBDIR	PKG_REGISTER_SHELLS
PKG_RESUME_TRANSFERS	PKG_SHELL
PKG_SUGGESTED_OPTIONS	PKG_SUPPORTED_OPTIONS
PKG_SYSCONFBASE	PKG_SYSCONFBASEDIR
PKG_SYSCONFDIR	PKG_SYSCONFDIR_PERMS
PKG_SYSCONFSUBDIR	PKG_SYSCONFVAR
PKG_TOOLS_BIN	PKG_UID
PKG_UPDATE_FONTS_DB	PKG_USERS
PKG_USERS_VARS	PKG_VERBOSE

PLEASE	PLIST
PLIST_AWK	PLIST_AWK_ENV
PLIST_SRC	PLIST_SUBST
PLIST_TYPE	PLIST_VARS
POPTOP_USE_MPPE	POSSIBLE_GFORTRAN_VERSION
POST_FETCH_HOOK	PREFER
PREFER_NATIVE	PREFER_NATIVE_PTHREADS
PREFER_PKGSRC	PREFIX
PREPEND_PATH	PRE_ROOT_CMD
PRIVILEGED_STAGES	PS
PTHREAD_AUTO_VARS	PTHREAD_CFLAGS
PTHREAD_LDFLAGS	PTHREAD_LIBS
PTHREAD_OPTS	PTHREAD_TYPE
PVM_SSH	PYPKGPREFIX
PYSOABISUFFIX	PYTHON_27_ACCEPTED
PYTHON_FOR_BUILD_ONLY	PYTHON_SELF_CONFLICT
PYTHON_VERSION	PYTHON_VERSIONED_DEPENDENCIES
PYTHON_VERSIONS_ACCEPTED	PYTHON_VERSIONS_INCOMPATIBLE
PYTHON_VERSION_DEFAULT	PYTHON_VERSION_REQD
PYVERSSUFFIX	QMAILDIR
QMAIL_ALIAS_USER	QMAIL_DAEMON_USER
QMAIL_LOG_USER	QMAIL_NOFILES_GROUP
QMAIL_PASSWD_USER	QMAIL_QMAIL_GROUP
QMAIL_QUEUE_DIR	QMAIL_QUEUE_EXTRA
QMAIL_QUEUE_USER	QMAIL_REMOTE_USER
QMAIL_ROOT_USER	QMAIL_SEND_USER
QORE_LATEST_MODULE_API	QORE_MODULE_API
QORE_MODULE_DIR	QORE_USER_MODULE_DIR
QORE_VERSION	QPOPPER_FAC
QPOPPER_SPOOL_DIR	QPOPPER_USER
RAKE_NAME	RASMOL_DEPTH
RCD_DIR	RCD_ORDER
RCD_SCRIPTS	RCD_SCRIPTS_DIR
RCD_SCRIPTS_EXAMPLEDIR	RCD_SCRIPTS_MODE
RCD_SCRIPTS_SHELL	RCD_SCRIPT_SRC
RCD_SUBR	RDOC

READLINE_DEFAULT	READLINE_TYPE
REAL_ROOT_GROUP	REAL_ROOT_USER
RECURSIVE_MAKE	RELAY_CTRL_DIR
RELRO_SUPPORTED	REPLACE_AWK
REPLACE_BASH	REPLACE_CSH
REPLACE_KSH	REPLACE_LUA
REPLACE_NODEJS	REPLACE_OCTAVE
REPLACE_PERL	REPLACE_PERL6
REPLACE_PHP	REPLACE_PYTHON
REPLACE_QORE	REPLACE_R
REPLACE_RUBY	REPLACE_RUBY_DIRS
REPLACE_RUBY_PAT	REPLACE_SH
REPLACE_TEXLUA	REPLACE_TOOL_PYTHON
REPLACE_WISH	REQD_DIRS
REQD_DIRS_PERMS	REQD_FILES
REQD_FILES_MODE	REQD_FILES_PERMS
RESOLV_AUTO_VARS	RESOLV_LDFLAGS
RESOLV_LIBS	RM
ROCKSPEC_NAME	ROCKSPEC_SPECFILE
ROOT_CMD	ROOT_GROUP
ROOT_USER	RPCGEN
RPM	RPM2PKG_PLIST
RPM2PKG_PREFIX	RPM2PKG_STAGE
RPM2PKG_STRIP	RPM2PKG_SUBPREFIX
RPMFILES	RPMIGNOREPATH
RPM_DB_PREFIX	RSSH_CVS_PATH
RSSH_RDIST_PATH	RSSH_RSYNC_PATH
RSSH_SCP_PATH	RSSH_SFTP_SERVER_PATH
RUBY	RUBYGEM
RUBYGEM_MANPAGES	RUBYGEM_NAME
RUBYGEM_OPTIONS	RUBYGEM_USE_MANPAGES
RUBYGEM_VERBOSE	RUBY_ABI_VERSION
RUBY_ALLOW_HAS_RDOC	RUBY_ARCH
RUBY_ARCHINC	RUBY_ARCHLIB
RUBY_BASE	RUBY_BASERIDIR
RUBY_BUILD_DOCUMENT	RUBY_DLEXT

RUBY_DOC	RUBY_DYNAMIC_DIRS
RUBY_EG	RUBY_EXTCONF
RUBY_EXTCONF_CHECK	RUBY_EXTCONF_DEBUG
RUBY_EXTCONF_MAKEFILE	RUBY_GEM_ARCH
RUBY_GEM_BASE	RUBY_INC
RUBY_LIB	RUBY_LIB_BASE
RUBY_NAME	RUBY_NOVERSION
RUBY_PKGPREFIX	RUBY_RAILS
RUBY_RAILS72_VERSION	RUBY_RAILS80_VERSION
RUBY_RAILS_ACCEPTED	RUBY_RAILS_DEFAULT
RUBY_RAILS_REQD	RUBY_RAILS_STRICT_DEP
RUBY_RIDIR	RUBY_SETUP
RUBY_SHLIB	RUBY_SHLIBALIAS
RUBY_SHLIBVER	RUBY_SIMPLE_INSTALL
RUBY_SITEARCHLIB	RUBY_SITELIB
RUBY_SITELIB_BASE	RUBY_SITERIDIR
RUBY_SLEXT	RUBY_SRCDIR
RUBY_STATICLIB	RUBY_SUFFIX
RUBY_SYSRIDIR	RUBY_USE_PTHREAD
RUBY_VENDORARCHLIB	RUBY_VENDORLIB
RUBY_VENDORLIB_BASE	RUBY_VER
RUBY_VERSION	RUBY_VERSIONS_ACCEPTED
RUBY_VERSIONS_INCOMPATIBLE	RUBY_VERSION_DEFAULT
RUBY_VERSION_REQD	RUBY_VER_DIR
RUN	RUN_LDCONFIG
RUST_TYPE	SCO
SCREWS_GROUP	SCREWS_USER
SCRIPTS_ENV	SCROLLKEEPER_DATADIR
SCROLLKEEPER_REBUILDDB	SCROLLKEEPER_UPDATEDB
SDIST_PAWD	SDL12_TYPE
SERIAL_DEVICES	SETGIDGAME
SETGID_GAMES_PERMS	SETUID_ROOT_PERMS
SH	SHAIRPORT_GROUP
SHAIRPORT_USER	SHLIB_EXT
SHORTNAME	SIGN_PACKAGES
SILC_CLIENT_WITH_PERL	SITE_SPECIFIC_PKGS

SKIP_DEPENDS	SMF_INSTANCES
SMF_MANIFEST	SMF_METHODS
SMF_METHOD_SHELL	SMF_METHOD_SRC
SMF_NAME	SMF_PREFIX
SMF_SRCDIR	SNIPROXY_GROUP
SNIPROXY_USER	SOURCE_BUFFSIZE
SPECIAL_PERMS	SPECIFIC_PKGS
SSH_SUID	SSLCERTBUNDLE
SSLCERTS	SSLDIR
SSLKEYS	SSP_SUPPORTED
SSYNC_PAWD	STEP_MSG
STRIP	STRIP_DBG
STRIP_DEBUG	STRIP_DEBUG_SUPPORTED
STRIP_FILES_SKIP	SU
SUBDIR	SUBST
SUBST_CLASSES	SUBST_FILES
SUBST_FILTER_CMD	SUBST_MESSAGE
SUBST_NOOP_OK	SUBST_SED
SUBST_SHOW_DIFF	SUBST_SKIP_TEXT_CHECK
SUBST_STAGE	SUBST_VARS
SUNWSPROBASE	SUSE_PREFER
SU_CMD	SVN_EXTRACTDIR
SVN_REPO	SVN_REPOSITORIES
SVN_REVISION	SYSCONFBASE
TARGET_MACHINE_ARCH	TBL
TERMCAP_TYPE	TERMINFO_DEFAULT
TERMINFO_TYPE	TEST
TEST_DEPENDS	TEST_DIRS
TEST_ENV	TEST_ENV_SHELL
TEST_MAKE_CMD	TEST_MAKE_FLAGS
TEST_TARGET	TEXLIVE_IGNORE_PATTERNS
TEXLIVE_REV	TEXLIVE_UNVERSIONED
TEXMFSITE	TEX_FORMATS
TEX_HYPHEN_DAT	TEX_HYPHEN_DEF
TEX_TEXMF_DIRS	THTTPD_LOG_FACILITY
TINYDYN_USER	TLS

TLSWRAPPER_CHROOT	TO
TOOLDIR	TOOLS_ALIASES
TOOLS_ALWAYS_WRAP	TOOLS_ARGS
TOOLS_BROKEN	TOOLS_CMD
TOOLS_CMDLINE_SED	TOOLS_CREATE
TOOLS_CROSS_DESTDIR	TOOLS_DIR
TOOLS_FAIL	TOOLS_GNU_MISSING
TOOLS_LDCONFIG	TOOLS_NOOP
TOOLS_PATH	TOOLS_SCRIPT
TOOLS_USE_CROSS_COMPILE	TOOL_DEPENDS
TTF_FONTDIR	TTF_FONTS_DIR
TYPE	UAC_REQD_EXECS
UCSPI_SSL_GROUP	UCSPI_SSL_USER
UDEV_REQUIRED	UDEV_TYPE
UNBOUND_GROUP	UNBOUND_LOG_FACILITY
UNBOUND_USER	UNLIMIT_RESOURCES
UNPRIVILEGED	UNPRIVILEGED_GROUP
UNPRIVILEGED_GROUPS	UNPRIVILEGED_USER
UNWRAP_FILES	UNWRAP_PATTERNS
UPDATE_GEMSPEC	UPDATE_TARGET
URI	USERGROUP_PHASE
USERPPP_GROUP	USER_SPECIFIC_PKGS
USE_ABI_DEPENDS	USE_ADA_FEATURES
USE_APR	USE_BSD_MAKEFILE
USE_BUILTIN	USE_CC_FEATURES
USE_CROSS_COMPILE	USE_CURSES
USE_CWRAPPERS	USE_CXX_FEATURES
USE_DB185	USE_FEATURES
USE_GAMESGROUP	USE_GCC_RUNTIME
USE_IMAKE	USE_INDIRECT_DEPENDS
USE_JAVA	USE_JAVA2
USE_LANGUAGES	USE_LIBTOOL
USE_NATIVE_GCC	USE_NETBSD_REPO
USE_PKGSRC_GCC	USE_PKGSRC_GCC_RUNTIME
USE_PKGTASKS	USE_PKG_ADMIN_DIGEST
USE_RUBY_EXTCONF	USE_RUBY_INSTALL

USE_RUBY_SETUP	USE_RUBY_SETUP_PKG
USE_TMPFILES	USE_TOOLS
UUCP_GROUP	UUCP_USER
VARBASE	VARNAME
VIM_EXTRA_OPTS	WARNING_MSG
WCALC_CGIDIR	WCALC_CGIPATH
WCALC_HTMLDIR	WCALC_HTMLPATH
WDM_MANAGERS	WRAPPER_CC
WRAPPER_REORDER_CMDS	WRKDIR
WRKDIR_BASENAME	WRKDIR_LOCKTYPE
WRKLOG	WRKOBJDIR
WRKSRC	X10_PORT
X11	X11BASE
X11_PKGSRCDIR	X11_TYPE
X509_CERTIFICATE	X509_KEY
XAW_TYPE	XLOCK_DEFAULT_MODE
XMKMF	XMKMF_FLAGS
XXX	XXXX
YES	ZSH_STATIC
__stdc__	_vargroups
accept	acquire-localbase-lock
acquire-lock	add
added	administrator
alloca	alternatives
aslr	asprintf
atlas	autoconf
automake	autoreconf
awk	barrier
bash	big-endian
bin-install	bind
binpkg-list	blas
bootstrap-depends	broken
broken_on_platform	bsd
bsd.prog.mk	build
build-env	buildlink-directories
buildlink-oss-soundcard-h	built-in

builtin	c
c++	ccache
cce	cdefs
ceil	changes
changes-entry	changes-entry-noupdate
check	check-clean
check-files	check-files-clean
check-hackage	check-vulnerable
checksum	checksum-phase
clean	clean-depends
cleandir	codeberg
commit	commit-changes-entry
compact	compiler
conf	config.guess
config.sub	configuration
configure	configure-env
configure-help	configure_args
connect	cos
cpe	cputime
create-usergroup	csh
ctf	cvs
debug	debug-barrier
declaration	declare
defined	depend
dependencies	depends
depends-checksum	depends-fetch
describe	destdir
disable	distclean
distfiles	distinfo
dl	dlopen
do-buildlink	do-clean
do-configure	do-configure-post-hook
do-extract	do-fetch
do-install	download
emul	emul-distinfo
emul-fetch	emulation

emulator	enable
endian	endif
enomem	ensurepip
err	errx
etc	exp
extract-rpm	fabs
feature	features
fetch	fetch-list
follows	forbids
form	format
fortify	fortify_source
fossil	friend
fts	fts_close
fts_open	fts_read
fts_set	gcc
gethostbyaddr	gethostbyname
gethostbyname2	getopt_long
getprogname	getservbyname
getservbyport	getservent
gettext	git
github	gitlab
glob	gnu
gnu_configure_strict	go
golang	guess-license
hashbang	heimdal
help	hg
imake	in-tree
increment	indirect
inet_aton	install
install-env	interp
interpreter	intl
ip4	ip6
ipv4	ipv6
iso	jobs
kerberos	krb
krb5	ksh

lapack	latex
libiconv	libintl_bindtextdomain
libintl_gettext	libintl_textdomain
libnbcompat	libs
libtool	licence
license	lintl
little-endian	lock
locking	lua
lvalue	machine_endian
make	makedistinfo
makepatchsum	makesum
mdi	memory
mercurial	meta
meta-package	meta_package
mit-krb5	mk.conf
mkl	mount
move	moved
mprotect	mps
mremap	native
nb	nbcompat
netlib	network
node	node.js
nodejs	obstack
obstack_ptr_grow	occurs
only	openblas
options	options.mk
order	override
override-intltool	override-message-intltool
package	parallel
path	pax
paxctl	pbulk-index
pc	perl
perl5	perms
php	pkg-build-options
pkg-config	pkg_build_options
pkgsrc	platform

plist	post-extract
post-fetch	post-wrapper
pre-build-checks-hook	pre-configure-checks-hook
pre-extract	pre-fetch
print-plist	print-summary-data
privileged-install-hook	pypi
python	r
readme-all	recursive
recv	recvfrom
regcomp	release-localbase-lock
release-lock	relro
remove	removed
rename	renamed
reorder	replace
replace_interpreter	reproducible
resolv	root
ruby	send
sendfile	sendto
setenv	setgid
setprogname	setuid
sh	shebang
show	show-all
show-build-defs	show-depends
show-depends-dirs	show-depends-pkgpaths
show-depends-recursive	show-deps
show-distfiles	show-downlevel
show-subdir-var	show-tools
show-var	show-vars
snprintf	socket
ssp	st_mode
stage-install	strcasestr
strict	strip
strndup	strnlen
strsep	subst
substitutions	subversion
sun	sunpro

sunwspro	svn
symlink	test
test-env	tex
texlive	tmp
todo	tool
tools	tools-libtool-m4-override
transitive	type
ulimit	undefined
undo-replace	unlimit
unprivileged	unprivileged-install-hook
unstripped	update
updated	upload
upload-distfiles	use_tools
user	utimes
vasprintf	verbose
vsnprintf	warn
warning	warnings
warnx	wattr_off
wattr_on	work
wrapper	wrkdir

Приложение F. Рекомендации по редактированию руководства pkgsrc

В этом разделе рассказывается о редактировании самого руководства pkgsrc.

F.1. Цели make

Исходный код руководства pkgsrc хранится в `pkgsrc/doc/guide/files`, и из него создаётся несколько файлов:

- `pkgsrc/doc/pkgsrc.txt`
- `pkgsrc/doc/pkgsrc.html`
- https://www.NetBSD.org/docs/pkgsrc/
- https://www.NetBSD.org/docs/pkgsrc/pkgsrc.pdf: версия руководства pkgsrc в формате PDF.
- https://www.NetBSD.org/docs/pkgsrc/pkgsrc.ps: версия руководства pkgsrc в формате PostScript.

F.2. Порядок действий

Порядок редактирования руководства pkgsrc таков:

1. Убедитесь, что вы извлекли репозиторий htdocs в каталог, соседний с вашим каталогом pkgsrc. Вам понадобится возможность фиксировать изменения как из pkgsrc, так и из htdocs.
2. Убедитесь, что у вас установлены пакеты, необходимые для повторной сборки руководства pkgsrc (а также другой документации NetBSD на основе XML). Они устанавливаются автоматически вместе с пакетом `meta-pkgs/pkgsrc-guide-tools`.
3. Выполните **cd doc/guide**, чтобы перейти в нужный каталог. Все дальнейшие действия выполняются здесь.
4. Отредактируйте XML-файлы в `files/`.
5. Выполните **bmake**, чтобы проверить руководство pkgsrc на корректность XML и собрать итоговые выходные файлы. Если на этом этапе возникнут ошибки, просто отредактируйте файлы: в рабочем каталоге находятся только символические ссылки, указывающие на файлы в `files/`.
6. **(cd files && cvs commit)**
7. Выполните **bmake clean && bmake**, чтобы заново создать выходные файлы с правильными идентификаторами RCS.
8. Выполните **bmake regen**, чтобы установить и зафиксировать файлы как в `pkgsrc/doc`, так и в `htdocs`.

> **ПРИМЕЧАНИЕ**
>
> Если вы добавили, удалили или переименовали какие-либо главы, их нужно синхронизировать с помощью **cvs add** или **cvs delete** в каталоге htdocs.